Data Mining Notes

Academic Year 2026/2027

Contents

1 Introduction to Data Mining and the KDD Process **1**
1.1 Foundations, Motivations, and Formal Definition 1
1.2 The Confluence of Disciplines and the KDD Process 3
1.3 Taxonomy of Data Mining Tasks 6
1.4 Open Challenges in Modern Data Mining 8

2 Data Understanding I: Data Types, Attributes, and Descriptive Statistics **10**
2.1 Understanding Data and Dataset Taxonomy 10
2.2 Attribute Types and Measurement Scales 12
2.3 Data Quality Issues 14
2.4 Univariate Distribution Inspection 14
2.5 Measures of Central Tendency and Dispersion 16
2.6 Five-Number Summary, Quartiles, and Box Plots 17

3 Data Understanding II – Multivariate Analysis, Correlation, and Outlier Detection **19**
3.1 Multidimensional Visualization Techniques 19
3.2 Bivariate Dependence Analysis: Covariance and Correlation . 24
3.3 Outlier Taxonomy and Detection Methodologies 28
3.4 Introduction to Missing Values 29

4 Data Preparation & Cleaning – Reduction, Discretization, and Transformation **31**
4.1 Distinction Between Data Understanding and Data Preparation 31
4.2 Aggregation and Sampling 32
4.3 Dimensionality Reduction 34
4.4 Principal Component Analysis (PCA) 36
4.5 Handling Missing Values 39
4.6 Discretization and Binarization Techniques 40
4.7 Functional Attribute Transformations and Variance Stabilization 42

4.8 Normalization Algorithms 44
4.9 Univariate Outlier Treatment 45

5 Python Lab: Data Understanding and Data Preparation 47
5.1 Environment, Ingestion, and Data Integration 48
5.2 Missing Value Diagnosis and Imputation 50
5.3 Multivariate Exploration and Correlation Analysis 51
5.4 Feature Transformation, Discretization, and Encoding 53
5.5 Dimensionality Reduction: PCA and t-SNE 55

6 Proximity Measures, Information Theory, and Cluster Analysis 58
6.1 Foundations of Similarity, Dissimilarity, and Proximity 58
6.2 Single-Attribute Proximity Measures 59
6.3 Vector Distances and Minkowski Metrics 61
6.4 Worked Example of Distance Matrix Computation 62
6.5 Metric Axioms and Mathematical Properties 63
6.6 Binary Proximity Measures: SMC vs Jaccard Index 64
6.7 Cosine Similarity for Document Data 65
6.8 Weighting, Missing Values, and Correlation Across Objects . . 66
6.9 Information Theory: Entropy and Mutual Information 67
6.10 Foundations and Objectives of Cluster Analysis 69
6.11 Clustering Taxonomies and Grouping Structures 70
6.12 Cluster Types and Mathematical Formulations 72
6.13 Input Data Characteristics and Algorithm Families 73

7 Cluster Analysis: K-Means and Hierarchical Clustering Algorithms 74
7.1 Introduction to Partitional and Hierarchical Cluster Analysis . 74
7.2 K-Means: Lloyd's Algorithm and Voronoi Geometry 75
7.3 SSE Objective Function and Proof of Convergence 79
7.4 Choice of Parameter K, Local Minima, and Initialization . . . 81
7.5 Initialization Heuristics and Variants of K-Means 83
7.6 Structural Limitations of K-Means and Adaptation Techniques 85
7.7 Foundations of Hierarchical Clustering 86
7.8 Agglomerative Algorithm and Linkage Criteria 88
7.9 MST-Based Divisive Hierarchical Clustering 93
7.10 Complexity, Structural Limits, and Synoptic Comparison . . . 94

8 Cluster Analysis: Density-Based Clustering (DBSCAN) 97
8.1 Introduction to the Density-Based Paradigm 98
8.2 Density Foundations and Point Taxonomy 100
8.3 Density-Connectivity Relations 103
8.4 Operational Architecture of DBSCAN 105
8.5 Spatial Data Case Study 109
8.6 Strengths and Geometric Capabilities 109
8.7 Critical Limitations and Application Challenges 110
8.8 Heuristic Parameter Selection: The k-Distance Plot 112
8.9 Global Comparative Synthesis: K-Means vs Hierarchical vs DBSCAN 114

9 K-Means Variants, BIC Criterion, Mixture Models, and EM 115
9.1 Scientific Context and Limitations of Standard K-Means . . . 116
9.2 The Bisecting K-Means Algorithm 118
9.3 The X-Means Algorithm and Model Selection 121
9.4 The Bayesian Information Criterion (BIC) 123
9.5 Mixture Models and the EM Algorithm 127
9.6 Normal Distributions and Statistical Estimation Foundations . 131
9.7 Case Study: Deconvolving a Two-Gaussian Mixture 132
9.8 Architectural and Systematic Comparison: K-Means vs. EM . 133

10 Cluster Validity: Metrics, Theory, and Methodologies 136
10.1 Foundations and Challenges of Cluster Validity 137
10.2 Taxonomy of Cluster Validation Measures 139
10.3 Clustering Tendency and the Hopkins Statistic 141
10.4 Internal Validation via Proximity Matrices 144
10.5 Cohesion and Separation: Theory and Algebraic Equivalence . 145
10.6 The Silhouette Coefficient 149
10.7 Internal Indices: Davies-Bouldin and Caliński-Harabasz 151
10.8 External Validation: Entropy, Purity, and Agreement 152
10.9 Statistical Framework and Null Models 154
10.10 Python Application Laboratory: Metrics and Model Selection 156

11 Association Analysis: Basic Concepts and Algorithms 160
11.1 Introduction to Association Rule Mining 160
11.2 Formal Definitions: Itemsets, Support, and Frequency 162
11.3 Association Rules and Fundamental Evaluation Metrics 163
11.4 The Mining Task and Computational Tractability 164
11.5 Decoupling the Problem: The Two-Step Approach 165
11.6 Itemset Lattice and Search Space Complexity 166

11.7 The Apriori Principle and Anti-Monotonicity 168
11.8 Step-by-Step Apriori Execution Trace 169
11.9 Apriori Pseudocode and Candidate Generation Techniques . . 171
11.10Support Counting via Hash Tree Structures 173
11.11Rule Generation from Frequent Itemsets 174
11.12Factors Governing Computational Complexity 176
11.13Compact Representations: Maximal and Closed Itemsets . . . 177
11.14Pattern Evaluation and Limitations of Confidence 179
11.15Objective Statistical Measures of Interest 181
11.16Handling Categorical and Continuous Attributes 183
11.17Concept Taxonomies and Multi-Level Association Rules 184

12 Sequential Pattern Mining: Models, Definitions, and Exercises 187
12.1 Introduction and Motivations 187
12.2 From the Transactional Paradigm to the Sequential Paradigm 190
12.3 Taxonomy and Sequence Representation 193
12.4 Formalization of Sequence Data 195
12.5 Containment Relationship and Subsequence Definition 197
12.6 Sequential Pattern Support and Multiple Instances 200
12.7 Guided Exercise: Full Matching and Instance Enumeration . . 202

Chapter 1

Introduction to Data Mining and the KDD Process

1.1 Foundations, Motivations, and Formal Definition

1.1.1 What is Data Mining?

Data Mining (knowledge extraction from data) is the application of efficient computational algorithms and techniques to the analysis of large-scale data collections, aiming to discover previously unknown, non-trivial, valid, and potentially actionable patterns, regularities, correlations, and structures.

Conceptually, Data Mining is neither simple database querying (as in SQL relational queries) nor basic descriptive statistical computation. Its core objective is uncovering **hidden knowledge**: latent dependency structures, implicit models, and predictive relationships that remain undetected through manual inspection or conventional analytical tools.

Definition 1.1.1 (Data Mining)**. Data Mining** is the computational process of automatically or semi-automatically discovering patterns, regularities, correlations, or latent structures within massive datasets (*Big Data*), yielding valid, novel, useful, and interpretable models for human decision-makers.

1.1.2 Application Context: Commercial and Scientific Drivers

The necessity of computational disciplines dedicated to pattern extraction stems from the exponential expansion of data generated, stored, and processed globally:

- **Commercial Perspective:** E-commerce transactions, web browsing logs, retail scanner logs, and financial flows (credit card transactions, wire transfers, POS logs) generate continuous data streams in the order of terabytes or petabytes daily. Modern enterprises and financial institutions cannot gain strategic leverage through passive archival; they require transforming raw logs into behavioral insights for Customer Relationship Management (CRM), personalized recommendations, churn prevention, and real-time fraud mitigation.
- **Scientific Perspective:** Remote sensing and advanced instrumentation produce unprecedented data volumes. Earth observation satellites (e.g., the NASA EOSDIS program) ingest petabytes of environmental measurements yearly; digital sky surveys scan billions of celestial objects; high-throughput genomics generates nucleotide sequences continuously; and computational physical and climate simulations yield terabytes within hours. Scientific hypothesis formulation and empirical analysis inherently require scalable automated synthesis.

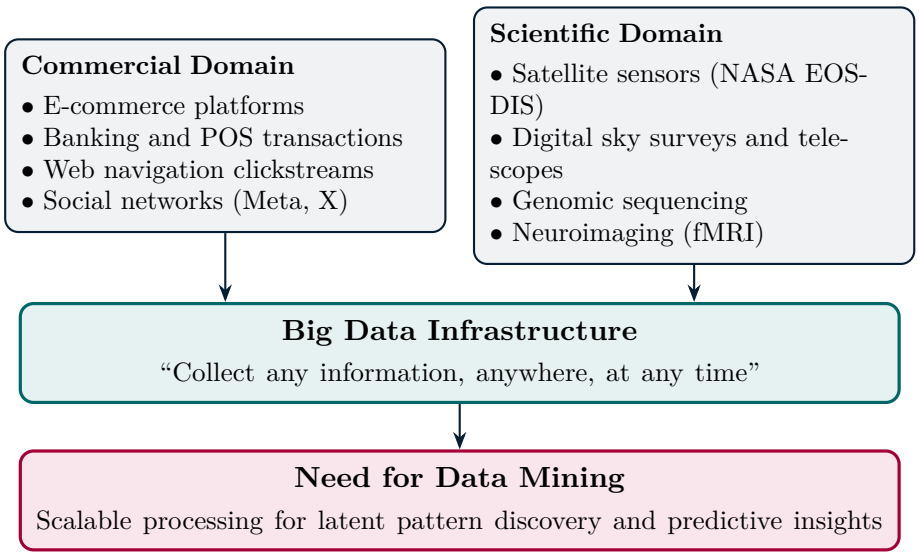


Figure 1.1: Genesis and evolution of Data Mining across commercial and scientific drivers.

1.1.3 Big Data as a Proxy for Human Activity and Global Challenges

Digital traces generated by connected devices, mobile nodes, and online services act as an empirical *quantitative proxy* for human behaviors and social

dynamics:

1. **Purchasing Patterns and Lifestyles:** Supermarket receipts, loyalty cards, and online baskets trace dietary habits, price elasticity, and household economics.

2. **Social Networks and Interactions:** Interaction graphs across social platforms reveal interpersonal contact topology and information diffusion dynamics.

3. **Public Sentiment and Collective Mood:** Search queries, microblogging posts, and collaborative encyclopedias measure public interests and sentiment trends in real time.

4. **Mobility and Transportation:** GPS sensors, cellular handovers, and air traffic matrices model mobility at both local levels (urban congestion) and global scales (pandemic pathogen dispersal tracking via platforms such as *BioDiaspora*).

These data-driven techniques enable society to tackle critical global challenges: healthcare efficiency, smart-grid energy optimization, climate change impact forecasting, and agricultural yield planning.

1.2 The Confluence of Disciplines and the KDD Process

1.2.1 A Multidisciplinary Confluence

Data Mining is inherently cross-disciplinary, emerging at the intersection of several core fields:

- **Database Systems:** Provide the infrastructure for scalable data storage, indexing, secondary-memory access, and distributed retrieval.

- **Statistics:** Furnishes mathematical foundations for uncertainty quantification, sampling, hypothesis testing, and probabilistic modeling.

- **Machine Learning:** Introduces adaptive architectures and optimization algorithms capable of learning empirical rules to generalize on unseen data.

- **Algorithms and Computational Theory:** Ensures analytical procedures remain computationally feasible with respect to time and memory complexities.
- **Visualization:** Enables intuitive perceptual data exploration and the effective communication of analytical findings.

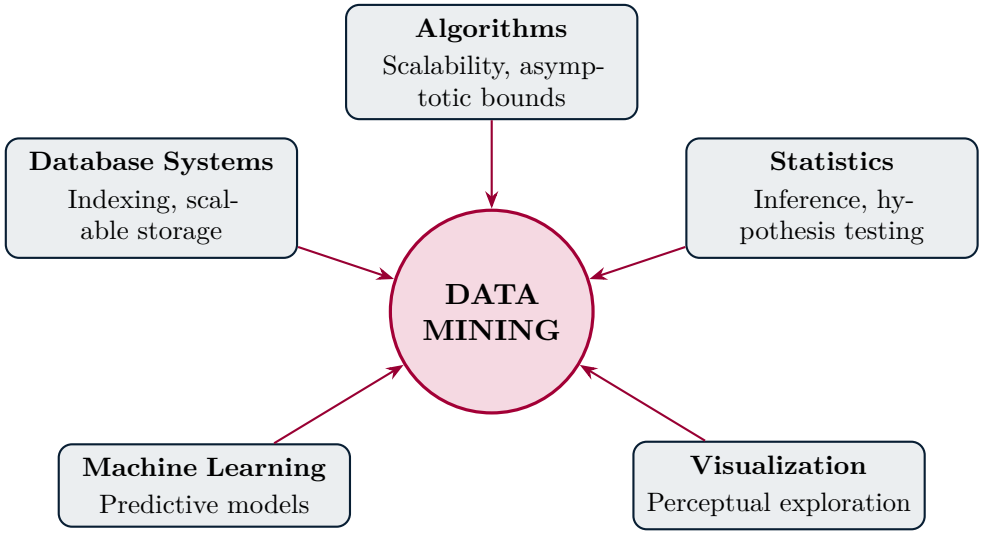


Figure 1.2: The multidisciplinary confluence of Data Mining across computational disciplines.

1.2.2 The KDD Process (Knowledge Discovery in Databases)

Data Mining is the algorithmic centerpiece of an iterative, multi-stage engineering methodology known as **KDD** (*Knowledge Discovery in Databases*):

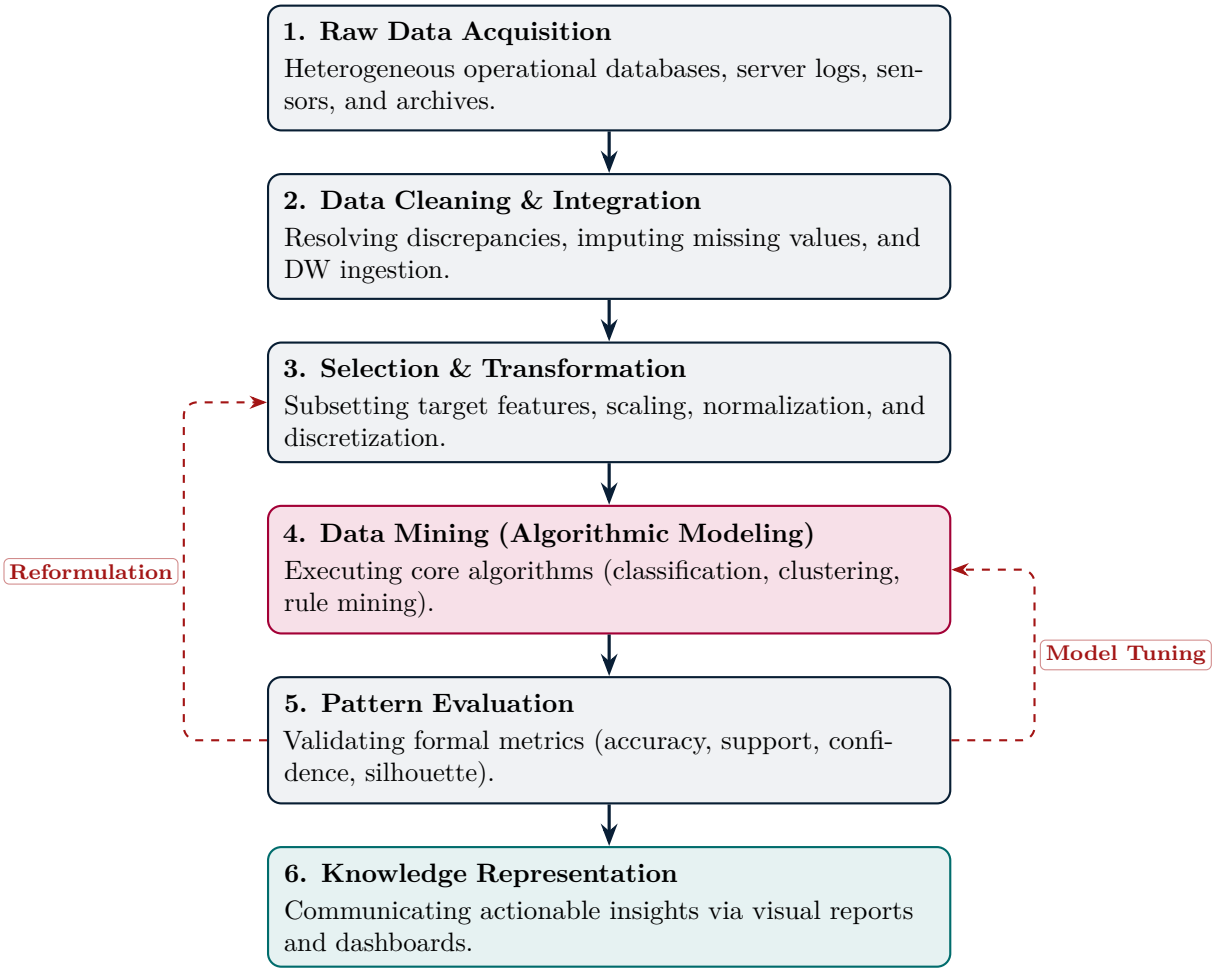


Figure 1.3: Step-by-step workflow of the KDD process with iterative feedback refinement loops.

Detailed Phases of the KDD Process:

1. **Business / Goal Understanding:** Formalizing domain objectives, defining KPIs, and setting operational constraints.

2. **Data Cleaning & Integration:** Reconciling fragmented data feeds, correcting noise and errors, and centralizing into a unified schema.

3. **Selection & Transformation:** Filtering target-specific entities and constructing engineered features.

4. **Data Mining:** Deploying supervised or unsupervised algorithms to uncover structural invariants.

5. **Pattern Evaluation:** Validating hypotheses against statistical benchmarks and domain expectations.

6. **Knowledge Representation:** Visualizing and operationalizing findings for stakeholder decision-making.

1.3 Taxonomy of Data Mining Tasks

Data Mining tasks are systematically categorized into two macro-classes:

1. **Predictive Methods:** Infer an inductive model mapping input predictors to an unknown continuous or categorical target attribute.

2. **Descriptive Methods:** Explore underlying groupings, correlations, and topological structures without an explicit target variable.

1.3.1 Classification (Predictive Modeling)

Classification maps an input vector $x \in \mathcal{X}$ to a discrete category label $y \in \mathcal{Y} = \{c_1, c_2, \ldots, c_k\}$. Given a labeled training set:

$$\mathcal{D}_{\text{train}} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\} \tag{1.1}$$

the induction algorithm estimates an optimal classification mapping:

$$f : \mathcal{X} \to \mathcal{Y} \tag{1.2}$$

that minimizes expected classification error on unseen test samples.

Example: Credit Worthiness Assessment Consider a toy loan-granting dataset:

TID	**Employed**	**Level of Education**	**Years at Address**	**Credit Worthy (Y)**
1	Yes	Graduate	5	**Yes**
2	Yes	High School	2	**No**
3	No	Undergrad	1	**No**
4	Yes	High School	10	**Yes**

Table 1.1: Toy dataset for inductive decision tree credit evaluation.

A decision tree induction algorithm induces the following hierarchical structure:

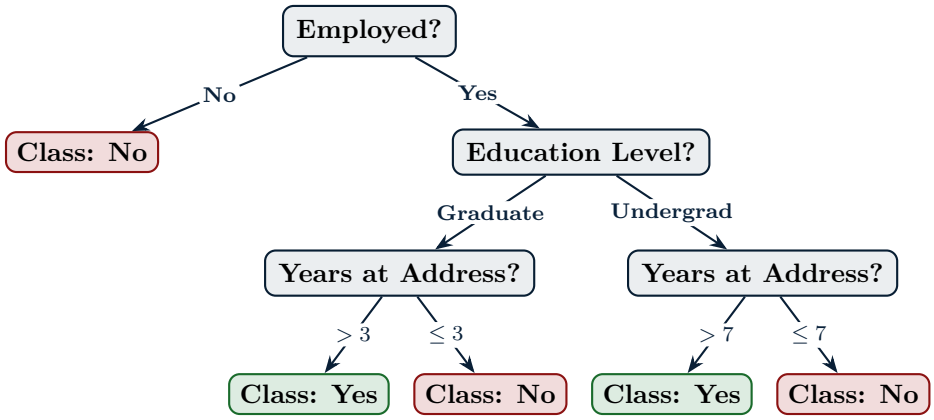


Figure 1.4: Decision tree inferred from the credit worthiness dataset.

1.3.2 Clustering (Unsupervised Partitioning)

Clustering partitions an unlabeled dataset $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$ into k disjoint subsets $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$ such that:

1. **Minimized Intra-Cluster Distance:** Items grouped within the same cluster C_i exhibit high mutual similarity.
2. **Maximized Inter-Cluster Distance:** Items in distinct clusters C_i and C_j $(i \neq j)$ are well-separated in feature space.



Figure 1.5: Conceptual clustering objective: intra-group cohesion and inter-group separation.

1.3.3 Association Rule Discovery

Association analysis identifies co-occurrence affinities within transactional databases. Given an itemset $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ and a collection of transactions $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ with $t_j \subseteq \mathcal{I}$, an **association rule** is defined as:

$$X \implies Y \quad \text{where } X, Y \subset \mathcal{I} \text{ and } X \cap Y = \emptyset \tag{1.3}$$

indicating that transactions containing X (antecedent) systematically contain Y (consequent).

TID	**Items Purchased**
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diapers, Milk
4	Beer, Bread, Diapers, Milk
5	Coke, Diapers, Milk

Table 1.2: Market Basket transaction log.

Prominent extracted rules include:

$$\{\text{Milk}\} \implies \{\text{Coke}\}, \qquad \{\text{Diapers}, \text{Milk}\} \implies \{\text{Beer}\} \tag{1.4}$$

1.3.4 Anomaly Detection (Deviation Detection)

Anomaly detection identifies rare samples or trajectories deviating substantially from standard data generation mechanisms, applied across network intrusion monitoring, payment fraud prevention, predictive equipment failure analysis, and ecological disturbance tracking.

1.4 Open Challenges in Modern Data Mining

1. **Scalability:** Sub-quadratic computational requirements to cope with petabyte-scale datasets.

2. **High Dimensionality:** Combating geometric volume inflation and distance concentration.

3. **Complex and Heterogeneous Data:** Harmonizing relational tables, graphs, textual corpora, temporal series, and high-frequency sensor streams.

4. **Distributed Ownership and Privacy:** Complying with regulatory frameworks and privacy constraints across distributed networks.

5. **Dynamic and Non-Stationary Environments:** Accommodating concept drift and online stream learning.

Chapter 2

Data Understanding I: Data Types, Attributes, and Descriptive Statistics

2.1 Understanding Data and Dataset Taxonomy

Data Understanding is the diagnostic investigation in which the analyst examines data quality, understands domain feature semantics, and uncovers initial indications of noise, extreme outliers, or missing values.

Core Objectives of Data Understanding

- What types of attributes constitute the dataset?
- What is the overall data quality, and what syntactic/semantic errors exist?
- Are there univariate or multivariate outliers present?
- What pairwise correlations and dependencies connect features?
- How are missing values distributed, and what mechanism generated them?
- Is feature transformation or feature engineering required prior to mining?

2.1.1 Fundamental Structure: Objects and Attributes

A dataset is a formal collection of **data objects** (also termed *records, points, samples, entities, instances, or cases*) described by a set of **attributes** (known as *variables, features, fields, or dimensions*).

- An **object** represents a single real-world entity (a customer, a bank transaction, a sensor reading).
- An **attribute** denotes a measurable property or characteristic of that entity (height, price, temperature, category).

2.1.2 Classification of Data Structures

Datasets differ fundamentally by their underlying structural topology:

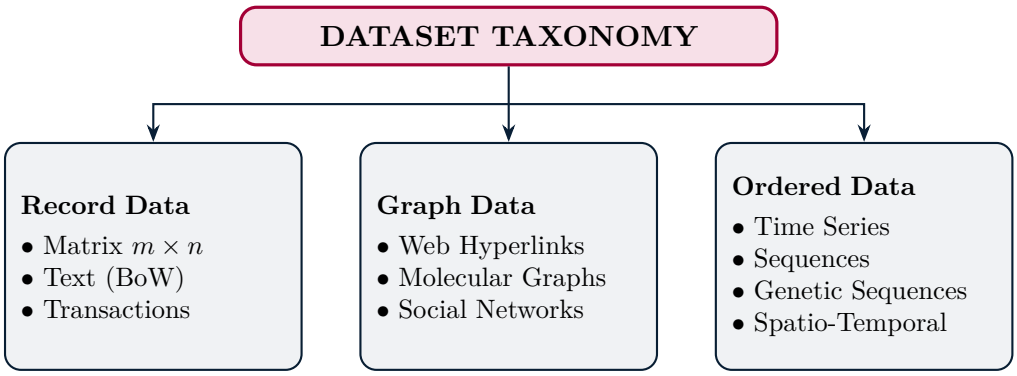


Figure 2.1: Structural taxonomy of major dataset formats.

1. Record Data

- **Data Matrix:** When all m objects share the same fixed set of n continuous numerical attributes, the dataset is modeled as an $m \times n$ matrix:

$$D = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{2.1}$$

  where each row i is a point in continuous Euclidean space $\mathbb{R}^n$, and each column is a coordinate axis.

- **Document Data (Document-Term Matrix):** Unstructured texts modeled via *Bag-of-Words*. Columns represent vocabulary terms and values record word frequencies:

Document	team	coach	play	ball	score	game	win	lost	timeout	season
Doc 1	3	0	5	0	2	6	0	2	0	2
Doc 2	0	7	0	2	1	0	0	3	0	0
Doc 3	0	1	0	0	1	2	2	0	3	0

Table 2.1: Document-Term Matrix representation for text collections.

- **Transaction Data:** Each record is identified by a unique TID and contains a variable-length set of discrete items (e.g., supermarket market basket).

2. Graph Data

Explicitly models pairwise entities: World Wide Web hyperlinks, molecular structures (e.g., benzene C_6H_6 covalent bonds), and citation networks.

3. Ordered Data

Data where temporal or spatial precedence governs informational semantics:

- **Time Series:** Continuous or discrete time-indexed readings with high autocorrelation.
- **Sequential Data:** Chronologically ordered event sequences (e.g., $(A, B) \rightarrow (D) \rightarrow (C, E)$).
- **Genomic Sequences:** Nucleotide polymer chains (‘A‘, ‘C‘, ‘G‘, ‘T‘).
- **Spatio-Temporal Data:** Space-time indexed matrices (e.g., global sea-surface temperature grids).

2.2 Attribute Types and Measurement Scales

Mathematical admissibility of operations on attribute domains:

- **Distinctness ($=$ or $\neq$):** Testing equality between values.
- **Order ($<$ or $>$):** Natural total or partial ordering.

- **Difference ($+$ or $-$):** Constant-metric measurable intervals.
- **Ratio ($*$ or $/$):** Natural absolute zero (total absence of measured property).

Scale	Distinct.	Order	Diff.	Ratio	Permissible Statistical Operators
Nominal	✓	–	–	–	Mode, entropy, χ^2 test
Ordinal	✓	✓	–	–	Median, quantiles, Spearman rank ρ
Interval	✓	✓	✓	–	Mean, standard deviation, Pearson r
Ratio	✓	✓	✓	✓	Geometric mean, coefficient of variation

Table 2.2: Hierarchy and mathematical properties of measurement scales.

2.2.1 Categorical (Qualitative) Attributes

1. **Nominal:** Allows only equality comparison ($=, \neq$). Examples: eye color, zip code, tax ID.
2. **Binary:** Nominal attributes with exactly two values $\{0, 1\}$.
    - *Symmetric Binary:* Both outcomes carry equal weight (e.g., gender Male/Female).
    - *Asymmetric Binary:* One outcome is rare or of focal interest ($1 =$ positive medical test, $0 =$ negative).
3. **Ordinal:** Natural ordering ($<, >$), but step distance is non-constant. Examples: Mohs hardness scale, ratings $\{\text{poor}, \text{fair}, \text{good}\}$, letter grades $\{A, B, C, D, F\}$.

2.2.2 Numeric (Quantitative) Attributes

1. **Interval-Scaled:** Differences are meaningful, but the scale **lacks an absolute zero**. Examples: Celsius or Fahrenheit temperature (20°C is not "twice as hot" as 10°C, since in Kelvin $293.15\,\text{K} \neq 2 \cdot 283.15\,\text{K}$); calendar dates.

2. **Ratio-Scaled:** Possesses a **natural absolute zero**. Examples: length, weight, age, Kelvin temperature (0 K absolute zero), monetary counts. Stating that a 3-hour match is 50% longer than a 2-hour match ($3/2 = 1.5$) is mathematically valid.

2.2.3 Discrete vs Continuous Variables

- **Discrete Attribute:** Finite or countably infinite set of values, represented by integers.
- **Continuous Attribute:** Real-valued domain ($\mathbb{R}$), represented by floating-point numbers.

2.3 Data Quality Issues

Flawed data invalidates downstream analytical pipelines (*Garbage In, Garbage Out*). Redman (2004) estimated poor data quality costs organizations $10\% \div 20\%$ of total revenue.

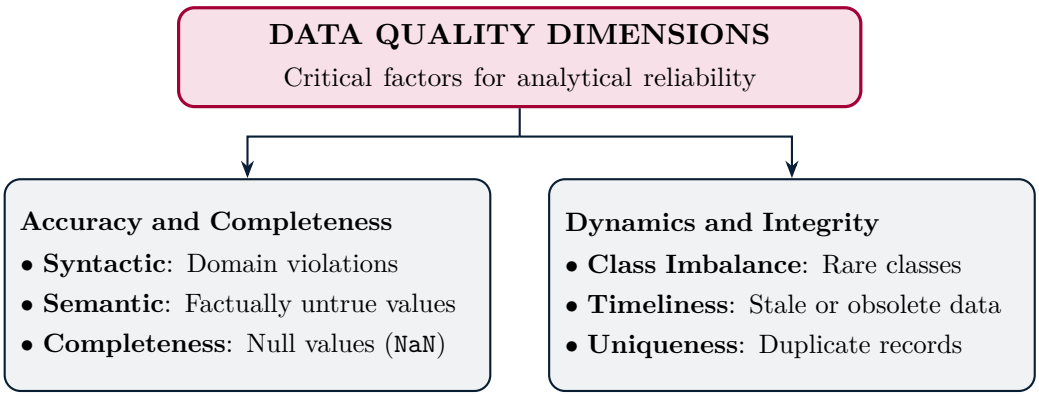


Figure 2.2: Core dimensions of data quality.

2.4 Univariate Distribution Inspection

2.4.1 Skewness and Multimodality

The shape of continuous probability density functions reveals underlying mechanisms:

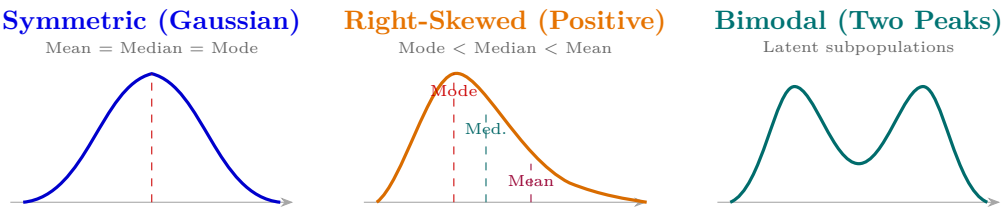


Figure 2.3: Univariate distribution profiles: symmetry, positive skewness, and multimodality.

- **Symmetric Distribution:** Reflectional symmetry about the center: Mean = Median = Mode.
- **Positively Skewed (Right-Skewed):** Long tail towards higher values (e.g., individual wealth):

$$\text{Mode} < \text{Median} < \text{Mean} \tag{2.2}$$

- **Negatively Skewed (Left-Skewed):** Tail stretching towards small values:

$$\text{Mean} < \text{Median} < \text{Mode} \tag{2.3}$$

- **Bimodal / Multimodal Distribution:** Signals overlapping subpopulations (e.g., adult height without conditioning on biological sex).

2.4.2 Visual Tools: Bar Charts vs Histograms

- **Bar Chart:** Plots discrete categories with visible spacing between bars.
- **Histogram:** Partitions continuous domains into contiguous intervals (*bins*); rectangle area is proportional to empirical sample counts.

Bin Sizing and Sturges' Rule: Under approximately normal distributions, Sturges' rule determines optimal bin counts k:

$$k = \lceil \log_2(n) + 1 \rceil \tag{2.4}$$

2.5 Measures of Central Tendency and Dispersion

2.5.1 Central Tendency

- **Sample Mean:**

$$\bar{x} = \frac{1}{m} \sum_{i=1}^{m} x_i \tag{2.5}$$

  Highly vulnerable to severe outliers.

- **Median:** Positional center in sorted arrays:

$$\text{median} = \begin{cases} x_{\left(\frac{m+1}{2}\right)} & \text{if } m \text{ is odd} \\ \dfrac{x_{\left(\frac{m}{2}\right)} + x_{\left(\frac{m}{2}+1\right)}}{2} & \text{if } m \text{ is even} \end{cases} \tag{2.6}$$

  A robust indicator resistant to extreme values.

- **Mode:** Most frequently observed value; uniquely defined across nominal attributes.

- **Trimmed Mean:** Evaluated by dropping a proportion α of extremes.

2.5.2 Measures of Dispersion

- **Range:** Range $= x_{\max} - x_{\min}$.

- **Sample Variance (s^2):**

$$s^2 = \frac{1}{m-1} \sum_{i=1}^{m} (x_i - \bar{x})^2 \tag{2.7}$$

  using Bessel's correction $(m-1)$ for unbiased population estimation.

- **Sample Standard Deviation (s):** $s = \sqrt{s^2}$.

Robust Dispersion Indicators:

- **Absolute Average Deviation (AAD):**

$$\text{AAD}(x) = \frac{1}{m} \sum_{i=1}^{m} |x_i - \bar{x}| \tag{2.8}$$

- **Median Absolute Deviation (MAD):**

$$\text{MAD}(x) = \text{median}\left(\left\{ |x_i - \text{median}(x)| \right\}_{i=1}^{m} \right) \tag{2.9}$$

2.6 Five-Number Summary, Quartiles, and Box Plots

2.6.1 The Five-Number Summary

The **Five-Number Summary** characterizes distributions using positional order statistics:

$$\left\{x_{\min},\ Q_1,\ \text{Median } (Q_2),\ Q_3,\ x_{\max}\right\} \tag{2.10}$$

- Q_1 (First Quartile): 25th percentile.
- Q_2 (Median): 50th percentile.
- Q_3 (Third Quartile): 75th percentile.
- **Interquartile Range (IQR):** $\text{IQR} = Q_3 - Q_1$.

2.6.2 Box Plot Anatomy

The **Box Plot** maps the five-number summary geometrically:

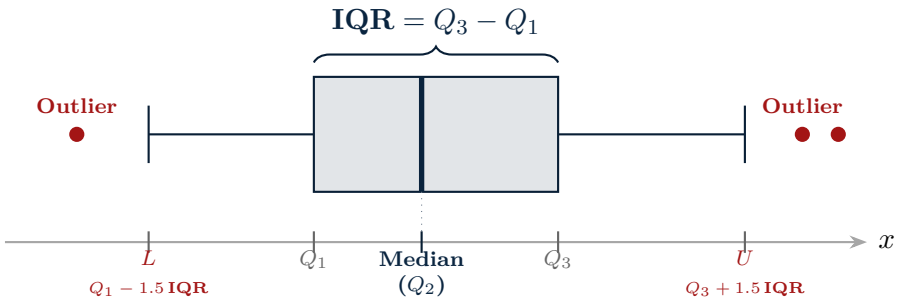


Figure 2.4: Box Plot anatomy with Tukey fences for univariate outlier demarcation.

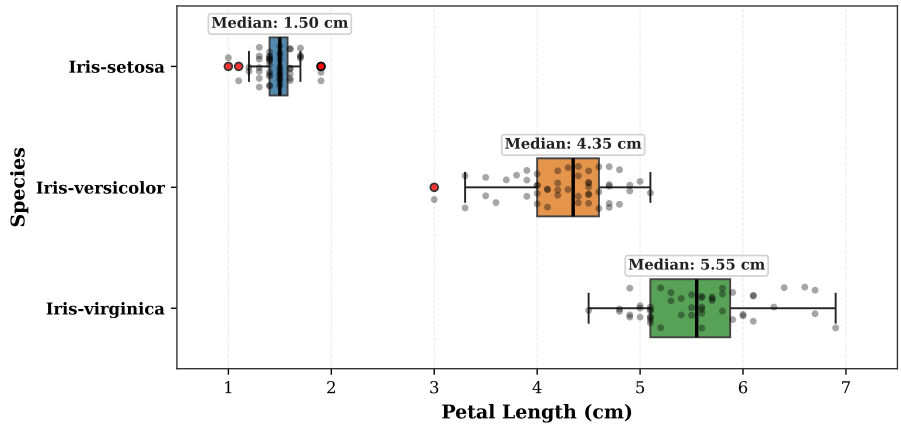


Figure 2.5: Empirical conditional box plots on the Iris dataset: petal length distributions partitioned by species.

Limitations of Box Plots Relative to Histograms: Box plots cannot detect multimodality. Identical five-number summaries can emerge from both bimodal and unimodal distributions; histograms remain necessary to reveal intermediate concavities.

Chapter 3

Data Understanding II – Multivariate Analysis, Correlation, and Outlier Detection

Multivariate analysis studies simultaneous relationships among two or more attributes within a dataset, overcoming the limitations of isolated univariate descriptions. In this chapter, we explore multidimensional visualization techniques, formal indices for linear and non-linear association and correlation, methodological criteria for outlier diagnosis, and an operational diagnostic checklist to conclude the Data Understanding phase.

The case study used as our benchmark is the famous **Iris** dataset (introduced by botanist Edgar Anderson and popularized by Ronald Fisher in 1936), composed of 150 observations evenly distributed across 3 balanced classes of 50 samples each (*Iris setosa*, *Iris versicolor*, *Iris virginica*) and described by 4 continuous features measured in centimeters:

- Sepal length (*Sepal Length*)
- Sepal width (*Sepal Width*)
- Petal length (*Petal Length*)
- Petal width (*Petal Width*)

3.1 Multidimensional Visualization Techniques

When exploratory analysis extends to bivariate and multivariate configurations, graphical representations must provide immediate visual insight

into geometric class separability, natural cluster formations, and structural anomalies.

3.1.1 Scatter Plot and Scatter Matrix

The **Scatter Plot** pairs two continuous attributes (X, Y), projecting observations as points with Cartesian coordinates $(x_i, y_i) \in \mathbb{R}^2$.

- **Key Capabilities**: Visually highlights clusters, direct or inverse correlation trends, and bivariate outliers that remain undetectable in univariate histograms.
- **Color Encoding**: By assigning distinct colors or markers according to class labels, practitioners can assess the geometric separability induced by the feature pair.
- **Evidence from Iris**: Crossing *Petal Length* and *Petal Width* reveals sharp class separation: *Setosa* forms an isolated compact cluster, whereas *Versicolor* and *Virginica* exhibit only modest boundary overlap (Figure 3.1). In contrast, the plane formed by *Sepal Length* and *Sepal Width* shows substantial inter-class confusion between *Versicolor* and *Virginica*.

When the number of numeric features p is moderate ($3 \leq p \leq 10$), a **Scatter Matrix (Scatter Plot Matrix - SPLOM)** is employed: a symmetric $p \times p$ grid containing all $\frac{p(p-1)}{2}$ distinct pairwise scatter plots. Univariate histograms or kernel density estimates (KDE) are typically placed along the main diagonal, enabling simultaneous inspection of univariate distributions and pairwise correlations (Figure 3.2).

The **Three-Dimensional (3D) Scatter Plot** extends Cartesian projection to three coordinates simultaneously (X_1, X_2, X_3). It allows identifying 3D outliers that appear completely normal in 2D projections, though planar displays suffer from visual occlusion and viewpoint dependence.

3.1.2 Visualizing High-Dimensional Spaces

In spaces where dimension count exceeds $p > 3$, standard orthogonal Cartesian projections fail to provide comprehensive views. Alternative coordinate geometries are required.

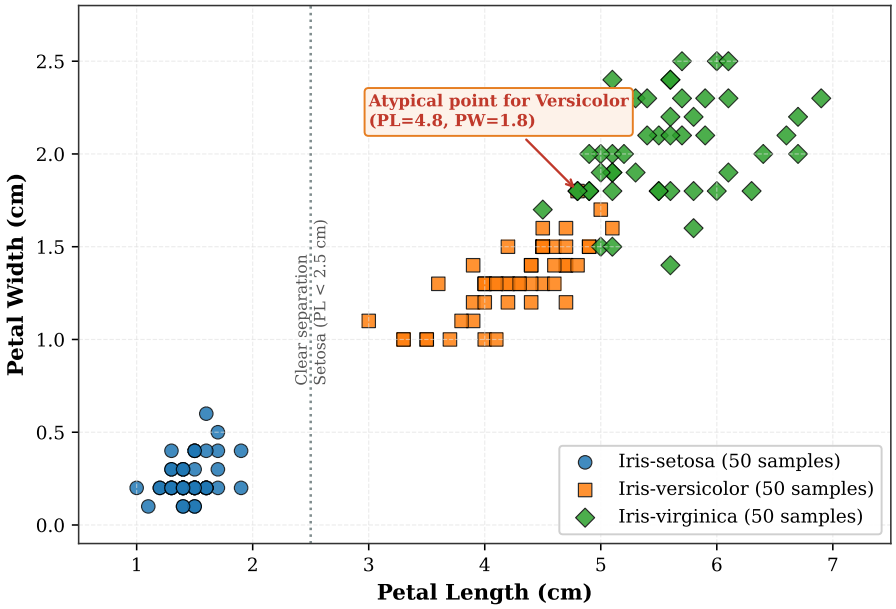


Figure 3.1: Bivariate scatter plots on the Iris dataset: comparing sepal and petal class separability.

Parallel Coordinates

In parallel coordinates, coordinate axes are not mutually orthogonal; instead, they are drawn as p equidistant, parallel vertical lines, each calibrated to the value range of its corresponding feature.

- **Geometric Mapping**: A p-dimensional observation $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip}) \in \mathbb{R}^p$ is represented not as a point, but as a continuous **polyline** crossing all parallel axes, intersecting the j-th vertical axis at height x_{ij}.
- **Diagnostic Traits**: Records sharing identical class labels tend to form compact bundles of polylines displaying coherent trajectories. An isolated polyline with an anomalous trajectory immediately flags a potential multivariate outlier.
- **Axis Ordering**: Visual legibility depends strongly on horizontal axis ordering. Placing strongly correlated attributes on adjacent axes reveals parallel bundles (positive correlation) or pencil-shaped X-crossings (negative correlation) (Figure 3.3).

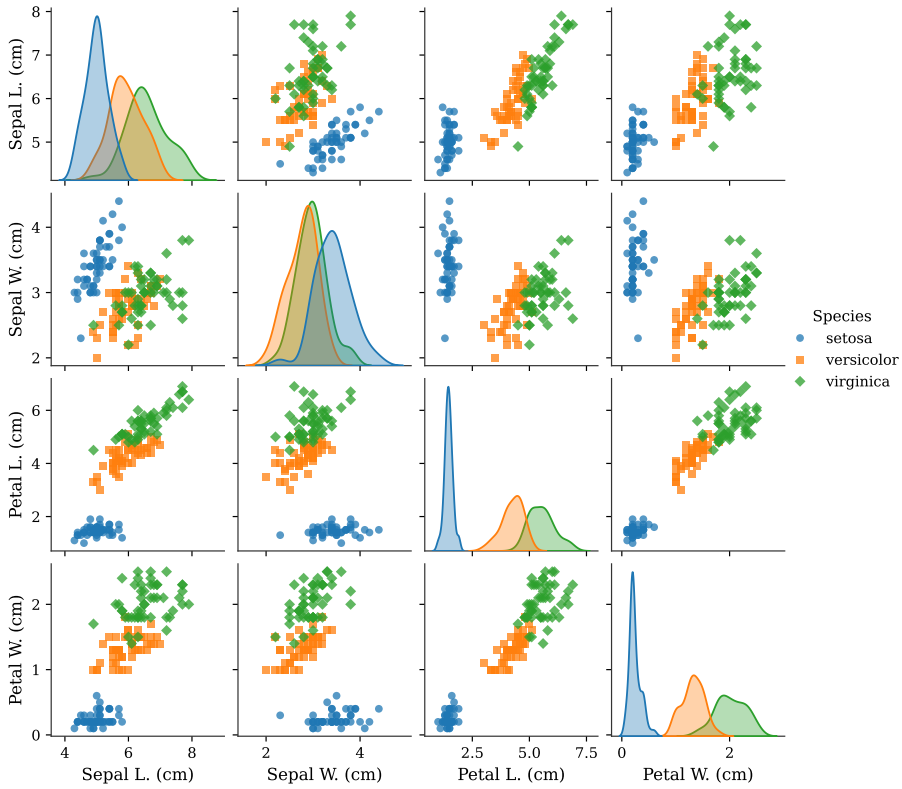


Figure 3.2: Complete Scatter Plot Matrix (SPLOM) across the four attributes of the Iris dataset.

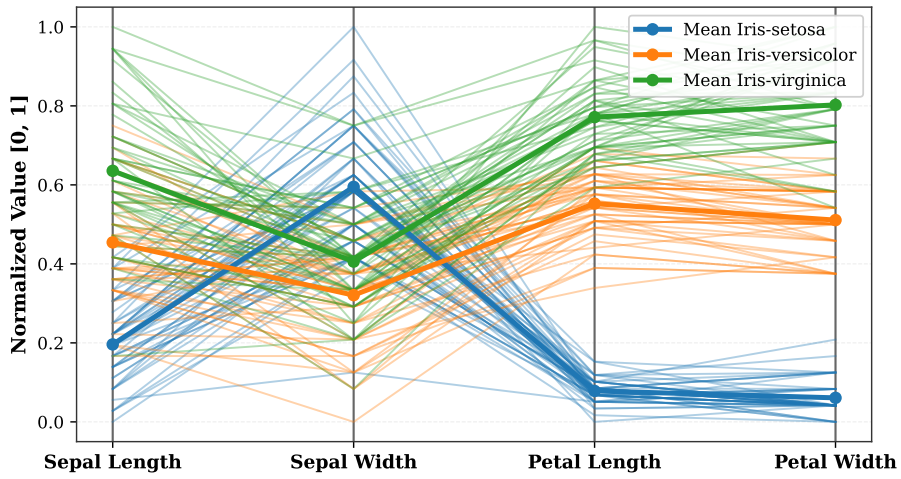


Figure 3.3: Parallel coordinates for the Iris dataset with class-stratified trajectories.

Radar Plot (Spider or Star Chart)

The **Radar Plot** shares the structural principle of parallel coordinates, but dimensional axes are arranged **radially emanating from a shared central pole**, separated by equi-spaced angles:

$$\theta_j = \frac{2\pi(j-1)}{p}, \quad j = 1, \ldots, p \tag{3.1}$$

For each data record, feature values determine the vertex distance along the corresponding radial spoke; connecting consecutive vertices produces a closed polygon. When radar plots are displayed as small individual multiples for each observation across a grid, they are called **Star Plots**, facilitating rapid geometric comparisons across multivariate instances (Figure 3.4).

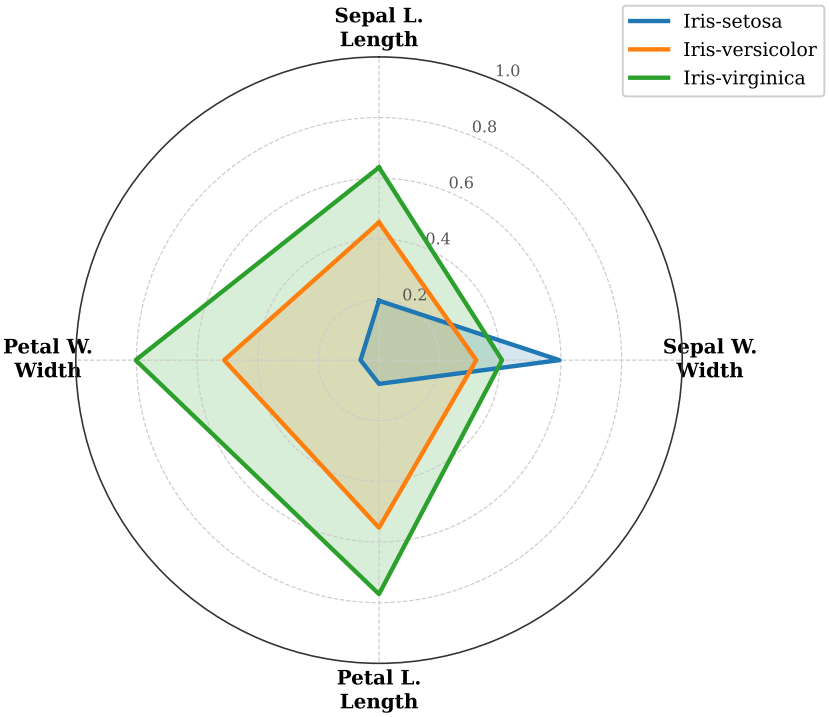


Figure 3.4: Class-average profiles for the Iris dataset rendered via Radar Plot.

3.2 Bivariate Dependence Analysis: Covariance and Correlation

Quantitative dependence analysis investigates the strength and direction with which two random variables co-vary.

3.2.1 From Variance to Covariance

While variance measures the dispersion of a single variable around its own expectation, **Sample Covariance** measures the joint tendency of two numerical variables X and Y to deviate synchronously from their sample means:

$$\text{Cov}(X, Y) = s_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \tag{3.2}$$

In the special case where $X = Y$, covariance equals sample variance:

$$\text{Cov}(X, X) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2 = s_x^2 \tag{3.3}$$

Covariance Matrix

For a set of p continuous attributes, joint variability is completely captured by the **Covariance Matrix** $V \in \mathbb{R}^{p \times p}$:

$$V = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_p) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \cdots & \text{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_p, X_1) & \text{Cov}(X_p, X_2) & \cdots & \text{Cov}(X_p, X_p) \end{bmatrix} \tag{3.4}$$

- Diagonal entries $V_{jj} = \text{Var}(X_j) = s_j^2$ represent marginal sample variances.
- Off-diagonal entries $V_{jk} = \text{Cov}(X_j, X_k)$ represent pairwise covariances.
- Since $\text{Cov}(X_j, X_k) = \text{Cov}(X_k, X_j)$, matrix V is **symmetric** ($V = V^T$) and positive semi-definite ($V \succeq 0$).

Geometric Interpretation of Deviation Products

The sign of sample covariance is governed by the quadrants formed around the bivariate mean $(\bar{x}, \bar{y})$ in Cartesian space:

- Points in Quadrants I and III contribute positive products (concordant deviations) $\implies \text{Cov}(X, Y) > 0$.

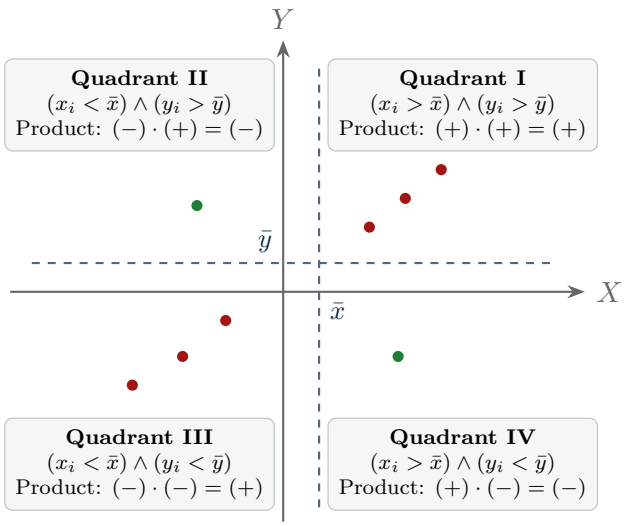


Figure 3.5: Geometric decomposition of bivariate deviation products across four mean-centered quadrants.

- Points in Quadrants II and IV contribute negative products (discordant deviations) $\implies \text{Cov}(X, Y) < 0$.
- Uniformly dispersed observations cancel out across quadrants, yielding $\text{Cov}(X, Y) \approx 0$.

Structural Limitation: Scale Dependence

Covariance lacks scale invariance: its magnitude scales linearly with measurement units. Converting income X from euros to thousands of euros ($X' = X/1000$) shrinks covariance by a factor of 1000, although the underlying statistical link remains identical. Hence, covariance cannot directly compare pairs with distinct units or scales.

3.2.2 Pearson Linear Correlation Coefficient

The **Pearson Linear Correlation Coefficient** (r_{xy}) standardizes covariance by dividing by the product of both sample standard deviations:

$$r_{xy} = \frac{\text{Cov}(X, Y)}{s_x s_y} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{3.5}$$

Mathematical Properties of r_{xy}

1. **Bounded Range**: By the Cauchy-Schwarz inequality on centered vectors, the coefficient is strictly bounded within:

$$-1 \leq r_{xy} \leq +1 \tag{3.6}$$

2. **Scale Invariance**: It is a dimensionless index invariant under positive affine transformations ($X' = aX + b, Y' = cY + d$ with $a, c > 0$).

3. **Perfect Linear Relationship**:

   - $r_{xy} = +1$: perfect positive linear correlation; all points lie exactly on a line with positive slope ($y = ax + b$, $a > 0$).
   - $r_{xy} = -1$: perfect negative linear correlation ($a < 0$).

4. **Absence of Linear Relationship**: A value of $r_{xy} \approx 0$ signifies **absence of linear association**, but does not imply statistical independence. Non-linear deterministic functions (e.g., $Y = X^2$ with X symmetrically distributed around zero) yield $r_{xy} = 0$, despite full underlying determinism.

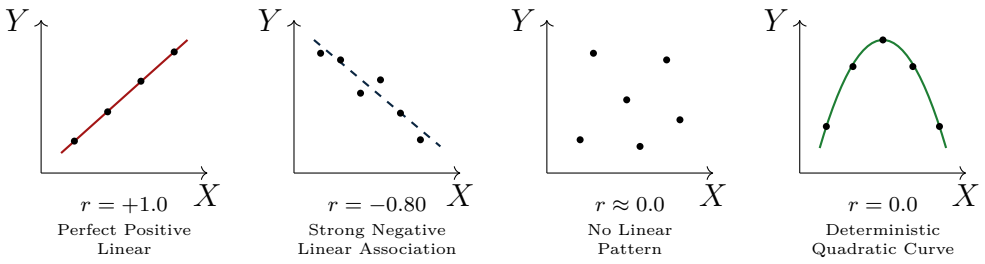


Figure 3.6: Archetypal scatter patterns and their corresponding Pearson correlation values.

3.2.3 Causal Fallacy and Confounding Variables

A cornerstone of data science states that **correlation does not imply causation**. A high observed $|r_{xy}|$ may arise from three distinct phenomena:

1. X is a direct causal driver of Y.

2. Y is a direct causal driver of X.

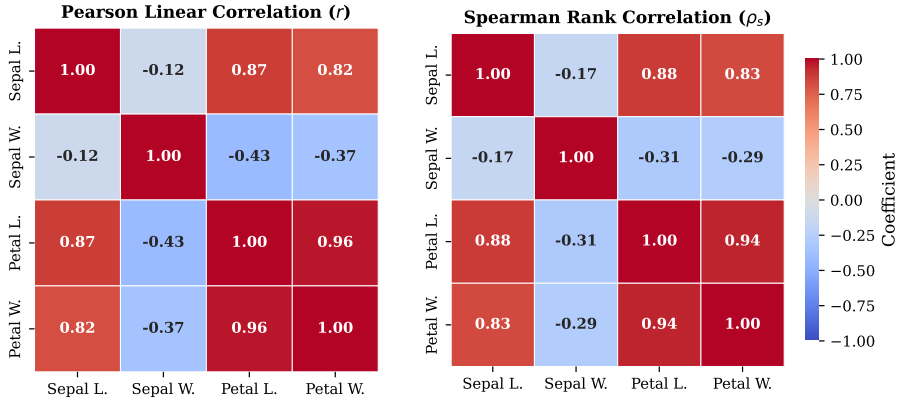


Figure 3.7: Correlation heatmaps for the overall Iris dataset and stratified by species.

3. X and Y are simultaneously driven by an unobserved or latent third factor, termed a **confounding variable**.

Classic Example: Ice Cream Sales and Drowning Incidents

During summer months, a high positive correlation ($r > 0.85$) is documented between ice cream sales (X) and drowning casualties (Y). Inferring that ice cream consumption increases drowning risk represents a classic causal fallacy: both variables are driven by an underlying exogenous confounder—high atmospheric temperature (*hot weather*)—which spurs both outdoor swimming and cold dessert purchases.

3.2.4 Spearman Rank Correlation Coefficient

When variables violate bivariate normality assumptions or exhibit non-linear yet strictly monotonic relationships, the **Spearman Rank Correlation Coefficient (ρ)** is preferred.

The method replaces continuous values x_i and y_i with their corresponding ordinal ranks $r(x_i), r(y_i) \in \{1, \ldots, n\}$. In the absence of ties, the explicit formulation simplifies to:

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} \Big(r(x_i) - r(y_i)\Big)^2}{n(n^2 - 1)} \tag{3.7}$$

- $\rho = +1$: indicates a perfect monotonic increasing relationship (as X increases, Y strictly increases, even along exponential or logarithmic curves).

- $\rho = -1$: indicates a perfect monotonic decreasing relationship.
- **Robustness**: Relying on ranks rather than raw metric differences renders Spearman significantly more resistant to extreme outliers in distribution tails.

3.3 Outlier Taxonomy and Detection Methodologies

An **outlier** is defined as an observation that deviates so substantially from the remaining data as to arouse suspicion that it was generated by a different mechanism or contaminated by measurement error.

3.3.1 The Dual Nature of Outliers: Noise vs. Primary Target

- **Outliers as Noise**: The anomaly stems from data entry typos, temporary sensor glitches, or spurious noise. In this role, outliers distort parameter estimation and should be identified, corrected, or removed during data preprocessing.
- **Outliers as the Primary Target (Analysis Goal)**: The explicit mission of the analysis is to uncover rare, high-impact events carrying crucial domain insights:
  - Intrusion detection in computer network security.
  - Financial fraud detection and anti-money laundering monitoring.
  - Predictive maintenance through anomalous vibration patterns in industrial turbines.

3.3.2 Outlier Identification Methodologies

Outlier identification methodologies can be systematically categorized into univariate and multidimensional approaches:

- **Categorical Attributes**: An anomalous observation belongs to a rare category occurring with near-zero empirical frequency ($< 0.1\%$).
- **Univariate Numeric Attributes**:
  - *Tukey IQR Rule*: Values outside the interval $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$.

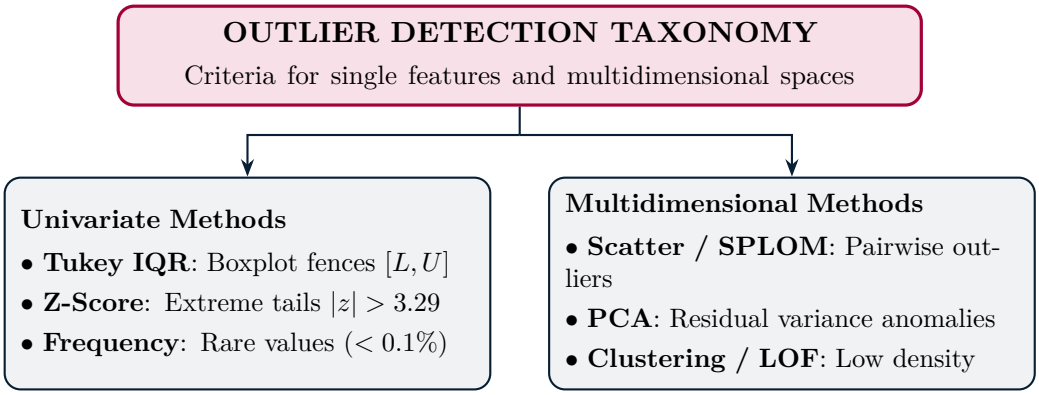


Figure 3.8: Taxonomy of outlier detection methodologies.

  - *Z-Score Rule*: Standardized values with $|z| > 3.29$ (representing the outermost 0.1% tails of a Gaussian distribution).

- **Multidimensional Spaces**:
  - *Bivariate Scatter Plots*: Visual identification of points detached from dense clusters.
  - *PCA Projections*: Observations projecting as outliers along secondary principal components.
  - *Clustering and Local Outlier Factor (LOF)*: Isolating samples residing in low-density neighborhoods relative to their k-nearest neighbors.

3.4 Introduction to Missing Values

A data instance contains **Missing Values** when no observation is recorded for one or more features (explicitly encoded via symbols like `NaN`, `NULL`, `?` or implicitly hidden under sentinel conventions such as `999` or `0`).

3.4.1 Root Causes of Missingness

1. **Voluntary Non-Response**: Respondents decline to disclose sensitive personal data (e.g., annual income, political affiliations, medical history).
2. **Conceptual Inapplicability**: The attribute is logically inapplicable for certain subpopulations (e.g., pregnancy counts for male subjects, or employment tenure for school-age minors).

3. **Hardware Failures or Dropped Connections**: Telemetry sensors freezing or disconnecting, producing null readings or reporting continuous dummy constants (e.g., a frozen wind sensor persistently reporting 0 m/s).

While the *Data Understanding* stage focuses on identifying the root causes and prevalence of missingness, operational strategies for listwise/pairwise deletion and statistical imputation are detailed in Chapter 4 (section 4.5) and implemented in Python in Chapter 5 (section 5.2).

3.4.2 Operational Data Understanding Checklist

Before proceeding to data cleaning and preparation, practitioners must systematically verify the following diagnostic checklist:

Diagnostic Check	Operational Verification Procedure
Syntactic Integrity	Validate data types, string/number formats, and domain constraints.
Outlier Inspection	Inspect extreme values using univariate Boxplots and bivariate Scatter Plots.
Missing Value Audit	Quantify null percentages and detect implicit sentinel values (`0`, `-1`, `999`).
Association & Redundancy	Compute Pearson and Spearman matrices to identify collinear or redundant features.
Distributional Assumptions	Check symmetry, normality, and variance stability prior to feature transformations.

Table 3.1: Operational diagnostic checklist for completing the Data Understanding phase.

Chapter 4

Data Preparation & Cleaning – Reduction, Discretization, and Transformation

While Data Understanding focuses on diagnosing anomalies and comprehending latent structures within a dataset, **Data Preparation** represents the operational, interventional component of the KDD lifecycle. During this phase, raw records are cleansed, compressed, scaled, and re-encoded to satisfy the algebraic and computational requirements of machine learning and data mining algorithms.

4.1 Distinction Between Data Understanding and Data Preparation

The transition from diagnostic exploration to operational data engineering is captured by the clear mapping between diagnostic findings and corresponding corrective actions:

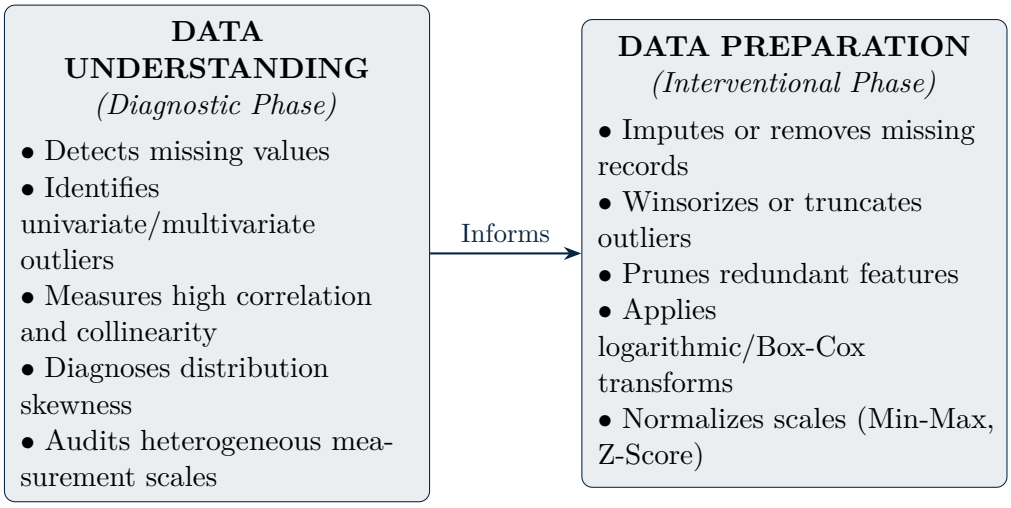


Figure 4.1: Sequential transition from Data Understanding (diagnosis) to Data Preparation (intervention).

4.2 Aggregation and Sampling

Reducing dataset volume can be achieved either by combining related observations or through statistical sampling of representative subsets.

4.2.1 Data Aggregation

Data aggregation merges two or more records or features into a single synthesized summary.

- **Core Objectives**:
    1. *Dimensional Reduction*: Reduces the total record count or feature cardinality, lowering computational complexity.
    2. *Scale Transformation*: Adjusts analysis to an appropriate conceptual level of granularity (e.g., aggregating hourly transactional logs into weekly sums, or regionalizing individual point-of-sale terminals).
    3. *Variance Stabilization*: Aggregated aggregates average out high-frequency noise, yielding lower-variance metrics and more robust statistical trends.
- **Case Study: Australian Precipitation (1982–1993)**: In historical Australian rainfall data, the standard deviation of monthly average

rainfall exhibits high dispersion with long tails reaching up to 15 cm. Conversely, when aggregating into annual averages, the standard deviation tightly clusters around low values with substantially reduced spread (Figure 4.2). To model seasonal agricultural patterns, the disaggregated monthly scale is essential, whereas comparing macroscopic regional climates benefits from the stable annual scale.

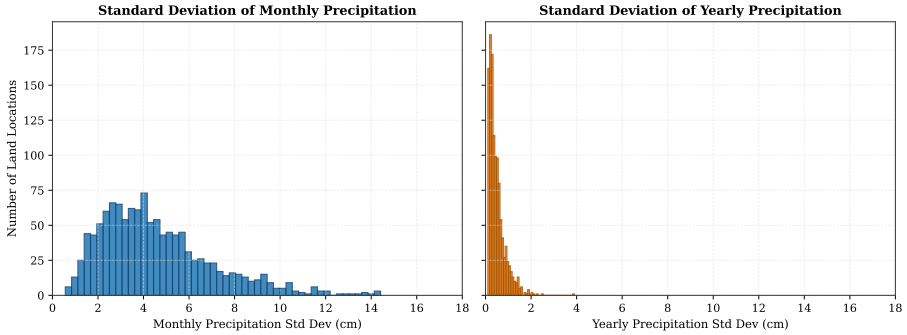


Figure 4.2: Dispersion comparison between monthly precipitation and aggregated annual precipitation in Australia.

4.2.2 Sampling Methodologies

Sampling selects a representative subset of size n from a large population of size N ($n \ll N$). A sample is deemed **representative** if it preserves the underlying statistical properties and distributional profile of the parent population.

1. **Simple Random Sampling**: Every instance in the parent dataset has identical a priori inclusion probability:

$$P(x_i) = \frac{1}{N} \tag{4.1}$$

   - *Without Replacement*: Extracted items are removed from the candidate pool and cannot be drawn again.
   - *With Replacement*: Extracted items remain in the pool and may appear multiple times in the sample.

2. **Stratified Sampling**: The dataset is partitioned into mutually disjoint, homogeneous subsets called *strata* (e.g., grouped by target class label or numeric percentiles). Independent random samples are extracted from each stratum proportional to its relative population size, preventing rare classes from being erased.

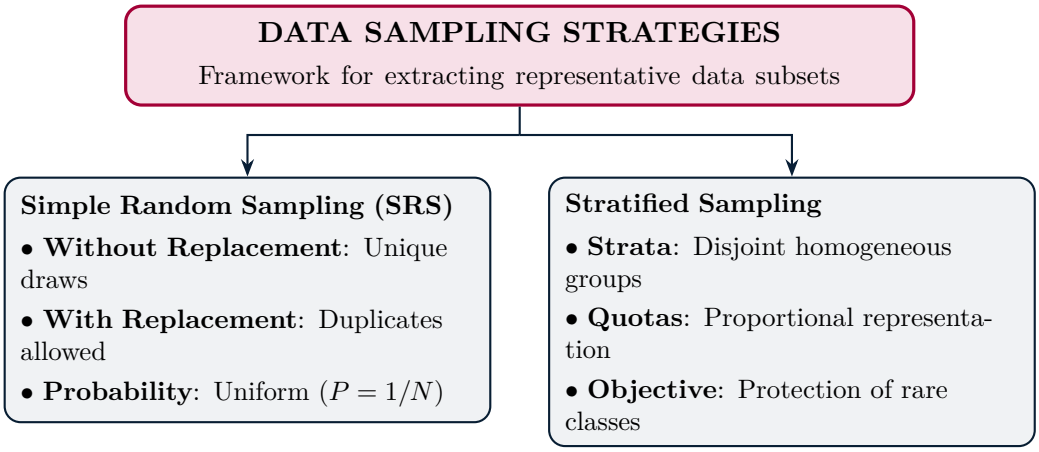


Figure 4.3: Taxonomy of data sampling strategies in data mining.

Selecting an optimal sample size involves a trade-off between computational processing cost and statistical fidelity, as depicted in Figure 4.4.

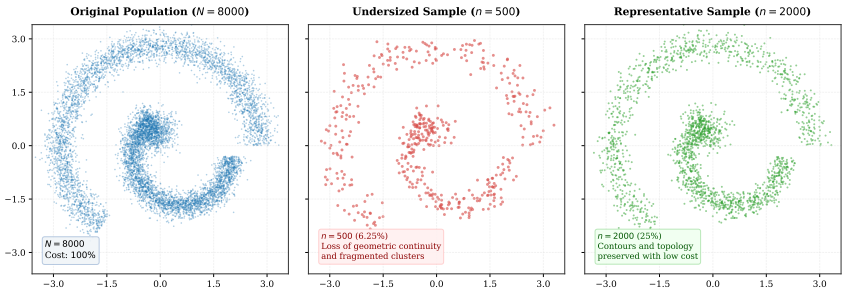


Figure 4.4: Trade-off curve between sample size and empirical parameter estimation error.

4.3 Dimensionality Reduction

High feature counts introduce significant computational and geometric bottlenecks.

4.3.1 The Curse of Dimensionality

As feature dimensionality p grows, the geometric volume of the embedding space $\mathbb{R}^p$ expands exponentially relative to the available sample count. Key consequences include:

1. **Extreme Sparsity**: Data observations become isolated in high-dimensional space; local point density vanishes towards zero, leaving the feature space virtually empty.

2. **Distance Concentration**: As $p \to \infty$, the contrast between the maximum Euclidean distance and minimum Euclidean distance to any query point collapses:

$$\lim_{p \to \infty} \frac{\text{dist}_{\text{max}} - \text{dist}_{\text{min}}}{\text{dist}_{\text{min}}} \to 0 \tag{4.2}$$

   This severely impairs distance-based algorithms (k-NN, K-Means clustering, local density estimators).

3. **Covariance Instability**: Reliably estimating a $p \times p$ sample covariance matrix requires sample sizes that scale exponentially with p to avoid rank deficiency and ill-conditioning.

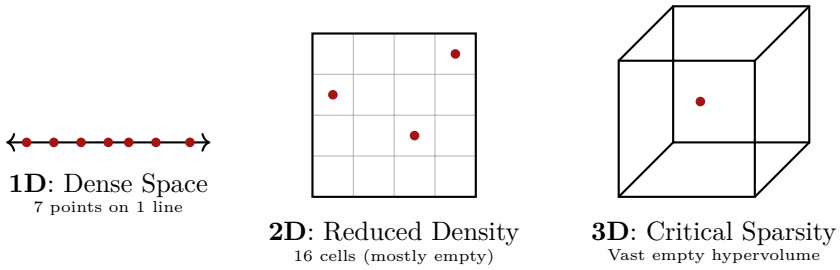


Figure 4.5: Visualizing exponential geometric dispersion of data samples with increasing dimensions.

4.3.2 Feature Selection

Feature Selection isolates an optimal subset of original attributes by discarding irrelevant (uninformative) and redundant (duplicated) features:

4.3.3 Feature Creation and Projection

- **Feature Construction**: Formulates novel descriptive attributes by combining raw features using domain knowledge (e.g., calculating density as mass/volume or productivity as completed/scheduled hours).

Paradigm	Selection Mechanism	Trade-offs
Filter	Ranks features a priori using univariate statistical metrics (ANOVA, Pearson r, χ^2), independent of learning models.	Blazing fast; ignores multivariate feature interactions.
Wrapper	Uses the target machine learning model as an evaluation oracle across candidate subsets (Forward Selection, Backward Elimination).	High predictive accuracy; computationally heavy (2^p potential subset evaluations).
Embedded	Selection occurs natively during algorithm optimization (e.g., decision tree branch splits, Lasso L_1 penalty regularization).	Excellent balance of computational efficiency and predictive accuracy.

Table 4.1: Comparative breakdown of Filter, Wrapper, and Embedded feature selection approaches.

- **Feature Projection**: Algebraically projects original vectors from $\mathbb{R}^p$ into a compact subspace $\mathbb{R}^k$ ($k \ll p$), preserving maximal variance or class separation (PCA, SVD, Linear Discriminant Analysis - LDA, Autoencoders).

4.4 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a linear, unsupervised technique that maps an original set of correlated features into a new coordinate system of mutually orthogonal (uncorrelated) variables, ordered in descending fashion by explained variance.

4.4.1 Step-by-Step PCA Mathematical Derivation

The formal mathematical workflow of PCA proceeds across five structured steps:

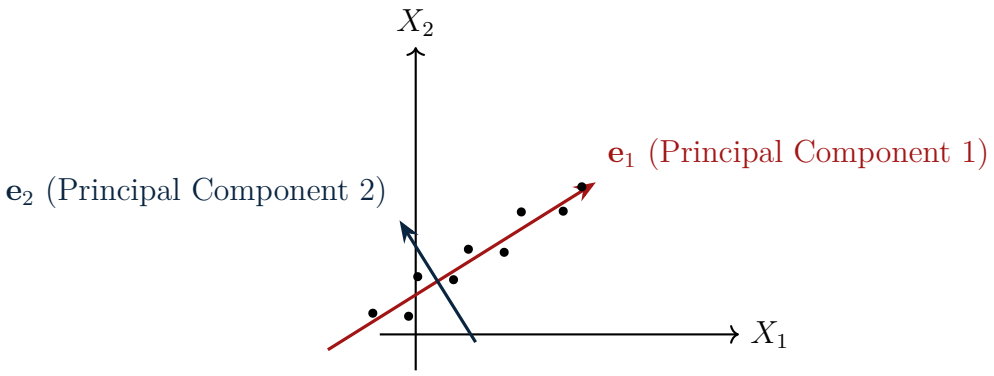


Figure 4.6: Alignment of orthogonal principal component axes along directions of maximal data variance.

Step 1: Centering and Standardizing the Data Matrix

Given a data matrix $D \in \mathbb{R}^{n \times p}$, compute the sample mean for each feature column:

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^{n} D_{ij} \tag{4.3}$$

Construct the **centered matrix** $C \in \mathbb{R}^{n \times p}$ by subtracting feature means from each observation:

$$C_{ij} = D_{ij} - \bar{x}_j \tag{4.4}$$

If attributes exhibit dissimilar scales or measurement units, construct the **standardized matrix** $Z \in \mathbb{R}^{n \times p}$ by dividing centered residuals by feature standard deviations s_j:

$$Z_{ij} = \frac{D_{ij} - \bar{x}_j}{s_j} \tag{4.5}$$

Step 2: Sample Covariance Matrix Computation

Compute the empirical covariance matrix $V \in \mathbb{R}^{p \times p}$ via matrix multiplication:

$$V = \frac{1}{n-1} C^T C \tag{4.6}$$

Step 3: Spectral Decomposition (Eigenvalues and Eigenvectors)

Solve the characteristic eigenvalue equation for symmetric matrix V:

$$V\mathbf{e}_j = \lambda_j \mathbf{e}_j, \quad j = 1, \ldots, p \tag{4.7}$$

- Each normalized unit-length **eigenvector** $\|\mathbf{e}_j\|_2 = 1$ establishes the spatial direction of the j-th principal axis. Because V is symmetric, eigenvectors belonging to distinct eigenvalues are mutually orthogonal ($\mathbf{e}_j^T \mathbf{e}_k = 0$ for $j \neq k$).
- Each corresponding **eigenvalue** $\lambda_j \geq 0$ quantifies the exact portion of sample variance explained along the axis $\mathbf{e}_j$.

Step 4: Sorting and Component Retention

Sort eigenvalues and associated eigenvectors in descending numerical order:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0 \tag{4.8}$$

The proportion of total variance explained by the j-th component equals:

$$\text{Explained Variance (\%)} = \frac{\lambda_j}{\sum_{m=1}^{p} \lambda_m} \times 100 \tag{4.9}$$

Retain the top k components ($k < p$) satisfying an aggregate cumulative variance threshold (e.g., 85% or 95%), discarding residual trailing components (Figure 4.7). Construct the projection matrix $F \in \mathbb{R}^{p \times k}$:

$$F = \begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & \cdots & \mathbf{e}_k \end{bmatrix} \tag{4.10}$$

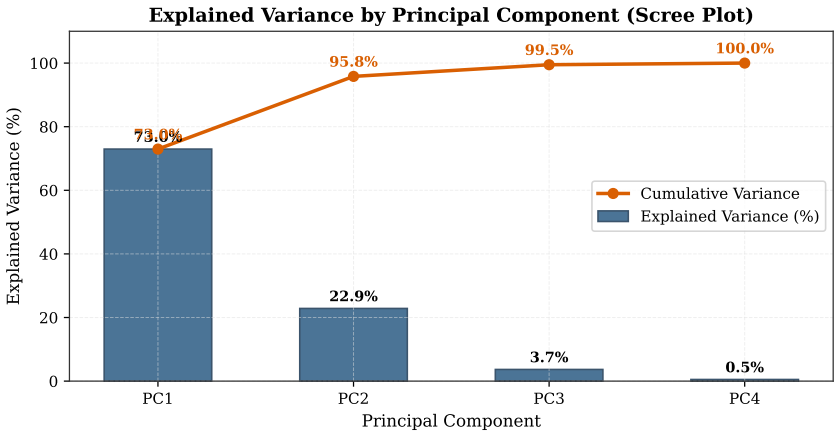


Figure 4.7: Eigenvalue scree plot and cumulative explained variance curve.

Step 5: Subspace Projection

Project the centered dataset onto the reduced k-dimensional coordinate system:

$$Z = CF \tag{4.11}$$

Figure 4.8 illustrates the 2D projection of the Iris dataset onto its first two principal components (PC1 and PC2), which capture over 95% of total dataset variance while achieving distinct species clustering.

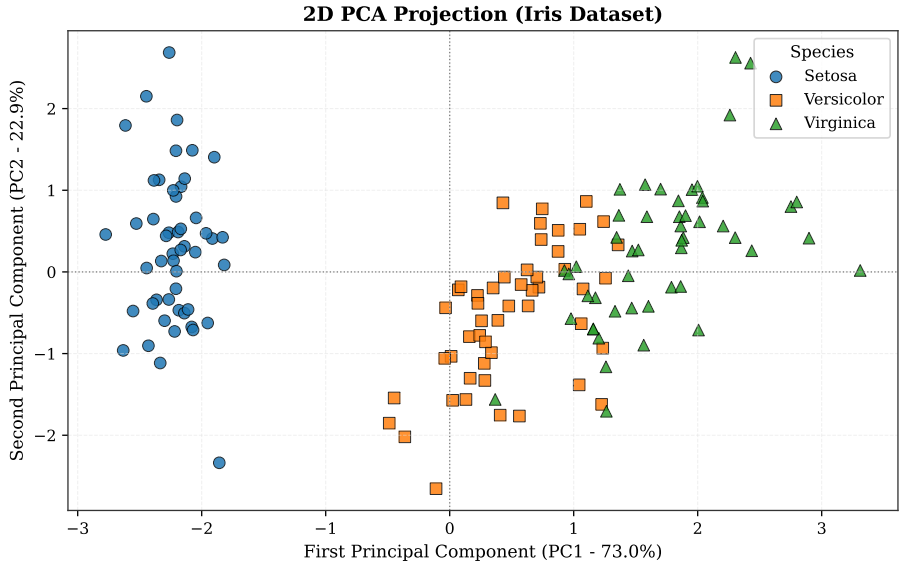


Figure 4.8: Two-dimensional projection of Iris samples onto the top two principal components.

4.5 Handling Missing Values

Operational strategies for incomplete records include:

1. **Listwise Deletion**: Discarding all records containing at least one missing attribute. Safe only when missing frequency is trivial ($< 3-5\%$) and mechanism is strictly *Missing Completely at Random (MCAR)*; otherwise, severe sampling bias ensues.

2. **Central Tendency Imputation**:

   - *Mean*: For symmetric numeric variables lacking outliers.

- *Median*: For skewed numeric features with heavy tails or outliers.
- *Mode*: For categorical attributes.
- *Caveat*: Artificially deflates feature variance and weakens inter-attribute correlation structures.

3. **Conditional Segmented Imputation**: Grouping instances into clusters or classes based on complete features, then imputing missing values using the mean or median calculated within that homogeneous segment.

4. **Model-Based Predictive Imputation**: Fitting a supervised model (k-NN, linear regression, Random Forest) where the missing feature acts as the target variable and complete attributes serve as predictors.

4.6 Discretization and Binarization Techniques

Discretization converts continuous quantitative variables into discrete ordinal attributes by partitioning the continuous support into a finite set of contiguous intervals (termed *bins*).

4.6.1 Unsupervised Discretization

1. Natural Binning (Equal-Width Binning)

Partitions the range $[x_{\min}, x_{\max}]$ into k equal-sized intervals of **constant width**:

$$\delta = \frac{x_{\max} - x_{\min}}{k} \tag{4.12}$$

The i-th bin ($i = 0, \ldots, k-1$) spans:

$$I_i = \Big[x_{\min} + i\delta,\ x_{\min} + (i+1)\delta\Big) \tag{4.13}$$

Numerical Example: 12 Beer Prices

Given $N = 12$ beer prices spanning from $x_{\min} = 100$ to $x_{\max} = 160$, choosing $k = 4$ bins yields interval width $\delta = \frac{160-100}{4} = 15$. Intervals and counts:

- Bin 1: $[100, 115) \to \{100, 110, 110\}$ (Count: 3)
- Bin 2: $[115, 130) \to \{120, 120, 125\}$ (Count: 3)
- Bin 3: $[130, 145) \to \{130, 130, 135, 140\}$ (Count: 4)
- Bin 4: $[145, 160] \to \{150, 160\}$ (Count: 2)

2. Equal-Frequency Binning (Equal-Depth Binning)

Sorts the data ascendingly and allocates an equal number of records into each bin:

$$f = \frac{N}{k} \tag{4.14}$$

For the same sorted 12 prices $\{100, 110, 110, 120, 120, 125, 130, 130, 135, 140, 150, 160\}$ with $k = 4$, target count is $f = 12/4 = 3$:

- Bin 1: $\{100, 110, 110\}$
- Bin 2: $\{120, 120, 125\}$
- Bin 3: $\{130, 130, 135\}$
- Bin 4: $\{140, 150, 160\}$

This balances density and dampens outlier skewness, though identical values straddling bin boundaries may be artificially split into adjacent categories.

Formulas for Optimal Bin Count and Width

Beyond Sturges' formula, the base-10 logarithmic rule provides:

$$C = 1 + \frac{10}{3}\log_{10}(N) \tag{4.15}$$

To optimize bin width h accounting jointly for sample spread and size, **Scott's Rule (1979)** provides:

$$h = \frac{3.5 \times s}{\sqrt[3]{N}} \tag{4.16}$$

where s is the sample standard deviation and N is sample size.

4.6.2 Supervised Entropy-Based Discretization

When a target class label $Y \in \{c_1, \ldots, c_m\}$ is available, split thresholds are selected to maximize class homogeneity. The Shannon entropy of dataset S is:

$$\text{Entropy}(S) = -\sum_{j=1}^{m} p_j \log_2(p_j) \tag{4.17}$$

where p_j is the proportion of samples belonging to class c_j. A candidate threshold T partitioning S into S_1 and S_2 yields weighted entropy:

$$E(S, T) = \frac{|S_1|}{|S|}\text{Entropy}(S_1) + \frac{|S_2|}{|S|}\text{Entropy}(S_2) \tag{4.18}$$

The optimal split T^* maximizes information gain $\text{Gain}(S, T) = \text{Entropy}(S) - E(S, T)$.

4.6.3 Binarization (Encoding)

Converts categorical attributes with m distinct levels into binary indicators $\{0, 1\}$:

- **One-Hot Encoding (Asymmetric Binary Encoding)**: Allocates a dedicated indicator feature for each category level. Avoids imposing artificial order:

Category	**Integer Code**	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

Table 4.2: One-Hot encoding for an attribute with 5 distinct qualitative categories.

- **Compact Binary Encoding**: Encodes category levels into positional binary integers using $n = \lceil \log_2(m) \rceil$ bits: *Drawback*: Induces synthetic

Category	**Integer Code**	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Table 4.3: Compact binary encoding on 3 bits for 5 categories.

  correlations and ordering artifacts among bits that lack real-world domain validity.

4.7 Functional Attribute Transformations and Variance Stabilization

A **functional transformation** applies a strictly monotonic, continuous function T to every value of feature X: $Y = T(X)$.

- **Variance Stabilization**: Enforces constant error variance across attribute ranges (*homoscedasticity*).
- **Distribution Normalization**: Transforms heavily skewed distributions toward approximate Gaussianity.
- **Relationship Linearization**: Transforms curvilinear relationships into linear ones suitable for correlation and regression.

4.7.1 Power and Logarithmic Transformation Family

The general parametric transformation is formulated as:

$$T_p(x) = \begin{cases} ax^p + b & \text{if } p \neq 0 \\ c\log(x) + d & \text{if } p = 0 \end{cases} \tag{4.19}$$

with $x > 0$.

- **Linear Transformation ($p = 1$)**: Alters only scale and origin without changing distribution shape (e.g., Fahrenheit to Celsius: $a = 5/9, b = -160/9$).
- **Logarithmic Transformation ($p = 0$)**: $T(x) = c\log(x) + d$. Compresses extreme upper-tail values while stretching small values, effectively correcting severe right skewness.

Bar	**Beer**	**Original Gain (X)**	**Transformed Gain** $\log_{10}(X)$
A	Bud	20	1.3010
A	Becks	10000	4.0000
C	Bud	300	2.4771
D	Bud	400	2.6021
D	Becks	5	0.6990
E	Becks	120	2.0792
E	Bud	120	2.0792
F	Bud	11000	4.0414
G	Bud	1300	3.1139
H	Bud	3200	3.5051
H	Becks	1000	3.0000
I	Bud	135	2.1303

Table 4.4: Empirical sample of 12 commercial Gain records before and after $\log_{10}$ transformation.

Statistical Metric	Raw Scale (X)	Logarithmic Scale ($Y = \log_{10} X$)
Mean	2481.82	2.5956
Standard Deviation	4079.02	1.0651
Minimum	5	0.6990
1st Quartile (Q_1)	120	2.0792
Median (Q_2)	400	2.6021
3rd Quartile (Q_3)	2250	3.3095
Maximum	11000	4.0414

Table 4.5: Descriptive metrics before and after logarithmic transformation.

In the raw scale, standard deviation (4079.02) dwarfs the mean (2481.82), heavily pulled by outliers at 10000 and 11000. Following $\log_{10}$ scaling, mean (2.5956) and median (2.6021) align closely, restoring symmetry.

4.7.2 Other Variance Stabilizing Transformations

- **Square Root ($p = 1/2$)**: $T(x) = \sqrt{x}$. Stabilizes variance for Poisson count data ($\sigma^2 = \mu$).
- **Reciprocal ($p = -1$)**: $T(x) = 1/x$. Applied when variance increases superlinearly relative to the mean.

4.8 Normalization Algorithms

Normalization scales diverse numerical features onto a shared, comparable range, preventing features with large magnitudes from dominating distance computations in k-NN, K-Means, SVM, or artificial neural networks.

4.8.1 Min-Max Normalization

Linearly transforms values from original range $[\min_A, \max_A]$ into a defined target interval $[\text{new_min}_A, \text{new_max}_A]$ (typically $[0, 1]$):

$$v' = \frac{v - \min_A}{\max_A - \min_A}\Big(\text{new_max}_A - \text{new_min}_A\Big) + \text{new_min}_A \tag{4.20}$$

Preserves exact linear proportions, but remains vulnerable to out-of-bounds future inputs and extreme outliers compressing standard values.

4.8.2 Z-Score Standardization

Shifts and scales values such that the transformed variable has **zero mean and unit variance**:

$$v' = \frac{v - \bar{x}_A}{s_A} \tag{4.21}$$

where $\bar{x}_A$ is the sample mean and s_A the sample standard deviation. It is the gold standard when data approximates a Gaussian distribution.

4.8.3 Decimal Scaling

Normalizes values by shifting the decimal point via powers of 10:

$$v' = \frac{v}{10^j} \tag{4.22}$$

where j is the smallest positive integer satisfying:

$$\max(|v'|) < 1 \iff 10^{j-1} \leq \max(|v|) < 10^j \tag{4.23}$$

Numerical Example: Decimal Scaling

Given the vector $V = \{-10, 201, 301, -401, 501, 601, 701\}$:

1. The absolute maximum is $m = \max(|v|) = 701$.
2. The minimal integer j where $10^j > 701$ is $j = 3$ ($10^3 = 1000$).
3. Dividing each value by 1000 yields:

$$V' = \{-0.01, 0.201, 0.301, -0.401, 0.501, 0.601, 0.701\} \tag{4.24}$$

Every transformed element strictly lies within the open interval $(-1, +1)$.

4.9 Univariate Outlier Treatment

Recalling the diagnostic criteria based on Boxplots, Interquartile Range (IQR), and Z-Scores formalized in Chapter 3 (section 3.3), the *Data Preparation* phase establishes operational rules to neutralize the distortive influence of isolated anomalies without necessarily dropping entire records:

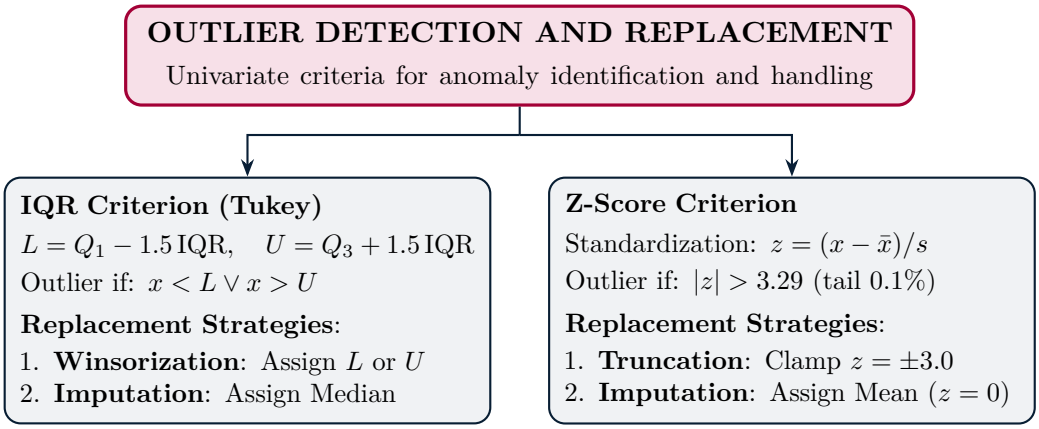


Figure 4.9: Decision hierarchy for univariate outlier detection and replacement strategies.

4.9.1 IQR-Based Outlier Treatment (Tukey)

Once Tukey fences $[L, U] = [Q_1 - 1.5\,\mathrm{IQR},\, Q_3 + 1.5\,\mathrm{IQR}]$ are computed, treatment options for outside values $x \notin [L, U]$ include:

- **Winsorization (Boundary Clamping)**: Set $x' = L$ if $x < L$, and $x' = U$ if $x > U$. This preserves ordinal rankings and original sample size while eliminating heavy tail distortion;
- **Median Imputation**: Replace flagged outliers with the sample median computed from non-outlier instances within $[L, U]$.

4.9.2 Z-Score Outlier Treatment

For approximately normal features where outliers are flagged by $|z| > 3.29$, two operational actions are standard:

1. 3σ **Truncation (Clamping)**: Clamp values to boundary $z = \pm 3.0$ ($x' = \bar{x} \pm 3.0\,s$), bringing instances into plausible bounds without erasing their status as extreme observations;
2. **Mean Imputation**: Substitute the outlier with $z = 0$, equivalent to the sample mean $\bar{x}$.

Chapter 5

Python Lab: Data Understanding and Data Preparation

This chapter presents the computational realization of the theoretical principles of Data Understanding (DU) and Data Preparation (DP), drawing upon the practical laboratory exercises implemented with Python's core data science ecosystem (`numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn`, and `scikit-learn`).

The laboratory uses the **Tips** dataset (augmented with the secondary **Tips_Age** table), describing restaurant customer transactions and tipping behaviors, to demonstrate the complete data engineering workflow: from ingestion, relational joining, and deduplication to missing value diagnosis, conditional imputation, feature engineering, normalization, and dimensionality reduction via PCA and t-SNE.

Interactive Environment and Laboratory Reproducibility

All examples, data preparation pipelines, and visualizations presented in this chapter are fully reproducible and interactively executable via Jupyter notebooks located in the `Data-Mining/notebooks/` directory. **Quick start with Docker**:

- Terminal execution (project root): `docker compose up jupyter`
- Windows shortcut: double-click `scripts/start-jupyter.cmd`
- Web interface: `http://localhost:8888/lab` (no token or password required).

To execute headless Python scripts directly from the command line:

- `docker compose run -rm app python3 Data-Mining/notebooks/generate_dispensa_figures.py`

5.1 Environment, Ingestion, and Data Integration

Remark 5.1.1 (Interactive Notebook Reference)**.** Code examples in Sections 5.1, 5.2, and 5.3 correspond to the interactive Jupyter **Notebook 1 (Data Understanding)** (file `1_basics_and_ understanding.ipynb`).

The scientific software stack employed throughout the laboratory exercises comprises:

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler,
    MinMaxScaler, KBinsDiscretizer, OneHotEncoder,
    TargetEncoder
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

5.1.1 Loading and Structural Audit

The primary dataset is ingested from a tab-separated values (`tsv`) file, setting the index to the first column:

```
tips = pd.read_csv('tips.csv', sep='\t', index_col=0)
print(tips.head())
tips.info()
```

The raw data initially contains 244 observations and 7 features: `total_bill` (total bill amount), `tip` (tip amount), `sex` (server/payer gender), `smoker` (smoking party indicator), `day` (day of week), `time` (meal timing: Lunch or Dinner), `size` (party size).

5.1.2 Data Integration and Relational Joining

In this lab, customer age data is stored in a separate table (`tips_age.csv`). The two sources are integrated using a left-join keyed on the shared primary identifier `ID`:

```
tips_age = pd.read_csv('tips_age.csv', sep='\t',
    index_col=0)
df = tips.join(tips_age, on='ID')
# Drop redundant columns originating from the secondary
    source
df = df.drop(columns=['gender'], errors='ignore')
```

5.1.3 Duplicate Row Detection and Filtering

Identifying duplicate rows requires inspecting exact equality across all attribute dimensions:

```
# Display all duplicate rows (keeping all duplicate
    instances)
duplicated_rows = df[df.duplicated(keep=False)]
print(f"Duplicate rows detected: {len(duplicated_rows)}"
    )

# Drop duplicate rows while retaining the first
    occurrence
df = df.drop_duplicates()
```

5.1.4 Cleaning Invalid Values and Type Casting

Real-world datasets frequently contain formatting anomalies or sentinel strings mistakenly entered into numeric fields (such as the dummy string `'aaaaa'` in the `total_bill` column). This forces Pandas to treat the column as a generic `object` type. Cleaning requires string replacement followed by explicit numerical casting:

```
# Replace invalid string with NaN
df["total_bill"] = df["total_bill"].replace("aaaaa", np.
    nan)

# Cast column to numeric float
df["total_bill"] = pd.to_numeric(df["total_bill"])
```

5.2 Missing Value Diagnosis and Imputation

5.2.1 Missing Value Audit

Missing values (`NaN`) are systematically audited by chaining `isnull()` and `sum()`:

```
# Summary of null counts per column
null_counts = df.isnull().sum()
print(null_counts[null_counts > 0])

# Filter records containing at least one missing
    attribute
incomplete_records = df[df.isnull().any(axis=1)]
```

5.2.2 Deletion vs. Statistical Imputation

- **Listwise Deletion**: Discarding all rows containing one or more null values:

```
df_cleaned = df.dropna()
```

- **Conditional Median Imputation**: When the `age` feature exhibits missing values or dummy zeros (0), global median imputation can mask real demographic disparities between meal times (*Lunch* vs. *Dinner*). Group-stratified imputation preserves local distributions:

```
df['age'] = df['age'].replace(0, np.nan)
# Impute using group-conditional median based on meal time
df['age'] = df['age'].fillna(df.groupby('time')['age'].transform('median'))
```

5.2.3 Advanced Imputation with Scikit-Learn

The `sklearn.impute` module provides transformer objects suitable for production pipelines:

```
# 1. SimpleImputer for numerical attributes (mean or median)
imp_mean = SimpleImputer(strategy='mean')
df['tip'] = imp_mean.fit_transform(df[['tip']])

# 2. SimpleImputer for categorical attributes (most frequent / mode)
imp_mode = SimpleImputer(strategy='most_frequent')
df['smoker'] = imp_mode.fit_transform(df[['smoker']]).ravel()

# 3. KNNImputer (multivariate k-nearest neighbors imputation)
knn_imp = KNNImputer(n_neighbors=2, weights='distance')
num_cols = ['total_bill', 'tip', 'size', 'age']
df[num_cols] = knn_imp.fit_transform(df[num_cols])
```

5.3 Multivariate Exploration and Correlation Analysis

5.3.1 Contingency Tables and Conditional Distributions

Evaluating dependencies across categorical features relies on row-normalized cross-tabulations:

```
# Joint frequency table between smoker status and gender
smoker_xt = pd.crosstab(df['smoker'], df['sex'])

```

```
# Row-wise normalization (proportional fractions)
smoker_pct = smoker_xt.div(smoker_xt.sum(1).astype(float), axis=0)
smoker_pct.plot(kind='bar', stacked=False, title='Gender per Smoker Category')
plt.ylabel('Relative Frequency')
plt.show()
```

5.3.2 Correlation Matrices and Heatmap Rendering

Pandas computes Pearson, Spearman, and Kendall correlation matrices across numeric attributes:

```
# Pearson linear correlation matrix
corr_pearson = df.corr(numeric_only=True, method='pearson')

# Spearman rank correlation matrix (robust monotonic measure)
corr_spearman = df.corr(numeric_only=True, method='spearman')

# Annotated heatmap rendering via Seaborn
plt.figure(figsize=(8, 6))
sns.heatmap(corr_pearson, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
plt.title('Pearson Correlation Heatmap (Tips Dataset)')
plt.show()
```

5.3.3 Scatter Matrix and Pairplots

Pairwise bivariate inspections are efficiently visualized using `scatter_matrix` or Seaborn `pairplot`:

```
# Pandas Scatter Matrix with KDE diagonal
pd.plotting.scatter_matrix(df[['total_bill', 'tip', 'size', 'age']], figsize=(9, 9), diagonal='kde')
plt.show()

# Seaborn Pairplot with class-based color coding
sns.pairplot(df, vars=['total_bill', 'tip', 'size'], hue='time', palette='Dark2')
plt.show()
```

5.3.4 Feature Engineering: Domain Ratio Construction

Domain expertise guides the derivation of novel features with enhanced explanatory power:

```
# 1. Effective tip ratio relative to total bill
df['tip_ratio'] = df['tip'] / df['total_bill']

# 2. Bill per person (BPP)
df['bpp'] = df['total_bill'] / df['size']

# Stratified Boxplot inspection across gender
df.boxplot(column=['bpp'], by='sex')
plt.title('Bill per Person Stratified by Gender')
plt.suptitle('')
plt.show()
```

5.4 Feature Transformation, Discretization, and Encoding

Remark 5.4.1 (Interactive Notebook Reference)**.** Code examples in Sections 5.4 and 5.5 correspond to the interactive Jupyter **Notebook 2 (Feature Engineering)**:

`2_feature_engineering_and_data_representation.ipynb`

5.4.1 Discretization with KBinsDiscretizer

Scikit-Learn's `KBinsDiscretizer` converts continuous attributes into discrete ordinal bins using three distinct binning strategies:

```
# Discretization into 4 bins across different strategies
disc_uniform = KBinsDiscretizer(n_bins=4, encode='
    ordinal', strategy='uniform')     # Equal-Width
disc_quantile = KBinsDiscretizer(n_bins=4, encode='
    ordinal', strategy='quantile')  # Equal-Frequency
disc_kmeans = KBinsDiscretizer(n_bins=4, encode='ordinal
    ', strategy='kmeans')       # Cluster-based

df['total_bill_bins'] = disc_uniform.fit_transform(df[['
    total_bill']])
print(df['total_bill_bins'].value_counts())
```

5.4.2 Feature Scaling: StandardScaler vs. MinMaxScaler

- **StandardScaler**: Standardizes features to zero mean and unit variance ($\mu = 0, \sigma = 1$).
- **MinMaxScaler**: Bounds values strictly into $[0, 1]$.

```
num_cols = ['total_bill', 'tip', 'age']

# Z-Score Standardization
scaler_std = StandardScaler()
df_standardized = df.copy()
df_standardized[num_cols] = scaler_std.fit_transform(df[num_cols])

# Min-Max Normalization
scaler_minmax = MinMaxScaler()
df_normalized = df.copy()
df_normalized[num_cols] = scaler_minmax.fit_transform(df[num_cols])

# Inverting transformation back to original scale
df_recovered = scaler_minmax.inverse_transform(df_normalized[num_cols])
```

5.4.3 Categorical Encoding: One-Hot vs. Target Encoding

```
# 1. One-Hot Encoding (prevents imposing synthetic order)
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
encoded_days = ohe.fit_transform(df[['day']])
encoded_df = pd.DataFrame(encoded_days, columns=ohe.get_feature_names_out(['day']))

# 2. Target Encoding (substitutes categories with target expectation)
target_enc = TargetEncoder(smooth='auto')
# Encode 'day' using mean 'tip'
```

```
df['day_target_encoded'] = target_enc.fit_transform(df[[
    'day']], df['tip'])
```

5.5 Dimensionality Reduction: PCA and t-SNE

5.5.1 Principal Component Analysis (PCA)

Features must be centered and scaled before fitting PCA so that high-variance variables do not arbitrarily dominate the spectral decomposition:

```
# Select quantitative predictors
X = df[['total_bill', 'tip', 'size', 'age', 'tip_ratio',
    'bpp']].dropna()
X_scaled = StandardScaler().fit_transform(X)

# Fit PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Eigenvalues (absolute variance explained per component
    )
eigenvalues = pca.explained_variance_
# Relative variance ratio
explained_var_ratio = pca.explained_variance_ratio_
# Cumulative explained variance
cumulative_var = np.cumsum(explained_var_ratio)
```

Scree Plot Visualization

The Scree Plot helps detect the characteristic "elbow" where incremental principal components contribute negligible variance:

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, len(eigenvalues) + 1), eigenvalues, 'o
    -', color='navy', label='Individual Eigenvalues')
plt.title('PCA Scree Plot (Eigenvalues)')
plt.xlabel('Principal Component Index')
plt.ylabel('Eigenvalue (Explained Variance)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```

5.5.2 Non-linear Projection via t-SNE

t-Distributed Stochastic Neighbor Embedding (t-SNE) is an unsupervised non-linear algorithm designed to preserve local neighborhood affinities in high-dimensional datasets:

```
tsne = TSNE(
    n_components=2,
    perplexity=30,       # Balance between local and
       global structure (typical range: 5-50)
    learning_rate=200,  # Gradient descent step size
    max_iter=1000,       # Maximum KL-divergence
       optimization iterations
    random_state=42
)
X_tsne = tsne.fit_transform(X_scaled)
```

5.5.3 Methodological Comparison: PCA vs. t-SNE

```
# Side-by-side comparative scatter plots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# PCA projection
ax1.scatter(X_pca[:, 0], X_pca[:, 1], c='navy', alpha
   =0.7, edgecolors='k')
ax1.set_title('Linear PCA Projection (PC1 vs PC2)')
ax1.set_xlabel('First Principal Component')
ax1.set_ylabel('Second Principal Component')
ax1.grid(True, linestyle='--', alpha=0.5)

# t-SNE projection
ax2.scatter(X_tsne[:, 0], X_tsne[:, 1], c='crimson',
   alpha=0.7, edgecolors='k')
ax2.set_title('Non-linear t-SNE Projection')
ax2.set_xlabel('t-SNE Dimension 1')
ax2.set_ylabel('t-SNE Dimension 2')
ax2.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

Criterion	**PCA (Principal Component Analysis)**	***t*-SNE (*t*-Distributed SNE)**
Nature	Linear, global, deterministic.	Non-linear, local, stochastic/probabilistic.
Objective	Maximizes projected global variance via eigenvalue decomposition.	Minimizes Kullback-Leibler divergence between neighborhood probability distributions.
Interpretability	Component axes are linear feature combinations with explicit loadings.	Axes possess no absolute physical meaning; only relative neighbor proximity matters.
Long-Range Distances	Preserves large-scale Euclidean spatial configurations.	Distorts global inter-cluster distances (cluster radii do not represent true cluster volume).
Standard Use	Dimensionality reduction for supervised learning, compression, decorrelation.	Exploratory manifold visualization and qualitative cluster identification (2D/3D).

Table 5.1: Comparative synthesis of PCA and *t*-SNE characteristics.

Chapter 6

Proximity Measures, Information Theory, and Cluster Analysis

This chapter establishes the computational and mathematical foundations underpinning unsupervised learning and instance-based reasoning in Data Mining:

1. Analytical formulation of **proximity, distance, and similarity** across continuous, discrete, and heterogeneous attributes;
2. Formal tools from **Shannon's Information Theory** (entropy and mutual information) for non-linear dependency and uncertainty quantification;
3. Core principles, taxonomies, and geometric formulations of **Cluster Analysis**.

6.1 Foundations of Similarity, Dissimilarity, and Proximity

Across Data Mining, Machine Learning, and Information Retrieval, pairwise comparisons form the algorithmic backbone of numerous paradigms: from nearest neighbor classification (k-NN) to cluster discovery, density estimation, and outlier detection.

Definition 6.1.1 (Similarity Measure)**.** A **similarity measure** is a real-valued function $s : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ quantifying the degree of affinity between two objects $\mathbf{x}$ and $\mathbf{y}$. This value increases monotonically as the objects grow closer: it reaches its upper bound when the two objects are identical and

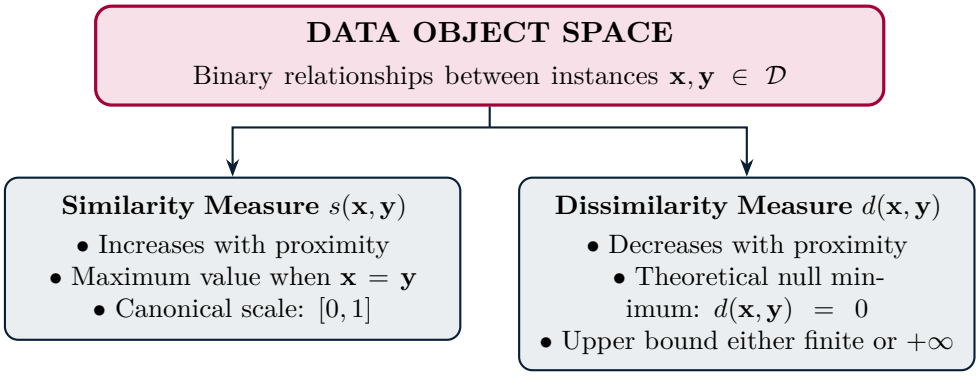


Figure 6.1: Conceptual hierarchy of proximity measures across the data domain.

decreases as they diverge. In most settings, it is normalized to $[0, 1]$, where 1 denotes total identity and 0 complete disjunction.

Definition 6.1.2 (Dissimilarity Measure)**.** A **dissimilarity measure** is a real-valued function $d : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ quantifying the discrepancy between two objects $\mathbf{x}$ and $\mathbf{y}$. It assumes lower values when objects are affine or identical, increasing monotonically as their difference widens. The lower bound is conventionally $d(\mathbf{x}, \mathbf{y}) = 0$ when $\mathbf{x} = \mathbf{y}$, whereas the upper bound may be bounded (1) or unbounded ($+\infty$).

The umbrella term **proximity** is used across the literature to refer interchangeably to either a similarity or a dissimilarity measure.

6.2 Single-Attribute Proximity Measures

Given two objects $\mathbf{x}$ and $\mathbf{y}$, let p be the scalar value of $\mathbf{x}$ along an attribute and q the value of $\mathbf{y}$ along the same attribute. The derivation of elementary dissimilarity $d(p, q)$ and dual similarity $s(p, q)$ depends on the attribute's measurement scale.

6.2.1 Nominal Attributes

Because qualitative nominal categories (e.g., color, department name) exhibit neither natural ordering nor metric properties, equality and inequality evaluations are the sole permissible operators:

$$d(p, q) = \begin{cases} 0 & \text{if } p = q \\ 1 & \text{if } p \neq q \end{cases}, \qquad s(p, q) = \begin{cases} 1 & \text{if } p = q \\ 0 & \text{if } p \neq q \end{cases} \tag{6.1}$$

Attribute Scale	**Dissimilarity** $d(p,q)$	**Dual Similarity** $s(p,q)$
Nominal	$d = 0$ if $p = q$; $d = 1$ if $p \neq q$	$s = 1$ if $p = q$; $s = 0$ if $p \neq q$
Ordinal	$d = \dfrac{\lvert p-q \rvert}{n-1}$ (with states mapped to $\{0, \ldots, n-1\}$)	$s = 1 - d = 1 - \dfrac{\lvert p-q \rvert}{n-1}$
Interval or Ratio	$d = \lvert p-q \rvert$	$s = -d$, $s = \dfrac{1}{1+d}$, or $s = 1 - \dfrac{d - d_{\min}}{d_{\max} - d_{\min}}$

Table 6.1: Formulation rules for elementary proximity across single attribute types.

6.2.2 Ordinal Attributes

Ordinal attributes possess an intrinsic rank composed of n distinct states. To obtain consistent distance values, discrete categories are mapped to positional integers $\{0, 1, \ldots, n-1\}$.

The absolute difference $|p-q|$ ranges from 0 to $n-1$. Dividing by the maximum observable spread guarantees normalization to $[0, 1]$:

$$d(p,q) = \frac{|p-q|}{n-1}, \qquad s(p,q) = 1 - d(p,q) = 1 - \frac{|p-q|}{n-1} \tag{6.2}$$

Example 6.2.1 (Similarity Between Ordinal Ratings)**.** Consider an academic performance ranking over $n = 5$ states:

$$\{\text{fail} = 0,\ \text{pass} = 1,\ \text{good} = 2,\ \text{very good} = 3,\ \text{excellent} = 4\} \tag{6.3}$$

For two candidates with ratings $p = 1$ (*pass*) and $q = 4$ (*excellent*):

$$d(1,4) = \frac{|1-4|}{5-1} = \frac{3}{4} = 0.75 \implies s(1,4) = 1 - 0.75 = 0.25 \tag{6.4}$$

6.2.3 Interval and Ratio Attributes

For continuous or discrete numeric variables, the elementary distance is the absolute difference $d(p,q) = |p-q|$. Converting to bounded similarity $[0, 1]$ follows three main formulations:

1. **Opposite Translation** (unbounded above): $s(p,q) = -|p-q|$;

2. **Asymptotic Inversion**: $s(p,q) = \dfrac{1}{1+|p-q|}$, yielding $s \to 1$ as $d \to 0$ and $s \to 0$ as $d \to \infty$;

3. **Linear Min-Max Rescaling**: $s(p,q) = 1 - \dfrac{d(p,q) - d_{\min}}{d_{\max} - d_{\min}}$, where $d_{\min} = 0$ and $d_{\max}$ represents the maximum observed attribute range.

6.3 Vector Distances and Minkowski Metrics

When objects are described by multidimensional continuous vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, overall dissimilarity aggregates dimensional discrepancies across all n orthogonal coordinates.

Definition 6.3.1 (Minkowski Distance (L_r Norm))**.** Given two vectors $\mathbf{x} = (x_1, \ldots, x_n)$ and $\mathbf{y} = (y_1, \ldots, y_n)$ in $\mathbb{R}^n$, the **Minkowski distance** with parameter $r \geq 1$ is defined as:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_r = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r} \tag{6.5}$$

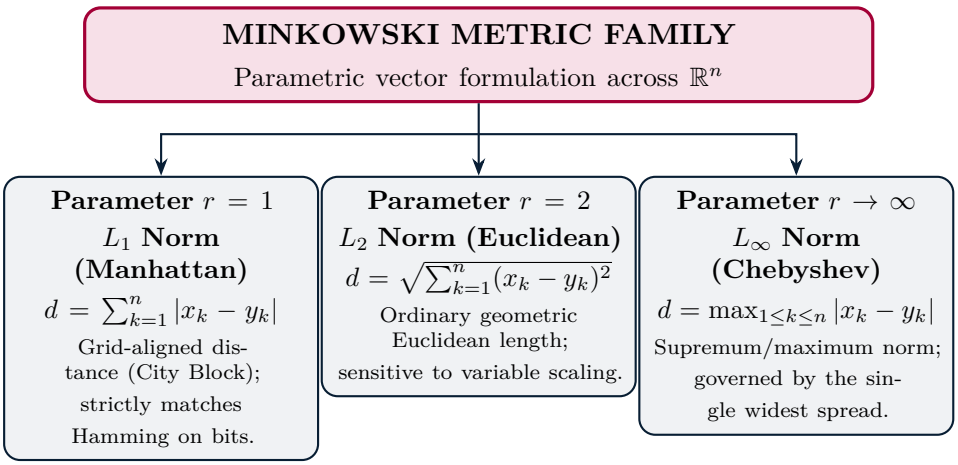


Figure 6.2: Structure of the Minkowski metric family across variations of parameter r.

1. **Manhattan Distance ($r = 1$, L_1 Norm, City Block)**:

$$d_{L_1}(\mathbf{x}, \mathbf{y}) = \sum_{k=1}^{n} |x_k - y_k| \tag{6.6}$$

Measures rectilinear paths along orthogonal coordinate axes. On binary indicator vectors, it is strictly equivalent to the **Hamming distance**.

2. **Euclidean Distance ($r = 2$, L_2 Norm)**:

$$d_{L_2}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2} \tag{6.7}$$

Corresponds to direct geometric straight-line length. Standardizing attributes via z-scores is mandatory when features possess disparate measurement units to avoid dominance by large variance dimensions.

3. **Supremum Distance ($r \to \infty$, L_∞ Norm, Chebyshev Distance)**:

$$d_{L_\infty}(\mathbf{x}, \mathbf{y}) = \lim_{r \to \infty} \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r} = \max_{1 \le k \le n} |x_k - y_k| \tag{6.8}$$

Isolates the single largest coordinate deviation between the pair of points.

6.4 Worked Example of Distance Matrix Computation

Consider the set of 4 two-dimensional points evaluated in lecture:

$$\mathbf{p}_1 = (0, 2), \quad \mathbf{p}_2 = (2, 0), \quad \mathbf{p}_3 = (3, 1), \quad \mathbf{p}_4 = (5, 1) \tag{6.9}$$

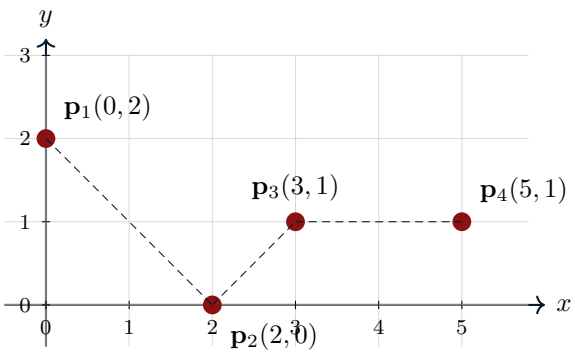


Figure 6.3: Geometric placement of four sample data points on the Cartesian plane.

Evaluating analytical distances across distinct pairs yields:

- $(\mathbf{p}_1, \mathbf{p}_2)$: $\Delta x = 2$, $\Delta y = 2 \implies L_1 = 4$, $L_2 = \sqrt{8} \approx 2.828$, $L_\infty = 2$;
- $(\mathbf{p}_1, \mathbf{p}_3)$: $\Delta x = 3$, $\Delta y = 1 \implies L_1 = 4$, $L_2 = \sqrt{10} \approx 3.162$, $L_\infty = 3$;
- $(\mathbf{p}_1, \mathbf{p}_4)$: $\Delta x = 5$, $\Delta y = 1 \implies L_1 = 6$, $L_2 = \sqrt{26} \approx 5.099$, $L_\infty = 5$;
- $(\mathbf{p}_2, \mathbf{p}_3)$: $\Delta x = 1$, $\Delta y = 1 \implies L_1 = 2$, $L_2 = \sqrt{2} \approx 1.414$, $L_\infty = 1$;
- $(\mathbf{p}_2, \mathbf{p}_4)$: $\Delta x = 3$, $\Delta y = 1 \implies L_1 = 4$, $L_2 = \sqrt{10} \approx 3.162$, $L_\infty = 3$;
- $(\mathbf{p}_3, \mathbf{p}_4)$: $\Delta x = 2$, $\Delta y = 0 \implies L_1 = 2$, $L_2 = \sqrt{4} = 2.000$, $L_\infty = 2$.

Since distance metrics enforce reflexivity ($d(\mathbf{p}_i, \mathbf{p}_i) = 0$) and symmetry ($d(\mathbf{p}_i, \mathbf{p}_j) = d(\mathbf{p}_j, \mathbf{p}_i)$), the complete distance matrices are symmetric with zero main diagonals:

$$\mathbf{D}_{L_1} = \begin{bmatrix} 0 & 4 & 4 & 6 \\ 4 & 0 & 2 & 4 \\ 4 & 2 & 0 & 2 \\ 6 & 4 & 2 & 0 \end{bmatrix}, \quad \mathbf{D}_{L_\infty} = \begin{bmatrix} 0 & 2 & 3 & 5 \\ 2 & 0 & 1 & 3 \\ 3 & 1 & 0 & 2 \\ 5 & 3 & 2 & 0 \end{bmatrix}, \tag{6.10}$$

$$\mathbf{D}_{L_2} = \begin{bmatrix} 0 & 2.828 & 3.162 & 5.099 \\ 2.828 & 0 & 1.414 & 3.162 \\ 3.162 & 1.414 & 0 & 2.000 \\ 5.099 & 3.162 & 2.000 & 0 \end{bmatrix} \tag{6.11}$$

6.5 Metric Axioms and Mathematical Properties

Definition 6.5.1 (Metric Distance Axioms)**.** A dissimilarity function $d : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ qualifies as a formal **metric** if and only if it satisfies three fundamental axioms for all triplets $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathcal{D}$:

1. **Positive Definiteness**:

$$d(\mathbf{x}, \mathbf{y}) \geq 0 \quad \forall \mathbf{x}, \mathbf{y}; \qquad d(\mathbf{x}, \mathbf{y}) = 0 \iff \mathbf{x} = \mathbf{y} \tag{6.12}$$

2. **Symmetry**:

$$d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x}) \quad \forall \mathbf{x}, \mathbf{y} \tag{6.13}$$

3. **Triangle Inequality**:

$$d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z}) \quad \forall \mathbf{x}, \mathbf{y}, \mathbf{z} \tag{6.14}$$

All Minkowski distance metrics ($r \geq 1$) strictly satisfy these three conditions (with the triangle inequality arising from Minkowski's inequality for sums and integrals).

Similarly, canonical similarity measures $s : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ satisfy:

1. **Maximum Identity**: $s(\mathbf{x}, \mathbf{y}) = 1$ (or maximum theoretical upper bound) if and only if $\mathbf{x} = \mathbf{y}$;
2. **Symmetry**: $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ for all pairs $\mathbf{x}, \mathbf{y}$.

6.6 Binary Proximity Measures: SMC vs Jaccard Index

In feature spaces spanned by asymmetric binary attributes (e.g., transaction baskets or vocabulary occurrence in text retrieval), evaluating concordance requires distinguishing between attribute presence (1) and absence (0).

For two binary vectors $\mathbf{p}$ and $\mathbf{q}$, pairwise state occurrences populate the 2×2 contingency table:

		Object q	
		1	0
Object p	1	M_{11} (joint presence)	M_{10} (present only in $\mathbf{p}$)
	0	M_{01} (present only in $\mathbf{q}$)	M_{00} (joint absence)

Table 6.2: Contingency table for binary attribute vectors.

The total number of binary attributes equals:

$$n = M_{11} + M_{10} + M_{01} + M_{00} \tag{6.15}$$

6.6.1 Coefficient Formulations

1. **Simple Matching Coefficient (SMC)**:

$$\text{SMC} = \frac{M_{11} + M_{00}}{M_{01} + M_{10} + M_{11} + M_{00}} \tag{6.16}$$

   Weights positive agreements (M_{11}) and negative agreements (M_{00}) equally. It is appropriate for symmetric binary features (such as biological sex or balanced binary survey indicators).

2. **Jaccard Index (J)**:

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}} \tag{6.17}$$

   Treats the binary feature as asymmetric: deliberately ignores joint absences (M_{00}).

Why Jaccard is Essential for Sparse Data

In high-dimensional and sparse domains (such as market basket analysis or text mining), any two records jointly lack thousands of items or terms. Using the SMC coefficient would cause M_{00} to dominate the fraction numerically, creating artificially inflated similarity (SMC $\to 1$) simply because both transactions did not purchase the vast majority of catalog products. Jaccard avoids this distortion by restricting evaluation strictly to active items.

Example 6.6.1 (SMC vs Jaccard Comparison on Sparse Vectors)**.** Given two binary vectors with $n = 10$ features:

$$\mathbf{p} = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0] \tag{6.18}$$

$$\mathbf{q} = [0, 0, 0, 0, 0, 0, 1, 0, 0, 1] \tag{6.19}$$

Inspecting indices yields:

$$M_{11} = 0, \quad M_{10} = 1, \quad M_{01} = 2, \quad M_{00} = 7 \tag{6.20}$$

The coefficients evaluate to:

$$\text{SMC} = \frac{0+7}{2+1+0+7} = \frac{7}{10} = 0.70, \qquad J = \frac{0}{2+1+0} = \frac{0}{3} = 0.00 \tag{6.21}$$

SMC indicates 70% affinity based entirely on shared non-occurrences, while Jaccard indicates 0% similarity since the two items share no active components.

6.7 Cosine Similarity for Document Data

In the Vector Space Model of Information Retrieval, text documents are represented as high-dimensional term frequency or TF-IDF vectors.

In this context, documents of differing lengths covering identical subject matter appear distant under the Euclidean metric due to differing vector magnitudes. **Cosine Similarity** captures directional alignment regardless of vector norm:

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\|_2 \|\mathbf{d}_2\|_2} = \frac{\sum_{k=1}^{n} d_{1,k} d_{2,k}}{\sqrt{\sum_{k=1}^{n} d_{1,k}^2} \sqrt{\sum_{k=1}^{n} d_{2,k}^2}} \tag{6.22}$$

Example 6.7.1 (Worked Example on Document Term Vectors)**.** Given two term count vectors:

$$\mathbf{d}_1 = [3, 2, 0, 5, 0, 0, 0, 2, 0, 0] \tag{6.23}$$

$$\mathbf{d}_2 = [1, 0, 0, 0, 0, 0, 0, 1, 0, 2] \tag{6.24}$$

1. **Dot Product**:

$$\mathbf{d}_1 \cdot \mathbf{d}_2 = (3\times 1)+(2\times 0)+(5\times 0)+(2\times 1)+(0\times 2) = 3+2 = 5 \quad (6.25)$$

2. **Euclidean Norms**:

$$\|\mathbf{d}_1\|_2 = \sqrt{3^2+2^2+5^2+2^2} = \sqrt{9+4+25+4} = \sqrt{42} \approx 6.4807 \quad (6.26)$$

$$\|\mathbf{d}_2\|_2 = \sqrt{1^2+1^2+2^2} = \sqrt{1+1+4} = \sqrt{6} \approx 2.4495 \quad (6.27)$$

3. **Cosine Similarity**:

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{5}{\sqrt{42}\times\sqrt{6}} = \frac{5}{\sqrt{252}} = \frac{5}{15.8745} \approx 0.3150 \quad (6.28)$$

The documents share roughly 31.5% angular thematic alignment.

6.8 Weighting, Missing Values, and Correlation Across Objects

6.8.1 Weighted Similarity with Missing Data Handling

When attributes carry uneven importance or contain null values, non-negative weights $\{\omega_k\}_{k=1}^n$ and a binary indicator variable $\delta_k \in \{0,1\}$ are incorporated ($\delta_k = 1$ if and only if feature k is present in both $\mathbf{x}$ and $\mathbf{y}$):

$$\text{similarity}(\mathbf{x},\mathbf{y}) = \frac{\sum_{k=1}^n \omega_k \delta_k s_k(\mathbf{x},\mathbf{y})}{\sum_{k=1}^n \omega_k \delta_k} \quad (6.29)$$

Analogously, the **weighted Minkowski distance** is formulated as:

$$d(\mathbf{x},\mathbf{y}) = \left(\sum_{k=1}^n \omega_k |x_k - y_k|^r\right)^{1/r} \quad (6.30)$$

6.8.2 Correlation Between Objects

Pearson's correlation coefficient can be calculated across rows to measure affinity between object profiles $\mathbf{p}$ and $\mathbf{q}$:

$$\text{corr}(\mathbf{p},\mathbf{q}) = \frac{\sum_{k=1}^n (p_k-\bar{p})(q_k-\bar{q})}{\sqrt{\sum_{k=1}^n (p_k-\bar{p})^2}\sqrt{\sum_{k=1}^n (q_k-\bar{q})^2}} = \mathbf{p}^* \cdot \mathbf{q}^* \quad (6.31)$$

where $\mathbf{p}^*$ and $\mathbf{q}^*$ denote standardized profiles with zero mean and unit variance.

6.9 Information Theory: Entropy and Mutual Information

Information Theory, pioneered by Claude Shannon (1948), supplies the mathematical foundation for evaluating disorder, uncertainty, and non-linear dependencies among discrete or binned variables.

6.9.1 Self-Information and Shannon Entropy

The information content associated with an event x_i occurring with probability $p_i = P(X = x_i)$ represents surprise:

$$I(x_i) = \log_2\left(\frac{1}{p_i}\right) = -\log_2(p_i) \tag{6.32}$$

Shannon Entropy $H(X)$ represents the expected value of self-information:

$$H(X) = \mathbb{E}[I(X)] = -\sum_{i=1}^{n} p_i \log_2(p_i) \tag{6.33}$$

with the continuity convention $\lim_{p\to 0^+} p\log_2(p) = 0$.

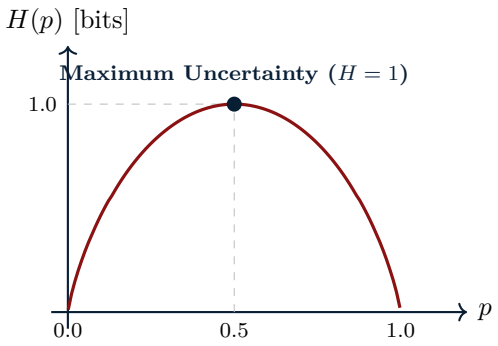


Figure 6.4: Shannon entropy curve for a Bernoulli random variable as a function of parameter p.

Entropy is strictly bounded: $0 \leq H(X) \leq \log_2(n)$. It vanishes if and only if the event is deterministic ($p_k = 1$), reaching its maximum $\log_2(n)$ under a uniform distribution ($p_i = 1/n$).

6.9.2 Mutual Information

Mutual Information $I(X;Y)$ quantifies shared information between variables (the reduction in uncertainty regarding X resulting from knowledge of

Y):

$$I(X;Y) = H(X) + H(Y) - H(X,Y) \tag{6.34}$$

where $H(X,Y)$ is the joint entropy over joint probabilities $p_{ij} = P(X = x_i, Y = y_j)$:

$$H(X,Y) = -\sum_{i=1}^{n_X}\sum_{j=1}^{n_Y} p_{ij}\log_2(p_{ij}) \tag{6.35}$$

Fundamental properties:

1. $I(X;Y) \geq 0$ and $I(X;Y) = I(Y;X)$ (non-negativity and symmetry);
2. $I(X;Y) = 0 \iff P(X,Y) = P(X)P(Y)$ (vanishes under statistical independence);
3. $I(X;Y) \leq \min\Big(H(X), H(Y)\Big) \leq \log_2(\min(n_X, n_Y))$.

6.9.3 Worked Calculation Example of Mutual Information

Consider the sample of $m = 100$ students evaluated in lecture, comparing **Academic Status** ($X \in \{\text{Undergrad}, \text{Grad}\}$) with **Grade** earned ($Y \in \{A, B, C\}$).

Status (X)	**Grade A**	**Grade B**	**Grade C**	**Total (X)**
Undergrad	5 (0.05)	30 (0.30)	10 (0.10)	45 (0.45)
Grad	30 (0.30)	20 (0.20)	5 (0.05)	55 (0.55)
Total (Y)	35 (0.35)	50 (0.50)	15 (0.15)	100 (1.00)

Table 6.3: Joint and marginal frequency distributions for academic status and course grades.

1. **Marginal Entropy of X**:

$$H(X) = -0.45\log_2(0.45) - 0.55\log_2(0.55) \approx 0.9928 \text{ bits} \tag{6.36}$$

2. **Marginal Entropy of Y**:

$$\begin{aligned} H(Y) &= -0.35\log_2(0.35) - 0.50\log_2(0.50) - 0.15\log_2(0.15) \\ &\approx 0.5301 + 0.5000 + 0.4105 = 1.4406 \text{ bits} \end{aligned} \tag{6.37}$$

3. **Joint Entropy** $H(X, Y)$:

$$H(X,Y) = -\sum_{i=1}^{2}\sum_{j=1}^{3} p_{ij}\log_2(p_{ij}) \approx 2(0.2161) + 2(0.5211) + 0.3322 + 0.4644 = 2.2710 \text{ bits} \tag{6.38}$$

4. **Mutual Information**:

$$I(X;Y) = H(X) + H(Y) - H(X,Y) = 0.9928 + 1.4406 - 2.2710 = 0.1624 \text{ bits} \tag{6.39}$$

Knowledge of the student's academic status reduces the uncertainty regarding their grade distribution by 0.1624 bits (accounting for 11.27% of the baseline grade entropy).

6.10 Foundations and Objectives of Cluster Analysis

Cluster Analysis (unsupervised classification) partitions an unlabelled dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ into homogeneous groups (**clusters**) according to two dual guiding principles:

1. **Maximizing Intra-Cluster Cohesion**: Objects assigned to the same cluster must share high reciprocal affinity and proximity;
2. **Maximizing Inter-Cluster Separation**: Objects located across different clusters must exhibit maximum metric distance.

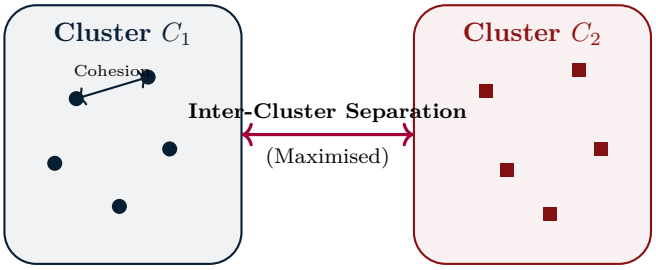


Figure 6.5: Geometric trade-off between internal cohesion and external separation in cluster analysis.

Unlike supervised learning, no target labels y are provided; structural boundaries are derived organically from sample geometry.

6.10.1 Applications: Understanding vs Summarization

Practical motivations for clustering fall into two complementary domains:

1. **Data Understanding**: Uncovering latent taxonomic patterns:
    - *Information Retrieval*: Hierarchical structuring of unstructured document libraries;
    - *Bioinformatics*: Grouping genes exhibiting synchronous functional co-expression profiles;
    - *Financial Markets*: Identifying co-moving equities (e.g., tech, energy, or banking clusters).
2. **Data Summarization**: Quantitative data compression where voluminous observations are condensed into representative prototypes (centroids or medoids), conserving aggregate statistical characteristics.

6.10.2 What Cluster Analysis is NOT

To maintain clear boundaries, clustering must be distinguished from related data tasks:

- **Manual Partitioning**: Arbitrary external assignment (e.g., grouping by name initial $A - L$ vs $M - Z$) reflects no natural data structure;
- **Database Querying**: SQL filter clauses (e.g., `WHERE Age > 30`) partition data by user-defined constraints rather than intrinsic topology;
- **Supervised Classification**: Optimizes explicit prediction against known target labels $y \in \mathcal{Y}$;
- **Pattern Mining**: Association rules discover local item implications across transactions, whereas clustering determines global structure across full feature vectors.

6.11 Clustering Taxonomies and Grouping Structures

6.11.1 Partitional vs Hierarchical Clustering

1. **Partitional Clustering**: Divides space into k disjoint subsets $\mathcal{C} = \{C_1, \ldots, C_k\}$ that fully cover the dataset:

$$\bigcup_{i=1}^{k} C_i = \mathcal{D}, \qquad C_i \cap C_j = \emptyset \quad \forall i \neq j \tag{6.40}$$

Each object belongs to exactly one cluster, requiring k to be specified upfront (e.g., K-Means).

2. **Hierarchical Clustering**: Produces a nested family of partitions organized as a metric tree called a **dendrogram**, eliminating the need to set k in advance.

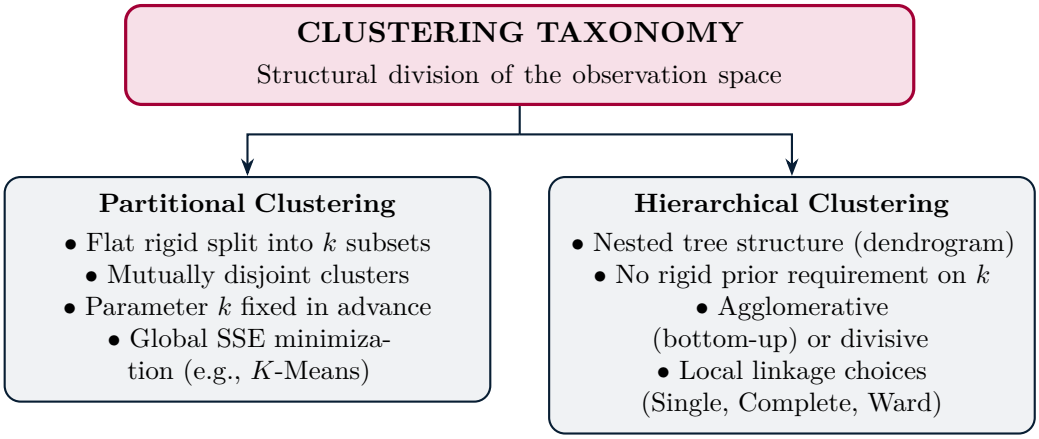


Figure 6.6: Structural comparison between partitional and hierarchical clustering.

6.11.2 Additional Structural Dimensions

- **Exclusive vs Non-Exclusive (Overlapping)**: In non-exclusive clustering, points may belong to multiple groups simultaneously (ideal for border observations or multi-topic texts).
- **Fuzzy (Soft) vs Non-Fuzzy (Hard)**: In fuzzy clustering, each instance $\mathbf{x}_i$ is assigned a continuous membership degree $w_{ik} \in [0, 1]$ satisfying $\sum_{k=1}^{K} w_{ik} = 1$.
- **Partial vs Complete**: Partial methods identify dense cohesive clusters while filtering out background noise and anomalies.
- **Homogeneous vs Heterogeneous**: Homogeneous algorithms presume similar geometric shapes and densities, whereas heterogeneous models accommodate arbitrary geometries and varying densities.

6.12 Cluster Types and Mathematical Formulations

The definition of a "cluster" differs significantly depending on the governing geometric concept:

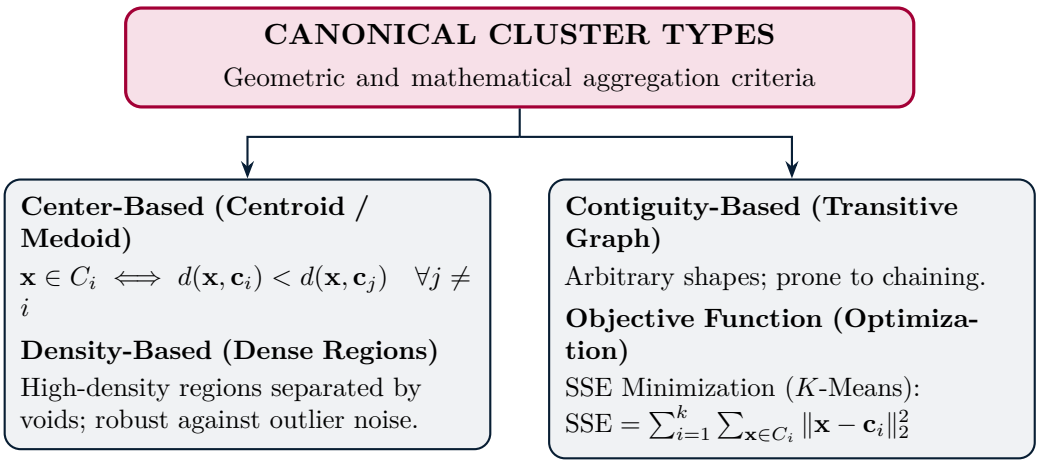


Figure 6.7: Overview of mathematical cluster formulations.

1. **Well-Separated Clusters**: Every point in a cluster is strictly closer to all cluster members than to any outside observation:

$$\forall \mathbf{x}, \mathbf{y} \in C_i, \ \forall \mathbf{z} \notin C_i \implies d(\mathbf{x}, \mathbf{y}) < d(\mathbf{x}, \mathbf{z}) \tag{6.41}$$

2. **Center-Based Clusters**: An object belongs to the cluster with the nearest prototype center:

$$\mathbf{x} \in C_i \iff d(\mathbf{x}, \mathbf{c}_i) < d(\mathbf{x}, \mathbf{c}_j) \quad \forall j \neq i \tag{6.42}$$

   The center can be the **centroid** (continuous arithmetic mean $\mathbf{c}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$) or the **medoid** (most central real instance $\mathbf{m}_i = \arg\min_{\mathbf{y} \in C_i} \sum_{\mathbf{x} \in C_i} d(\mathbf{x}, \mathbf{y})$).

3. **Contiguity-Based (Graph-Based) Clusters**: A point belongs to a cluster if it is closer to at least one cluster member than to any outside observation:

$$\forall \mathbf{x} \in C_i, \ \exists \mathbf{y} \in C_i \setminus \{\mathbf{x}\} \quad \text{s.t.} \quad d(\mathbf{x}, \mathbf{y}) < \min_{\mathbf{z} \notin C_i} d(\mathbf{x}, \mathbf{z}) \tag{6.43}$$

   Accommodates arbitrary non-convex geometries, but suffers from the **chaining effect**, where isolated noise observations bridge and improperly merge distinct groups.

4. **Density-Based Clusters**: Dense regions bounded and separated by regions of low point density (e.g., DBSCAN). Resilient to noise and unconstrained by spherical geometry.

5. **Objective-Function-Defined Clusters**: Framed as optimization problems. Exhaustive global discovery across N points and k clusters scales with the Stirling numbers of the second kind:

$$S(N,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^N \approx \frac{k^N}{k!} \tag{6.44}$$

As the problem is inherently **NP-Hard**, iterative descent heuristics such as K-Means minimize the Sum of Squared Errors (SSE):

$$\text{SSE} = \sum_{i=1}^{k}\sum_{\mathbf{x}\in C_i}\|\mathbf{x}-\mathbf{c}_i\|_2^2 \tag{6.45}$$

6.13 Input Data Characteristics and Algorithm Families

Clustering effectiveness depends on input space characteristics:

- **Curse of Dimensionality**: In high-dimensional spaces $\mathbb{R}^n$, pairwise distances concentrate and equalize, diminishing the contrast between near and far neighbors;
- **Sparsity and Heterogeneity**: Sparse vectors require asymmetric metrics (Jaccard, Cosine); mixed data types require hybrid metrics (Gower);
- **Noise and Outliers**: Distort prototype centroids in center-based methods and trigger spurious bridging in contiguity-based methods.

The three principal algorithm families are structured as follows:

1. **K-Means and Variants**: Partitional, exclusive, complete. Iteratively minimizes SSE across compact spherical clusters of comparable density;
2. **Agglomerative Hierarchical Clustering**: Hierarchical, nested. Iteratively merges closest cluster pairs according to linkage criteria (Single, Complete, Average, Ward), producing an interpretable dendrogram;
3. **Density-Based Clustering (DBSCAN)**: Local density-based, partial, non-parametric with respect to k. Naturally isolates noise points and detects clusters of arbitrary shape.

Chapter 7

Cluster Analysis: K-Means and Hierarchical Clustering Algorithms

In this chapter, we present the theoretical foundations, algebraic formulations, and operational dynamics of the two fundamental paradigms of unsupervised learning based on partitioning and hierarchy building:

1. The **K-Means Algorithm**: analytical formulation, Lloyd's dynamics, minimization of the sum of squared errors (SSE), formal proof of convergence via Coordinate Descent, initialization heuristics (K-Means++, Bisecting K-Means), and structural limitations;
2. **Hierarchical Clustering**: agglomerative and divisive approaches, dendrogram interpretation, inter-cluster linkage criteria (MIN, MAX, Group Average, Ward's method with analytical derivation), universal Lance-Williams recurrence relation, and graph-theoretic equivalence with the Minimum Spanning Tree (MST).

7.1 Introduction to Partitional and Hierarchical Cluster Analysis

As established in Chapter 6 (section 6.10), **Cluster Analysis** partitions a multidimensional dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$ by jointly maximizing intra-cluster cohesion and inter-cluster separation. Building upon the foundational taxonomy outlined in section 6.11, this chapter investigates the two primary operational paradigms of unsupervised pattern discovery: prototype-based partitional clustering (K-Means) and proximity-based hierarchical clustering.

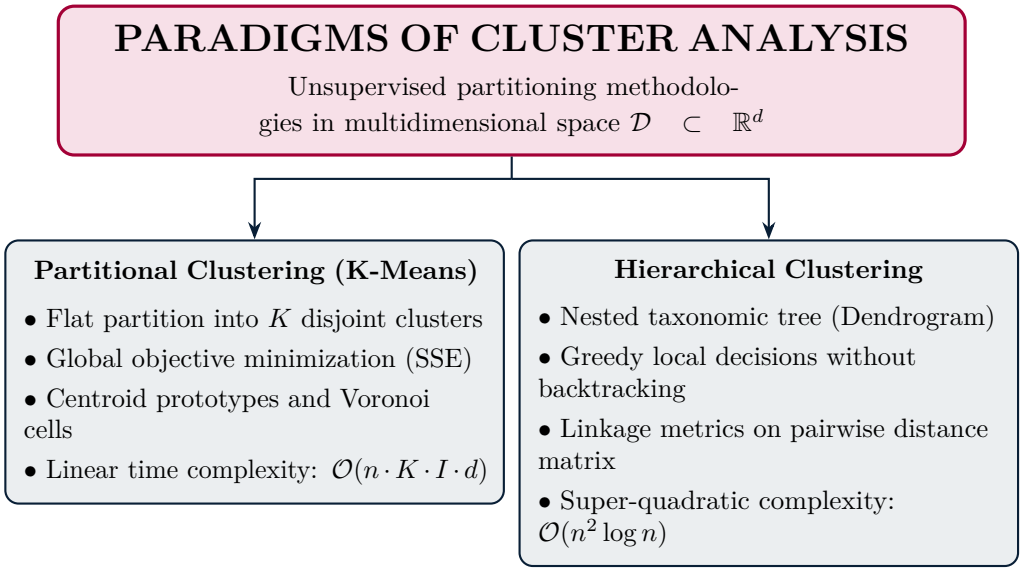


Figure 7.1: Taxonomy of the two core clustering paradigms covered in this chapter.

The two paradigms approach grouping with distinct structural philosophies: partitional clustering fixes the number of groups K a priori and pursues an optimal polygonal tessellation of space via prototypes; hierarchical clustering generates a nested sequence of partitions, deferring the choice of K to post-hoc dendrogram inspection.

7.2 K-Means: Lloyd's Algorithm and Voronoi Geometry

The **K-Means** algorithm represents the archetype of prototype-based partitional methods. Historically introduced by Hugo Steinhaus in 1956, formalized by Stuart Lloyd (1957/1982) in signal quantization, and named by James MacQueen (1967), the method produces a hard clustering where every data point is uniquely assigned to exactly one cluster.

Definition 7.2.1 (Hard Partition and Centroid)**.** Given a dataset of n points $\mathcal{D} \subset \mathbb{R}^d$ and a fixed positive integer $K \in \mathbb{N}^+$, a hard partition is a family of subsets $\{C_1, \ldots, C_K\}$ such that:

$$\bigcup_{i=1}^{K} C_i = \mathcal{D}, \qquad C_i \cap C_j = \emptyset \quad (\forall i \neq j), \qquad C_i \neq \emptyset \quad (\forall i \in \{1, \ldots, K\}) \tag{7.1}$$

Each cluster C_i is associated with a prototype vector termed the **centroid** $\mathbf{m}_i \in \mathbb{R}^d$ (often denoted as $\mathbf{c}_i$), which for Euclidean distance corresponds to the arithmetic sample mean of the assigned points:

$$\mathbf{m}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x} \tag{7.2}$$

where $|C_i|$ denotes the cardinality of cluster C_i.

7.2.1 Pseudocode of Lloyd's Algorithm

The standard algorithm alternates recursively between two deterministic phases until convergence, as formulated in Algorithm 1.

7.2.2 Geometric Interpretation: Voronoi Diagram

The assignment step partitions continuous Euclidean space $\mathbb{R}^d$ into K convex polyhedral regions bounded by hyperplanes, known as the **Voronoi tessellation** generated by the prototypes $\{\mathbf{m}_1, \ldots, \mathbf{m}_K\}$:

$$V(C_i) = \left\{ \mathbf{x} \in \mathbb{R}^d : \|\mathbf{x} - \mathbf{m}_i\|_2 \leq \|\mathbf{x} - \mathbf{m}_j\|_2 \quad \forall j \neq i \right\} \tag{7.3}$$

Each separating hyperplane between cluster C_i and cluster C_j is orthogonal to the line segment connecting $\mathbf{m}_i$ and $\mathbf{m}_j$ and passes through their midpoint. Consequently, every cluster identified by K-Means is a convex polyhedron; data lying on non-convex or manifold geometries cannot be captured without fracturing.

7.2.3 Temporal Dynamics Across Iterations

Analyzing centroid trajectories across iterations $t = 1, 2, \ldots$:

- **Iterations 1–3**: Large macroscopic migration occurs. Initial centroids rapidly escape spurious positions and move toward the centers of high-density regions.
- **Iterations 4–6 and onward**: Fine-tuning phase. Only a few boundary points swap across Voronoi boundaries until centroid shifts fall below the tolerance ε.

7.2.4 Generalization to Other Proximity Measures

While K-Means is traditionally formulated for Euclidean distance, the two-phase framework generalizes to other metrics provided the prototype is appropriately chosen:

Algorithm 1: Standard K-Means algorithm (Lloyd).

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$, number of clusters K, tolerance threshold $\varepsilon \geq 0$.

Output: Final partition $\{C_1, \ldots, C_K\}$ and centroids $\{\mathbf{m}_1, \ldots, \mathbf{m}_K\}$.

Initialize K centroids $\{\mathbf{m}_1^{(0)}, \ldots, \mathbf{m}_K^{(0)}\}$ via random uniform selection from $\mathcal{D}$

$t \leftarrow 0$

repeat

  $t \leftarrow t + 1$

  `// Phase 1: Assignment to Voronoi cells`

  Initialize $C_i^{(t)} \leftarrow \emptyset$ for each $i \in \{1, \ldots, K\}$

  foreach $\mathbf{x} \in \mathcal{D}$ **do**

    Determine index of closest centroid: $k^* = \arg\min_{j \in \{1,\ldots,K\}} \|\mathbf{x} - \mathbf{m}_j^{(t-1)}\|^2$

    Assign $\mathbf{x}$ to corresponding cluster: $C_{k^*}^{(t)} \leftarrow C_{k^*}^{(t)} \cup \{\mathbf{x}\}$

  end

  `// Phase 2: Prototype recomputation (sample means)`

  foreach $i \in \{1, \ldots, K\}$ **do**

    if $|C_i^{(t)}| > 0$ **then**

      $\mathbf{m}_i^{(t)} \leftarrow \dfrac{1}{|C_i^{(t)}|} \sum_{\mathbf{x} \in C_i^{(t)}} \mathbf{x}$

    end

    else

      Handle empty cluster by reallocating an orphan centroid

    end

  end

until *centroids do not change significantly:* $\sum_{i=1}^{K} \|\mathbf{m}_i^{(t+1)} - \mathbf{m}_i^{(t)}\|^2 \leq \varepsilon$

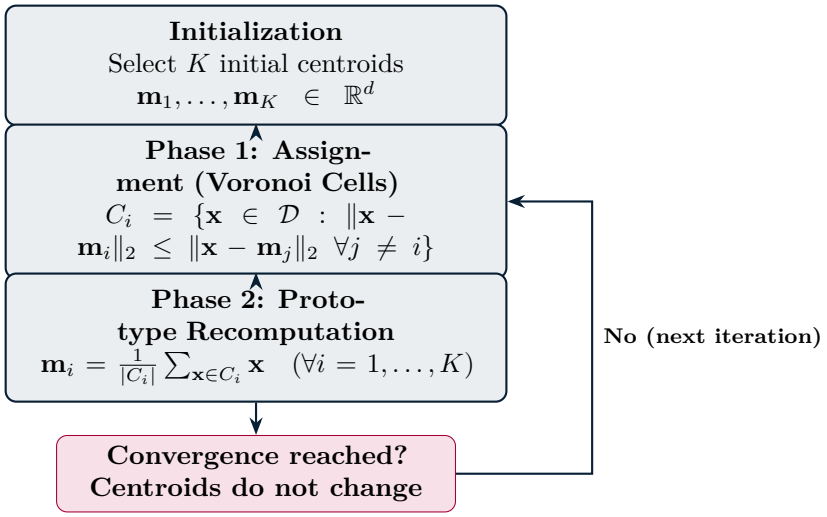


Figure 7.2: Two-phase iterative update cycle of the K-Means algorithm.

- **Euclidean Distance (L_2)**: Prototype = **Mean** (centroid). Minimizes the sum of squared errors (SSE).
- **Manhattan Distance (L_1)**: Prototype = Component-wise **Median** (**K-Medians** algorithm). Minimizes the sum of absolute errors (SAE = $\sum_i \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_1$).
- **Cosine Similarity**: Prototype = **Normalized Mean** to unit norm ($\mathbf{m}_i / \|\mathbf{m}_i\|_2$). Maximizes cumulative cosine similarity (widely used in text mining).
- **Pearson Correlation**: Prototype = **Standardized Vector** (zero mean, unit variance).

7.2.5 Computational Complexity Analysis

Lloyd's algorithm exhibits exceptional computational efficiency:

- **Time Complexity**: In each iteration, assigning n points to K centroids in d dimensions requires $\mathcal{O}(n \cdot K \cdot d)$ operations. Recomputing centroids takes $\mathcal{O}(n \cdot d)$. Denoting by I the number of iterations to convergence, total time complexity is:

$$\mathcal{T}_{\text{K-Means}} = \mathcal{O}(n \cdot K \cdot I \cdot d) \tag{7.4}$$

  Since empirically $K \ll n$, $d \ll n$, and $I \ll n$ (often $I \leq 50$), time scales linearly with dataset size n.

- **Space Complexity**: Storing the data matrix $n \times d$, K centroid vectors of dimension d, and the cluster assignment vector $n \times 1$ requires:

$$\mathcal{S}_{\text{K-Means}} = \mathcal{O}((n + K) \cdot d) \tag{7.5}$$

making the algorithm readily scalable to massive datasets residing in main memory.

7.3 SSE Objective Function and Proof of Convergence

The analytical stability of K-Means stems from the presence of a well-defined scalar objective function that is strictly minimized by the alternating steps of Lloyd's method.

Definition 7.3.1 (Sum of Squared Errors)**.** For a partition $\{C_1, \ldots, C_K\}$ with prototypes $\{\mathbf{m}_1, \ldots, \mathbf{m}_K\}$, the **sum of squared errors** (SSE, or within-cluster inertia) is defined as:

$$\text{SSE} = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_2^2 = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} \sum_{j=1}^{d} (x_j - m_{ij})^2 \tag{7.6}$$

7.3.1 Proof: The Sample Mean Strictly Minimizes SSE

Theorem 7.3.1 (Optimality of the Centroid Mean)**.** *Given a cluster C_k containing $|C_k|$ points in $\mathbb{R}^d$, the prototype $\mathbf{m}_k \in \mathbb{R}^d$ that minimizes the sum of squared Euclidean distances to all points in C_k is uniquely the arithmetic sample mean of those points.*

Proof. The squared error for a single cluster C_k decomposes independently across the d orthogonal dimensions:

$$\text{SSE}_k = \sum_{\mathbf{x} \in C_k} \sum_{j=1}^{d} (x_j - m_{kj})^2 = \sum_{j=1}^{d} \left(\sum_{\mathbf{x} \in C_k} (x_j - m_{kj})^2 \right) \tag{7.7}$$

To find the minimum with respect to each scalar coordinate m_{kj} of centroid $\mathbf{m}_k$, we compute the first partial derivative and set it to zero:

$$\begin{aligned} \frac{\partial \, \text{SSE}_k}{\partial m_{kj}} &= \frac{\partial}{\partial m_{kj}} \sum_{\mathbf{x} \in C_k} (x_j - m_{kj})^2 \\ &= \sum_{\mathbf{x} \in C_k} 2(x_j - m_{kj}) \cdot (-1) \\ &= -2 \sum_{\mathbf{x} \in C_k} x_j + 2 \sum_{\mathbf{x} \in C_k} m_{kj} = 0 \end{aligned} \tag{7.8}$$

Since m_{kj} is constant with respect to the summation over $\mathbf{x} \in C_k$, the second sum evaluates to $|C_k| \cdot m_{kj}$:

$$2|C_k|\, m_{kj} = 2 \sum_{\mathbf{x} \in C_k} x_j \implies m_{kj} = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} x_j \tag{7.9}$$

Verifying the second derivative:

$$\frac{\partial^2 \mathrm{SSE}_k}{\partial m_{kj}^2} = 2|C_k| > 0 \tag{7.10}$$

The corresponding Hessian matrix is a strictly positive-definite scalar diagonal matrix:

$$\nabla^2 \mathrm{SSE}_k = 2|C_k|\, \mathbf{I}_d \succ 0 \tag{7.11}$$

Therefore, the sample mean $\mathbf{m}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$ is the unique strict global minimizer of SSE_k. □

7.3.2 Coordinate Descent Formulation and Convergence Proof

Lloyd's algorithm can be rigorously formulated as **Coordinate Descent** on a bivariate objective function.

Let $\mathbf{W} \in \{0,1\}^{n \times K}$ be the binary assignment matrix with $w_{ik} = 1$ if instance $\mathbf{x}_i \in C_k$ and $w_{ik} = 0$ otherwise, subject to $\sum_{k=1}^{K} w_{ik} = 1$ for all i. Let $\mathbf{S} = (\mathbf{m}_1, \ldots, \mathbf{m}_K) \in \mathbb{R}^{d \times K}$ represent the matrix of centroids. The global objective function is:

$$G(\mathbf{W}, \mathbf{S}) = \sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik} \|\mathbf{x}_i - \mathbf{m}_k\|_2^2 \tag{7.12}$$

Theorem 7.3.2 (Monotone Convergence to a Local Minimum)**.** *The K-Means algorithm monotonically decreases the objective function $G(\mathbf{W}, \mathbf{S})$ at every half-step and converges in a finite number of iterations to a local minimum or stationary point.*

Proof. Execution alternates between two optimization steps:

1. **Step 1 (Assignment)**: Holding centroids $\mathbf{S}^{(t-1)}$ fixed, minimize $G(\mathbf{W}, \mathbf{S}^{(t-1)})$ over $\mathbf{W}$. Because each observation's contribution is decoupled, the objective is minimized by setting:

$$w_{ik}^{(t)} = \begin{cases} 1 & \text{if } k = \arg\min_j \|\mathbf{x}_i - \mathbf{m}_j^{(t-1)}\|_2^2 \\ 0 & \text{otherwise} \end{cases} \tag{7.13}$$

Consequently:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t-1)}) \leq G(\mathbf{W}^{(t-1)}, \mathbf{S}^{(t-1)}) \tag{7.14}$$

2. **Step 2 (Centroid Update)**: Holding assignments $\mathbf{W}^{(t)}$ fixed, minimize $G(\mathbf{W}^{(t)}, \mathbf{S})$ over $\mathbf{S}$. By the centroid optimality theorem, the optimal choice for each centroid $\mathbf{m}_k$ is:

$$\mathbf{m}_k^{(t)} = \frac{\sum_{i=1}^n w_{ik}^{(t)} \mathbf{x}_i}{\sum_{i=1}^n w_{ik}^{(t)}} = \frac{1}{|C_k^{(t)}|} \sum_{\mathbf{x} \in C_k^{(t)}} \mathbf{x} \tag{7.15}$$

Consequently:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t)}) \leq G(\mathbf{W}^{(t)}, \mathbf{S}^{(t-1)}) \tag{7.16}$$

Combining both inequalities gives a monotonically non-increasing sequence:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t)}) \leq G(\mathbf{W}^{(t-1)}, \mathbf{S}^{(t-1)}) \tag{7.17}$$

Since $G(\mathbf{W}, \mathbf{S}) \geq 0$ is bounded below by zero, and the set of possible discrete partition matrices $\mathbf{W}$ is finite (given by the Stirling numbers of the second kind $S(n, K)$), the algorithm cannot cycle indefinitely and must reach convergence to a stationary point (local minimum) in a finite number of steps. □

7.4 Choice of Parameter K, Local Minima, and Initialization

Despite guaranteed convergence, the optimization landscape of K-Means is non-convex and laden with sub-optimal local minima. In fact, finding the globally optimal clustering that minimizes SSE is known to be **NP-hard** even for $K = 2$ in general dimensions, or for $d = 2$ with general K.

7.4.1 Determining the Number of Clusters: The Elbow Method

Because SSE(K) monotonically decreases as K increases (trivially reaching zero at $K = n$), one cannot minimize SSE directly with respect to K.

The standard empirical approach is the **Elbow Method**:

1. Run K-Means across a range of candidate values $K \in \{1, 2, \ldots, K_{\max}\}$;
2. Plot the resulting SSE against K;

3. Identify the inflection point ("elbow") where the marginal reduction in error sharply diminishes.

Analytically, the elbow corresponds to the value of K that maximizes the discrete second derivative of SSE:

$$\Delta^2 \mathrm{SSE}(K) = \mathrm{SSE}(K+1) - 2\,\mathrm{SSE}(K) + \mathrm{SSE}(K-1) \tag{7.18}$$

Beyond this point, introducing further clusters splits natural groups without providing substantial explanatory gain.

7.4.2 Sensitivity to Initial Centroids and Combinatorial Proof

The final clustering quality produced by K-Means depends critically on the initial centroid placement $\mathbf{m}_1^{(0)}, \ldots, \mathbf{m}_K^{(0)}$. If two centroids land in the same true cluster, an adjacent natural group is left without a dedicated prototype, trapping the algorithm in an inferior local minimum.

Theorem 7.4.1 (Combinatorial Probability of Pure Random Initialization)**.** *Consider an ideal dataset containing K well-separated natural clusters of equal size N/K. Under uniform random sampling with replacement, the probability $P(K)$ that each selected initial centroid falls into a distinct natural cluster is:*

$$P(K) = \frac{K!}{K^K} \tag{7.19}$$

Proof. We evaluate the probability as a product of conditional probabilities step by step:

1. The first centroid can land in any of the K clusters: $P_1 = \frac{K}{K} = 1$.
2. For the second centroid to land in a different cluster, it must be drawn from one of the remaining $K-1$ unoccupied clusters. Since all clusters have equal size, $P_2 = \frac{K-1}{K}$.
3. The third centroid must fall into one of the remaining $K-2$ unoccupied clusters: $P_3 = \frac{K-2}{K}$.
4. Continuing up to the K-th centroid yields $P_K = \frac{1}{K}$.

Multiplying these independent probabilities:

$$P(K) = \prod_{j=1}^{K} \frac{K-j+1}{K} = \frac{K \cdot (K-1) \cdot (K-2) \cdots 1}{K^K} = \frac{K!}{K^K} \tag{7.20}$$

□

Evaluating equation (7.19) for increasing values of K:

- $K = 2 \implies P(2) = \frac{2}{4} = 0.50$ (50%);
- $K = 3 \implies P(3) = \frac{6}{27} \approx 0.222$ (22.2%);
- $K = 5 \implies P(5) = \frac{120}{3125} \approx 0.0384$ (3.84%);
- $K = 10 \implies P(10) = \frac{3\,628\,800}{10^{10}} \approx 0.00036$ (a mere **0.036%**).

Applying Stirling's approximation $K! \approx \sqrt{2\pi K}\left(\frac{K}{e}\right)^K$:

$$P(K) \approx \frac{\sqrt{2\pi K}\left(\frac{K}{e}\right)^K}{K^K} = \sqrt{2\pi K} \cdot e^{-K} \tag{7.21}$$

The probability of successful random initialization decays **exponentially** toward zero as K grows. For $K \geq 10$, naive random initialization almost never lands one centroid per true cluster.

7.5 Initialization Heuristics and Variants of K-Means

To counteract the fragility of naive initialization, robust initialization heuristics and structural variants have been developed.

7.5.1 Heuristic Initialization Strategies

1. **Random Restarts**: Run the algorithm R times (e.g., $R = 50$) with different random seeds, retaining the partition yielding minimum SSE. Effective for small K, but computationally prohibitive and insufficient for large K.

2. **Sampling with Hierarchical Pre-Clustering**: Extract a small random sample ($\approx 1\% - 5\%$) of the data and apply agglomerative hierarchical clustering with Ward's method to identify K stable groups, using their centroids as seeds for full-scale K-Means.

3. **K-Means++ (D^2-Weighting)**: Introduced by Arthur and Vassilvitskii (2007), chooses the first centroid uniformly at random, and each subsequent centroid with probability proportional to the squared distance to the nearest existing centroid:

$$p(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{y}\in\mathcal{D}} D(\mathbf{y})^2}, \qquad D(\mathbf{x}) = \min_{j\in\{1,\ldots,m\}} \|\mathbf{x} - \mathbf{m}_j\|_2 \tag{7.22}$$

Theoretical guarantee: K-Means++ guarantees that the expected final SSE is within a logarithmic factor of the global optimum:

$$\mathbb{E}[\text{SSE}] \leq 8(\ln K + 2) \cdot \text{SSE}_{\text{opt}} = \mathcal{O}(\log K) \cdot \text{SSE}_{\text{opt}} \tag{7.23}$$

4. **Bisecting K-Means**: A hybrid divisive approach relying on recursive binary bisections ($K = 2$):

   - Start with the entire dataset treated as a single macro-cluster;
   - At each step, select the cluster with the highest within-cluster SSE (or largest cardinality);
   - Apply standard K-Means with $K = 2$ to split this cluster into two sub-clusters;
   - Repeat until exactly K clusters are formed.

   Bisecting K-Means is substantially less sensitive to initialization than Lloyd's method and inherently produces a hierarchical tree of clusters.

7.5.2 Incremental Centroid Update (Online K-Means)

In MacQueen's online formulation (1967), centroids are updated *immediately* after every individual point reassignment, rather than at the end of each epoch:

Property 7.5.1 (MacQueen's Incremental Update Formulas)**.** *When point* $\mathbf{x}$ *is removed from cluster* C_j *and added to cluster* C_i*:*

$$\mathbf{m}_j \leftarrow \mathbf{m}_j - \frac{\mathbf{x} - \mathbf{m}_j}{|C_j| - 1} \tag{7.24}$$

$$\mathbf{m}_i \leftarrow \mathbf{m}_i + \frac{\mathbf{x} - \mathbf{m}_i}{|C_i| + 1} \tag{7.25}$$

This incremental update guarantees that SSE decreases with every individual movement, eliminates empty cluster occurrences, but introduces sensitivity to data presentation order.

7.5.3 Handling Empty Clusters

In batch Lloyd's algorithm, an assignment phase may leave a centroid with no nearest points ($|C_k| = 0$, an orphan centroid). Standard remediation strategies:

1. **Maximum Residual Selection**: Identify point $\mathbf{x}^* \in \mathcal{D}$ with maximum squared error from its assigned centroid $\|\mathbf{x}^* - \mathbf{m}_{c(\mathbf{x}^*)}\|^2$, and assign it as the new isolated centroid of the empty cluster. This yields the maximum immediate marginal reduction in SSE.

2. **Splitting the Cluster with Maximum SSE**: Find the cluster contributing the highest cumulative SSE and split it by moving its most distant point into the empty cluster.

7.6 Structural Limitations of K-Means and Adaptation Techniques

K-Means implicitly assumes spherical, compact, well-separated clusters of similar volumes and densities. In the presence of complex geometries, the algorithm systematically degrades.

1. Differing Sizes
Centroids divide space evenly; smaller cluster centroid absorbs peripheral points from the larger cluster.

2. Differing Densities
Sparse clusters are fragmented, while compact high-density clusters are improperly merged together.

3. Non-Globular Geometries
Concave or intertwined shapes (*two moons*, concentric rings) are sliced by linear Voronoi boundaries.

4. Outlier Sensitivity
Squared error penalties $\|\mathbf{x} - \mathbf{m}\|^2$ heavily distort centroid locations toward extreme outliers.

Figure 7.3: Four primary structural failure modes of the standard K-Means algorithm.

7.6.1 Analysis of Three Geometric Failure Modes

1. **Differing Sizes**: Because SSE penalizes squared deviations without factoring in intrinsic volume, the centroid of a compact cluster absorbs points from a dispersed cluster to lower overall SSE, distorting cluster boundaries.

2. **Differing Densities**: Equidistant hyperplanes between centroids do not account for density gradients. A sparse cluster requires multiple centroids to cover, forcing adjacent dense clusters to merge into a single group.

3. **Non-Globular Shapes (e.g., Two Moons)**: Voronoi cells are convex polyhedra. For intertwined non-convex manifolds (two interlocking crescents), linear hyperplanes cut across both curves, failing entirely to separate them.

Remediation Strategy: Over-Clustering

To model complex manifolds or uneven densities, one runs K-Means with an over-specified number of micro-clusters ($K' \gg K$). A chain of small centroids provides a piecewise linear approximation along the manifold curve. Subsequent graph-based or linkage routines merge these micro-clusters into the desired non-linear clusters.

7.6.2 Pre-processing and Post-processing: The ISODATA Paradigm

To mitigate these limitations in practice, pre- and post-processing routines formalized in the classic **ISODATA** algorithm (*Iterative Self-Organizing Data Analysis Technique*) are commonly used:

- **Pre-processing**:
  - Feature normalization or standardization (z-score) to prevent large-scale attributes from dominating Euclidean distance calculations;
  - Outlier detection and filtering prior to centroid initialization.
- **Post-processing (ISODATA Dynamics)**:
  - **Split**: If a cluster's standard deviation or SSE exceeds a maximum threshold θ_{split} and its size is sufficient ($|C_i| \geq 2\theta_{\text{min}}$), split it along its coordinate of maximum variance.
  - **Merge**: If the Euclidean distance between two centroids $\|\mathbf{m}_i - \mathbf{m}_j\|_2$ falls below θ_{merge}, merge them into a single centroid.
  - **Prune**: If $|C_i| < \theta_{\text{min}}$, eliminate the cluster and reassign its members.

7.7 Foundations of Hierarchical Clustering

Hierarchical Clustering overcomes the limitation of flat partitions by generating a continuous nested sequence of clusters.

Definition 7.7.1 (Dendrogram)**.** A **dendrogram** is a rooted binary tree visualization where:

- The **leaves** at level $h = 0$ represent individual data points (singleton clusters);
- The **internal nodes** represent successive pairwise cluster merges;
- The **height** h of each junction equals the distance or dissimilarity at which the two clusters were merged.

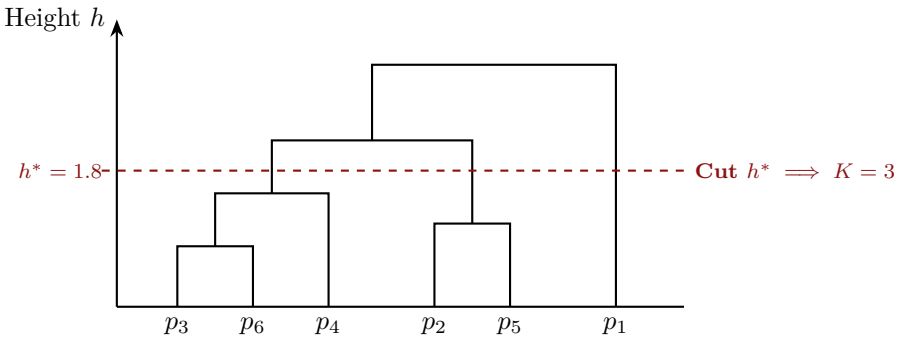


Figure 7.4: Dendrogram structure: junction heights represent merge distances. A horizontal cut yields a flat partition with $K = 3$ clusters.

7.7.1 Distinct Strengths of Hierarchical Clustering

- **No a priori constraint on** K: The user does not need to specify cluster count beforehand. A full visual taxonomy is revealed, and K is chosen post-hoc by drawing a horizontal cut at height h^*.
- **Modeling natural hierarchies**: Biological phylogenies, taxonomies, and linguistic trees possess natural hierarchical structures faithfully captured by dendrograms.
- **Determinism**: Standard hierarchical algorithms are fully deterministic, with no sensitivity to random seeds.

7.7.2 Taxonomy of Hierarchical Algorithms

- **Agglomerative (*Bottom-up*)**: Begin with n singleton clusters and iteratively merge the closest pair of clusters in $n - 1$ steps until a single root cluster remains.
- **Divisive (*Top-down*)**: Begin with the entire dataset in a single cluster and recursively split clusters into sub-clusters until n singletons are reached.

7.8 Agglomerative Algorithm and Linkage Criteria

The standard agglomerative procedure requires maintaining an inter-cluster proximity matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$, following Algorithm 2.

Algorithm 2: Standard agglomerative hierarchical clustering algorithm.

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, distance metric $d(\cdot, \cdot)$.
Output: Complete hierarchical dendrogram of merges.

Initialize n singleton clusters: $C_i = \{\mathbf{x}_i\}$ for each $i \in \{1, \ldots, n\}$
Compute initial proximity matrix $\mathbf{D}^{(0)} \in \mathbb{R}^{n \times n}$ where $D_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$
for $t = 1$ **to** $n - 1$ **do**
    Find the closest pair of clusters: $(A, B) = \arg\min_{i \neq j} D_{ij}^{(t-1)}$
    Merge clusters A and B into $A \cup B$ at height $h_t = D_{AB}^{(t-1)}$
    Remove rows and columns corresponding to A and B from the matrix
    Compute distance between new cluster $A \cup B$ and every remaining cluster Q according to chosen linkage criterion
    Update proximity matrix $\mathbf{D}^{(t)}$ with the new row and column for $A \cup B$
end

7.8.1 The Four Principal Linkage Criteria

The definition of inter-cluster distance $D(A, B)$ governs the geometry and properties of the resulting hierarchy.

1. MIN (Single Link)

Distance between two clusters equals the minimum distance between any pair of constituent points:

$$D_{\text{MIN}}(A, B) = \min_{\mathbf{x} \in A,\, \mathbf{y} \in B} d(\mathbf{x}, \mathbf{y}) \tag{7.26}$$

- **Strengths**: Capable of detecting arbitrary non-convex, elongated, or concentric shapes (isomorphic to Kruskal's Minimum Spanning Tree construction).

- **Weaknesses (*Chaining Effect*)**: Highly susceptible to noise. A single bridge of noise points between two well-separated clusters causes them to prematurely merge into a long chain.

2. MAX (Complete Link)

Distance equals the maximum distance between any pair of constituent points:

$$D_{\text{MAX}}(A, B) = \max_{\mathbf{x}\in A,\, \mathbf{y}\in B} d(\mathbf{x}, \mathbf{y}) \tag{7.27}$$

- **Strengths**: Immune to chaining; enforces compact, spherical clusters by strictly bounding the cluster diameter: $\text{diam}(A \cup B) \leq h$.
- **Weaknesses**: Tends to fragment elongated clusters and is sensitive to peripheral outliers.

3. Group Average (Average Link / UPGMA)

Distance is defined as the average distance between all pairs of cross-cluster points:

$$D_{\text{AVG}}(A, B) = \frac{1}{|A| \cdot |B|} \sum_{\mathbf{x}\in A} \sum_{\mathbf{y}\in B} d(\mathbf{x}, \mathbf{y}) \tag{7.28}$$

A robust compromise between MIN and MAX. Less sensitive to noise than Single Link because individual outliers are averaged across all pairwise distances.

4. Ward's Method (Variance Minimization)

Introduced by Joe H. Ward Jr. (1963), proximity between two clusters is measured by the **increase in total squared error** (ΔSSE) resulting from their merge:

$$\Delta\text{SSE}_{AB} = \text{SSE}_{A\cup B} - (\text{SSE}_A + \text{SSE}_B) \tag{7.29}$$

Theorem 7.8.1 (Closed-Form Expression for Ward's Method)**.** *The increase in sum of squared errors ΔSSE_{AB} caused by merging clusters A and B equals the squared Euclidean distance between their centroids, scaled by the harmonic product of their cardinalities:*

$$\Delta\text{SSE}_{AB} = \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|_2^2 \tag{7.30}$$

Proof. Let A and B be disjoint clusters with cardinalities $|A|, |B|$ and centroids $\mathbf{m}_A, \mathbf{m}_B$. The centroid of the merged cluster $A \cup B$ is:

$$\mathbf{m}_{A\cup B} = \frac{|A|\mathbf{m}_A + |B|\mathbf{m}_B}{|A| + |B|} \tag{7.31}$$

The displacement vector between the merged centroid and $\mathbf{m}_A$ is:

$$\mathbf{m}_{A\cup B} - \mathbf{m}_A = \frac{|B|(\mathbf{m}_B - \mathbf{m}_A)}{|A| + |B|} \tag{7.32}$$

Symmetrically for cluster B:

$$\mathbf{m}_{A\cup B} - \mathbf{m}_B = \frac{|A|(\mathbf{m}_A - \mathbf{m}_B)}{|A| + |B|} \tag{7.33}$$

The total squared error of the union is:

$$\mathrm{SSE}_{A\cup B} = \sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 + \sum_{\mathbf{y}\in B} \|\mathbf{y} - \mathbf{m}_{A\cup B}\|^2 \tag{7.34}$$

Expanding the first summation by inserting and subtracting $\mathbf{m}_A$:

$$\begin{aligned}\sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 &= \sum_{\mathbf{x}\in A} \|(\mathbf{x} - \mathbf{m}_A) - (\mathbf{m}_{A\cup B} - \mathbf{m}_A)\|^2 \\ &= \sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_A\|^2 + \sum_{\mathbf{x}\in A} \|\mathbf{m}_{A\cup B} - \mathbf{m}_A\|^2 \\ &\quad - 2\left(\sum_{\mathbf{x}\in A}(\mathbf{x} - \mathbf{m}_A)\right) \cdot (\mathbf{m}_{A\cup B} - \mathbf{m}_A)\end{aligned} \tag{7.35}$$

Since $\sum_{\mathbf{x}\in A}(\mathbf{x}-\mathbf{m}_A) = \mathbf{0}$ by definition of the centroid, the cross-term vanishes:

$$\sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 = \mathrm{SSE}_A + |A| \cdot \|\mathbf{m}_{A\cup B} - \mathbf{m}_A\|^2 \tag{7.36}$$

Substituting the displacement expression:

$$\begin{aligned}\sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 &= \mathrm{SSE}_A + |A| \left\| \frac{|B|(\mathbf{m}_B - \mathbf{m}_A)}{|A| + |B|} \right\|^2 \\ &= \mathrm{SSE}_A + \frac{|A| \cdot |B|^2}{(|A| + |B|)^2} \|\mathbf{m}_A - \mathbf{m}_B\|^2\end{aligned} \tag{7.37}$$

Repeating for cluster B:

$$\sum_{\mathbf{y}\in B} \|\mathbf{y} - \mathbf{m}_{A\cup B}\|^2 = \mathrm{SSE}_B + \frac{|A|^2 \cdot |B|}{(|A| + |B|)^2} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \tag{7.38}$$

Summing both contributions yields $\mathrm{SSE}_{A\cup B}$:

$$\begin{aligned}\mathrm{SSE}_{A\cup B} &= \mathrm{SSE}_A + \mathrm{SSE}_B + \left[\frac{|A| \cdot |B|^2 + |A|^2 \cdot |B|}{(|A| + |B|)^2}\right] \|\mathbf{m}_A - \mathbf{m}_B\|^2 \\ &= \mathrm{SSE}_A + \mathrm{SSE}_B + \left[\frac{|A| \cdot |B|(|B| + |A|)}{(|A| + |B|)^2}\right] \|\mathbf{m}_A - \mathbf{m}_B\|^2 \\ &= \mathrm{SSE}_A + \mathrm{SSE}_B + \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|^2\end{aligned} \tag{7.39}$$

Subtracting $(\mathrm{SSE}_A + \mathrm{SSE}_B)$ proves the claim:

$$\Delta\mathrm{SSE}_{AB} = \mathrm{SSE}_{A\cup B} - (\mathrm{SSE}_A + \mathrm{SSE}_B) = \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \quad (7.40)$$

□

Ward's method serves as the exact hierarchical analogue of the K-Means objective function: it produces compact, spherical, balanced clusters and exhibits strong noise tolerance.

7.8.2 Worked Numerical Example on 6 Points: MIN vs MAX

Consider a dataset of 6 points $\{p_1, \ldots, p_6\}$ with initial distance matrix:

$$\mathbf{D} = \begin{pmatrix} & p_1 & p_2 & p_3 & p_4 & p_5 & p_6 \\ p_1 & 0.00 & 0.24 & 0.22 & 0.37 & 0.34 & 0.23 \\ p_2 & 0.24 & 0.00 & 0.15 & 0.20 & 0.14 & 0.25 \\ p_3 & 0.22 & 0.15 & 0.00 & 0.15 & 0.28 & 0.11 \\ p_4 & 0.37 & 0.20 & 0.15 & 0.00 & 0.29 & 0.22 \\ p_5 & 0.34 & 0.14 & 0.28 & 0.29 & 0.00 & 0.39 \\ p_6 & 0.23 & 0.25 & 0.11 & 0.22 & 0.39 & 0.00 \end{pmatrix} \quad (7.41)$$

Resolution with MIN (Single Link)

1. Global minimum: $d(p_3, p_6) = 0.11 \implies$ Merge $\{p_3, p_6\}$ at $h = 0.11$.
2. Next minimum: $d(p_2, p_5) = 0.14 \implies$ Merge $\{p_2, p_5\}$ at $h = 0.14$.
3. Distance from $\{p_3, p_6\}$ to p_4: $\min(d(p_3, p_4), d(p_6, p_4)) = 0.15 \implies$ Merge into $\{p_3, p_6, p_4\}$ at $h = 0.15$.
4. Distance between $\{p_3, p_6, p_4\}$ and $\{p_2, p_5\}$:

$$\begin{aligned} \min_{\mathbf{x}\in\{3,6,4\},\, \mathbf{y}\in\{2,5\}} d(\mathbf{x}, \mathbf{y}) &= \min(0.15, 0.25, 0.20, 0.28, 0.39, 0.29) \\ &= d(p_3, p_2) = 0.15 \end{aligned} \quad (7.42)$$

   The two blocks merge immediately at height $h = 0.15$ via link p_3–p_2.
5. Distance to p_1: $\min(d(p_1, p_j)) = d(p_1, p_3) = 0.22 \implies$ Final merge at $h = 0.22$.

Resolution with MAX (Complete Link)

1. First two steps match MIN: merge $\{p_3, p_6\}$ at $h = 0.11$, and $\{p_2, p_5\}$ at $h = 0.14$.

2. MAX distance from $\{p_3, p_6\}$ to p_4: $\max(d(p_3, p_4), d(p_6, p_4)) = 0.22$.

3. MAX distance between $\{p_3, p_6\}$ and $\{p_2, p_5\}$:

$$\begin{aligned} \max_{\mathbf{x}\in\{3,6\},\,\mathbf{y}\in\{2,5\}} d(\mathbf{x}, \mathbf{y}) &= \max(0.15, 0.25, 0.28, 0.39) \\ &= d(p_6, p_5) = 0.39 \end{aligned} \tag{7.43}$$

   Since $0.22 < 0.39$, the preferred merge is $\{p_3, p_6\}$ with p_4 at $h = 0.22$.

4. Distance between $\{p_3, p_6, p_4\}$ and $\{p_2, p_5\}$ remains high (0.39). In contrast, distance from p_1 to $\{p_2, p_5\}$ is $\max(0.24, 0.34) = 0.34$. Thus, MAX postpones merging the two large groups, avoiding premature linkage chains.

7.8.3 Lance-Williams Recurrence Formula

The recurrence formula by Lance and Williams (1967) enables updating proximity matrices algebraically without recomputing individual point distances.

Theorem 7.8.2 (Lance-Williams Formula)**.** *When clusters A and B merge into $A \cup B$, the distance to any remaining cluster Q satisfies:*

$$D(A\cup B,\, Q) = \alpha_A\, D(A, Q) + \alpha_B\, D(B, Q) + \beta\, D(A, B) + \gamma\, |D(A, Q) - D(B, Q)| \tag{7.44}$$

where parameters $(\alpha_A, \alpha_B, \beta, \gamma)$ define the specific linkage metric.

Linkage Criterion	$\alpha_{\mathbf{A}}$	$\alpha_{\mathbf{B}}$	β	γ
MIN (Single Link)	$1/2$	$1/2$	0	$-1/2$
MAX (Complete Link)	$1/2$	$1/2$	0	$+1/2$
Group Average	$\frac{\lvert A\rvert}{\lvert A\rvert+\lvert B\rvert}$	$\frac{\lvert B\rvert}{\lvert A\rvert+\lvert B\rvert}$	0	0
Centroid Linkage	$\frac{\lvert A\rvert}{\lvert A\rvert+\lvert B\rvert}$	$\frac{\lvert B\rvert}{\lvert A\rvert+\lvert B\rvert}$	$-\frac{\lvert A\rvert\lvert B\rvert}{(\lvert A\rvert+\lvert B\rvert)^2}$	0
Ward's Method	$\frac{\lvert A\rvert+\lvert Q\rvert}{\lvert A\rvert+\lvert B\rvert+\lvert Q\rvert}$	$\frac{\lvert B\rvert+\lvert Q\rvert}{\lvert A\rvert+\lvert B\rvert+\lvert Q\rvert}$	$-\frac{\lvert Q\rvert}{\lvert A\rvert+\lvert B\rvert+\lvert Q\rvert}$	0

Table 7.1: Lance-Williams recurrence coefficients for major hierarchical methods.

Consistency Proof for Group Average. Substituting the Lance-Williams coefficients for Group Average:

$$\sum_{\mathbf{x}\in A\cup B}\sum_{\mathbf{z}\in Q} d(\mathbf{x},\mathbf{z}) = \sum_{\mathbf{x}\in A}\sum_{\mathbf{z}\in Q} d(\mathbf{x},\mathbf{z}) + \sum_{\mathbf{y}\in B}\sum_{\mathbf{z}\in Q} d(\mathbf{y},\mathbf{z})$$
$$= |A|\cdot|Q|\cdot D_{\text{AVG}}(A,Q) + |B|\cdot|Q|\cdot D_{\text{AVG}}(B,Q) \quad (7.45)$$

Dividing by $|A\cup B|\cdot|Q| = (|A|+|B|)\cdot|Q|$ and simplifying $|Q|>0$:

$$D_{\text{AVG}}(A\cup B,\, Q) = \frac{|A|}{|A|+|B|}D_{\text{AVG}}(A,Q) + \frac{|B|}{|A|+|B|}D_{\text{AVG}}(B,Q) \quad (7.46)$$

matching the coefficients in Table 7.1. □

7.8.4 Synoptic Comparison of Linkage Criteria

Criterion	Objective	Geometry	Outliers	Properties
MIN (Single)	$\min_{\mathbf{x},\mathbf{y}} d(\mathbf{x},\mathbf{y})$	Arbitrary, chaining	Very high (*chaining*)	Isomorphic to MST
MAX (Complete)	$\max_{\mathbf{x},\mathbf{y}} d(\mathbf{x},\mathbf{y})$	Spherical, compact	Low at core	Diameter $\leq h$
Group Average	Pairwise $\frac{\sum\sum d}{\lvert A\rvert\lvert B\rvert}$	Globular tolerant	Moderate	Balanced
Ward's Method	ΔSSE_{AB}	Spherical balanced	Very low	K-Means analogue

Table 7.2: Analytical comparison across the four principal linkage criteria in hierarchical clustering.

7.9 MST-Based Divisive Hierarchical Clustering

While naive divisive clustering is computationally intractable (evaluating all $2^{n-1}-1$ possible binary splits), graph theory offers an elegant and polynomial formulation using the **Minimum Spanning Tree** (MST).

Definition 7.9.1 (Minimum Spanning Tree of Data Points)**.** Given complete graph $G=(V,E)$ where vertices V represent the n data points and edge weights are $w(u,v)=d(\mathbf{x}_u,\mathbf{x}_v)$, an MST is a cycle-free subgraph connecting all vertices with minimum total weight:

$$w(\text{MST}) = \sum_{e\in\text{MST}} w(e) \to \min \quad (7.47)$$

7.9.1 Divisive Algorithm via Edge Deletion

Algorithm 3: Divisive clustering based on Minimum Spanning Tree.

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, desired number of clusters K.
Output: Partition $\{C_1, \ldots, C_K\}$.

```
Construct complete weighted graph G = (V, E) from pairwise distances
Extract Minimum Spanning Tree T_MST using Prim's or Kruskal's algorithm
Sort the n − 1 tree edges in non-increasing order: w(e_1) ≥ w(e_2) ≥ ··· ≥ w(e_{n−1})
for k = 1 to K − 1 do
    Delete longest remaining edge e_k from the spanning tree
    The tree splits into two disjoint connected components
end
Return the resulting K connected components as the final clusters
```

Property 7.9.1 (Equivalence with Single Link Agglomerative Clustering)**.** *The partition produced by removing the $K-1$ heaviest edges from an* MST *is* ***strictly identical*** *to the partition obtained by running Single Link agglomerative clustering down to K clusters. Consequently, Prim's algorithm ($\mathcal{O}(n^2)$) provides an efficient direct path to construct the Single Link hierarchy.*

7.10 Complexity, Structural Limits, and Synoptic Comparison

7.10.1 Asymptotic Complexity Analysis

- **Space Complexity**: Hierarchical clustering requires storing the full $n \times n$ proximity matrix:

$$\mathcal{S}_{\text{Hierarchical}} = \mathcal{O}(n^2) \tag{7.48}$$

  This presents a formidable bottleneck in Big Data scenarios. For $n = 100\,000$ points with double precision (8 bytes), the matrix requires approximately 40 GB of RAM.

- **Naive Time Complexity**: With $n-1$ merge iterations and linear search across an $\mathcal{O}(n^2)$ matrix at each step:

$$\mathcal{T}_{\text{Naive}} = \sum_{t=1}^{n-1} \mathcal{O}((n-t)^2) = \mathcal{O}(n^3) \tag{7.49}$$

- **Priority Queue (Heap) Optimization**: Maintaining distances in min-heaps reduces merge search and updates to $\mathcal{O}(n \log n)$ per step, lowering overall complexity to:

$$\mathcal{T}_{\text{Heap}} = \mathcal{O}(n^2 \log n) \tag{7.50}$$

  (For specific criteria such as Single Link and Complete Link, specialized algorithms like SLINK and CLINK achieve $\mathcal{O}(n^2)$ time with $\mathcal{O}(n)$ auxiliary space).

7.10.2 Structural Limitations of Hierarchical Clustering

1. **Irreversibility of Greedy Decisions**: Merges and splits are permanent without backtracking. If two points merge prematurely due to local noise, they remain tied at all subsequent levels.

2. **Absence of a Global Objective Function**: Except for Ward's method (which minimizes local variance growth), linkage metrics do not optimize a global mathematical cost function across the entire dataset.

7.10.3 Final Synoptic Schema: K-Means vs Hierarchical Clustering

Table 7.3 contrasts the architectural, algebraic, and operational properties of the two paradigms.

Operational Feature	**K-Means Clustering**	**Hierarchical Clustering**
Structure Type	Flat partition (*hard clustering*)	Nested taxonomic tree (Dendrogram)
Parameter K	Mandatory a priori	Derivable post-hoc by drawing a cut line
Objective Function	Global SSE minimization	Greedy local decisions without global cost
Time Complexity	$\mathcal{O}(n \cdot K \cdot I \cdot d)$ (Linear in n)	$\mathcal{O}(n^2 \log n)$ to $\mathcal{O}(n^3)$ (Super-quadratic)
Space Complexity	$\mathcal{O}((n + K) \cdot d)$ (Linear in n)	$\mathcal{O}(n^2)$ (Quadratic memory bottleneck)
Data Scalability	Very high (millions of records)	Low ($n \leq 20\,000$ in standard RAM)
Initialization Sensitivity	Severe (susceptible to local minima)	Fully deterministic (no random seed)
Shape Flexibility	Spherical/globular only (Voronoi hyperplanes)	Arbitrary with Single Link; spherical with Ward
Decision Reversibility	Yes (points reassign at each iteration)	No (permanent merges/splits without backtracking)

Table 7.3: Global synoptic comparison between K-Means and Hierarchical Clustering.

Chapter 8

Cluster Analysis: Density-Based Clustering (DBSCAN)

In this chapter, we investigate the paradigm of **density-based clustering** and, in particular, the foundational **DBSCAN** algorithm (*Density-Based Spatial Clustering of Applications with Noise*), originally introduced by Ester, Kriegel, Sander, and Xu [1] and detailed in [2]. The presentation rigorously covers:

1. The **geometric and computational limitations** of prototype-based (K-Means) and hierarchical models, and the resulting paradigm shift toward spatial connectivity via local density;
2. The **analytical foundations of local density**: neighborhood radius ε (*Eps*), minimum cardinality threshold $MinPts$, and the disjoint classification of points (*Core*, *Border*, and *Noise*);
3. The **algebraic relations of density connectivity**: directly density-reachable, density-reachable via core point chains, and symmetric density-connected;
4. The **operational architecture of the DBSCAN algorithm**: the three procedural phases (initial point classification, core backbone connecting, border point absorption and noise filtering), detailed pseudocode, and computational complexity analysis with spatial index trees (k-d tree, R^*-tree);
5. The phenomenon of **non-determinism for shared border points** and independence from an a priori number of clusters K;
6. Key **strengths** (arbitrary non-convex shapes, heterogeneous cluster sizes, natural noise rejection) and **structural limitations** (varying densities within the same dataset, severe performance degradation under the curse of dimensionality);

7. The **heuristic parameter calibration methodology**: construction and geometric interpretation of the *k-distance plot* (k-NN distance graph) for identifying the critical inflection point (*knee point*);
8. A **comprehensive comparative framework**: K-Means, Hierarchical Clustering, and DBSCAN contrasted across topological assumptions, algebraic constraints, and operational properties.

8.1 Introduction to the Density-Based Paradigm

In previous chapters, two historical pillars of unsupervised clustering were examined:

- **Prototype-based partitional methods** (e.g., K-Means), where each cluster is represented by its centroid $\mathbf{m}_k$ and Euclidean space is partitioned into Voronoi hyper-polyhedral cells. This model implicitly assumes hyper-spherical, convex clusters with comparable variances, failing completely when real-world patterns exhibit elongated, concave, crescent-shaped, spiral, or concentric ring geometries. Furthermore, global minimization of the sum of squared errors (SSE) forces every single instance — including noise and isolated outliers — into a cluster, distorting centroid positions.

- **Hierarchical methods** (agglomerative or divisive), which build a dendrogram based on global pairwise distance matrices. Although single linkage can theoretically follow chains of arbitrary shapes, it suffers from severe chaining effects, where even a single noisy point bridging two distinct clusters collapses the entire hierarchy.

Density-based clustering resolves these limitations at their root by redefining the very notion of a cluster: no longer as proximity to a central prototype, but as a **contiguous, connected region of high local density**, separated from other clusters by **regions of low density or empty space** (which house background noise and outliers).

From this perspective, DBSCAN introduces a local aggregation mechanism capable of:

1. Discovering clusters with arbitrary spatial morphologies (concave, ring-shaped, filamentary);

2. Naturally isolating and discarding observations that do not belong to any dense aggregation (*noise filtering*);

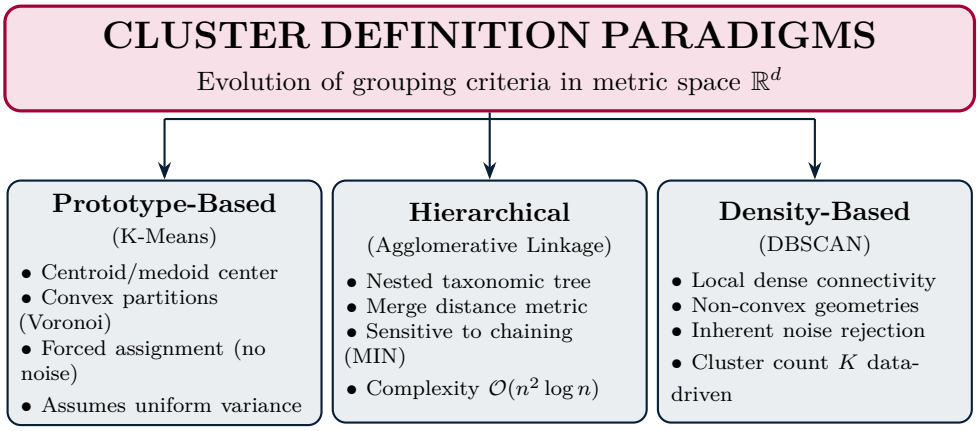


Figure 8.1: Conceptual positioning of density-based clustering relative to prototype-based and hierarchical paradigms.

3. Requiring no a priori specification of the expected number of clusters K, extracting it as an emergent property of the data topology.

8.2 Density Foundations and Point Taxonomy

Let $\mathcal{D} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n\} \subset \mathbb{R}^d$ be a multidimensional dataset equipped with a distance metric $\text{dist} : \mathcal{D} \times \mathcal{D} \to \mathbb{R}_{\geq 0}$ (typically Euclidean distance L_2, though any metric or semimetric is applicable).

DBSCAN parameterizes spatial density via two global hyperparameters:

- $\varepsilon \in \mathbb{R}^+$ (**Eps**): the radius threshold defining the extent of the local spherical neighborhood centered on each point;
- $MinPts \in \mathbb{N}^+$: the minimum positive integer number of points that must reside within an ε-neighborhood for that region to be classified as dense.

8.2.1 Neighborhood Definition and Local Density

Definition 8.2.1 (ε-Neighborhood / Eps-Neighborhood)**.** Given an observation $\mathbf{p} \in \mathcal{D}$ and a scalar radius $\varepsilon > 0$, the **ε-neighborhood** of $\mathbf{p}$, denoted by $N_\varepsilon(\mathbf{p})$ or $N_{\text{Eps}}(\mathbf{p})$, is the subset of observations in $\mathcal{D}$ whose distance from $\mathbf{p}$ does not exceed ε:

$$N_\varepsilon(\mathbf{p}) = \Big\{ \mathbf{q} \in \mathcal{D} \;\Big|\; \text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon \Big\} \tag{8.1}$$

Remark 8.2.1 (Reflexive Inclusion of the Point Itself)**.** Since any metric satisfies identity of indiscernibles $\text{dist}(\mathbf{p}, \mathbf{p}) = 0 \leq \varepsilon$, for any choice of $\varepsilon > 0$ we have:

$$\mathbf{p} \in N_\varepsilon(\mathbf{p}) \tag{8.2}$$

Therefore, when computing local density, the center point $\mathbf{p}$ is **always counted as part of its own neighborhood** (*"counts the point itself"*).

Definition 8.2.2 (Local Density)**.** The **local density** of a point $\mathbf{p} \in \mathcal{D}$ with respect to ε is defined discretely by the cardinality of its ε-neighborhood:

$$\text{Density}(\mathbf{p}) = |N_\varepsilon(\mathbf{p})| \tag{8.3}$$

In continuous Euclidean space $\mathbb{R}^d$, the corresponding volumetric density is $|N_\varepsilon(\mathbf{p})|/V_d(\varepsilon)$, where $V_d(\varepsilon) = \frac{\pi^{d/2}}{\Gamma(d/2+1)}\varepsilon^d$ is the volume of the hypersphere of radius ε. For fixed d and ε, $V_d(\varepsilon)$ is constant, and cardinality $|N_\varepsilon(\mathbf{p})|$ directly tracks local mass concentration.

8.2.2 Disjoint Taxonomic Classification of Points

Comparing local density Density($\mathbf{p}$) against the critical threshold $MinPts$, DBSCAN unambiguously partitions $\mathcal{D}$ into three mutually disjoint subsets:

Definition 8.2.3 (Core Point)**.** A point $\mathbf{p} \in \mathcal{D}$ is a **Core Point** ($\mathbf{p} \in \mathcal{C}$) if and only if its ε-neighborhood contains at least $MinPts$ points:

$$\mathbf{p} \in \mathcal{C} \iff |N_\varepsilon(\mathbf{p})| \geq MinPts \tag{8.4}$$

Core points reside firmly in the interior of dense regions and serve as structural cluster seeds.

Definition 8.2.4 (Border Point)**.** A point $\mathbf{p} \in \mathcal{D}$ is a **Border Point** ($\mathbf{p} \in \mathcal{B}$) if it fails the core point condition, but falls within the ε-neighborhood of at least one existing core point:

$$\mathbf{p} \in \mathcal{B} \iff \Big(|N_\varepsilon(\mathbf{p})| < MinPts\Big) \wedge \Big(\exists \mathbf{q} \in \mathcal{C} \text{ such that } \mathbf{p} \in N_\varepsilon(\mathbf{q})\Big) \tag{8.5}$$

Border points do not possess sufficient local density to generate a cluster on their own, but form the outer boundary perimeter of the dense component they border.

Definition 8.2.5 (Noise Point / Outlier)**.** A point $\mathbf{p} \in \mathcal{D}$ is a **Noise Point** ($\mathbf{p} \in \mathcal{N}$) if it is neither a core point nor a border point:

$$\mathbf{p} \in \mathcal{N} \iff (\mathbf{p} \notin \mathcal{C}) \wedge (\mathbf{p} \notin \mathcal{B}) \tag{8.6}$$

Equivalently in terms of distances:

$$\mathbf{p} \in \mathcal{N} \iff \Big(|N_\varepsilon(\mathbf{p})| < MinPts\Big) \wedge \Big(\forall \mathbf{q} \in \mathcal{C},\ \mathrm{dist}(\mathbf{p}, \mathbf{q}) > \varepsilon\Big) \tag{8.7}$$

Noise points represent isolated background samples or sampling artifacts located in the inter-cluster void.

Property 8.2.1 (Partition of the Point Set)**.** *For any fixed pair of parameters $(\varepsilon, MinPts)$, dataset $\mathcal{D}$ is partitioned into three pairwise disjoint classes:*

$$\mathcal{D} = \mathcal{C} \cup \mathcal{B} \cup \mathcal{N}, \qquad \text{with } \mathcal{C} \cap \mathcal{B} = \emptyset,\ \mathcal{C} \cap \mathcal{N} = \emptyset,\ \mathcal{B} \cap \mathcal{N} = \emptyset \tag{8.8}$$

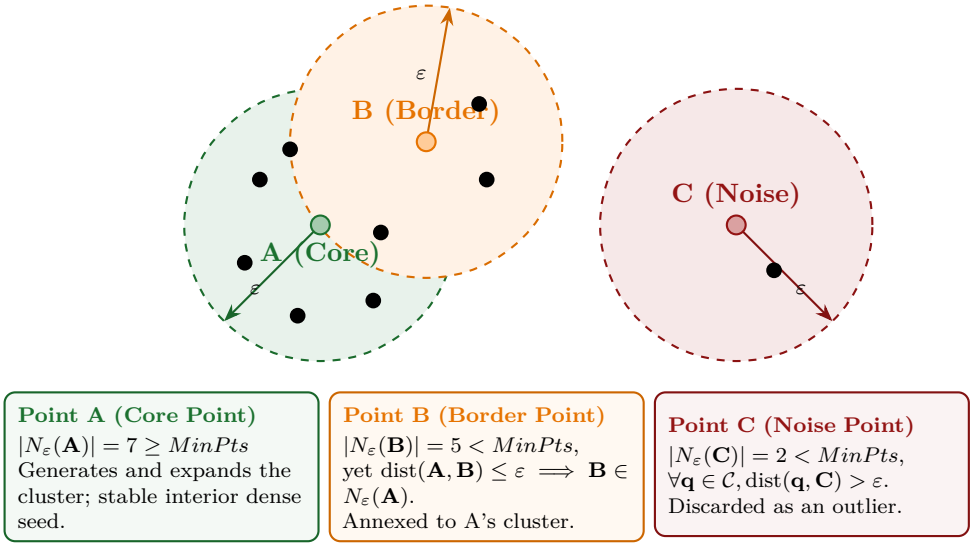


Figure 8.2: Geometric taxonomy of points in DBSCAN ($MinPts = 7$). Point A is a Core Point ($|N_\varepsilon(A)| \geq 7$), B is a Border Point ($|N_\varepsilon(B)| < 7$, but within radius ε of core A), and C is Noise (isolated, far from all cores).

8.2.3 Geometric Analysis: 2D Example with $MinPts = 7$

Figure 8.2 illustrates this classification in a two-dimensional plane with threshold $MinPts = 7$.

The geometric properties shown are unambiguous:

1. **Point A (Core Point)**: Drawing a radius-ε circle around A, the point count including A itself equals 7. Because $|N_\varepsilon(A)| \geq 7$, A is verified as a core point.

2. **Point B (Border Point)**: Centering the same radius-ε circle at B yields only 5 points ($|N_\varepsilon(B)| = 5 < 7$). However, the Euclidean distance between A and B satisfies $\text{dist}(A, B) \leq \varepsilon$, so $B \in N_\varepsilon(A)$. Because A is a core point, B is classified as a border point associated with A.

3. **Point C (Noise Point)**: Centered on C, the neighborhood captures only 2 observations ($|N_\varepsilon(C)| = 2 \ll 7$). Furthermore, distance from C to every core point exceeds ε. Hence, C is rejected as background noise.

8.3 Density-Connectivity Relations

To aggregate dense points into clusters without geometric failures, DBSCAN establishes three algebraic connectivity relations:

8.3.1 Directly Density-Reachable

Definition 8.3.1 (Directly Density-Reachable)**.** A point $\mathbf{q} \in \mathcal{D}$ is **directly density-reachable** from a point $\mathbf{p} \in \mathcal{D}$ with respect to ε and $MinPts$ (notation $\mathbf{p} \to_{\text{dir}} \mathbf{q}$) if:

1. $\mathbf{p}$ is a Core Point: $|N_\varepsilon(\mathbf{p})| \geq MinPts$;
2. $\mathbf{q}$ lies in $\mathbf{p}$'s neighborhood: $\mathbf{q} \in N_\varepsilon(\mathbf{p})$ ($\text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon$).

Property 8.3.1 (Asymmetry of Direct Reachability)**.** *Direct density-reachability is generally* ***asymmetric****:*

$$\mathbf{p} \to_{\text{dir}} \mathbf{q} \implies \mathbf{q} \to_{\text{dir}} \mathbf{p} \tag{8.9}$$

If $\mathbf{p} \in \mathcal{C}$ and $\mathbf{q} \in \mathcal{B}$, then $\mathbf{q}$ is directly reachable from $\mathbf{p}$. However, since $\mathbf{q}$ is not a core point ($|N_\varepsilon(\mathbf{q})| < MinPts$), no point can be directly reachable from $\mathbf{q}$. The relation is ***symmetric*** *if and only if both points are core points:*

$$\Big(\mathbf{p} \in \mathcal{C} \ \wedge \ \mathbf{q} \in \mathcal{C} \ \wedge \ \text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon\Big) \iff \Big(\mathbf{p} \to_{\text{dir}} \mathbf{q} \ \wedge \ \mathbf{q} \to_{\text{dir}} \mathbf{p}\Big) \tag{8.10}$$

8.3.2 Density-Reachable

Definition 8.3.2 (Density-Reachable)**.** A point $\mathbf{q} \in \mathcal{D}$ is **density-reachable** from a point $\mathbf{p} \in \mathcal{D}$ with respect to ε and $MinPts$ (notation $\mathbf{p} \rightsquigarrow \mathbf{q}$) if there exists a finite chain of points in $\mathcal{D}$:

$$\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_m \tag{8.11}$$

such that:

1. $\mathbf{p}_1 = \mathbf{p}$;
2. $\mathbf{p}_m = \mathbf{q}$;
3. For each $i \in \{1, 2, \ldots, m-1\}$, point $\mathbf{p}_{i+1}$ is directly density-reachable from $\mathbf{p}_i$ ($\mathbf{p}_i \to_{\text{dir}} \mathbf{p}_{i+1}$).

Remark 8.3.1 (Role of Intermediate Chain Points)**.** All intermediate points $\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_{m-1}$ along the chain must be **Core Points**. Only the terminal node $\mathbf{p}_m = \mathbf{q}$ may be a Border Point. Consequently, density-reachability $\mathbf{p} \rightsquigarrow \mathbf{q}$ is non-symmetric when the end node is a border point: one can reach the border from a core, but reachability halts at a border point.

8.3.3 Density-Connected

Because density-reachability is asymmetric, it cannot directly serve as an equivalence relation for defining symmetric clusters. DBSCAN resolves this by introducing density-connectivity:

Definition 8.3.3 (Density-Connected)**.** Two points $\mathbf{p}, \mathbf{q} \in \mathcal{D}$ are **density-connected** with respect to ε and $MinPts$ (notation $\mathbf{p} \sim_{\text{dens}} \mathbf{q}$) if there exists an intermediate seed point $\mathbf{o} \in \mathcal{D}$ (necessarily a Core Point) from which both $\mathbf{p}$ and $\mathbf{q}$ are density-reachable:

$$\mathbf{p} \sim_{\text{dens}} \mathbf{q} \iff \exists \mathbf{o} \in \mathcal{D} \quad \text{such that} \quad (\mathbf{o} \rightsquigarrow \mathbf{p}) \wedge (\mathbf{o} \rightsquigarrow \mathbf{q}) \tag{8.12}$$

Property 8.3.2 (Properties of Density-Connectivity)**.** *The density-connectivity relation* $\sim_{\text{dens}}$ *satisfies:*

1. ***Reflexivity****: For all points* $\mathbf{p}$ *belonging to a dense cluster,* $\mathbf{p} \sim_{\text{dens}} \mathbf{p}$*;*
2. ***Symmetry****:* $\mathbf{p} \sim_{\text{dens}} \mathbf{q} \iff \mathbf{q} \sim_{\text{dens}} \mathbf{p}$*;*
3. ***Transitivity on Core Points****: If* $\mathbf{p}_1 \sim_{\text{dens}} \mathbf{p}_2$ *and* $\mathbf{p}_2 \sim_{\text{dens}} \mathbf{p}_3$ *with* $\mathbf{p}_2 \in \mathcal{C}$*, then* $\mathbf{p}_1 \sim_{\text{dens}} \mathbf{p}_3$*.*

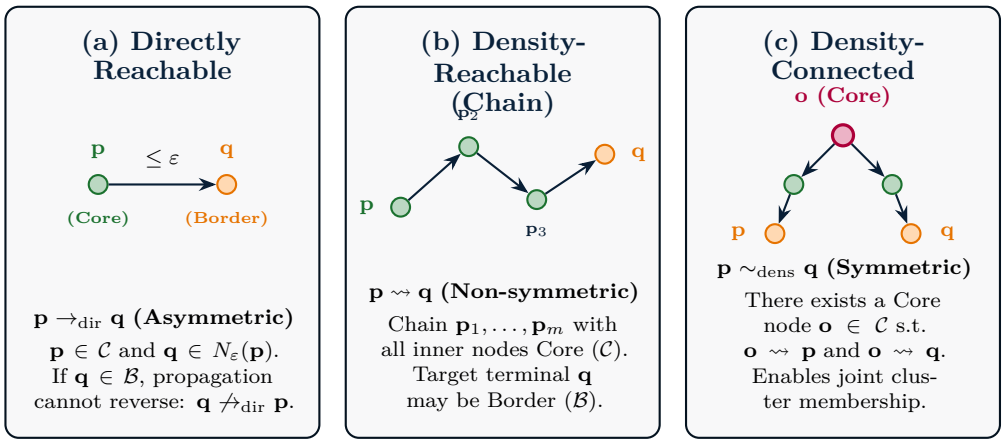


Figure 8.3: Algebraic formalization of the three density spatial relations in DBSCAN.

8.3.4 Formal Definition of a Density-Based Cluster

Using these relations, DBSCAN defines a cluster with axiomatic precision [1]:

Definition 8.3.4 (Density-Based Cluster)**.** Given dataset $\mathcal{D}$, a **cluster** $\mathcal{C}_k \subseteq \mathcal{D}$ with respect to ε and $MinPts$ is a non-empty subset satisfying two conditions:

1. **Maximality**: If $\mathbf{p} \in \mathcal{C}_k$ and $\mathbf{q} \in \mathcal{D}$ is density-reachable from $\mathbf{p}$ ($\mathbf{p} \rightsquigarrow \mathbf{q}$), then $\mathbf{q} \in \mathcal{C}_k$;

2. **Connectivity**: For every pair of points $\mathbf{p}, \mathbf{q} \in \mathcal{C}_k$, $\mathbf{p}$ and $\mathbf{q}$ are density-connected ($\mathbf{p} \sim_{\text{dens}} \mathbf{q}$).

Theorem 8.3.1 (Cluster Characterization by Core Points)**.** *Let $\mathcal{C}_k$ be a cluster in $\mathcal{D}$ and let $\mathbf{p} \in \mathcal{C}_k$ be any of its Core Points. Then $\mathcal{C}_k$ is uniquely generated as the set of all points density-reachable from $\mathbf{p}$:*

$$\mathcal{C}_k = \left\{ \mathbf{q} \in \mathcal{D} \;\middle|\; \mathbf{p} \rightsquigarrow \mathbf{q} \right\} \tag{8.13}$$

Theorem 8.3.1 underpins the operational logic of the algorithm: discovering an entire cluster requires only picking an arbitrary unvisited core point $\mathbf{p}$ and taking the transitive closure of all points density-reachable from it.

8.4 Operational Architecture of DBSCAN

The theoretical density formulation maps into an efficient procedural workflow structured in three successive phases.

8.4.1 The Three Operational Phases

1. **Phase 1: Initial Point Classification (*Point Labeling*)**

   - For every observation $\mathbf{p} \in \mathcal{D}$, execute a neighborhood range query $N_\varepsilon(\mathbf{p}) = \{\mathbf{q} \in \mathcal{D} \mid \text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon\}$.
   - If $|N_\varepsilon(\mathbf{p})| \geq MinPts$, assign the label `CORE`;
   - Otherwise, assign provisional state `UNVISITED` or `NOISE` (subject to being recognized later as a border point).

2. **Phase 2: Connecting Core Objects (*Core Point Graph Clustering*)**

   - Consider the undirected subgraph $G_{\text{core}} = (\mathcal{C}, E_{\text{core}})$ induced on core points $\mathcal{C}$, where an edge $(\mathbf{u}, \mathbf{v}) \in E_{\text{core}}$ exists if and only if $\text{dist}(\mathbf{u}, \mathbf{v}) \leq \varepsilon$.
   - The maximal connected components of G_{core} determine the distinct backbones of the clusters.

3. **Phase 3: Border Point Assignment and Noise Filtering**

   - For every non-core point $\mathbf{b} \in \mathcal{B}$ within the ε-neighborhood of at least one core point $\mathbf{c} \in \mathcal{C}$, bind $\mathbf{b}$ to the cluster containing $\mathbf{c}$.
   - All unassigned points remaining outside any core neighborhood are permanently designated as `NOISE` (marked with symbol $\times$ and discarded).

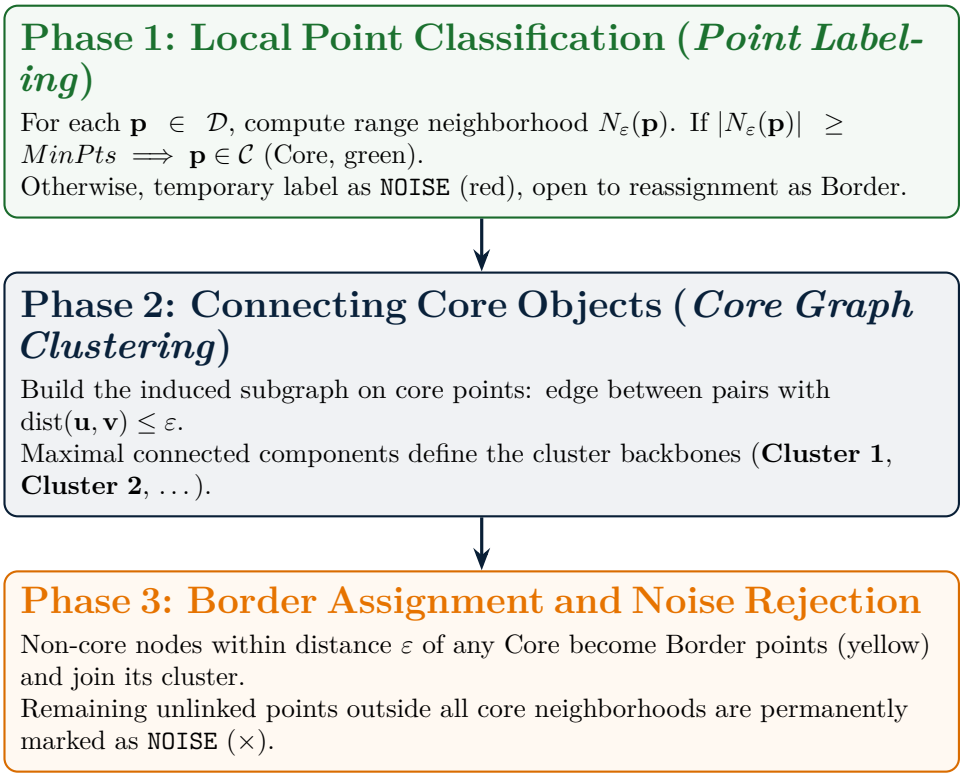


Figure 8.4: The three computational phases in DBSCAN cluster formation.

8.4.2 Non-Determinism in Shared Border Points

While core point partitioning and noise exclusion are strictly deterministic, assigning **contested border points** exhibits order dependence:

Property 8.4.1 (Border Point Assignment Non-Determinism)**.** *Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be two distinct clusters generated by disjoint components of core points. If a border point $\mathbf{b} \in \mathcal{B}$ simultaneously satisfies:*

$$\begin{aligned} &\exists \mathbf{c}_1 \in \mathcal{C}_1 \cap \mathcal{C} \quad s.t. \quad \operatorname{dist}(\mathbf{b}, \mathbf{c}_1) \leq \varepsilon, \\ &\exists \mathbf{c}_2 \in \mathcal{C}_2 \cap \mathcal{C} \quad s.t. \quad \operatorname{dist}(\mathbf{b}, \mathbf{c}_2) \leq \varepsilon, \end{aligned} \tag{8.14}$$

then $\mathbf{b}$ *is density-reachable from both clusters. Crucially, because* $\mathbf{b}$ *is not a core point (*$|N_\varepsilon(\mathbf{b})| < MinPts$*), it* ***cannot act as a propagation bridge*** *between* $\mathbf{c}_1$ *and* $\mathbf{c}_2$*; the two clusters do not merge.*

Consequently, whether $\mathbf{b}$ *is assigned to Cluster 1 or Cluster 2 depends solely on the* ***data processing order****: the cluster whose core point scans* $\mathbf{b}$ *first claims it.*

In practice, the percentage of shared border points in real-world data is negligible, leaving macro-clustering structures intact.

8.4.3 DBSCAN Pseudocode

Algorithm 4 presents the unified traversal algorithm using an expansion queue/frontier (BFS/DFS).

8.4.4 Computational Complexity Analysis

Execution speed is dominated by neighborhood range queries $\textsc{RangeQuery}(\mathcal{D}, \mathbf{p}, \varepsilon)$ performed across the dataset.

- **Naive Linear Scan**: Without spatial index structures, computing $N_\varepsilon(\mathbf{p})$ requires calculating distances between $\mathbf{p}$ and all $n-1$ other points in $\mathcal{O}(n \cdot d)$ time. Applied across all n points, total time complexity is quadratic:
$$\mathcal{T}_{\text{naive}}(n) = \mathcal{O}(n^2 \cdot d) \tag{8.15}$$
- **Spatial Index Trees (k-d Tree, R^*-Tree)**: In low-to-moderate dimensions ($d \leq 10$), indexing data in spatial trees reduces average query lookup to $\mathcal{O}(\log n)$, yielding an overall time complexity of:
$$\mathcal{T}_{\text{tree}}(n) = \mathcal{O}(n \log n) \tag{8.16}$$
- **High-Dimensional Curse**: When dimensionality grows ($d \gg 10$), spatial tree partitioning degenerates; trees must inspect nearly all nodes, reverting query time back to $\mathcal{O}(n)$ and total runtime to $\mathcal{O}(n^2)$.
- **Space Complexity**: Memory requirements encompass storing dataset $\mathcal{D}$, label array y, and the active frontier queue $\mathcal{S}$, which is strictly linear:
$$\mathcal{S}(n) = \mathcal{O}(n \cdot d) \tag{8.17}$$

Algorithm 4: Standard DBSCAN algorithm with density queue expansion.

Input: Dataset $\mathcal{D} = \{\mathbf{p}_1, \ldots, \mathbf{p}_n\}$, metric radius $\varepsilon > 0$, neighborhood threshold $MinPts \in \mathbb{N}^+$.

Output: Label array $\mathbf{y} \in (\mathbb{N} \cup \{\texttt{NOISE}\})^n$.

Initialize all labels: $y[\mathbf{p}] \leftarrow \texttt{UNVISITED}$ for all $\mathbf{p} \in \mathcal{D}$
cluster_id $\leftarrow 0$
foreach $\mathbf{p} \in \mathcal{D}$ **do**
    if $y[\mathbf{p}] \neq$ *UNVISITED* **then**
        continue
        `// Already processed in an earlier cluster expansion`
    end
    $\mathcal{N}_\mathbf{p} \leftarrow \textsc{RangeQuery}(\mathcal{D}, \mathbf{p}, \varepsilon)$
    if $|\mathcal{N}_\mathbf{p}| < MinPts$ **then**
        $y[\mathbf{p}] \leftarrow \texttt{NOISE}$
        `// Provisional label: may still become a Border point`
    end
    else
        cluster_id $\leftarrow$ cluster_id $+ 1$
        $y[\mathbf{p}] \leftarrow$ cluster_id
        $\mathcal{S} \leftarrow \mathcal{N}_\mathbf{p} \setminus \{\mathbf{p}\}$
        `// Expansion set (queue or seed list)`
        while $\mathcal{S} \neq \emptyset$ **do**
            Extract point $\mathbf{q}$ from $\mathcal{S}$
            if $y[\mathbf{q}] ==$ *NOISE* **then**
                $y[\mathbf{q}] \leftarrow$ cluster_id
                `// Relabel Noise point to Border point`
            end
            if $y[\mathbf{q}] \neq$ *UNVISITED* **then**
                continue
            end
            $y[\mathbf{q}] \leftarrow$ cluster_id
            $\mathcal{N}_\mathbf{q} \leftarrow \textsc{RangeQuery}(\mathcal{D}, \mathbf{q}, \varepsilon)$
            if $|\mathcal{N}_\mathbf{q}| \geq MinPts$ **then**
                $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{N}_\mathbf{q}$
                `// Propagate expansion: q is also a Core Point`
            end
        end
    end
end

8.5 Spatial Data Case Study

Consider the continuous 2D distribution discussed in [2], where two high-density bodies are connected by a slender neck (*bridge of points*) and surrounded by sparse, scattered points.

8.5.1 Behavior with $\varepsilon = 10$ and $MinPts = 4$

Setting $\varepsilon = 10$ and $MinPts = 4$:

1. **Interior Backbone**: Inside the dense masses, average pairwise distances are far below 10. Inner points capture 8 to 25 neighbors within radius 10, easily exceeding $MinPts = 4$. They are classified as Core Points (dark blue) and form the cluster backbone.

2. **Boundary Definition**: Moving outward, local point counts decrease. Along cluster margins, points have fewer than 4 neighbors within radius 10, but sit within 10 units of an inner core point. They become Border Points (red), crisply delineating the cluster boundary.

3. **Handling the Narrow Bridge**: If points along the connecting neck have mutual distances ≤ 10 and each contains at least 4 neighbors, they qualify as Core Points and chain the two bodies into one cluster. If neck density drops below 4 points per radius 10, the bridge breaks and DBSCAN preserves two separate clusters.

4. **Outlier Isolation**: Sparse points scattered in empty space sit > 10 units away from any core point with ≤ 2 local neighbors. They are tagged as Noise (light gray crosses), preventing distortion of cluster boundaries.

8.6 Strengths and Geometric Capabilities

DBSCAN provides fundamental capabilities that resolve core vulnerabilities of prototype-based clustering:

1. **Discovery of Arbitrarily Shaped Clusters**: K-Means imposes convex Voronoi partitions. DBSCAN tracks topological density continuity, reliably segmenting non-convex geometries such as interlocking crescents and rings.

1. Inherent Noise Resistance

Outliers and background points are not forced into clusters: they are formally isolated as `NOISE` without perturbing true cluster geometry or centroids.

2. Arbitrary Non-Convex Morphologies

Successfully identifies concentric rings, nested crescents, spirals, and elongated filaments where K-Means fractures data.

3. Heterogeneous Cluster Sizes

No equipartition bias (SSE). As long as local density exceeds the threshold, tiny and giant clusters comfortably coexist.

4. No Prior K Specification

The final cluster count is an emergent topological property; users need not guess K or test random restarts.

Figure 8.5: Four key architectural strengths of DBSCAN over prototype-based models.

2. **Cluster Size Invariance**: K-Means minimizes global SSE, showing a strong bias toward equally sized clusters. DBSCAN evaluates purely local intensive density $MinPts/V_d(\varepsilon)$, enabling small and large clusters to coexist.

3. **Robust Noise Filtering**: Unlike algorithms requiring hard complete partitions ($\bigcup_k C_k = \mathcal{D}$), DBSCAN explicitly accommodates an unassigned noise set ($\mathcal{N} \neq \emptyset$), keeping corrupted data from distorting valid structures.

4. **No Preset Number of Clusters**: The user is not required to specify K upfront; the cluster count emerges directly from spatial density connections.

8.7 Critical Limitations and Application Challenges

Despite its strengths, DBSCAN faces two major operational hurdles:

8.7.1 1. The Varying Densities Dilemma

Because ε and $MinPts$ are **global constants**, DBSCAN struggles when natural clusters within the same dataset possess starkly different densities:

In the benchmark experiment presented in [2]:

Attempt A: Excessively Large ε

- Accurately captures sparse (low-density) clusters.
- **Failure**: Denser neighboring clusters coalesce and merge into one giant amorphous mass.

Attempt B: Excessively Small ε

- Correctly separates compact high-density clusters.
- **Failure**: Valid but sparser clusters fall below the threshold and are discarded as `NOISE`.

Figure 8.6: The global threshold dilemma when handling heterogeneous cluster densities.

- Setting $MinPts = 4, \varepsilon = 9.92$: DBSCAN detects the sparse ellipse on the left, but merges the entire right-hand section (diffuse cloud plus dense sub-clusters) into one single cluster.
- Lowering the radius slightly to $\varepsilon = 9.75$ ($MinPts = 4$): DBSCAN isolates the three hyper-dense micro-clusters on the right, but completely discards the left ellipse and central cluster as noise (`NOISE`).

This limitation inspired density hierarchy algorithms:

- **OPTICS** (*Ordering Points To Identify the Clustering Structure*) [3]: produces an augmented ordering based on reachability distance, visualized via a reachability plot;
- **HDBSCAN** (*Hierarchical DBSCAN*) [4]: integrates local density with hierarchical extraction, automating cluster selection across varying density scales by optimizing persistence (*excess of mass*).

8.7.2 2. The Curse of Dimensionality

When the feature dimensionality d is high ($d \gg 10$), DBSCAN suffers significantly. As extensively discussed in Chapter 4 (subsection 4.3.1), hypervolumetric dilution and relative distance concentration in high-dimensional spaces cause any metric hypersphere of radius ε to be either entirely empty or, with a minute increase in ε, swallow nearly the entire dataset. Consequently, in high dimensionality it becomes virtually impossible to identify a metric threshold ε that effectively separates dense regions from sparse ones, causing the concept of local Euclidean density to lose operational meaning.

8.8 Heuristic Parameter Selection: The k-Distance Plot

Choosing parameters $(\varepsilon, MinPts)$ blindly typically results in either total cluster collapse or near-complete data rejection. The standard, data-driven approach is the **k-nearest neighbor distance plot** (*k-distance plot*) [1, 2].

8.8.1 Core Heuristic Principle

The heuristic exploits the structural gap between cluster points and noise:

- **Points in dense clusters**: their distance to their k-th nearest neighbor is small and relatively consistent.
- **Noise and outlier points**: their distance to their k-th nearest neighbor is substantially larger.

8.8.2 Step-by-Step Construction

1. **Fixing Parameter k ($MinPts$)**: Set $k = MinPts$. As a rule of thumb for dimension d:
$$MinPts \geq d + 1 \tag{8.18}$$
To smooth sampling fluctuations, $MinPts = 4$ is common for 2D data ($MinPts \approx 2d$ in general).

2. **Computing k-NN Distances**: For each point $\mathbf{p}_i \in \mathcal{D}$ $(i = 1, \ldots, n)$:
   - Compute distances to all other $n - 1$ points;
   - Sort distances in ascending order:
   $$d_{(1)}(\mathbf{p}_i) \leq d_{(2)}(\mathbf{p}_i) \leq \cdots \leq d_{(k)}(\mathbf{p}_i) \leq \cdots \leq d_{(n-1)}(\mathbf{p}_i) \tag{8.19}$$
   - Extract $D_i = d_{(k)}(\mathbf{p}_i)$, the distance to the k-th nearest neighbor.

3. **Global Sorting and Curve Plotting**: Sort the array $\{D_1, \ldots, D_n\}$ in ascending order:
$$D_{\pi(1)} \leq D_{\pi(2)} \leq \cdots \leq D_{\pi(n)} \tag{8.20}$$
Plot points with index m on the X-axis and $D_{\pi(m)}$ on the Y-axis.

4. **Locating the "Knee Point"**: The curve displays three distinct segments (Figure 8.7):

- **Initial Flat Plateau** ($D \leq 8$): points inside dense clusters;
- **The Knee Point**: point of maximum curvature marking the phase transition from intra-cluster density to noise;
- **Steep Tail** ($D \geq 15$): isolated noise points whose k-NN distance escalates sharply.

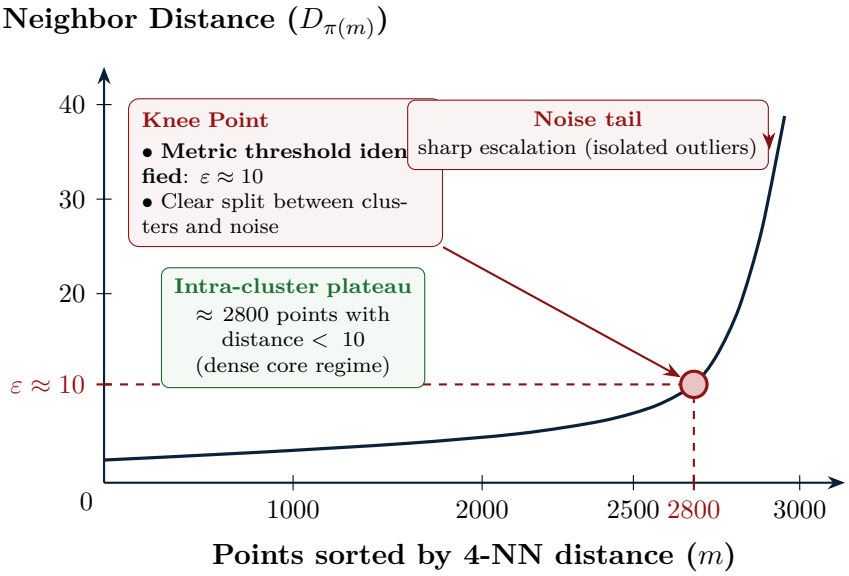


Figure 8.7: The k-distance plot ($k = 4$) for approximately 3000 points. The optimal radius threshold ε is identified at the knee point along the Y-axis, here located at $\varepsilon \approx 10$.

8.8.3 Interpretation and Selection of $\varepsilon \approx 10$

In Figure 8.7:

- Roughly 2800 out of 3000 points have their 4-th nearest neighbor within a narrow range between 2 and 8.
- The sharp inflection (knee) sits precisely at Y-value $\varepsilon \approx 10$.
- Choosing $\varepsilon = 10$ ensures that points prior to the knee have at least 4 neighbors within distance 10 (becoming Core points), while points in the steep tail have their 4th neighbor beyond 10 and are classified as `NOISE`.

This analytical calibration matches the parameters used in the Section 8.5 case study ($\varepsilon = 10, MinPts = 4$).

8.9 Global Comparative Synthesis: K-Means vs Hierarchical vs DBSCAN

Table 8.1 synthesizes the topological, computational, and practical tradeoffs among all three primary clustering paradigms studied in Chapters 7 and 8.

Operational Feature	**K-Means Clustering**	**Hierarchical Clustering**	**DBSCAN (Density-Based)**
Core Model	Partitional centroid prototype	Agglomerative/Divisive nested tree	Local high-density connectivity
Number of Clusters K	Mandatory a priori	Derived post-hoc by tree cutting	Emerges dynamically from data
Cluster Geometry	Exclusively convex/hyper-spherical (Voronoi)	Arbitrary with Single Link; compact/spherical with Ward	Arbitrary (concave, rings, spirals, filaments)
Noise Sensitivity	High (outliers drag centroid locations)	Linkage-dependent (Single Link prone to chaining)	Inherent noise immunity (formally labeled `NOISE`)
Cluster Sizes/Densities	Assumes similar size, volume, and variance	Flexible, but greedy merges are irreversible	Handles diverse sizes; fails under heterogeneous densities
Time Complexity	$\mathcal{O}(n \cdot K \cdot I \cdot d)$ (Linear in n)	$\mathcal{O}(n^2 \log n)$ to $\mathcal{O}(n^3)$ (Super-quadratic)	$\mathcal{O}(n \log n)$ with spatial index; $\mathcal{O}(n^2 \cdot d)$ naive
Space Complexity	$\mathcal{O}((n + K) \cdot d)$ (Linear)	$\mathcal{O}(n^2)$ (Pairwise distance matrix)	$\mathcal{O}(n \cdot d)$ (Linear)
Data Scalability	Excellent (tens of millions of records)	Very low ($n \leq 20\,000$ bound by RAM)	Good to moderate (efficient up to $n \leq 10^6$ in low dimensions)
High-Dimensional Impact	Moderate degradation under Euclidean metric	Global pairwise distortion	Severe degradation (loss of density contrast)
Determinism	Non-deterministic (restart dependent)	Strictly deterministic	Deterministic for Core/Noise; order-dependent on shared Border

Table 8.1: Comprehensive comparative synthesis across Partitional (K-Means), Hierarchical, and Density-Based (DBSCAN) clustering paradigms.

Chapter 9

K-Means Variants, BIC Criterion, Mixture Models, and EM

In this chapter, we explore the theoretical and algorithmic evolution that overcomes the intrinsic limitations of basic standard K-Means (Lloyd's algorithm), introducing advanced divisive partitioning techniques, formal information-theoretic criteria for automatic model complexity selection, and probabilistic generative models founded on the *Expectation-Maximization* (EM) paradigm. The presentation rigorously covers:

1. The **structural and geometric limitations of standard K-Means**: extreme sensitivity to initial centroid seeding, the rigidity of deterministic assignment (*hard clustering*), the assumption of hyperspherical clusters with isotropic and homogeneous variances, and the requirement to specify the hyperparameter K a priori;
2. The **Bisecting K-Means** algorithm: the transition to the divisive hierarchical (*top-down*) paradigm, decomposing the global partitioning problem into a sequence of local two-way bisections ($K = 2$), multi-trial execution with minimization of intra-cluster sum of squared errors (SSE), split selection heuristics, boundary/singleton cases, and post-processing global Lloyd refinement;
3. The **X-Means** algorithm (Pelleg and Moore [5]): modeling clustering as a statistical model selection problem over the interval $[K_{\min}, K_{\max}]$, local bisection testing guided by principal axis perturbations, local 2-means optimization, and information gain validation via ΔBIC;
4. The **Bayesian Information Criterion (BIC)** (Schwarz [6]): asymptotic derivation from the marginal likelihood via Laplace's approxi-

mation, the analytical trade-off between log-likelihood and complexity penalty (Occam's razor), and precise counting of free parameters under spherical, diagonal, and full covariance matrices;
5. The **Gaussian Mixture Model (GMM)** framework: extending clustering to the probabilistic domain (*soft clustering*), generative formulation using latent categorical variables, incomplete vs. complete data log-likelihood, and Maximum Likelihood Estimation (MLE) vs. Maximum A Posteriori (MAP);
6. The foundational **Expectation-Maximization (EM)** algorithm (Dempster, Laird, and Rubin [7]): Jensen's inequality, the variational evidence lower bound, the auxiliary $\mathcal{Q}$-function, and the cyclic alternation between the E-step (analytical posterior responsibilities γ_{ik}) and the M-step (closed-form weighted updates for mixing proportions π_k, mean vectors $\boldsymbol{\mu}_k$, and covariance matrices $\boldsymbol{\Sigma}_k$);
7. A **concrete analytical case study on a two-Gaussian mixture**: deconvolution of overlapping demographic populations (e.g., human heights), comparison between theoretical continuous PDF and empirical 20,000-sample histogram, critical saddle point analysis ($x = 0$), and soft uncertainty assignment;
8. A **comprehensive architectural comparison between K-Means and EM**: geometric and energetic taxonomy, operational strengths, computational vulnerabilities, covariance singularities, and high-dimensional regularization.

9.1 Scientific Context and Limitations of Standard K-Means

Standard K-Means (Lloyd's algorithm), formalized in Chapter 7 (section 7.2), operates by locating K centroids $\mathbf{C} = \{\mathbf{c}_1, \ldots, \mathbf{c}_K\}$ that minimize the global intra-cluster Sum of Squared Errors SSE (Equation 7.6). While computationally linear per iteration ($\mathcal{O}(N \cdot K \cdot d)$), the structural analysis in section 7.6 highlighted three primary vulnerabilities that constrain its utility in practical applications:

1. **Extreme sensitivity to initialization**: Continuous SSE minimization is non-convex and $\mathcal{NP}$-hard. Lloyd's coordinate descent guarantees convergence only to a local minimum: seeds placed sub-optimally or within the same natural dense region trap the algorithm in fragmented, high-error states;

2. **Requirement of predetermining parameter K**: Standard K-Means lacks internal model selection. Because SSE decreases monotonically with K, it cannot independently guide the selection of the natural cluster count without complexity penalties;

3. **Geometric rigidity and deterministic assignment (*hard clustering*)**: K-Means forces every instance into a single, mutually exclusive cluster, carving out linear Voronoi cells. This formulation implicitly assumes clusters are hyperspherical, convex, equally sized, and endowed with identical isotropic variances.

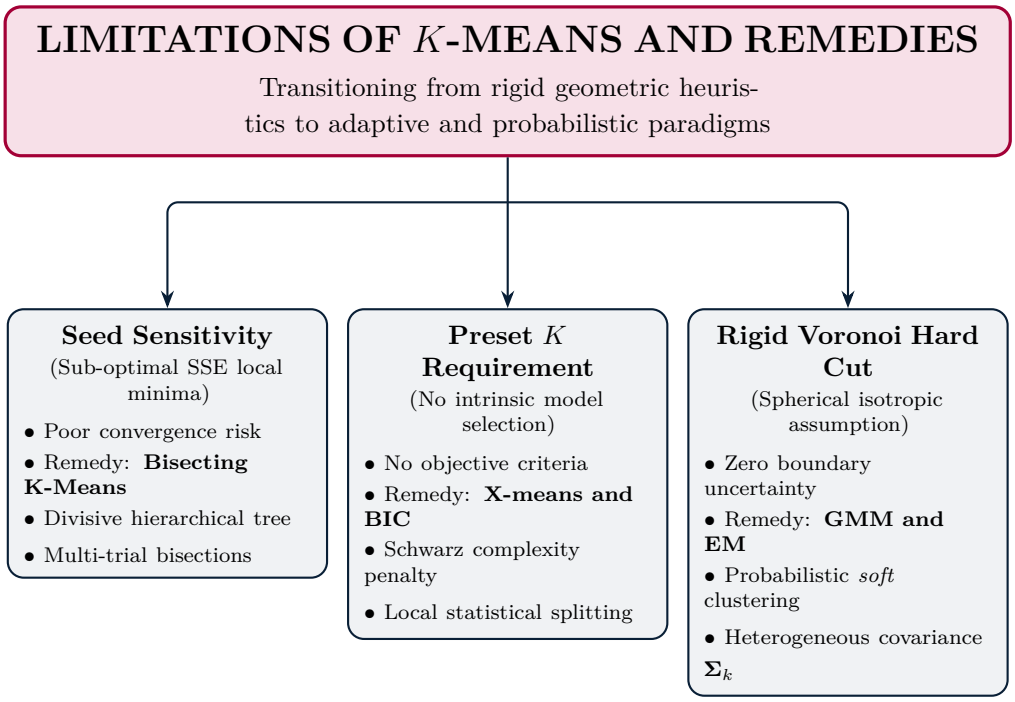


Figure 9.1: Conceptual overview of the foundational limitations of standard K-Means and the three complementary algorithmic strategies designed to address them.

To overcome these structural hurdles, data mining research has developed three complementary methodologies illustrated in Figure 9.1:

- **Bisecting K-Means**: a top-down divisive hierarchical framework that drastically mitigates initialization fragility by converting global clustering into a sequence of tractable binary sub-problems ($K = 2$);

- **X-Means and the BIC Criterion**: incorporating formal information-theoretic regularization to autonomously discover the intrinsic number of clusters;
- **Gaussian Mixture Models and Expectation-Maximization**: replacing rigid geometric tessellations with a generative probabilistic model featuring continuous degree-of-belonging values (*posterior responsibilities*).

9.2 The Bisecting K-Means Algorithm

The **Bisecting K-Means** algorithm (formalized as a divisive variant in [2]) addresses the initialization problem by replacing simultaneous global seeding with a top-down hierarchical procedure founded on the *divide-and-conquer* principle. Instead of positioning K centroids simultaneously in high-dimensional space, the algorithm decomposes the challenge into $K-1$ sequential, independent two-way bisections ($K=2$).

9.2.1 Mathematical Formulation of Error and Partition Strategy

Given a subset of instances $\mathcal{C} \subseteq \mathcal{D}$ with cardinality $|\mathcal{C}| = n_{\mathcal{C}}$, the intra-cluster error $\text{SSE}(\mathcal{C})$ is computed relative to its centroid $\mathbf{c} = \frac{1}{n_{\mathcal{C}}} \sum_{\mathbf{x} \in \mathcal{C}} \mathbf{x}$:

$$\text{SSE}(\mathcal{C}) = \sum_{\mathbf{x} \in \mathcal{C}} \|\mathbf{x} - \mathbf{c}\|_2^2 \tag{9.1}$$

If the current partition of the space is represented by an active list of clusters $\mathcal{L} = \{\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_m\}$, the total sum of squared errors across the configuration is:

$$\text{SSE}_{\text{total}}(\mathcal{L}) = \sum_{j=1}^{m} \text{SSE}(\mathcal{C}_j) \tag{9.2}$$

When a candidate cluster $\mathcal{C}^* \in \mathcal{L}$ is selected, extracted, and bisected into two disjoint child clusters $\mathcal{A}$ and $\mathcal{B}$ (such that $\mathcal{A} \cup \mathcal{B} = \mathcal{C}^*$ and $\mathcal{A} \cap \mathcal{B} = \emptyset$), the updated global sum of squared errors becomes:

$$\text{SSE}_{\text{total}}(\mathcal{L}') = \text{SSE}_{\text{total}}(\mathcal{L}) - \text{SSE}(\mathcal{C}^*) + \Big(\text{SSE}(\mathcal{A}) + \text{SSE}(\mathcal{B})\Big) \tag{9.3}$$

By the Huygens-Steiner theorem of variance decomposition, the dispersion about two independent centroids is strictly no greater than the dispersion about a single pooled centroid: $\text{SSE}(\mathcal{A}) + \text{SSE}(\mathcal{B}) \leq \text{SSE}(\mathcal{C}^*)$. Consequently, every bisection step monotonically decreases the global residual error.

9.2.2 Pseudocode and Procedural Analysis

Algorithm 5 formalizes the procedural workflow of Bisecting K-Means.

Algorithm 5: Bisecting K-Means Algorithm

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$, desired number of clusters K, number of bisection trials T
Output: Final list of K clusters $\mathcal{L} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$

Initialize cluster list $\mathcal{L} \leftarrow \{\mathcal{D}\}$ containing the entire dataset as a single root cluster
while $|\mathcal{L}| < K$ **do**
    Select and remove a candidate cluster $\mathcal{C}^*$ from $\mathcal{L}$ using a split heuristic (e.g., maximum SSE or maximum cardinality)
    $\mathcal{L} \leftarrow \mathcal{L} \setminus \{\mathcal{C}^*\}$
    best_sse $\leftarrow +\infty$
    $(\mathcal{A}^*, \mathcal{B}^*) \leftarrow (\emptyset, \emptyset)$
    for $i \leftarrow 1$ **to** T **do**
        Randomly initialize two centroids within $\mathcal{C}^*$
        Execute basic K-Means with $K = 2$ on $\mathcal{C}^*$, yielding child clusters $(\mathcal{A}_i, \mathcal{B}_i)$
        current_sse $\leftarrow \mathrm{SSE}(\mathcal{A}_i) + \mathrm{SSE}(\mathcal{B}_i)$
        if *current_sse* $<$ *best_sse* **then**
            best_sse $\leftarrow$ current_sse
            $(\mathcal{A}^*, \mathcal{B}^*) \leftarrow (\mathcal{A}_i, \mathcal{B}_i)$
    Insert optimal child clusters into list: $\mathcal{L} \leftarrow \mathcal{L} \cup \{\mathcal{A}^*, \mathcal{B}^*\}$
return $\mathcal{L}$

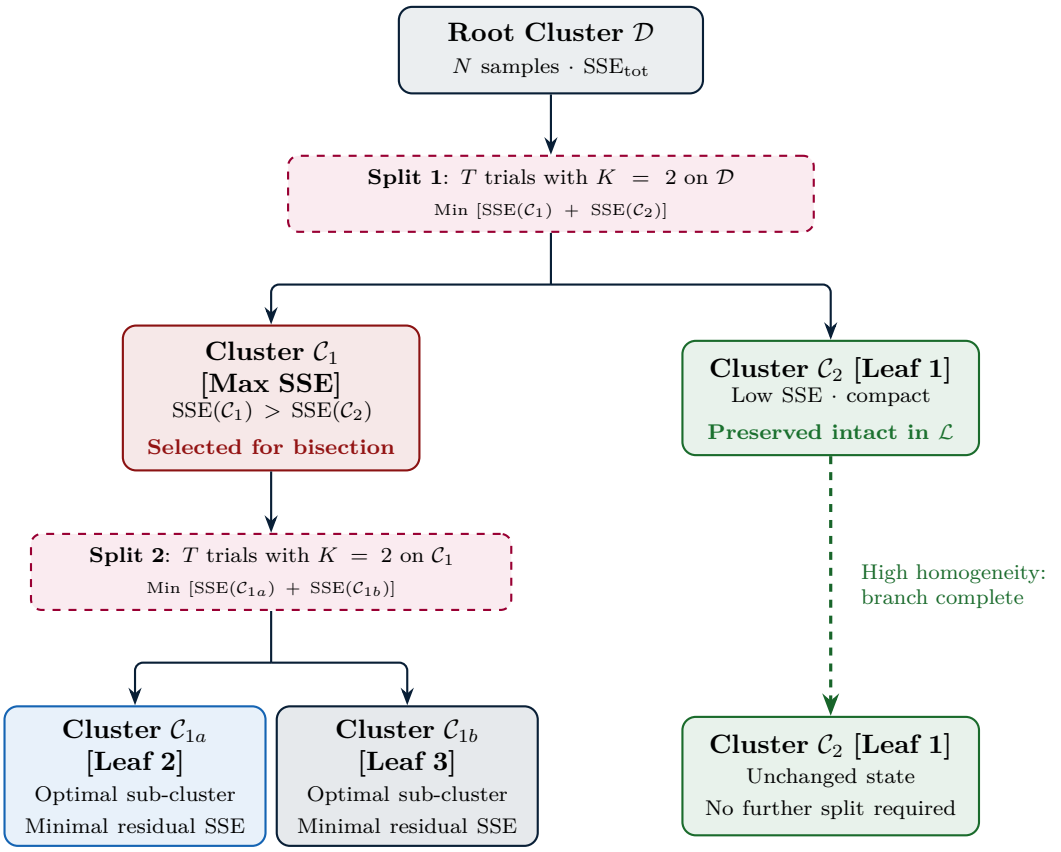
9.2.3 Cluster Selection Heuristics and Multi-Trial Execution

The algorithm provides two essential degrees of freedom:

1. **Candidate Cluster Selection Strategy $\mathcal{C}^*$:**
    - *Maximum Individual SSE* ($\mathcal{C}^* = \arg\max_{\mathcal{C} \in \mathcal{L}} \mathrm{SSE}(\mathcal{C})$): The standard recommended strategy in literature. By splitting the cluster exhibiting the highest internal variance, one maximizes the local error reduction $\mathrm{SSE}(\mathcal{C}^*) - (\mathrm{SSE}(\mathcal{A}) + \mathrm{SSE}(\mathcal{B}))$, actively promoting variance equalization across clusters.

- *Maximum Cardinality* ($\mathcal{C}^* = \arg\max_{\mathcal{C}\in\mathcal{L}} |\mathcal{C}|$): Selects the most populous group, particularly effective when data contains significant cluster size imbalances or oversized clusters.

2. **Role of Multiple Trials (T Trials)**: Because basic 2-means remains an iterative heuristic sensitive to initial seed placement within $\mathcal{C}^*$, performing T independent trials (typically $T \in [5, 20]$) and retaining the partition minimizing $\text{SSE}(\mathcal{A}) + \text{SSE}(\mathcal{B})$ exponentially reduces the likelihood of adopting a sub-optimal split.



Top-down divisive progression: greedily select the cluster with maximum SSE from $\mathcal{D}$. Execute T trials with $K = 2$, choosing the split that minimizes $\text{SSE}(\mathcal{A}) + \text{SSE}(\mathcal{B})$. Homogeneous clusters remain leaves, prioritizing dispersed groups until reaching K clusters.

Figure 9.2: Hierarchical binary tree representation of the divisive progression of Bisecting K-Means with multiple bisection trials and minimum SSE selection.

9.2.4 Computational Complexity and Boundary Cases

Computational analysis reveals significant operational advantages:

- **Per-Bisection Complexity**: Running 2-means on a cluster of n_k points for I_2 iterations requires time $\mathcal{O}(I_2 \cdot 2 \cdot n_k \cdot d)$. Because at any tree depth the sum of cluster sizes is $\sum n_k = N$, if the divisive tree is reasonably balanced the overall depth is $\mathcal{O}(\log K)$. The total cost amounts to $\mathcal{O}(T \cdot I_2 \cdot N \cdot d \cdot \log K)$, which is significantly faster than standard global K-Means when K is large.
- **Degenerate Clusters and Singletons**: If a candidate cluster contains only one point ($|\mathcal{C}| = 1$) or has zero variance (identical points), it cannot be bisected. The algorithm skips such clusters and inspects the next candidate ordered by SSE.

Global Lloyd Refinement

Because partitioning decisions in Bisecting K-Means are made greedily and locally (points in $\mathcal{C}^*$ are separated without interacting with points in other existing clusters), decision boundaries may slightly diverge from global optimality. It is standard best practice to initialize a final run of standard K-Means using the K centroids produced by Bisecting K-Means. This refinement typically converges in very few iterations (2–4 iterations), cleaning up boundary assignments without any risk of getting trapped in bad local minima.

9.3 The X-Means Algorithm and Model Selection

The **X-Means** algorithm, introduced by Dan Pelleg and Andrew Moore [5], addresses the fundamental problem of choosing K automatically and rigorously. Instead of asking the user for a rigid fixed value, X-Means requires only a bounding search range $[K_{\min}, K_{\max}]$, dynamically exploring model complexity via local statistical tests guided by the **Bayesian Information Criterion** (BIC).

9.3.1 Procedural Flow and Local Split Decisions

X-Means alternates between global clustering iterations and statistical split evaluation:

1. **Initialization and Global Convergence**: The process initializes at $K = K_{\min}$ and executes conventional K-Means to convergence.

2. **Centroid Perturbation along Principal Direction**: For each active cluster $\mathcal{C}_k$ with centroid $\mathbf{c}_k$, the algorithm evaluates whether $\mathcal{C}_k$ should remain a single entity (null hypothesis $\mathcal{M}_1$) or be divided into two child clusters (alternative hypothesis $\mathcal{M}_2$). Two candidate seeds $\mathbf{c}_k^{(1)}$ and $\mathbf{c}_k^{(2)}$ are generated by perturbing the centroid along direction $\mathbf{v}$ proportional to the local principal axis of variance:

$$\mathbf{c}_k^{(1)} = \mathbf{c}_k - \alpha\mathbf{v}, \qquad \mathbf{c}_k^{(2)} = \mathbf{c}_k + \alpha\mathbf{v} \tag{9.4}$$

   where $\alpha = s \cdot \sqrt{\lambda_{\max}}$, with $\lambda_{\max}$ being the maximum eigenvalue of the local covariance and s a scaling constant.

3. **Local 2-means Clustering**: A local instance of 2-means is executed strictly on points belonging to $\mathcal{C}_k$, yielding refined child centroids $\hat{\mathbf{c}}_k^{(1)}$ and $\hat{\mathbf{c}}_k^{(2)}$.

4. **BIC Model Evaluation**: The Bayesian Information Criterion is computed for the single-cluster hypothesis $\mathrm{BIC}(\mathcal{M}_1)$ and the two-cluster hypothesis $\mathrm{BIC}(\mathcal{M}_2)$. The information gain is evaluated as:

$$\Delta\mathrm{BIC} = \mathrm{BIC}(\mathcal{M}_2) - \mathrm{BIC}(\mathcal{M}_1) \tag{9.5}$$

   If $\Delta\mathrm{BIC} > 0$, the gain in likelihood outweighs the complexity penalty of additional parameters: the split is accepted, and $\mathbf{c}_k$ is replaced by the pair $(\hat{\mathbf{c}}_k^{(1)}, \hat{\mathbf{c}}_k^{(2)})$. Otherwise, the split is rejected.

5. **Termination Check**: If no clusters are split or if the total number of centroids reaches $K_{\max}$, the algorithm terminates. Otherwise, global K-Means is re-run with the expanded centroid set, and the cycle repeats.

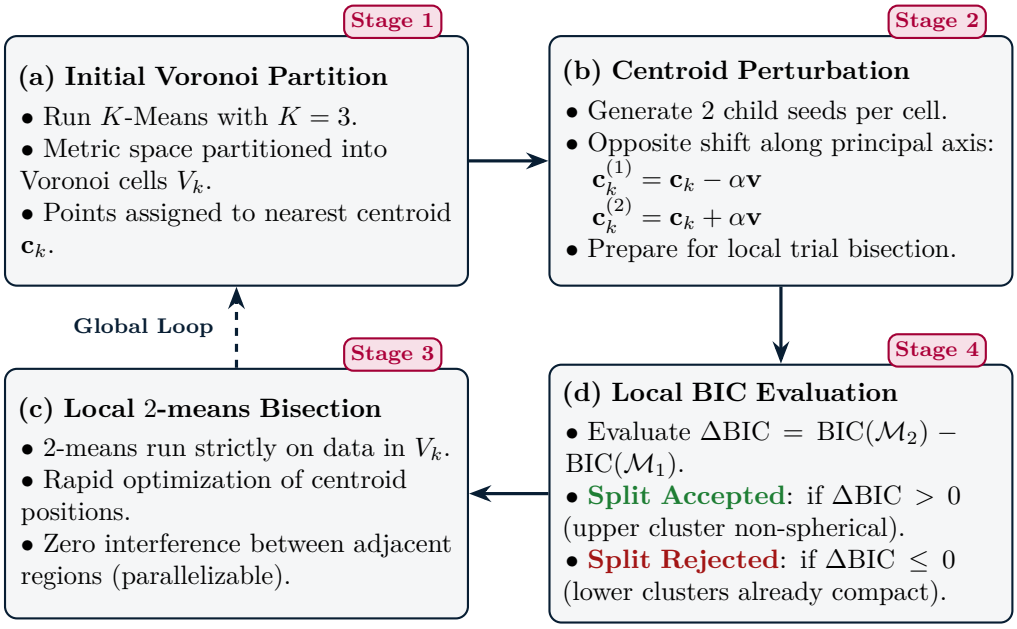


Figure 9.3: Four-stage operational cycle of X-Means: initial Voronoi partition, directional seed perturbation, local bisection, and statistical validation via the information-theoretic ΔBIC test.

9.4 The Bayesian Information Criterion (BIC)

The Bayesian Information Criterion (BIC), formulated by Gideon Schwarz [6], is grounded in asymptotic Bayesian model selection theory. It provides a principled mathematical mechanism for trading off **goodness of fit** against **parametric model complexity** (*parsimony or Occam's razor*).

9.4.1 Asymptotic Derivation from Marginal Likelihood

Consider an observed dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_R\}$ and a parametric model family $\mathcal{M}$ indexed by parameter vector $\boldsymbol{\theta} \in \mathbb{R}^p$. In Bayesian inference, the posterior probability of model $\mathcal{M}$ given $\mathcal{D}$ is given by Bayes' theorem:

$$p(\mathcal{M}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathcal{M})\, p(\mathcal{M})}{p(\mathcal{D})} \propto p(\mathcal{D}|\mathcal{M})\, p(\mathcal{M}) \tag{9.6}$$

Under a uniform model prior ($p(\mathcal{M})$ constant), optimal model selection corresponds to maximizing the **marginal likelihood** (or model evidence):

$$p(\mathcal{D}|\mathcal{M}) = \int_{\Theta} p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M})\, p(\boldsymbol{\theta}|\mathcal{M})\, d\boldsymbol{\theta} \tag{9.7}$$

Because the integral (9.7) is analytically intractable in general, Schwarz applied the **Laplace approximation** as sample size $R \to \infty$. Expanding the log-likelihood function $\ell(\boldsymbol{\theta}) = \ln p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M})$ in a second-order Taylor series around the Maximum Likelihood Estimator (MLE) $\hat{\boldsymbol{\theta}}$:

$$\ell(\boldsymbol{\theta}) \approx \ell(\hat{\boldsymbol{\theta}}) - \frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) \tag{9.8}$$

where $\mathbf{H} = -\nabla^2 \ell(\hat{\boldsymbol{\theta}})$ is the observed Fisher information matrix. Substituting this Gaussian quadratic form into the integral and evaluating the asymptotic determinant for large R gives:

$$\ln p(\mathcal{D}|\mathcal{M}) = \ell(\hat{\boldsymbol{\theta}}) - \frac{p}{2} \ln R + \mathcal{O}(1) \tag{9.9}$$

Defining $\text{BIC}(\mathcal{M})$ proportional to the log marginal likelihood yields the standard formulation:

$$\text{BIC}(\mathcal{M}) = \ell(\hat{\boldsymbol{\theta}}) - \frac{p}{2} \ln R \tag{9.10}$$

where:

- $\ell(\hat{\boldsymbol{\theta}}) = \sum_{i=1}^{R} \ln p(\mathbf{x}_i|\hat{\boldsymbol{\theta}})$ is the maximized log-likelihood of the data under the model;
- p is the total count of free estimated parameters;
- R is the number of observations;
- $-\frac{p}{2} \ln R$ is **Schwarz's complexity penalty**, growing logarithmically with sample size and linearly with parametric degrees of freedom.

Remark 9.4.1 (Sign Conventions in Statistical Literature)**.** Two sign conventions coexist in literature:

1. **Regularized Likelihood Convention** (standard in data mining and Pelleg-Moore): $\text{BIC} = \ell(\hat{\boldsymbol{\theta}}) - \frac{p}{2} \ln R$, which is **maximized** (higher is better).
2. **Deviance Convention** (common in econometrics): $\text{BIC}_{\text{dev}} = -2\ell(\hat{\boldsymbol{\theta}}) + p \ln R$, which is **minimized**. Both conventions are strictly equivalent as $\text{BIC}_{\text{dev}} = -2 \cdot \text{BIC}$.

9.4.2 Formalization of BIC for Spherical Clustering

In the X-Means framework [5], each cluster $k \in \{1, \ldots, K\}$ is modeled as an isotropic spherical Gaussian in d dimensions with shared variance $\mathbf{\Sigma}_k = \sigma^2 \mathbf{I}_d$. The probability density function of point $\mathbf{x}$ generated by cluster $\mathcal{C}_k$ is:

$$p(\mathbf{x}|\mathbf{c}_k, \sigma^2) = \frac{1}{(2\pi)^{d/2}\sigma^d} \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_k\|_2^2}{2\sigma^2}\right) \tag{9.11}$$

The log-likelihood of R points partitioned across K clusters is given analytically by:

$$\ell(\hat{\boldsymbol{\theta}}) = -\frac{R \cdot d}{2}\ln(2\pi) - \frac{R \cdot d}{2}\ln(\hat{\sigma}^2) - \frac{1}{2\hat{\sigma}^2}\sum_{k=1}^{K}\mathrm{SSE}(\mathcal{C}_k) + \sum_{k=1}^{K} R_k \ln\left(\frac{R_k}{R}\right) \tag{9.12}$$

where $R_k = |\mathcal{C}_k|$, and the unbiased estimator of the pooled residual variance $\hat{\sigma}^2$ accounts for the degrees of freedom:

$$\hat{\sigma}^2 = \frac{1}{R - K}\frac{1}{d}\sum_{k=1}^{K}\mathrm{SSE}(\mathcal{C}_k) \tag{9.13}$$

Substituting $\hat{\sigma}^2$ into (9.12) simplifies the third term to exactly $-\frac{(R-K)\cdot d}{2}$.

9.4.3 Rigorous Counting of Free Parameters

The parameter budget p_K for a model with K clusters consists of:

- $K \cdot d$ parameters for the spatial coordinates of the K centroid vectors $\mathbf{c}_k \in \mathbb{R}^d$;
- 1 parameter for the shared isotropic variance $\hat{\sigma}^2$;
- $K-1$ free parameters for cluster prior weights $\pi_k = \frac{R_k}{R}$ (since $\sum_{k=1}^{K}\pi_k = 1$).

Thus, the total parameter count is:

$$p_K = K \cdot d + 1 + (K - 1) = K(d + 1) \tag{9.14}$$

When prior weights are assumed uniform or empirically fixed, literature often uses the reduced budget $p_K = K \cdot d + 1$.

Table 9.1 highlights the quadratic explosion in complexity ($\mathcal{O}(K \cdot d^2)$) when transitioning from spherical to full covariance matrices, demonstrating why X-Means employs the spherical formulation for computational scalability and numerical stability in high dimensions.

Covariance Model	**Form of $\boldsymbol{\Sigma}_k$**	**Parameters p**	**Geometric Properties**
Shared Spherical	$\sigma^2\mathbf{I}_d$	$K \cdot d + 1$	Isotropic spheres of identical radius across all clusters.
Distinct Spherical	$\sigma_k^2\mathbf{I}_d$	$K(d+1)$	Spheres with heterogeneous volumes across clusters.
Diagonal (Independent)	$\mathrm{diag}(\sigma_{k1}^2, \ldots, \sigma_{kd}^2)$	$K \cdot 2d$	Hyper-ellipsoids aligned with coordinate axes.
Full (General)	Arbitrary $\boldsymbol{\Sigma}_k \succ 0$	$K\left(d + \frac{d(d+1)}{2}\right)$	Ellipsoids with arbitrary orientation, rotation, and eccentricity.

Table 9.1: Comparison of free parameter counts p as a function of Gaussian covariance parameterization for K components in dimension d.

9.5 Mixture Models and the Expectation-Maximization (EM) Algorithm

While geometric partitioning methods assign points deterministically, the probabilistic generative paradigm founded on **Mixture Models** and the **Expectation-Maximization** (EM) algorithm [7] views data as realizations of a random variable generated by a convex combination of elementary probability distributions.

9.5.1 Generative Formulation and Latent Variables

Each observation $\mathbf{x}_i \in \mathbb{R}^d$ $(i = 1, \ldots, N)$ is assumed to arise from the following stochastic process:

1. A latent categorical component $z_i \in \{1, \ldots, K\}$ is drawn according to prior probabilities:

$$p(z_i = k) = \pi_k, \qquad \text{with } \sum_{k=1}^{K} \pi_k = 1 \quad \text{and} \quad \pi_k \geq 0 \tag{9.15}$$

   The values π_k are termed **mixing proportions** (or weights).

2. Conditioned on component k, the observation $\mathbf{x}_i$ is sampled from distribution $p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)$:

$$\mathbf{x}_i \sim p(\mathbf{x}_i|z_i = k, \boldsymbol{\theta}_k) \tag{9.16}$$

The marginal probability density of observation $\mathbf{x}_i$ is obtained via the law of total probability:

$$p(\mathbf{x}_i|\boldsymbol{\Theta}) = \sum_{k=1}^{K} p(z_i = k)\, p(\mathbf{x}_i|z_i = k, \boldsymbol{\theta}_k) = \sum_{k=1}^{K} \pi_k\, p_k(\mathbf{x}_i|\boldsymbol{\theta}_k) \tag{9.17}$$

where $\boldsymbol{\Theta} = \{\pi_1, \ldots, \pi_K, \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K\}$ represents the full parameter set.

9.5.2 Incomplete vs. Complete Log-Likelihood

Given an i.i.d. dataset $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, the observed **incomplete log-likelihood** is:

$$\ln p(\mathbf{X}|\boldsymbol{\Theta}) = \sum_{i=1}^{N} \ln \left(\sum_{k=1}^{K} \pi_k\, p_k(\mathbf{x}_i|\boldsymbol{\theta}_k) \right) \tag{9.18}$$

The presence of the logarithm of a summation in (9.18) couples all parameters, preventing closed-form solutions for $\nabla_{\boldsymbol{\Theta}} \ln p(\mathbf{X}|\boldsymbol{\Theta}) = \mathbf{0}$.

If the latent indicator variables $\mathbf{Z} = \{\mathbf{z}_1, \ldots, \mathbf{z}_N\}$ were observed, using indicator variables $z_{ik} = \mathbb{I}(z_i = k)$, the joint **complete log-likelihood** of $(\mathbf{X}, \mathbf{Z})$ would take the decoupled form:

$$\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\Theta}) = \sum_{i=1}^{N} \sum_{k=1}^{K} z_{ik} \left[\ln \pi_k + \ln p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)\right] \tag{9.19}$$

where the logarithm acts directly on individual component densities, permitting closed-form independent optimization.

9.5.3 Variational Foundation of the EM Algorithm

The EM algorithm optimizes the incomplete log-likelihood by applying Jensen's inequality to establish a variational lower bound computed under the posterior distribution of latent variables.

Let $\boldsymbol{\Theta}^{(t)}$ denote parameter estimates at step t. The auxiliary function $\mathcal{Q}(\boldsymbol{\Theta}, \boldsymbol{\Theta}^{(t)})$ is defined as the conditional expectation of the complete log-likelihood:

$$\begin{aligned}
\mathcal{Q}(\boldsymbol{\Theta}, \boldsymbol{\Theta}^{(t)}) &= \mathbb{E}_{\mathbf{Z}|\mathbf{X}, \boldsymbol{\Theta}^{(t)}} \left[\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\Theta})\right] \\
&= \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbb{E}[z_{ik}|\mathbf{x}_i, \boldsymbol{\Theta}^{(t)}] \left[\ln \pi_k + \ln p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)\right] \\
&= \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \left[\ln \pi_k + \ln p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)\right]
\end{aligned} \tag{9.20}$$

where $\gamma_{ik} \equiv \mathbb{E}[z_{ik}|\mathbf{x}_i, \boldsymbol{\Theta}^{(t)}] = p(z_i = k|\mathbf{x}_i, \boldsymbol{\Theta}^{(t)})$ represents the **posterior responsibility** that component k takes for generating instance $\mathbf{x}_i$.

Theorem 9.5.1 (Monotonicity of Likelihood in EM)**.** *For any sequence of parameter vectors $\boldsymbol{\Theta}^{(t)}$ and $\boldsymbol{\Theta}^{(t+1)}$ generated by EM, the likelihood satisfies the monotonic ascent property:*

$$\ln p(\mathbf{X}|\boldsymbol{\Theta}^{(t+1)}) \geq \ln p(\mathbf{X}|\boldsymbol{\Theta}^{(t)}) \tag{9.21}$$

The algorithm converges monotonically to a stationary point (local maximum or saddle point) of the incomplete log-likelihood landscape.

9.5.4 Operational Cycle: E-Step and M-Step

EM iterates through two steps until numerical convergence:

1. Expectation Step (E-step) Holding parameters $\boldsymbol{\Theta}^{(t)}$ fixed, apply Bayes' rule to compute posterior responsibilities:

$$\gamma_{ik} = \frac{\pi_k^{(t)}\, p_k(\mathbf{x}_i|\boldsymbol{\theta}_k^{(t)})}{\sum_{j=1}^{K} \pi_j^{(t)}\, p_j(\mathbf{x}_i|\boldsymbol{\theta}_j^{(t)})} \tag{9.22}$$

Note that $\sum_{k=1}^{K} \gamma_{ik} = 1$ for every observation i. The **effective sample size** belonging to cluster k is:

$$N_k = \sum_{i=1}^{N} \gamma_{ik} \tag{9.23}$$

2. Maximization Step (M-step) Holding responsibilities γ_{ik} fixed, obtain updated parameters $\boldsymbol{\Theta}^{(t+1)}$ by maximizing $\mathcal{Q}$:

$$\boldsymbol{\Theta}^{(t+1)} = \arg\max_{\boldsymbol{\Theta}} \mathcal{Q}(\boldsymbol{\Theta}, \boldsymbol{\Theta}^{(t)}) \tag{9.24}$$

For Gaussian Mixture Models (GMM), closed-form updates for mixing weights, means, and covariances are:

$$\pi_k^{(t+1)} = \frac{N_k}{N} = \frac{1}{N} \sum_{i=1}^{N} \gamma_{ik} \tag{9.25}$$

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} \mathbf{x}_i \tag{9.26}$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})^T \tag{9.27}$$

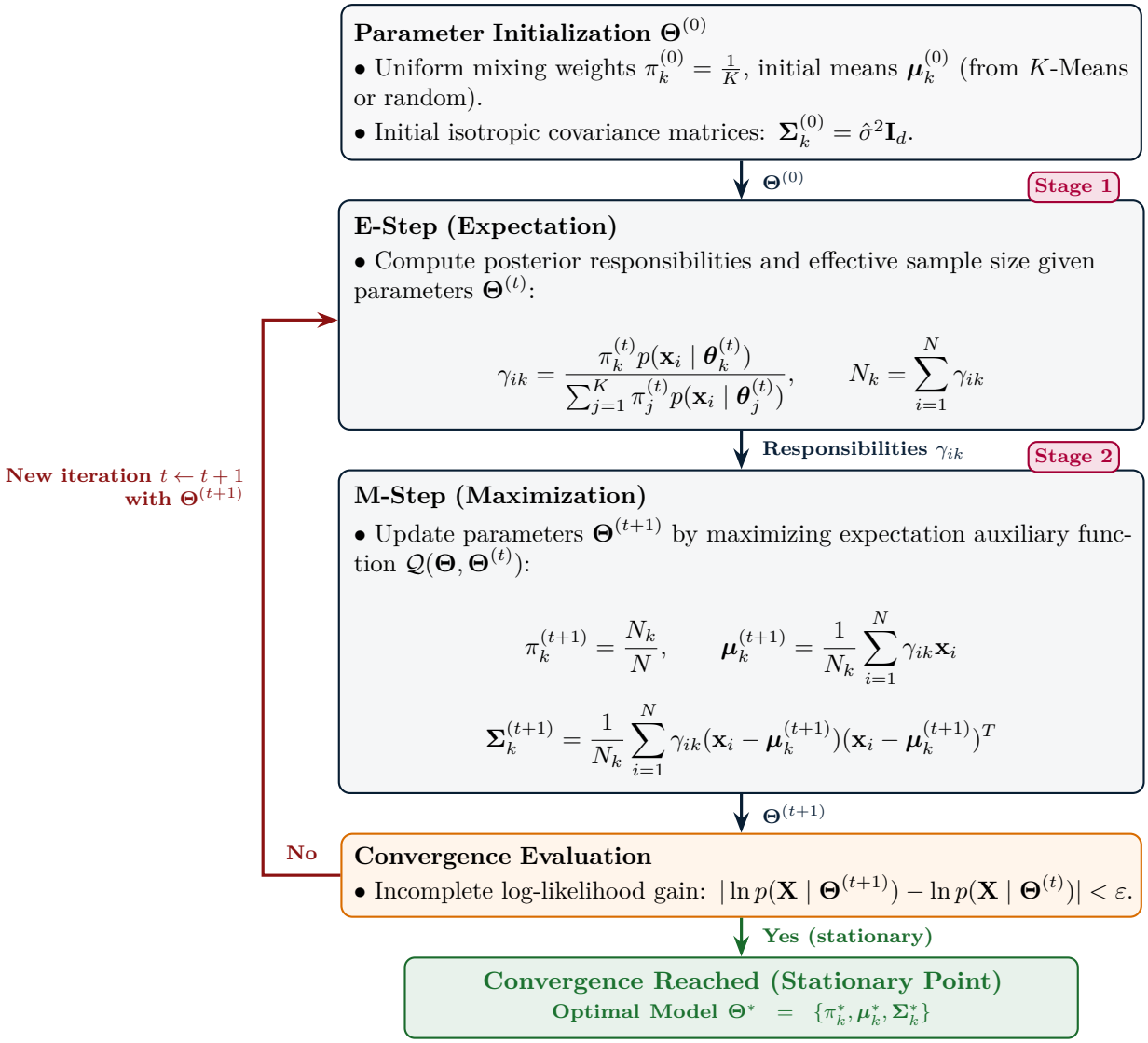


Figure 9.4: Operational flowchart of the Expectation-Maximization (EM) algorithm.

9.6 Normal Distributions and Statistical Estimation Foundations

To fully contextualize the M-step updates in GMMs, we review Maximum Likelihood Estimation (MLE) for a univariate Gaussian.

The probability density function for a univariate normal random variable $x \sim \mathcal{N}(\mu, \sigma^2)$ is:

$$p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{9.28}$$

Given an i.i.d. sample of N scalar observations $\{x_1, \ldots, x_N\}$, the log-likelihood function is:

$$\ell(\mu, \sigma^2) = \sum_{i=1}^{N} \ln p(x_i|\mu, \sigma^2) = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(x_i-\mu)^2 \tag{9.29}$$

Setting the partial derivative with respect to μ to zero:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2}\sum_{i=1}^{N}(x_i - \mu) = 0 \implies \hat{\mu}_{\text{MLE}} = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{9.30}$$

yields the sample arithmetic mean. Substituting $\hat{\mu}$ and setting the partial derivative with respect to σ^2 to zero:

$$\frac{\partial \ell}{\partial (\sigma^2)} = -\frac{N}{2\sigma^2} + \frac{1}{2(\sigma^2)^2}\sum_{i=1}^{N}(x_i - \hat{\mu})^2 = 0 \implies \hat{\sigma}^2_{\text{MLE}} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{\mu})^2 \tag{9.31}$$

In GMMs, the M-step extends these derivations directly by substituting simple arithmetic averages with **weighted averages** governed by posterior responsibilities γ_{ik} (Equations (9.26) and (9.27)).

Bayesian MAP vs. MLE Foundations While MLE maximizes only the data likelihood $\hat{\boldsymbol{\Theta}}_{\text{MLE}} = \arg\max \ln p(\mathbf{X}|\boldsymbol{\Theta})$, **Maximum A Posteriori (MAP)** estimation incorporates conjugate parameter priors $p(\boldsymbol{\Theta})$:

$$\hat{\boldsymbol{\Theta}}_{\text{MAP}} = \arg\max_{\boldsymbol{\Theta}} \Big[\ln p(\mathbf{X}|\boldsymbol{\Theta}) + \ln p(\boldsymbol{\Theta})\Big] \tag{9.32}$$

In high-dimensional mixture modeling, MAP estimation with Normal-Inverse-Wishart priors prevents singular covariance determinants ($|\boldsymbol{\Sigma}_k| \to 0$), acting as structural regularization (*ridge shrinkage*).

9.7 Case Study: Deconvolving a Two-Gaussian Mixture

To gain concrete intuition for probabilistic soft clustering, consider the motivating educational scenario discussed in [2]: modeling human height distributions in a mixed demographic population (e.g., overlapping subpopulations with distinct average stature).

9.7.1 Definition of the Bimodal Mixture Model

Consider a balanced univariate mixture ($K = 2$) of two normal distributions with identical variance but shifted centers of mass:

- **Component 1** ($\mathcal{C}_1$): Mean $\mu_1 = -4$, standard deviation $\sigma_1 = 2$, mixing weight $\pi_1 = 0.5$;
- **Component 2** ($\mathcal{C}_2$): Mean $\mu_2 = +4$, standard deviation $\sigma_2 = 2$, mixing weight $\pi_2 = 0.5$.

The marginal probability density of the combined population is:

$$\begin{aligned} p(x) &= 0.5 \cdot \mathcal{N}(x \mid -4, 4) + 0.5 \cdot \mathcal{N}(x \mid +4, 4) \\ &= \frac{1}{2\sqrt{8\pi}} \left[\exp\left(-\frac{(x+4)^2}{8} \right) + \exp\left(-\frac{(x-4)^2}{8} \right) \right] \end{aligned} \tag{9.33}$$

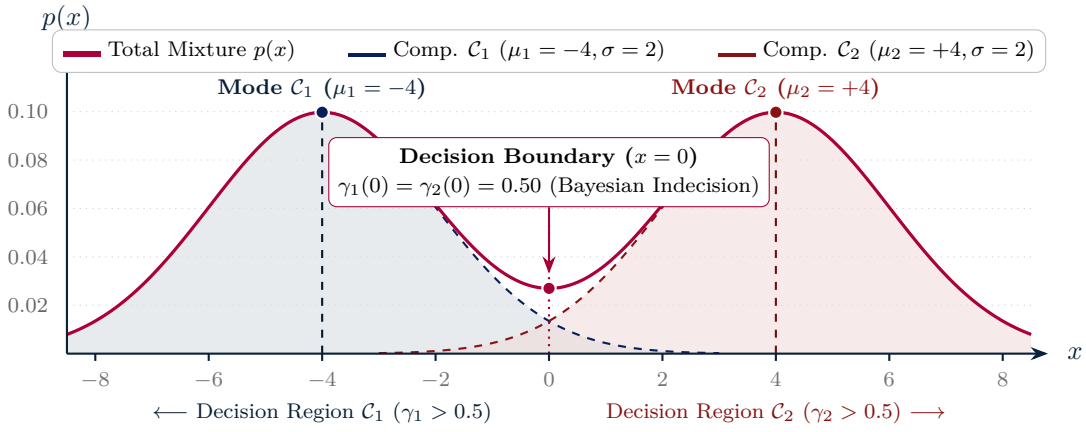


Figure 9.5: Analytical deconvolution of a bimodal two-Gaussian mixture with equal variance ($\sigma = 2$) and separated centers ($\Delta\mu = 8$).

9.7.2 Overlap Analysis and Posterior Responsibilities

Figure 9.5 illustrates the key mathematical properties:

1. **Two Distinct Local Maxima**: Clear peaks appear near $x = -4$ and $x = +4$.

2. **Saddle Point and Overlap Uncertainty at** $x = 0$: At the midpoint $x = 0$, both conditional densities are identical:

$$p_1(0) = \frac{1}{\sqrt{8\pi}} \exp\left(-\frac{16}{8}\right) = \frac{e^{-2}}{\sqrt{8\pi}}, \qquad p_2(0) = \frac{e^{-2}}{\sqrt{8\pi}} \tag{9.34}$$

   Consequently, by Equation (9.22), posterior responsibilities are equal:

$$\gamma_1(0) = \frac{0.5 \cdot p_1(0)}{0.5 \cdot p_1(0) + 0.5 \cdot p_2(0)} = 0.5, \qquad \gamma_2(0) = 0.5 \tag{9.35}$$

   At this location, any rigid deterministic cutoff (assigning $\mathcal{C}_1$ if $x < 0$ and $\mathcal{C}_2$ if $x \geq 0$) artificially conceals epistemic ambiguity. EM transparently reflects this uncertainty through continuous responsibilities $\gamma_{ik} = 0.5$.

3. **Validation on 20,000 Empirical Samples**: When $N = 20\,000$ synthetic points are drawn from mixture (9.33), the empirical histogram matches the continuous theoretical curve within Central Limit Theorem bounds. EM initialized with perturbed values accurately recovers true generating parameters $(\mu_1, \mu_2, \sigma_1, \sigma_2, \pi_1, \pi_2)$ with minimal parameter estimation error ($< 10^{-3}$).

9.8 Architectural and Systematic Comparison: K-Means vs. EM

Lloyd's algorithm (K-Means) and the EM algorithm for Gaussian Mixture Models share a formally isomorphic iterative architecture: both start with parameter estimates and alternate between updating point cluster memberships and recalculating cluster parameters. However, they diverge fundamentally in their geometric, energetic, and statistical foundations.

9.8.1 Covariance Singularities in EM

A significant vulnerability of EM with unconstrained covariance matrices $\mathbf{\Sigma}_k$ is the existence of **singularities in the likelihood surface**. If a Gaussian

Dimension	Standard K-Means (Lloyd)	Gaussian Mixture (EM)
Assignment Paradigm	**Hard clustering**: deterministic and disjoint ($r_{ik} \in \{0, 1\}$).	**Soft clustering**: probabilistic and continuous ($\gamma_{ik} \in [0, 1]$).
Cluster Model	Barycentric centroid vector $\mathbf{c}_k \in \mathbb{R}^d$.	Parameter tuple $(\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.
Geometric Shape	Hyperspherical, isotropic, homogeneous variance.	Hyper-ellipsoidal with arbitrary orientation, size, and shape.
Objective Function	Minimizing intra-cluster Sum of Squared Errors (SSE).	Maximizing incomplete log-likelihood ($\ln p(\mathbf{X}\|\boldsymbol{\Theta})$).
Phase 1 (Point State)	Voronoi assignment to nearest centroid (L_2).	E-step: analytical posterior responsibilities γ_{ik} (Bayes).
Phase 2 (Update)	Unweighted arithmetic mean of assigned points.	M-step: weighted mean and covariance using weights γ_{ik}.
Computational Cost	$\mathcal{O}(N \cdot K \cdot d)$	$\mathcal{O}(N \cdot K \cdot d^2 + K \cdot d^3)$ (matrix inversion).
Vulnerabilities	Seed sensitivity, spherical bias, zero uncertainty quantification.	Covariance singularities, $\mathcal{O}(d^3)$ cost, local optima.

Table 9.2: Comprehensive architectural comparison between K-Means and Expectation-Maximization (EM).

component collapses onto an isolated singleton point $\mathbf{x}_m$, its sample covariance shrinks to zero:

$$\boldsymbol{\Sigma}_k \to \mathbf{0} \implies |\boldsymbol{\Sigma}_k| \to 0 \tag{9.36}$$

Because the multivariate Gaussian density includes $|\boldsymbol{\Sigma}_k|^{1/2}$ in its denominator:

$$p_k(\mathbf{x}_m|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \to +\infty \implies \ln p(\mathbf{X}|\boldsymbol{\Theta}) \to +\infty \tag{9.37}$$

The likelihood function is unbounded from above, generating infinite spikes on the surface that represent pathological overfitting to single observations rather than meaningful clusters. Common remedies include:

- **Diagonal Jitter**: Adding a small positive constant $\varepsilon\mathbf{I}_d$ to covariance matrices at every step ($\boldsymbol{\Sigma}'_k = \boldsymbol{\Sigma}_k + \epsilon\mathbf{I}_d$).
- **MAP Regularization**: Imposing an Inverse-Wishart prior on $\boldsymbol{\Sigma}_k$ to penalize shrinking determinants.

9.8.2 Summary of the Methodological Path

The conceptual journey presented in this chapter charts the modern evolution of clustering:

- Divisive hierarchical clustering (**Bisecting K-Means**) converts an unstable global heuristic into a stable sequence of low-complexity binary decisions.
- Statistical information theory (X**-Means** and **BIC**) replaces subjective guesses for K with objective complexity optimization rooted in Occam's razor.
- Probabilistic generative modeling (**GMM** and **EM**) elevates spatial partitioning into rigorous statistical inference, quantifying ambiguity and modeling arbitrary oriented geometries in continuous space.

Chapter 10

Cluster Validity: Metrics, Theory, and Methodologies

This chapter provides a rigorous and comprehensive treatment of **Cluster Validity**, encompassing the theoretical foundations, quantitative metrics, and inferential statistical methodologies used to assess the reliability, significance, and quality of clusterings produced by unsupervised learning algorithms [2, 8].

The presentation systematically covers:

1. The **foundations and challenges of cluster validation**: the inherent ambiguity of the unsupervised paradigm, comparison with supervised validation, the propensity of clustering algorithms to discover spurious patterns in pure noise (demonstrated on uniform spatial distributions), and methodological motivations for evaluation;
2. The **classical taxonomy of validation measures**: unsupervised (*internal*) criteria, supervised (*external*) criteria, *relative* criteria, and clustering tendency tests;
3. The **Hopkins statistic**: mathematical formulation of clustering tendency, stochastic sampling procedures, nearest-neighbor distance metrics, and probabilistic interpretation of divergence from spatial uniformity;
4. **Internal validation via proximity matrices**: Pearson correlation between similarity matrices and ideal incidence matrices, visual inspection of reordered distance heatmaps, and comparative behavior across K-Means, Complete Link, DBSCAN, and non-convex shapes;
5. **Cluster cohesion and separation**: graph-based versus prototype-based formulations, the total variance decomposition TSS = WSS+BSS, a detailed algebraic proof of the conservation theorem, and a step-by-step 1D numerical walkthrough;

6. The **Silhouette coefficient** (Rousseeuw [9]): pointwise geometric formulation ($a(i)$, $b(i)$, $s(i)$), derivation of boundary regimes, global mean Silhouette Score, and fine-grained cluster diagnostics via silhouette plots;
7. **Complementary laboratory internal indices**: the Davies-Bouldin index (DB) and the Caliński-Harabasz ratio (CH), their geometric interpretations, and computational trade-offs;
8. **External validation measures**: empirical joint distributions p_{ij}, formal definitions of conditional entropy and weighted purity, a full case study on the *LA Documents* text corpus, and contingency matrices;
9. The **inferential statistical framework and null models**: formulation of the randomness null hypothesis H_0, Monte Carlo simulation over empirical distributions, and computation of p-values and critical values for SSE and matrix correlation;
10. The **Python application laboratory**: practical index implementations using `scikit-learn`, empirical analysis on the course dataset, optimal cluster parameter K^* estimation using the knee/elbow method, and cross-tabulation (*crosstab*) with external categorical variables.

10.1 Foundations and Challenges of Cluster Validity

Cluster analysis is a cornerstone of unsupervised machine learning. Its primary goal is to partition a collection of objects or attribute vectors $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$ into disjoint or overlapping subsets (called *clusters*), such that instances belonging to the same group exhibit high mutual similarity (**intra-cluster cohesion**), while elements belonging to different groups are distinct and well-isolated (**inter-cluster separation**) [2].

10.1.1 Divergence Between Supervised and Unsupervised Learning

In supervised learning (such as classification or regression), the presence of a known target label (*ground truth*) provides an objective, direct evaluation benchmark: model predictive quality is evaluated by comparing predictions against true values via established metrics such as Accuracy, Precision, Recall, F-measure, or the Area Under the ROC Curve (AUC).

In stark contrast, the unsupervised paradigm provides no external class labels:

"Clustering is in the eye of the beholder."

There is no unique or universally accepted definition of what constitutes a "natural cluster". The geometric structure identified depends intrinsically on the chosen proximity metric and the algorithmic family employed:

- For K-**Means**, a cluster is conceived as a compact, hyper-spherical, convex grouping centered around an average barycentric centroid;
- For **DBSCAN**, a cluster is a connected high-density component separated from other clusters by low-density regions or noise, capable of adopting arbitrary shapes;
- For **Single Linkage Hierarchical Clustering**, a cluster corresponds to a contiguous chain of mutually proximate points, prone to the chaining phenomenon.

10.1.2 The Critical Dilemma: Clustering Pure Noise

The most compelling motivation for a rigorous theory of cluster validity is that **all clustering algorithms produce clusters even when applied to pure noise**.

Consider an experiment conducted on a continuous 2D domain $\Omega = [0, 1]^2$, where N points are sampled independently from a uniform distribution:

$$\mathbf{x}_i \sim \mathcal{U}\left([0, 1]^2\right), \quad \forall i \in \{1, \ldots, N\} \tag{10.1}$$

By stochastic definition, this dataset contains no cluster structure, attractors, or density peaks. Nevertheless:

1. Executing K-Means with $K = 3$ partitions the stochastic domain into three compact Voronoi cells, positioning centroids to minimize local intra-cluster variance and returning an apparently plausible grouping;

2. Executing an agglomerative hierarchical algorithm with Complete Link produces a complete dendrogram which, when cut at an arbitrary horizontal threshold, yields clusters with well-defined diameters;

3. Applying DBSCAN with uncalibrated $(\varepsilon, \text{MinPts})$ parameters can aggregate random Poisson density fluctuations into spurious dense micro-clusters.

Fundamental Axiom of Cluster Validity

Every clustering algorithm is a geometric optimizer designed to aggregate data. It will invariably output clusters, even in an informational void. Consequently, no clustering result should ever be accepted without first verifying:

1. Whether the original dataset possesses an actual **clustering tendency** compared to a null hypothesis of uniform noise;
2. Whether the discovered partition is **robust, statistically significant**, and not an artifact of random fluctuations or algorithmic biases.

10.1.3 Methodological Goals of Cluster Validity

Within the Knowledge Discovery in Databases (KDD) lifecycle, cluster validation pursues four distinct operational objectives:

1. **Noise filtering and tendency testing**: Determining beforehand whether the dataset exhibits a non-random propensity to cluster, discarding datasets that lack structural geometric signal;

2. **Comparing heterogeneous algorithms**: Evaluating which paradigm (partitional, hierarchical, density-based, or probabilistic) most faithfully models the intrinsic geometry of the observations;

3. **Model selection and hyperparameter tuning**: Identifying optimal parametrizations, such as the true number of clusters K^* in K-Means or the neighborhood radius ε in DBSCAN;

4. **Local evaluation of individual clusters**: Examining the quality of each cluster within a partition, distinguishing highly cohesive and separated cores from marginal, spurious, or noise-contaminated groups.

10.2 Taxonomy of Cluster Validation Measures

Following the classical formulation by Jain and Dubes [8] and Tan et al. [2], validation metrics and methodologies are divided into four broad categories according to the availability of auxiliary information and the analytical purpose.

As summarized in Figure 10.1:

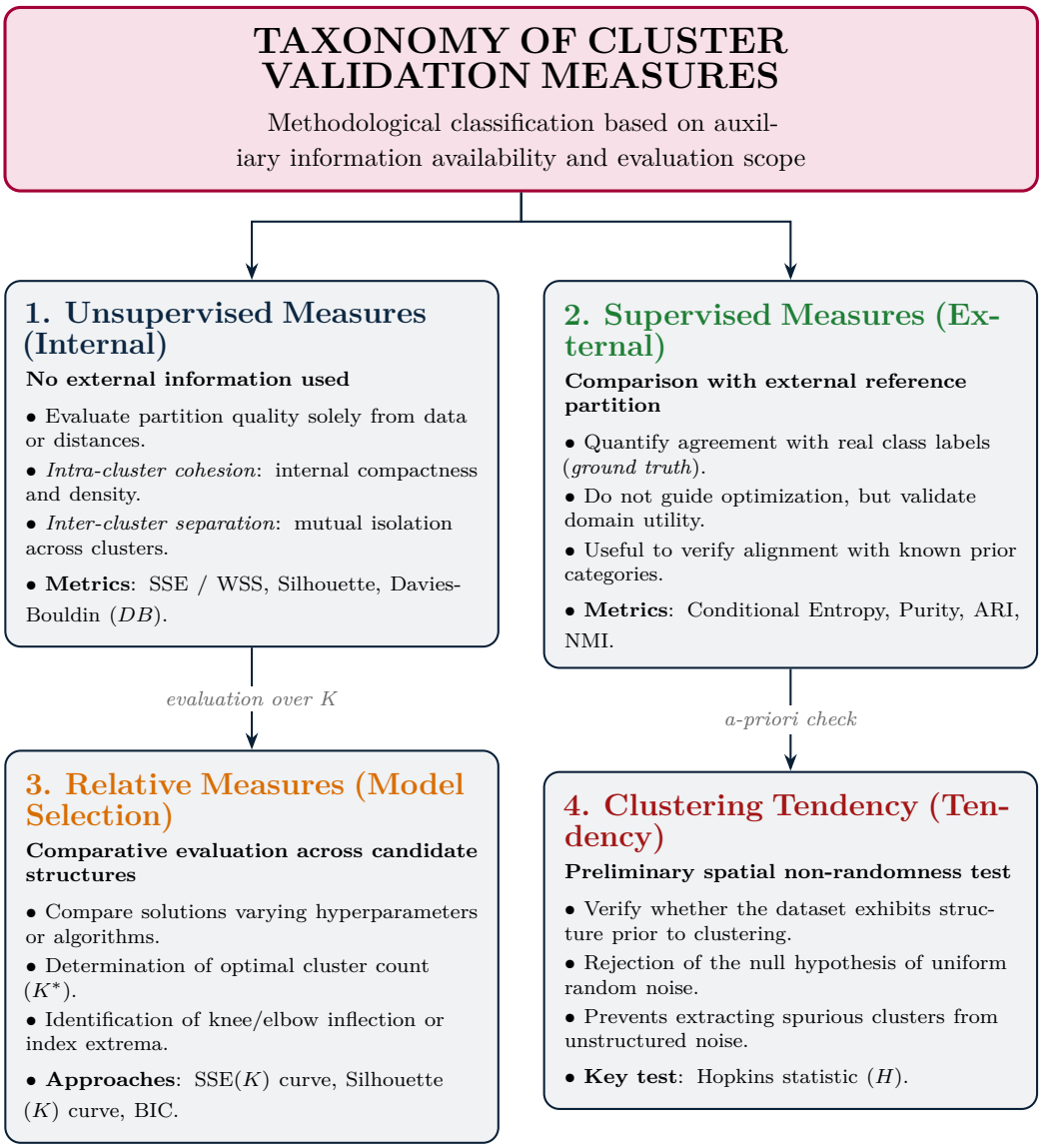


Figure 10.1: Taxonomy of cluster validation measures categorized by data nature and analytical objective.

1. **Internal Measures**: operate entirely in the unsupervised space. They evaluate whether clusters are compact and well-separated;

2. **External Measures**: quantify correspondence between discovered clusters and domain-expert ground truth labels;

3. **Relative Measures**: provide comparative criteria across candidate models to choose appropriate hyperparameters (such as K^*);

4. **Clustering Tendency**: serves as an upfront sanity check applied prior to clustering.

10.3 Clustering Tendency and the Hopkins Statistic

Before investing computational effort in optimizing clustering algorithms, one should determine whether the data exhibits non-uniform structure. This problem is termed **Clustering Tendency** [2]. The standard quantitative formulation is the **Hopkins Statistic** [10, 8].

10.3.1 Mathematical Formulation of the Hopkins Statistic

Let $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$ be a multivariate dataset. The Hopkins procedure evaluates the discrepancy between the empirical nearest-neighbor distance distribution of real data and the expected nearest-neighbor distribution under a uniform stochastic process.

The procedure operates as follows:

1. Sample randomly without replacement a small subset of p real data points from $\mathcal{D}$, denoted by:

$$\mathcal{X} = \{\mathbf{x}_1^*, \mathbf{x}_2^*, \ldots, \mathbf{x}_p^*\} \subset \mathcal{D}, \quad \text{with } p \ll N \tag{10.2}$$

   Typically, $p \approx 0.05 \cdot N$ or $p \approx 0.10 \cdot N$ to ensure statistical independence.

2. For each selected point $\mathbf{x}_i^* \in \mathcal{X}$, compute its minimum Euclidean distance to its nearest neighbor in the remainder of the dataset $\mathcal{D} \setminus \{\mathbf{x}_i^*\}$:

$$w_i = \min_{\mathbf{x} \in \mathcal{D} \setminus \{\mathbf{x}_i^*\}} \|\mathbf{x}_i^* - \mathbf{x}\|_2 \tag{10.3}$$

3. Construct a hyper-rectangular bounding box $\Omega \subset \mathbb{R}^d$ enclosing the data support:

$$\Omega = \prod_{j=1}^{d} \left[\min_{k=1}^{N} x_{kj},\ \max_{k=1}^{N} x_{kj} \right] \tag{10.4}$$

Inside this bounding box Ω, generate p synthetic points distributed uniformly:

$$\mathcal{Y} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_p\} \overset{\text{i.i.d.}}{\sim} \mathcal{U}(\Omega) \tag{10.5}$$

4. For each synthetic point $\mathbf{y}_i \in \mathcal{Y}$, compute its minimum distance to the nearest real point in $\mathcal{D}$:

$$u_i = \min_{\mathbf{x} \in \mathcal{D}} \|\mathbf{y}_i - \mathbf{x}\|_2 \tag{10.6}$$

5. The **Hopkins statistic** H is defined as the normalized ratio:

$$H = \frac{\sum_{i=1}^{p} u_i}{\sum_{i=1}^{p} u_i + \sum_{i=1}^{p} w_i} \tag{10.7}$$

10.3.2 Probabilistic Interpretation of Hopkins Values

The statistic H is constrained to the interval $[0, 1]$. Its behavior across different spatial regimes is detailed in Table 10.1.

Remark 10.3.1 (Operational Decision Threshold)**.** In multivariate analysis practice, a dataset is considered to display statistically significant clustering tendency if repeated independent estimates of the Hopkins statistic yield:

$$\bar{H} > 0.70 \quad \text{(preferably } \bar{H} \geq 0.75 \text{ or } 0.80\text{)} \tag{10.8}$$

If $H \in [0.45, 0.55]$, the data cannot be distinguished from uniform noise, and partitional clustering should not be relied upon.

Value of H	**Behavior of u_i vs w_i**	**Spatial Diagnostics and Clustering Tendency**
$H \approx 0.5$	$\sum u_i \approx \sum w_i$	**Uniformly Distributed Data (No Clusters)**: distances from random points $\mathbf{y}_i$ to real data are statistically indistinguishable from distances between adjacent real points. The dataset possesses no natural cluster structure.
$H \to 1.0$	$\sum u_i \gg \sum w_i$	**Strong Clustering Tendency (Dense Clusters)**: real data points reside in dense clusters ($w_i \approx 0$). Uniform points $\mathbf{y}_i$ fall predominantly in sparse inter-cluster spaces, producing large distances u_i.
$H \to 0.0$	$\sum u_i \ll \sum w_i$	**Regularly Spaced Data (Hypercubic Lattice)**: real points repel each other, producing a periodic lattice arrangement (maximal dispersion).

Table 10.1: Quantitative interpretation of the Hopkins statistic for evaluating clustering tendency.

10.4 Internal Validation via Proximity Matrices

A general and intuitive strategy for evaluating clustering validity is to assess the agreement between the observed proximity matrix and an ideal incidence matrix derived from the partition [2].

10.4.1 Proximity Matrix vs Ideal Incidence Matrix

Let $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ be a collection of objects and let $P \in \mathbb{R}^{N \times N}$ be the corresponding **proximity matrix**, where $P_{ij} = \text{dist}(\mathbf{x}_i, \mathbf{x}_j)$ or $P_{ij} = \text{sim}(\mathbf{x}_i, \mathbf{x}_j)$.

Given a partition $\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$, the **ideal incidence matrix** $Y \in \{0,1\}^{N \times N}$ is defined as:

$$Y_{ij} = \begin{cases} 1, & \text{if } \mathbf{x}_i, \mathbf{x}_j \in \mathcal{C}_k \text{ (same cluster)}, \\ 0, & \text{if } \mathbf{x}_i \in \mathcal{C}_k, \mathbf{x}_j \in \mathcal{C}_l \text{ with } k \neq l \text{ (distinct clusters)}. \end{cases} \tag{10.9}$$

If P_{ij} represents normalized similarity in $[0,1]$, an ideal partition implies that within-cluster pairs should have similarity 1, whereas between-cluster pairs should have similarity 0.

10.4.2 Pearson Matrix Correlation Coefficient

Because P and Y are symmetric and have constant diagonal entries ($P_{ii} = 1$, $Y_{ii} = 1$), their agreement is quantified by computing the Pearson correlation across all $M = \frac{N(N-1)}{2}$ unique off-diagonal pairs (i,j) with $i < j$:

$$\rho(P, Y) = \frac{\sum_{i<j} \left(P_{ij} - \bar{P}\right)\left(Y_{ij} - \bar{Y}\right)}{\sqrt{\sum_{i<j} \left(P_{ij} - \bar{P}\right)^2 \sum_{i<j} \left(Y_{ij} - \bar{Y}\right)^2}} \tag{10.10}$$

where $\bar{P}$ and $\bar{Y}$ denote off-diagonal empirical means:

$$\bar{P} = \frac{2}{N(N-1)} \sum_{i<j} P_{ij}, \qquad \bar{Y} = \frac{2}{N(N-1)} \sum_{i<j} Y_{ij} \tag{10.11}$$

10.4.3 Visual Inspection of Reordered Distance Matrices

A powerful diagnostic tool consists in visualizing the distance matrix as a heatmap after **reordering instance indices by cluster membership**:

1. Group indices so that all elements of cluster $\mathcal{C}_1$ appear first, followed by $\mathcal{C}_2$, up to $\mathcal{C}_K$;
2. Within each cluster block, sort points by ascending distance to their cluster centroid or by hierarchical leaf order.

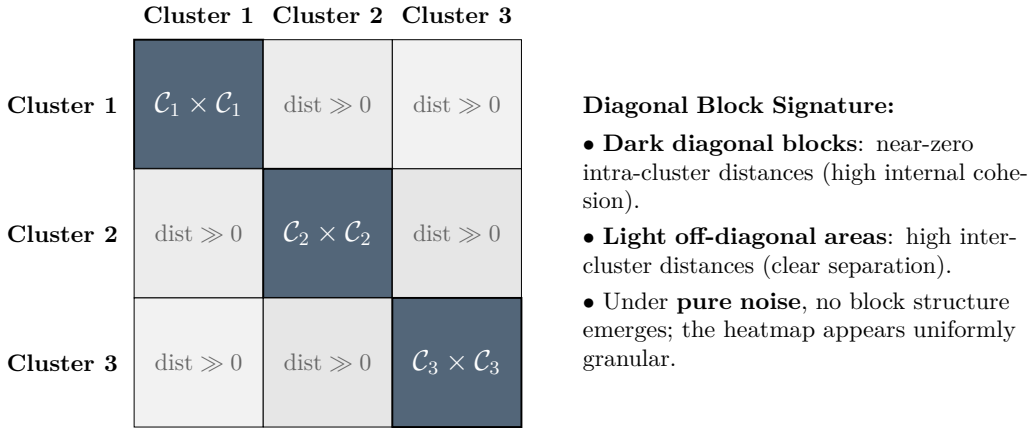


Figure 10.2: Reordered distance matrix schematic: prominent square blocks along the main diagonal visually confirm cluster validity.

10.4.4 Sensitivity to Non-Convex Cluster Geometry

Matrix correlation assumes spherical geometry:

- **Compact, spherical clusters**: for K-Means or Complete Link, within-cluster distances are uniformly low, and between-cluster distances are consistently large, yielding high matrix correlation ($\rho \approx 0.85$–0.90);
- **Elongated or concentric non-convex clusters**: for shapes like concentric rings handled by DBSCAN, Euclidean distances between opposite ends of the same ring are very large. While $Y_{ij} = 1$, P_{ij} records large Euclidean distances. Consequently, matrix correlation $\rho(P, Y)$ drops misleadingly, despite an optimal geometric grouping.

10.5 Cohesion and Separation: Theory and Algebraic Equivalence

The two fundamental properties of clustering quality are **cohesion** (compactness) and **separation** (isolation) [2].

10.5.1 Graph-Based vs Prototype-Based Formulations

1. **Graph-based formulation**: Data points are vertices in a weighted graph with edge weights $s(\mathbf{x}_i, \mathbf{x}_j)$ or $d(\mathbf{x}_i, \mathbf{x}_j)$. Cohesion is the sum of internal edge weights, while separation is the sum of cut edge weights across clusters;

2. **Prototype-based formulation**: Each cluster $\mathcal{C}_k$ is represented by its geometric centroid $\mathbf{c}_k$. Cohesion is measured by intra-cluster squared deviations from centroids (WSS), while separation is measured by centroid deviations from the grand dataset mean (BSS).

10.5.2 Prototype-Based Formulation: WSS, BSS, and TSS

Let $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$ partitioned into K disjoint clusters $\mathcal{C}_1, \ldots, \mathcal{C}_K$, with cluster sizes $n_k = |\mathcal{C}_k|$.

Define the **cluster centroid** $\mathbf{c}_k$ and **grand dataset mean** $\mathbf{m}$:

$$\mathbf{c}_k = \frac{1}{n_k} \sum_{\mathbf{x} \in \mathcal{C}_k} \mathbf{x}, \qquad \mathbf{m} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{x}_i = \frac{1}{N} \sum_{k=1}^{K} n_k \mathbf{c}_k \tag{10.12}$$

Define the three sums of squares:

1. **Within-Cluster Sum of Squares (**WSS **or** SSE**)**:

$$\text{WSS} = \sum_{k=1}^{K} \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 \tag{10.13}$$

2. **Between-Cluster Sum of Squares (**BSS**)**:

$$\text{BSS} = \sum_{k=1}^{K} n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2 \tag{10.14}$$

3. **Total Sum of Squares (**TSS**)**:

$$\text{TSS} = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}\|_2^2 \tag{10.15}$$

10.5.3 Rigorous Proof of the Total Variance Conservation Theorem

Theorem 10.5.1 (Conservation of Total Sum of Squares)**.** *For any partition of dataset $\mathcal{D}$ into K disjoint clusters with centroids defined as arithmetic means, the total sum of squares (*TSS*) equals the sum of within-cluster dispersion (*WSS*) and between-cluster dispersion (*BSS*):*

$$\text{TSS} = \text{WSS} + \text{BSS} \tag{10.16}$$

and TSS *is constant for a given dataset, independent of the clustering.*

Proof. Expand TSS by splitting the summation over all N points into sums over the K clusters:

$$\text{TSS} = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}\|_2^2 = \sum_{k=1}^{K} \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{m}\|_2^2 \tag{10.17}$$

Add and subtract the cluster centroid $\mathbf{c}_k$ within each term:

$$\mathbf{x} - \mathbf{m} = (\mathbf{x} - \mathbf{c}_k) + (\mathbf{c}_k - \mathbf{m}) \tag{10.18}$$

Expanding the squared Euclidean norm via inner products:

$$\|\mathbf{x} - \mathbf{m}\|_2^2 = \|\mathbf{x} - \mathbf{c}_k\|_2^2 + \|\mathbf{c}_k - \mathbf{m}\|_2^2 + 2(\mathbf{x} - \mathbf{c}_k)^\top (\mathbf{c}_k - \mathbf{m}) \tag{10.19}$$

Summing over all points in cluster $\mathcal{C}_k$:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{m}\|_2^2 = \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 + \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{c}_k - \mathbf{m}\|_2^2 + 2 \sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k)^\top (\mathbf{c}_k - \mathbf{m}) \tag{10.20}$$

Examining the three terms:

1. The first term summed over all k yields WSS:

$$\sum_{k=1}^{K} \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 = \text{WSS} \tag{10.21}$$

2. In the second term, $\|\mathbf{c}_k - \mathbf{m}\|_2^2$ is constant with respect to $\mathbf{x}$. Summing over n_k elements gives $n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2$, whose sum over k yields BSS:

$$\sum_{k=1}^{K} n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2 = \text{BSS} \tag{10.22}$$

3. For the cross-product term, factor out $(\mathbf{c}_k - \mathbf{m})$:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k)^\top (\mathbf{c}_k - \mathbf{m}) = \left[\sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k) \right]^\top (\mathbf{c}_k - \mathbf{m}) \tag{10.23}$$

By the definition of the centroid as the arithmetic mean:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k) = \sum_{\mathbf{x} \in \mathcal{C}_k} \mathbf{x} - n_k \mathbf{c}_k = \mathbf{0} \tag{10.24}$$

Thus, the cross-product vanishes identically for every cluster:

$$2 \cdot \mathbf{0}^\top (\mathbf{c}_k - \mathbf{m}) = 0 \tag{10.25}$$

Summing over all K clusters yields:

$$\text{TSS} = \text{WSS} + \text{BSS} \tag{10.26}$$

completing the proof. □

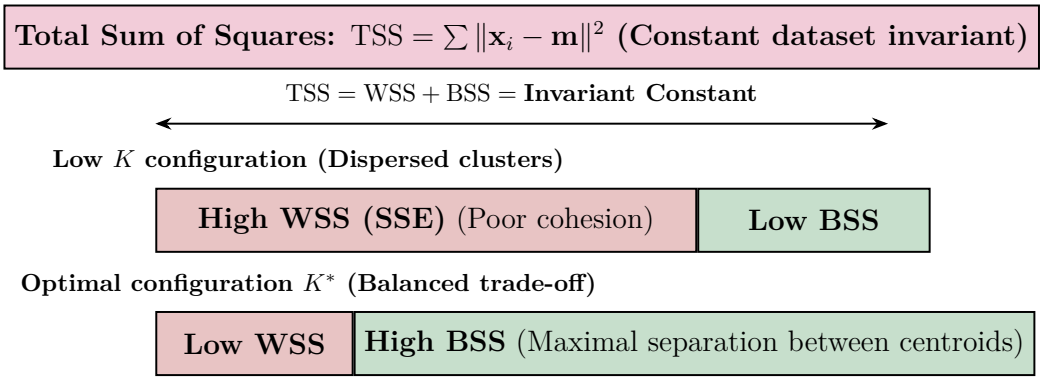


Figure 10.3: Total variance conservation decomposition TSS = WSS + BSS: minimizing within-cluster dispersion WSS is mathematically identical to maximizing between-cluster separation BSS.

Core Optimization Takeaway

Theorem 10.5.1 demonstrates that:

$$\min \text{WSS} \iff \max \text{BSS} \tag{10.27}$$

Minimizing SSE in K-Means inherently maximizes the weighted distance between cluster centroids and the grand mean. However, because WSS $\to 0$ monotonically as $K \to N$, BSS increases monotonically toward TSS. Therefore, neither metric can be used alone to select the optimal K.

10.5.4 1D Educational Walkthrough

Consider the 1D dataset:

$$\mathcal{D} = \{1, 2, 4, 5\}, \quad N = 4 \tag{10.28}$$

The global mean is $\mathbf{m} = \frac{1+2+4+5}{4} = 3$. The total sum of squares is:

$$\text{TSS} = (1-3)^2 + (2-3)^2 + (4-3)^2 + (5-3)^2 = 4 + 1 + 1 + 4 = 10 \tag{10.29}$$

- **Case** $K = 1$ ($\mathcal{C}_1 = \{1, 2, 4, 5\}$): $\mathbf{c}_1 = 3 \implies$ WSS $= 10$, BSS $= 4 \cdot (3-3)^2 = 0 \implies$ TSS $= 10 + 0 = 10$.
- **Case** $K = 2$ ($\mathcal{C}_1 = \{1, 2\}, \mathcal{C}_2 = \{4, 5\}$): Centroids are $\mathbf{c}_1 = 1.5$ and $\mathbf{c}_2 = 4.5$.

$$\text{SSE}(\mathcal{C}_1) = (1-1.5)^2 + (2-1.5)^2 = 0.25 + 0.25 = 0.5 \tag{10.30}$$
$$\text{SSE}(\mathcal{C}_2) = (4-4.5)^2 + (5-4.5)^2 = 0.25 + 0.25 = 0.5 \tag{10.31}$$
$$\text{WSS} = 0.5 + 0.5 = 1.0 \tag{10.32}$$
$$\text{BSS} = 2(1.5-3)^2 + 2(4.5-3)^2 = 2(2.25) + 2(2.25) = 9.0 \tag{10.33}$$

Conservation check: WSS + BSS $= 1.0 + 9.0 = 10.0 =$ TSS.

10.6 The Silhouette Coefficient

Introduced by Peter Rousseeuw in 1987 [9], the **Silhouette Coefficient** evaluates both cohesion and separation at the level of **individual data points**, without assuming spherical geometry or relying exclusively on cluster centroids [2].

10.6.1 Pointwise Mathematical Formulation

For a partition into $K \geq 2$ clusters and a point $\mathbf{x}_i \in \mathcal{C}_k$:

1. **Mean intra-cluster distance** $a(i)$:

$$a(i) = \frac{1}{|\mathcal{C}_k| - 1} \sum_{\mathbf{x}_j \in \mathcal{C}_k, j \neq i} \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \tag{10.34}$$

   Smaller values indicate higher cohesion.

2. **Mean distance to nearest neighboring cluster** $b(i)$:

$$b(i) = \min_{l \neq k} \left(\frac{1}{|\mathcal{C}_l|} \sum_{\mathbf{x}_j \in \mathcal{C}_l} \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \right) \tag{10.35}$$

   Larger values indicate greater separation.

3. **Silhouette coefficient** $s(i)$:

$$s(i) = \frac{b(i) - a(i)}{\max\left(a(i),\ b(i)\right)} \tag{10.36}$$

10.6.2 Boundary Regimes and Interpretation

By construction, $s(i) \in [-1, 1]$:

- $s(i) \to +1$ ($a(i) \ll b(i)$): the point is well-matched to its own cluster and well-separated from neighbors;
- $s(i) \approx 0$ ($a(i) \approx b(i)$): the point lies on the decision boundary between two clusters;
- $s(i) \to -1$ ($a(i) \gg b(i)$): the point is closer to a neighboring cluster, indicating probable misassignment.

The **mean Silhouette Score** across all N samples is $\bar{s} = \frac{1}{N} \sum_{i=1}^{N} s(i)$. Values $\bar{s} > 0.7$ indicate strong structure; $[0.5, 0.7]$ reasonable structure; and < 0.25 no substantial structure.

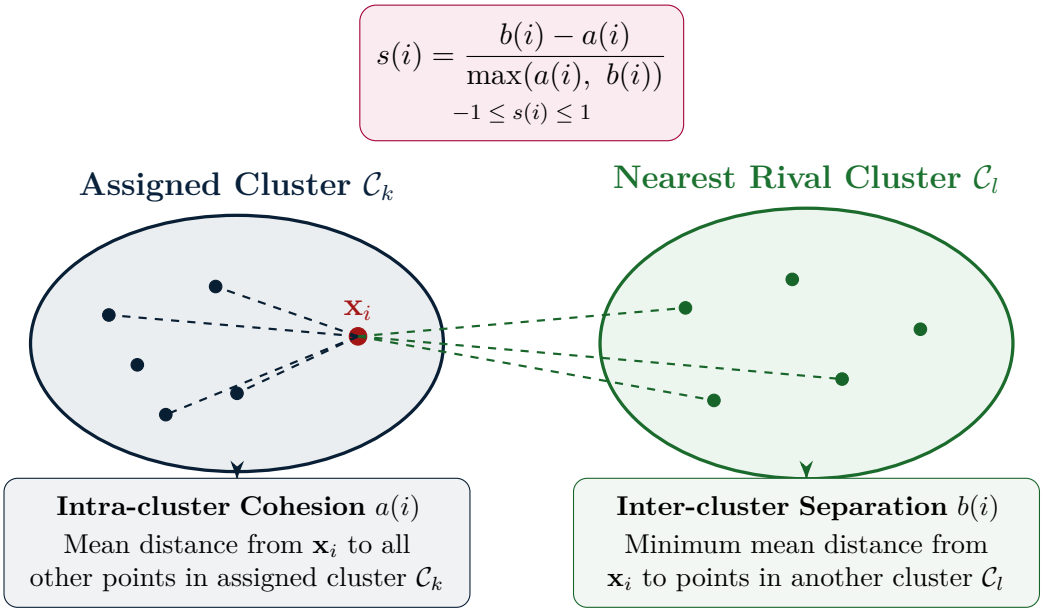


Figure 10.4: Silhouette coefficient geometry: visual comparison between internal cohesion $a(i)$ and separation from the nearest rival cluster $b(i)$.

10.7 Internal Indices: Davies-Bouldin and Caliński-Harabasz

In data mining laboratories (such as notebook `3_clustering.ipynb` [2]), alternative internal metrics are used that provide better scalability than the quadratic $\mathcal{O}(N^2)$ distance requirements of Silhouette.

10.7.1 Davies-Bouldin Index (DB Score)

Introduced by Davies and Bouldin (1979) [11], the DB index measures the average worst-case similarity ratio between each cluster and its most similar rival.

Let σ_i denote average intra-cluster distance to centroid $\mathbf{c}_i$:

$$\sigma_i = \frac{1}{|\mathcal{C}_i|} \sum_{\mathbf{x} \in \mathcal{C}_i} \|\mathbf{x} - \mathbf{c}_i\|_2 \tag{10.37}$$

The similarity between clusters $\mathcal{C}_i$ and $\mathcal{C}_j$ is:

$$R_{ij} = \frac{\sigma_i + \sigma_j}{\|\mathbf{c}_i - \mathbf{c}_j\|_2} \tag{10.38}$$

The **Davies-Bouldin index** is the mean over maximum cluster similarities:

$$DB = \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i} R_{ij} \tag{10.39}$$

Property 10.7.1 (Properties of the DB Index)**.** $DB \geq 0$. *Lower values indicate superior clustering (compact clusters with small σ_i and well-separated centroids). It computes in $\mathcal{O}(Nd + K^2 d)$ time, making it significantly faster than Silhouette on large datasets.*

10.7.2 Caliński-Harabasz Index (Variance Ratio Criterion)

Proposed by Caliński and Harabasz (1974) [12], the CH index is the ratio of between-cluster to within-cluster dispersion, adjusted for degrees of freedom:

$$CH = \frac{\text{BSS}/(K-1)}{\text{WSS}/(N-K)} \tag{10.40}$$

Higher CH values indicate better defined clusters. The optimal K^* often corresponds to a local maximum in the $CH(K)$ curve.

10.8 External Validation: Entropy, Purity, and Agreement

When independent class labels (*ground truth*) are available, supervised metrics evaluate how closely discovered clusters recover true categories [2].

10.8.1 Contingency Matrix and Joint Distributions

Let $\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$ be the cluster partition and $\mathcal{L} = \{\mathcal{L}_1, \ldots, \mathcal{L}_L\}$ the ground truth classes. The contingency table counts intersections:

$$m_{ij} = |\mathcal{C}_i \cap \mathcal{L}_j|, \qquad m_i = |\mathcal{C}_i| = \sum_{j=1}^{L} m_{ij} \tag{10.41}$$

The conditional probability that a point in cluster $\mathcal{C}_i$ belongs to class $\mathcal{L}_j$ is:

$$p_{ij} = \frac{m_{ij}}{m_i} \tag{10.42}$$

10.8.2 Entropy and Purity

1. **Cluster Entropy** e_i: Applying the foundational definition of Shannon Entropy formalized in Chapter 6 (section 6.9), individual cluster entropy measures disorder relative to the external class partition:

$$e_i = -\sum_{j=1}^{L} p_{ij} \log_2(p_{ij}) \tag{10.43}$$

   (with $0 \log_2(0) = 0$). Lower values indicate higher homogeneity (with $e_i = 0$ for a pure cluster and $\log_2 L$ for uniform distribution).

2. **Global Weighted Entropy** E:

$$E = \sum_{i=1}^{K} \frac{m_i}{N} e_i \tag{10.44}$$

3. **Cluster Purity** p_i:

$$p_i = \max_j p_{ij} \tag{10.45}$$

4. **Global Weighted Purity** P:

$$P = \sum_{i=1}^{K} \frac{m_i}{N} p_i = \frac{1}{N} \sum_{i=1}^{K} \max_j m_{ij} \tag{10.46}$$

   Values range in $[0, 1]$; values near 1 indicate dominant majority classes within each cluster.

10.8.3 Analytical Case Study: The "LA Documents" Text Corpus

Tan et al. [2] analyze the *Los Angeles Times Documents* corpus ($N = 3\,204$ articles across 6 news topics: Entertainment, Financial, Foreign, Politics, Sports, Technology). Applying K-Means with $K = 9$ yields the contingency table in Table 10.2.

Clusters $\mathcal{C}_4$ (Sports) and $\mathcal{C}_6$ (Finance) attain exceptionally high purity ($> 97\%$) and low entropy (< 0.24). Entertainment is split into two pure sub-clusters ($\mathcal{C}_7$ and $\mathcal{C}_8$), highlighting natural sub-topical separation.

Cluster	Ent.	Fin.	For.	Pol.	Spo.	Tec.	Total	Purity	Entropy
$\mathcal{C}_1$	3	5	40	506	0	3	557	0.908	0.505
$\mathcal{C}_2$	17	7	280	29	0	0	333	0.841	0.778
$\mathcal{C}_3$	2	1	1	7	0	2	13	0.538	1.761
$\mathcal{C}_4$	7	1	1	0	422	3	434	0.972	0.239
$\mathcal{C}_5$	40	180	20	10	2	3	255	0.706	1.341
$\mathcal{C}_6$	4	505	6	0	0	4	519	0.973	0.208
$\mathcal{C}_7$	423	3	12	3	0	1	442	0.957	0.320
$\mathcal{C}_8$	105	3	5	1	0	0	114	0.921	0.449
$\mathcal{C}_9$	60	17	85	8	0	367	537	0.683	1.258
Global							**3204**	**0.887**	**0.672**

Table 10.2: Contingency table, purity, and entropy for $K = 9$ clustering on *LA Documents* (Tan et al. [2]).

10.9 Statistical Framework and Null Models

A clustering metric value (such as SSE $= 24.5$ or $\rho = 0.42$) is uninterpretable without comparing it against the expected value under a reference random distribution [2, 8].

10.9.1 Formulation of the Randomness Null Hypothesis

The null hypothesis is:

$$H_0 : \text{Data contains no cluster structure and is randomly generated.} \tag{10.47}$$

For continuous data, H_0 is modeled as a uniform Poisson spatial process across the bounding hyper-rectangle of the observed data:

$$\mathbf{x} \sim \mathcal{U}\left(\prod_{j=1}^{d}[a_j, b_j]\right) \tag{10.48}$$

or via random attribute coordinate shuffling.

10.9.2 Monte Carlo Simulation and Empirical Null Distribution

Because closed-form distributions for metrics like SSE under H_0 are generally intractable:

1. Generate B independent synthetic datasets $\mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(B)}$ (typically $B = 500$ or $1\,000$) with size N and dimension d under H_0;

2. Execute the clustering algorithm on each $\mathcal{D}^{(b)}$ under identical settings;

3. Compute the target statistic $\theta^{(b)}$ to construct the empirical null distribution $f(\theta \mid H_0)$.

10.9.3 Empirical p-Value Calculation

For metrics minimized for quality (such as SSE):

$$p\text{-value} = \frac{1}{B} \sum_{b=1}^{B} \mathbb{I}\left(\theta^{(b)} \leq \theta_{\text{obs}}\right) \tag{10.49}$$

For metrics maximized for quality (such as Silhouette or matrix correlation ρ):

$$p\text{-value} = \frac{1}{B} \sum_{b=1}^{B} \mathbb{I}\left(\theta^{(b)} \geq \theta_{\text{obs}}\right) \tag{10.50}$$

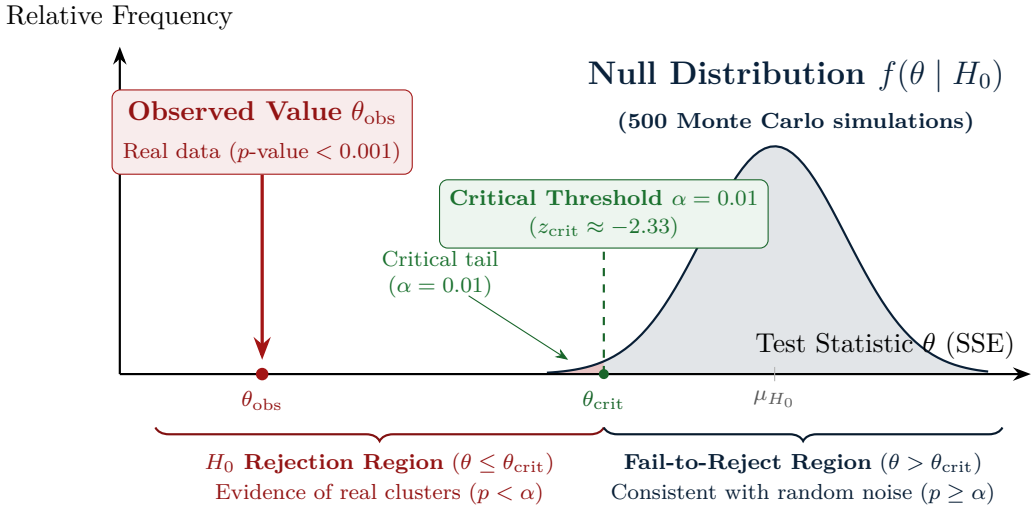


Figure 10.5: Null model hypothesis testing framework: when observed SSE falls far to the left of the empirical null distribution, the randomness hypothesis is formally rejected.

10.10 Python Application Laboratory: Metrics and Model Selection

This section summarizes practical methodologies from the course data mining laboratory (notebook `3_clustering.ipynb` [2]), illustrating `scikit-learn` implementations for internal and external validity metrics as well as optimal K^* selection.

Interactive Environment and Laboratory Reproducibility

All computational routines, validation metrics, and model selection procedures presented in this section are directly executable and reproducible via the official course Jupyter environment:

- Interactive notebook: **Notebook 3 (Clustering)** (file `Data-Mining/notebooks/3_clustering.ipynb`);
- Quick start with Docker: `docker compose up jupyter` (or Windows script `scripts/start-jupyter.cmd`);
- Web interface: `http://localhost:8888/lab` (no token or login required).

Remark 10.10.1 (Interactive Notebook Reference)**.** The code in the following subsections maps directly to the cells under *Metrics to Evaluate Clustering* and *K parameter estimation* in the interactive notebook **3_clustering.ipynb**:

- **Internal metrics** (SSE, Davies-Bouldin, Silhouette): cells 33–34;
- **Elbow method** (scanning $K \in [2, 40]$ and highlighting K^*): cells 38–40;
- **External validation** (contingency matrix and `day` profiling): cells 35–37.

10.10.1 Internal Metrics in Scikit-Learn

In the laboratory workflow (cell 34 of **Notebook 3**), after fitting K-Means on standardized numerical features X, compactness (SSE), separation (Davies-Bouldin), and balance (Silhouette) are calculated directly:

Listing 10.1: Joint calculation of SSE, Davies-Bouldin, and Silhouette Score in Python.

```
from sklearn.cluster import KMeans
from sklearn import metrics
from sklearn.metrics import silhouette_score
```

```
4
5  # Fit K-Means with specified K
6  kmeans = KMeans(n_clusters=4, n_init=10, max_iter=300,
       random_state=42)
7  kmeans.fit(X)
8
9  # 1. Intra-cluster cohesion: Inertia (SSE / WSS)
10 # Built-in attribute measuring sum of squared distances
       to centroids
11 sse = kmeans.inertia_
12 print(f'SSE (Inertia / WSS): {sse:.3f}')
13
14 # 2. Inter-cluster separation: Davies-Bouldin Score
15 # Evaluates worst-case similarity ratio; optimal near 0
16 db_score = metrics.davies_bouldin_score(X, kmeans.
       labels_)
17 print(f'Davies-Bouldin Score: {db_score:.3f}')
18
19 # 3. Global Silhouette Score: balance between cohesion
       and separation
20 # Range [-1, 1]; optimal near +1
21 sil_score = silhouette_score(X, kmeans.labels_)
22 print(f'Silhouette Score: {sil_score:.3f}')
```

10.10.2 Estimating Optimal K: Knee/Elbow Method

As formalized in Chapter 7 (subsection 7.4.1), heuristic estimation of K^* relies on identifying the knee/elbow point where marginal variance reduction sharply flattens.

Listing 10.2 (drawn from cells 38–40 of **Notebook 3**) demonstrates the programmatic scan over $K \in [2, 40]$ and subsequent visualization of the knee threshold K^*:

Listing 10.2: Systematic scan and elbow plot visualization for optimal K^* selection.

```
1  import matplotlib.pyplot as plt
2
3  sse_list = []
4  max_k = 40
5
6  # Scan range of cluster complexity
7  for k in range(2, max_k + 1):
```

```
    km = KMeans(n_clusters=k, n_init=10, max_iter=100,
        random_state=42)
    km.fit(X)
    sse_list.append(km.inertia_)

# Identify knee inflection point
k_star = 10
sse_star = sse_list[k_star - 2]  # Zero-based index
    adjustment

# Visualize elbow curve
plt.figure(figsize=(9, 5))
plt.plot(range(2, max_k + 1), sse_list, marker='o',
    color='navy', lw=2)
plt.ylabel('SSE (Inertia)', fontsize=14)
plt.xlabel('Number of Clusters (K)', fontsize=14)
plt.title('Elbow Method for Estimating K*', fontsize=16)

# Anchor lines at optimal point
plt.vlines(x=k_star, ymin=0, ymax=sse_star, colors='
    crimson', linestyles='dashed', lw=1.5)
plt.hlines(y=sse_star, xmin=0, xmax=k_star, colors='
    crimson', linestyles='dashed', lw=1.5)
plt.scatter(k_star, sse_star, color='crimson', s=90,
    zorder=5, label=f'Knee K*={k_star}')

plt.legend(fontsize=12)
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

10.10.3 External Validation via Cross-Tabulation

In the laboratory exercises (cells 35–37 of **Notebook 3**), to analyze whether unsupervised clusters align with external categorical attributes not used in training (such as `day` or `time` in the `tips` dataset), `pd.crosstab` builds the empirical contingency table:

Listing 10.3: Contingency analysis and cluster profiling with Pandas.

```
import pandas as pd

# Contingency matrix between cluster assignments and
    external variable
```

```
cross_df = pd.crosstab(kmeans.labels_, df_cat['day'])
print("Contingency Matrix: Cluster vs Day:")
print(cross_df)

# Visualize distribution across clusters
cross_df.plot(kind='bar', stacked=False, figsize=(10, 5)
    , colormap='viridis')
plt.title('Distribution of external variable "day"
    across clusters')
plt.xlabel('Cluster ID')
plt.ylabel('Instance Count')
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.show()
```

The contingency table allows direct computation of cluster purity and conditional entropy with respect to domain variables, completing the quantitative cluster validation workflow.

Chapter 11

Association Analysis: Basic Concepts and Algorithms

Association analysis represents one of the foundational methodologies in unsupervised data mining [2]. Originating in retail market-basket analysis to discover customer purchasing patterns, association analysis provides a rigorous formal, algorithmic, and statistical framework for uncovering latent relationships, co-occurrences, and cross-item dependencies within large transactional datasets [13, 14].

This chapter provides a comprehensive treatment of the mathematical formulation of association mining, the combinatorial structure of the candidate itemset search space, the Apriori anti-monotonicity principle, advanced indexing data structures (such as hash trees) for efficient frequency counting, compact lossless and lossy pattern representations (closed and maximal itemsets), the intrinsic limitations of the standard confidence metric, objective statistical correlation measures (Lift, Leverage, and Pearson's ϕ), and methodological extensions for handling continuous attributes, high-cardinality categorical variables, and multi-level concept taxonomies [2].

11.1 Introduction to Association Rule Mining

In unsupervised learning, association analysis differs fundamentally from both supervised classification (where a target label is known *a priori*) and geometric clustering (where data instances are grouped by proximity in vector space). Its objective is discovering intrinsic empirical regularities expressed as **association rules**—logical implications asserting that the presence of certain items in a transaction predicts the presence of other items [2, 13].

11.1.1 Problem Context and Transactional Model

The canonical data model consists of a collection of discrete transaction records. In retail environments, a transaction corresponds to an individual customer basket or checkout invoice.

TID (Transaction ID)	Items (Purchased Itemset)
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Table 11.1: Toy transactional database for market-basket analysis.

In Table 11.1, each row represents a unique transaction identified by its `TID`, consisting of a non-empty subset of items drawn from the overall retail inventory.

11.1.2 Examples of Association Rules

Empirical observation of transactional records yields rules of the form:

$$\{\text{Diaper}\} \longrightarrow \{\text{Beer}\} \tag{11.1}$$

$$\{\text{Milk, Bread}\} \longrightarrow \{\text{Eggs, Coke}\} \tag{11.2}$$

$$\{\text{Beer, Bread}\} \longrightarrow \{\text{Milk}\} \tag{11.3}$$

Fundamental Principle: Statistical Co-occurrence vs Causality

The implication expressed by an association rule $X \rightarrow Y$ does not represent a cause-and-effect relationship, nor does it posit a temporal or sequential ordering. It describes purely a **statistical co-occurrence** within a single shopping session [2].
Stating that $\{\text{Diaper}\} \rightarrow \{\text{Beer}\}$ means solely that customers who purchase diapers show a strong statistical propensity to also purchase beer in the same transaction; it in no way implies that purchasing diapers causes the purchase of beer.

11.2 Formal Definitions: Itemsets, Support, and Frequency

To formalize the problem mathematically, we define the item universe and the transaction database [2, 14].

Definition 11.2.1 (Item Universe and Transaction Database)**.** Let $I = \{i_1, i_2, \ldots, i_d\}$ denote the finite set of all d distinct items in the application domain. Let $T = \{t_1, t_2, \ldots, t_N\}$ denote the set of N transactions, where each transaction $t_j \subseteq I$ is a non-empty item subset.

Definition 11.2.2 (k-Itemset)**.** An arbitrary subset of items $X \subseteq I$ is called an **itemset**. If the cardinality of the itemset is $|X| = k$, it is referred to as a **k-itemset**.

For instance, in Table 11.1, the set $X = \{\text{Milk, Bread, Diaper}\}$ contains 3 distinct items and constitutes a 3-itemset.

11.2.1 Support Count and Relative Support

The statistical significance of an itemset is evaluated through its frequency of occurrence across the database.

Definition 11.2.3 (Support Count (σ))**.** Given an itemset $X \subseteq I$, its support count $\sigma(X)$ is the absolute number of transactions in T that contain X as a subset:

$$\sigma(X) = \left|\{t_j \in T \mid X \subseteq t_j\}\right| \tag{11.4}$$

Verifying this definition for $X = \{\text{Milk, Bread, Diaper}\}$ on Table 11.1:

- Transaction 1 (t_1): {Bread, Milk} (missing Diaper) $\rightarrow$ does not contain X;
- Transaction 2 (t_2): {Bread, Diaper, Beer, Eggs} (missing Milk) $\rightarrow$ does not contain X;
- Transaction 3 (t_3): {Milk, Diaper, Beer, Coke} (missing Bread) $\rightarrow$ does not contain X;
- Transaction 4 (t_4): {Bread, Milk, Diaper, Beer} $\rightarrow$ contains X;
- Transaction 5 (t_5): {Bread, Milk, Diaper, Coke} $\rightarrow$ contains X.

Thus, the absolute frequency is $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$.

Definition 11.2.4 (Relative Support (s))**.** The relative support $s(X)$ is the fraction or proportion of transactions in T that contain X, normalized by the total transaction count $|T| = N$:

$$s(X) = \frac{\sigma(X)}{|T|} = \frac{\sigma(X)}{N} \tag{11.5}$$

In our toy dataset with $N = 5$:

$$s(\{\text{Milk, Bread, Diaper}\}) = \frac{2}{5} = 0.40 \quad (40\%) \tag{11.6}$$

Definition 11.2.5 (Frequent Itemset and minsup Threshold)**.** Given a user-specified minimum support threshold minsup $\in (0, 1]$, an itemset X is defined as **frequent** if and only if its relative support satisfies:

$$s(X) \geq \text{minsup} \quad \Longleftrightarrow \quad \sigma(X) \geq \lceil \text{minsup} \times |T| \rceil \tag{11.7}$$

11.3 Association Rules and Fundamental Evaluation Metrics

With itemsets defined, we formalize association rules and their core metrics.

Definition 11.3.1 (Association Rule)**.** An association rule is an implication expression of the form:

$$X \longrightarrow Y \tag{11.8}$$

where $X, Y \subset I$ are disjoint itemsets ($X \cap Y = \emptyset$). Itemset X is termed the **antecedent** (or left-hand side, *LHS*), while Y is termed the **consequent** (or right-hand side, *RHS*).

11.3.1 Primary Metrics: Support and Confidence

Rule utility and certainty are traditionally measured using support and confidence [13, 2]:

Definition 11.3.2 (Rule Support)**.** The support of rule $X \to Y$ measures the proportion of transactions in the database containing both antecedent and consequent simultaneously, matching the empirical joint probability $P(X \cup Y)$:

$$s(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{|T|} = P(X \cup Y) \tag{11.9}$$

Definition 11.3.3 (Rule Confidence)**.** The confidence of rule $X \to Y$ quantifies inference reliability, representing the conditional probability that a transaction contains Y given that it contains X:

$$c(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)} = \frac{s(X \cup Y)}{s(X)} = P(Y \mid X) \tag{11.10}$$

11.3.2 Detailed Analytical Calculation Example

Consider the candidate rule:

$$\{\text{Milk, Diaper}\} \longrightarrow \{\text{Beer}\} \tag{11.11}$$

From Table 11.1:

- Antecedent: $X = \{\text{Milk, Diaper}\}$;
- Consequent: $Y = \{\text{Beer}\}$;
- Joint set: $X \cup Y = \{\text{Milk, Diaper, Beer}\}$.

Evaluating transaction counts:

1. $X \cup Y$ appears in transactions 3 and 4 $\implies \sigma(X \cup Y) = 2$;
2. X appears in transactions 3, 4, and 5 $\implies \sigma(X) = 3$;
3. Total transactions: $N = |T| = 5$.

Applying formulas (11.9) and (11.10):

$$s(\{\text{Milk, Diaper}\} \to \{\text{Beer}\}) = \frac{2}{5} = 0.40 \quad (40\%) \tag{11.12}$$

$$c(\{\text{Milk, Diaper}\} \to \{\text{Beer}\}) = \frac{2}{3} \approx 0.6667 \quad (66.67\%) \tag{11.13}$$

11.4 The Mining Task and Computational Tractability

Definition 11.4.1 (Association Rule Mining Problem)**.** Given transaction database T, support threshold minsup $\in (0, 1]$, and confidence threshold minconf $\in (0, 1]$, the mining problem consists of discovering all association rules $X \to Y$ satisfying:

$$\begin{cases} s(X \longrightarrow Y) \geq \text{minsup} \\ c(X \longrightarrow Y) \geq \text{minconf} \end{cases} \tag{11.14}$$

11.4.1 The Brute-Force Approach and Combinatorial Explosion

A naive brute-force algorithm would enumerate every possible rule derivable from item universe I, compute its support and confidence across database T, and discard rules failing the thresholds.

For a universe of d distinct items, the total number of non-empty disjoint association rules R is given by:

$$R = \sum_{k=2}^{d} \binom{d}{k} \sum_{j=1}^{k-1} \binom{k}{j} = 3^d - 2^{d+1} + 1 \tag{11.15}$$

Proof. Each item $i \in I$ can take one of three mutually exclusive roles relative to a candidate rule:

1. Belongs to antecedent X;
2. Belongs to consequent Y;
3. Is excluded from the rule ($i \notin X \cup Y$).

This yields 3^d assignments. Subtracting invalid degenerate cases:

- Antecedent is empty ($X = \emptyset$): item is in Y or excluded (2^d cases);
- Consequent is empty ($Y = \emptyset$): item is in X or excluded (2^d cases);
- Both X and Y empty ($X = \emptyset, Y = \emptyset$): double-subtracted, restored once ($+1$).

Thus $R = 3^d - 2 \cdot 2^d + 1 = 3^d - 2^{d+1} + 1$. □

For a supermarket catalog with $d = 100$ items, $3^{100} \approx 5.15 \times 10^{47}$, rendering naive enumeration completely intractable.

11.5 Decoupling the Problem: The Two-Step Approach

To bypass combinatorial explosion, the association mining literature adopts a two-phase decomposition [13, 14, 2].

11.5.1 Properties of Binary Partitions of an Itemset

Consider all possible candidate rules generated by partitioning 3-itemset $L = \{\text{Milk, Diaper, Beer}\}$:

Table 11.2 reveals two key properties:

Candidate Rule ($X \to Y$)	$\sigma(X \cup Y)$	$\sigma(X)$	**Support** s	**Confidence** c
{Milk, Diaper} $\to$ {Beer}	2	3	$2/5 = 0.40$	$2/3 \approx 0.67$
{Milk, Beer} $\to$ {Diaper}	2	2	$2/5 = 0.40$	$2/2 = 1.00$
{Diaper, Beer} $\to$ {Milk}	2	3	$2/5 = 0.40$	$2/3 \approx 0.67$
{Beer} $\to$ {Milk, Diaper}	2	3	$2/5 = 0.40$	$2/3 \approx 0.67$
{Diaper} $\to$ {Milk, Beer}	2	4	$2/5 = 0.40$	$2/4 = 0.50$
{Milk} $\to$ {Diaper, Beer}	2	4	$2/5 = 0.40$	$2/4 = 0.50$

Table 11.2: Rule properties across binary partitions of {Milk, Diaper, Beer}.

1. **Support Invariance**: All rules derived from the same underlying itemset L share identical support, equal to the support of L:

$$s(X \longrightarrow L \setminus X) = s(L) = \frac{\sigma(L)}{|T|} \tag{11.16}$$

   If L is infrequent, none of its derived rules can possibly satisfy the support threshold.

2. **Confidence Variation**: Rule confidence varies widely across partitions, as the denominator $\sigma(X)$ depends solely on the specific antecedent.

11.5.2 Sequential Two-Step Decomposition

The overall mining problem is consequently decoupled into two sequential phases:

1. **Step 1: Frequent Itemset Generation**: Identify all itemsets $L \subseteq I$ satisfying $s(L) \geq \text{minsup}$.

2. **Step 2: Rule Generation**: For each frequent itemset L found in Step 1, generate all binary partitions $X \to L \setminus X$ satisfying $c \geq \text{minconf}$.

Step 1 is the computationally dominant phase of the pipeline.

11.6 Itemset Lattice and Search Space Complexity

Given d items, the candidate itemset search space forms a **Boolean lattice** (the power set $\mathcal{P}(I)$) of cardinality:

$$|\mathcal{P}(I)| = 2^d \tag{11.17}$$

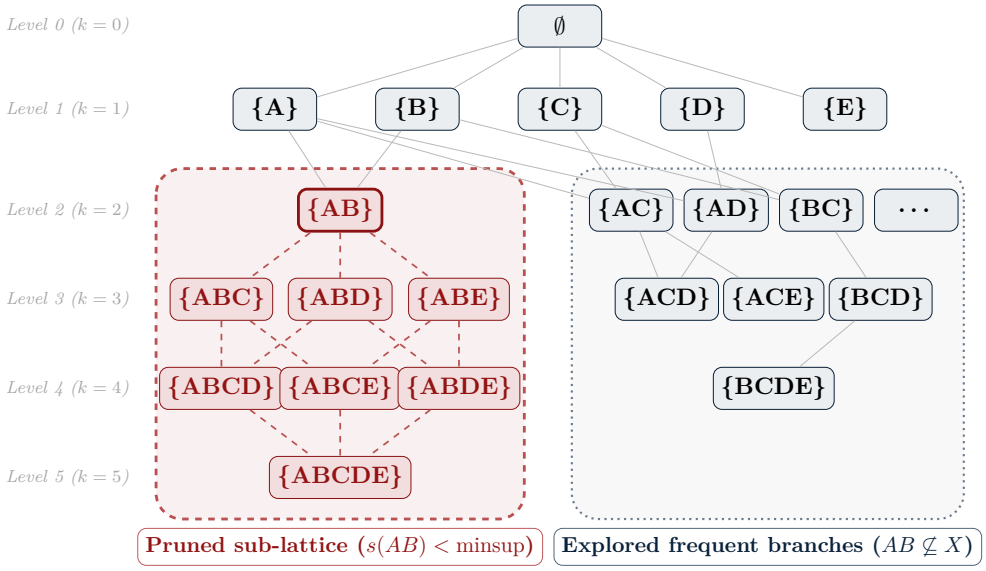


Figure 11.1: Itemset lattice representation for $d = 5$ items. As soon as $\{AB\}$ is found infrequent ($s(AB) < \text{minsup}$), the entire sub-lattice of its $2^{5-2} = 8$ supersets (highlighted on the left in red) is pruned *a priori*, while the remaining branches not containing $\{AB\}$ (on the right) continue to be explored.

11.6.1 Search Complexity and Optimization Strategies

Without pruning, matching all $M = 2^d$ candidate itemsets against N transactions with average length w costs:

$$\mathcal{O}\left(N \times 2^d \times w\right) \tag{11.18}$$

Three orthogonal optimization strategies make itemset generation feasible [2]:

1. **Reduce Candidate Count (M)**: Systematically prune search space branches using anti-monotonicity;
2. **Reduce Transaction Count (N)**: Drop transactions containing fewer items than current candidate size k;
3. **Reduce Comparison Count ($N \times M$)**: Use hash trees to index transactions or candidates efficiently.

11.7 The Apriori Principle and the Anti-Monotonicity Property

Theorem 11.7.1 (Apriori Principle)**.** *If an itemset is frequent, all of its subsets must also be frequent:*

$$s(Y) \geq \text{minsup} \quad \Longrightarrow \quad \forall X \subseteq Y,\ s(X) \geq \text{minsup} \tag{11.19}$$

11.7.1 Proof of Support Anti-Monotonicity

The principle rests on the **anti-monotonicity** of the support function with respect to set inclusion [2].

Theorem 11.7.2 (Anti-monotonicity of Support)**.** *The support function is monotonically non-increasing under set inclusion:*

$$\forall X, Y \subseteq I: \quad (X \subseteq Y) \Longrightarrow s(X) \geq s(Y) \tag{11.20}$$

Proof. Let $T(X) = \{t \in T \mid X \subseteq t\}$ be the transaction set containing itemset X. If $X \subseteq Y$, any transaction containing Y must necessarily contain X, yielding the inclusion:

$$T(Y) \subseteq T(X) \tag{11.21}$$

By cardinality monotonicity on finite sets:

$$|T(Y)| \leq |T(X)| \quad \Longleftrightarrow \quad \sigma(Y) \leq \sigma(X) \tag{11.22}$$

Dividing by positive constant $N = |T|$ gives:

$$\frac{\sigma(Y)}{|T|} \leq \frac{\sigma(X)}{|T|} \quad \Longleftrightarrow \quad s(Y) \leq s(X) \tag{11.23}$$

Thus support cannot increase as itemset size increases. □

11.7.2 Support-Based Pruning via Logical Contraposition

Applying logical contraposition to Theorem 11.7.1 yields the operational pruning rule:

Corollary 11.7.1 (Apriori Pruning Rule)**.** *If an itemset is infrequent, all of its supersets are guaranteed to be infrequent and can be pruned immediately:*

$$s(X) < \text{minsup} \quad \Longrightarrow \quad \forall Y \supseteq X,\ s(Y) \leq s(X) < \text{minsup} \tag{11.24}$$

As shown in Figure 11.1, once $\{AB\}$ is found infrequent, candidates $\{ABC\}$, $\{ABD\}$, $\{ABE\}$, $\{ABCD\}$, $\{ABCE\}$, $\{ABDE\}$, and $\{ABCDE\}$ are immediately pruned without database scans.

11.8 Step-by-Step Execution Trace of the Apriori Algorithm

We trace Apriori execution on Table 11.1 with absolute support threshold:

$$\sigma_{\min} = 3 \quad \left(\text{minsup} = \frac{3}{5} = 0.60\right) \tag{11.25}$$

11.8.1 Iteration 1: Frequent 1-Itemset Generation ($k = 1$)

Scanning the database yields the frequency counts for individual items:

Item (1-itemset)	**Count σ**	**Status relative to $\sigma_{\min} = 3$**
{Bread}	4	Frequent (≥ 3) $\rightarrow$ Retained
{Coke}	2	Infrequent (< 3) $\rightarrow$ **PRUNED**
{Milk}	4	Frequent (≥ 3) $\rightarrow$ Retained
{Beer}	3	Frequent (≥ 3) $\rightarrow$ Retained
{Diaper}	4	Frequent (≥ 3) $\rightarrow$ Retained
{Eggs}	1	Infrequent (< 3) $\rightarrow$ **PRUNED**

Table 11.3: Support counting for candidate set C_1 and discovery of F_1.

The frequent 1-itemset collection is:

$$F_1 = \{\{\text{Bread}\}, \{\text{Milk}\}, \{\text{Beer}\}, \{\text{Diaper}\}\} \tag{11.26}$$

Items Coke and Eggs are permanently discarded.

11.8.2 Iteration 2: 2-Itemset Generation and Validation ($k = 2$)

Forming pairwise combinations from F_1 yields $\binom{4}{2} = 6$ candidates in C_2. Database scanning gives:

The frequent 2-itemset collection is:

$$F_2 = \{\{\text{Bread, Milk}\}, \{\text{Bread, Diaper}\}, \{\text{Milk, Diaper}\}, \{\text{Beer, Diaper}\}\} \tag{11.27}$$

Candidate (2-itemset)	**Count σ**	**Status relative to $\sigma_{\min} = 3$**
{Bread, Milk}	3	Frequent (≥ 3) $\rightarrow$ Retained
{Bread, Beer}	2	Infrequent (< 3) $\rightarrow$ **PRUNED**
{Bread, Diaper}	3	Frequent (≥ 3) $\rightarrow$ Retained
{Milk, Beer}	2	Infrequent (< 3) $\rightarrow$ **PRUNED**
{Milk, Diaper}	3	Frequent (≥ 3) $\rightarrow$ Retained
{Beer, Diaper}	3	Frequent (≥ 3) $\rightarrow$ Retained

Table 11.4: Support counting for candidate set C_2 and discovery of F_2.

11.8.3 Iteration 3: 3-Itemset Generation, Pruning, and Counting ($k = 3$)

1. **Candidate Generation (C_3)**: Combining elements in F_2 yields 4 potential triplets:

$$\begin{aligned} &\{\text{Beer, Diaper, Milk}\}, \quad \{\text{Beer, Bread, Diaper}\}, \\ &\{\text{Bread, Diaper, Milk}\}, \quad \{\text{Beer, Bread, Milk}\} \end{aligned} \tag{11.28}$$

2. **Candidate Pruning**: Verify that every 2-subset of each candidate belongs to F_2:

   - {Beer, Diaper, Milk}: {Beer, Milk} $\notin F_2 \implies$ **PRUNED**;
   - {Beer, Bread, Diaper}: {Beer, Bread} $\notin F_2 \implies$ **PRUNED**;
   - {Beer, Bread, Milk}: {Beer, Bread} $\notin F_2 \implies$ **PRUNED**;
   - {Bread, Diaper, Milk}: all subsets are in $F_2 \implies$ **Survives pruning**.

   Thus, $C_3 = \{\{\text{Bread, Diaper, Milk}\}\}$.

3. **Support Counting**: Scanning database transactions:

$$\sigma(\{\text{Bread, Diaper, Milk}\}) = 2 \tag{11.29}$$

   Because $2 < \sigma_{\min} = 3$, this candidate is infrequent.

Since $F_3 = \emptyset$, Apriori terminates. The complete frequent itemset set is $F = F_1 \cup F_2$.

11.9 Apriori Pseudocode and Candidate Generation Techniques

Algorithm 6 outlines the formal structure of Apriori [14, 2].

Algorithm 6: Apriori Algorithm for Frequent Itemset Mining

Input: Transaction database T, minimum support threshold $\text{minsup} \in (0, 1]$.

Output: Complete frequent itemset collection $F = \bigcup_k F_k$.

$k \leftarrow 1$

$F_1 \leftarrow \{i \in I \mid s(\{i\}) \geq \text{minsup}\}$ `// Generate frequent 1-itemsets`

repeat

  $k \leftarrow k + 1$

  $C_k \leftarrow \text{Candidate-Generation}(F_{k-1})$ `// Common-prefix join`

  $C_k \leftarrow \text{Candidate-Pruning}(C_k, F_{k-1})$ `// Apriori subset pruning`

  foreach *transaction* $t \in T$ **do**

    $C_t \leftarrow \text{Subsets}(C_k, t)$ `// Candidates contained in` t

    foreach *candidate* $c \in C_t$ **do**

      $c.\text{count} \leftarrow c.\text{count} + 1$

    end

  end

  $F_k \leftarrow \{c \in C_k \mid c.\text{count} \geq \lceil \text{minsup} \times |T| \rceil\}$

until $F_k = \emptyset$

return $\bigcup_k F_k$

11.9.1 Common-Prefix Join Method ($F_{k-1} \times F_{k-1}$)

To generate C_k without duplicates, itemset elements are lexicographically ordered [14]. Two itemsets $p, q \in F_{k-1}$ are merged if and only if they share a common $(k-2)$-prefix and differ only in their last element:

$$\begin{cases} p[1] = q[1] \\ p[2] = q[2] \\ \quad \vdots \\ p[k-2] = q[k-2] \\ p[k-1] < q[k-1] \end{cases} \tag{11.30}$$

The merged candidate is:

$$c = \{p[1], p[2], \ldots, p[k-2], p[k-1], q[k-1]\} \tag{11.31}$$

Common-Prefix Execution Example

Given frequent 3-itemset set:

$$F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\} \tag{11.32}$$

Prefix length is $k - 2 = 1$:

- Merge$(ABC, ABD) \rightarrow$ common prefix AB, $C < D \implies ABCD$ generated;
- Merge$(ABC, ABE) \rightarrow$ common prefix AB, $C < E \implies ABCE$ generated;
- Merge$(ABD, ABE) \rightarrow$ common prefix AB, $D < E \implies ABDE$ generated;
- Merge$(ABD, ACD) \rightarrow$ differing length-2 prefixes $(AB \neq AC) \implies$ not merged.

Candidate set:

$$C_4 = \{ABCD, ABCE, ABDE\} \tag{11.33}$$

Candidate Pruning Phase

Checking whether all $\binom{k}{k-1} = 4$ length-3 subsets belong to F_3:

- For $ABCD$: subsets $\{ABC, ABD, ACD, BCD\}$ are all in $F_3 \implies$ **Retained**;
- For $ABCE$: subsets $\{ACE, BCE\} \notin F_3 \implies$ **PRUNED**;
- For $ABDE$: subset $\{ADE\} \notin F_3 \implies$ **PRUNED**.

The pruned candidate set is $C_4 = \{ABCD\}$.

11.9.2 Alternative $F_{k-1} \times F_{k-1}$ Suffix-Prefix Join

Two itemsets $p, q \in F_{k-1}$ are merged if the last $k-2$ elements of p equal the first $k-2$ elements of q:

$$p[2] = q[1], \quad p[3] = q[2], \quad \ldots, \quad p[k-1] = q[k-2] \tag{11.34}$$

Candidate: $c = \{p[1], p[2], \ldots, p[k-1], q[k-1]\}$. Applying this to F_3 yields:

- Merge($A\underline{BC}, \underline{BC}D) \rightarrow ABCD$;
- Merge($A\underline{BD}, \underline{BD}E) \rightarrow ABDE$;
- Merge($A\underline{CD}, \underline{CD}E) \rightarrow ACDE$;
- Merge($B\underline{CD}, \underline{CD}E) \rightarrow BCDE$.

Subsequent subset pruning eliminates $ABDE$, $ACDE$, and $BCDE$, converging to the identical minimal set $C_4 = \{ABCD\}$.

11.10 Support Counting via Hash Tree Structures

During Apriori execution, support counting verifies which candidates in C_k appear in each transaction $t \in T$.

11.10.1 Combinatorial Complexity of Direct Matching

For transaction t, the number of length-k subsets contained within it is:

$$\binom{|t|}{k} \tag{11.35}$$

For $|t| = 20$ items and candidate size $k = 3$, the transaction generates $\binom{20}{3} = 1\,140$ combinations. Direct linear matching against thousands of candidates is computationally prohibitive [2].

11.10.2 Hash Tree Architecture

To minimize comparisons, candidates C_k are stored in a **Hash Tree** [14, 2]:

- **Internal Nodes**: Contain hash tables with partition function $h(i) = i \pmod B$. At tree level j, branching depends on hashing the j-th item of the candidate;
- **Leaf Nodes**: Store buckets of candidate itemsets up to a fixed maximum capacity.

When processing transaction t, the algorithm descends recursively through the tree exploring ordered item subsets. Direct subset matching occurs only against candidates residing in reached leaves, reducing comparison costs substantially [2].

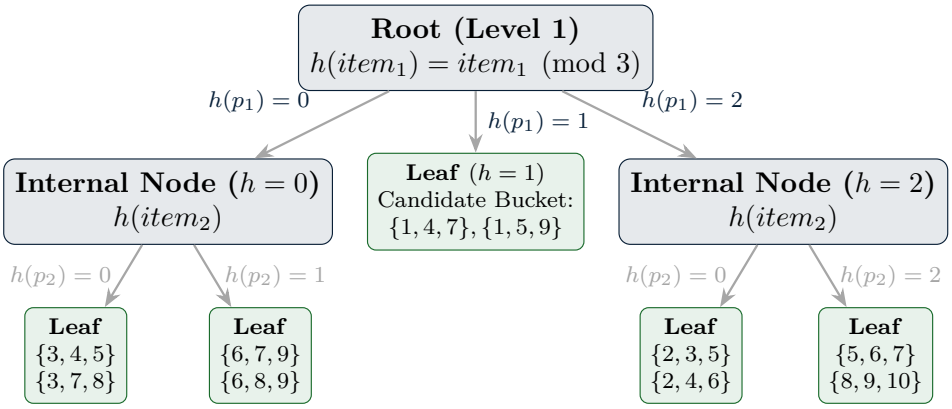


Figure 11.2: Hash tree indexing architecture for efficient candidate support counting.

11.11 Rule Generation from Frequent Itemsets

Following Step 1, Step 2 extracts high-confidence association rules.

11.11.1 Combinatorial Rule Decomposition

For frequent itemset L with cardinality $|L| = k$, the goal is identifying all non-empty subsets $X \subset L$ such that $X \to L \setminus X$ satisfies:

$$c(X \longrightarrow L \setminus X) = \frac{\sigma(L)}{\sigma(X)} \geq \text{minconf} \tag{11.36}$$

Excluding empty antecedent and consequent cases, the number of candidate rules is:

$$2^k - 2 \tag{11.37}$$

For frequent 4-itemset $\{A, B, C, D\}$, $2^4 - 2 = 14$ candidate rules arise:

- **1 item in RHS (4 rules)**: $ABC \to D$, $ABD \to C$, $ACD \to B$, $BCD \to A$;
- **2 items in RHS (6 rules)**: $AB \to CD$, $AC \to BD$, $AD \to BC$, $BC \to AD$, $BD \to AC$, $CD \to AB$;
- **3 items in RHS (4 rules)**: $A \to BCD$, $B \to ACD$, $C \to ABD$, $D \to ABC$.

11.11.2 Confidence Anti-Monotonicity Property

Across different itemsets, confidence is generally not anti-monotone. However, for rules generated from the **same frequent itemset** L, confidence exhibits a key algebraic property [2].

Theorem 11.11.1 (Confidence Anti-Monotonicity on the Rule Lattice)**.** *For rules R_1 and R_2 derived from identical frequent itemset L:*

$$R_1 : X \longrightarrow L \setminus X, \qquad R_2 : X' \longrightarrow L \setminus X' \quad \textit{with } X' \subset X \subset L \tag{11.38}$$

the following inequality holds:

$$c(X \longrightarrow L \setminus X) \geq c(X' \longrightarrow L \setminus X') \tag{11.39}$$

Proof. By confidence definition:

$$c(R_1) = \frac{\sigma(L)}{\sigma(X)}, \qquad c(R_2) = \frac{\sigma(L)}{\sigma(X')} \tag{11.40}$$

Since $X' \subset X$, by support anti-monotonicity (Theorem 11.7.1):

$$\sigma(X') \geq \sigma(X) \tag{11.41}$$

Inverting the ratio with constant numerator $\sigma(L) > 0$:

$$\frac{\sigma(L)}{\sigma(X')} \leq \frac{\sigma(L)}{\sigma(X)} \quad \Longleftrightarrow \quad c(R_2) \leq c(R_1) \tag{11.42}$$

□

Corollary 11.11.1 (Confidence-Based Rule Pruning)**.** *Rule confidence is* ***anti-monotone with respect to the number of items in the consequent*** *(RHS). If a rule fails the* minconf *threshold, any rule derived from the same itemset moving additional items from antecedent to consequent will have even lower confidence and can be pruned:*

$$c(X \longrightarrow Y) < \text{minconf} \quad \Longrightarrow \quad \forall Y' \supset Y,\ c(L \setminus Y' \longrightarrow Y') < \text{minconf} \tag{11.43}$$

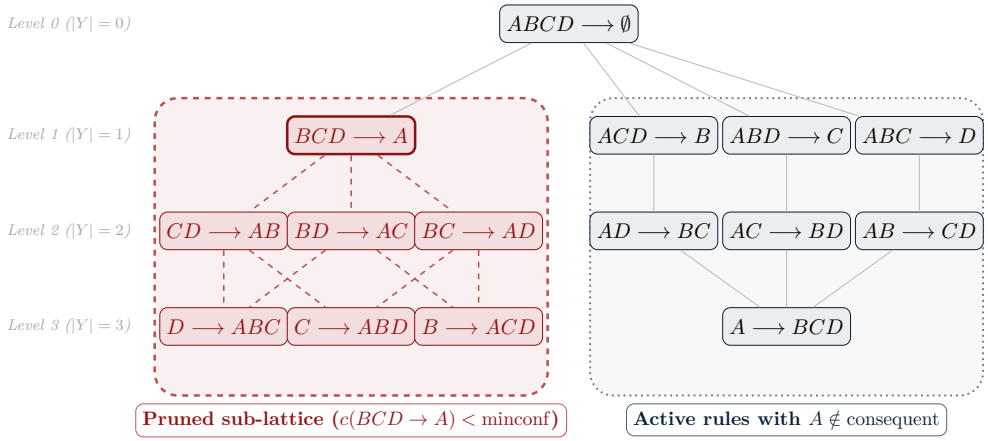


Figure 11.3: Rule lattice generated from $\{A, B, C, D\}$ organized by consequent cardinality ($|Y|$). As soon as rule $BCD \to A$ falls below minconf, all of its descendants (having A in their consequent, highlighted on the left) are pruned *a priori* by confidence anti-monotonicity.

11.12 Factors Governing Computational Complexity

Apriori algorithm scalability is governed by four primary parameters [2]:

1. **Support Threshold** (minsup): Lowering minsup exponentially increases the number of frequent itemsets, maximum pattern length $k_{\max}$, candidate counts, and required database passes;

2. **Item Universe Cardinality** ($d = |I|$): Higher item counts enlarge candidate tracking memory, with level-2 candidate pairs scaling quadratically as $\mathcal{O}(d^2)$;

3. **Database Size** ($N = |T|$): Because each iteration performs a full sequential database scan, computational cost scales linearly with N;

4. **Average Transaction Width** (w): Wider transactions produce longer patterns. Furthermore, the number of candidate subsets per transaction, $\binom{w}{k}$, grows combinatorially with w.

11.13 Compact Representations: Maximal and Closed Frequent Itemsets

Dense datasets generate explosive candidate counts: a frequent itemset of size 20 yields $2^{20}-1 \approx 10^6$ frequent sub-patterns. To eliminate redundancy without loss of essential information, two compact representations are employed: **maximal itemsets** and **closed itemsets** [2].

11.13.1 Maximal Frequent Itemsets

Definition 11.13.1 (Maximal Frequent Itemset)**.** An itemset X is **maximal frequent** if $s(X) \geq \text{minsup}$ and **no immediate superset of** X is frequent:

$$X \text{ is maximal frequent} \iff s(X) \geq \text{minsup} \quad \wedge \quad \forall Y \supset X,\ s(Y) < \text{minsup} \tag{11.44}$$

Maximal frequent itemsets delineate the boundary between frequent and infrequent space in the lattice. However, while they identify all frequent itemsets, they **do not retain exact subset support values**, representing a lossy compression.

11.13.2 Closed and Closed Frequent Itemsets

Definition 11.13.2 (Closed Itemset)**.** An itemset X is **closed** if none of its immediate supersets has the identical support count as X:

$$X \text{ is closed} \iff \forall Y \supset X,\ \sigma(Y) < \sigma(X) \tag{11.45}$$

Equivalently, X is not closed if there exists an immediate superset $Y \supset X$ such that $\sigma(Y) = \sigma(X)$.

Definition 11.13.3 (Closed Frequent Itemset)**.** An itemset is **closed frequent** if it is both closed and frequent ($s(X) \geq \text{minsup}$).

Theorem 11.13.1 (Lossless Compression of Closed Frequent Itemsets)**.** *The collection of closed frequent itemsets forms a* ***lossless compact representation****. The exact support count of any arbitrary frequent itemset S can be reconstructed without consulting the database:*

$$\sigma(S) = \max\left\{\sigma(C) \mid C \textit{ is closed frequent and } S \subseteq C\right\} \tag{11.46}$$

TID	Transaction
1	$\{A, B\}$
2	$\{B, C, D\}$
3	$\{A, B, C, D\}$
4	$\{A, B, D\}$
5	$\{A, B, C, D\}$

Table 11.5: Transaction database for diagnosing closed and maximal itemsets.

11.13.3 Analytical Case Study on Tabular Data

Consider the transactional database in Table 11.5:

Evaluating closure:

1. Itemset $\{A\}$: occurs in TID 1, 3, 4, 5 $\implies \sigma(\{A\}) = 4$. Immediate superset $\{A, B\}$ also has $\sigma(\{A, B\}) = 4$. Because support matches, $\{A\}$ is **NOT closed**;

2. Itemset $\{B\}$: occurs in all 5 transactions $\implies \sigma(\{B\}) = 5$. Immediate supersets have supports $\sigma(\{A, B\}) = 4$, $\sigma(\{B, C\}) = 3$, $\sigma(\{B, D\}) = 4$. Since no superset has support 5, $\{B\}$ is **CLOSED**;

3. Itemset $\{A, B\}$: $\sigma(\{A, B\}) = 4$. Immediate supersets have supports $\sigma(\{A, B, C\}) = 2$ and $\sigma(\{A, B, D\}) = 3$. Since both are strictly less than 4, $\{A, B\}$ is **CLOSED**;

4. Itemset $\{C\}$: $\sigma(\{C\}) = 3$. Superset $\{B, C\}$ has $\sigma(\{B, C\}) = 3$. Thus $\{C\}$ is **NOT closed**;

5. Itemset $\{A, B, C, D\}$: $\sigma(\{A, B, C, D\}) = 2$. No supersets exist with support ≥ 2, hence $\{A, B, C, D\}$ is **CLOSED**.

11.13.4 Set Inclusion Relationships

The structural hierarchy between itemset classes is shown in Figure 11.4:

$$\text{Maximal Frequent} \subseteq \text{Closed Frequent} \subseteq \text{Frequent Itemsets} \tag{11.47}$$

$$\text{Closed Frequent} = \text{Frequent} \cap \text{Closed} \tag{11.48}$$

Every maximal frequent itemset is necessarily closed frequent, but the converse is not generally true.

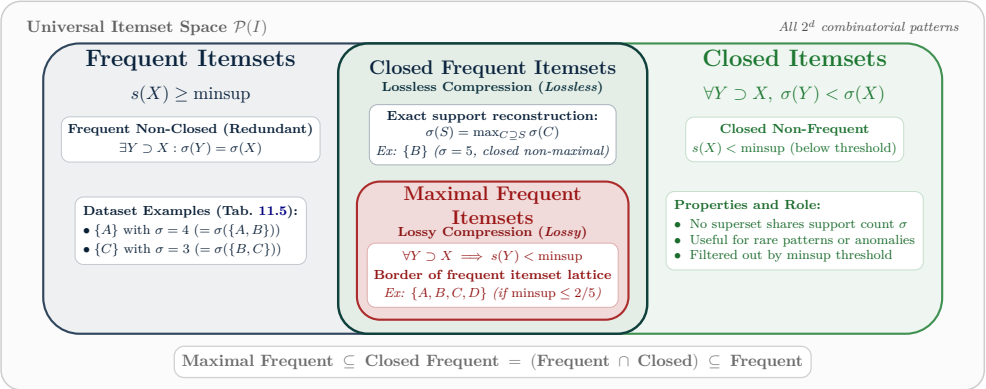


Figure 11.4: Euler-Venn diagram illustrating inclusion relationships between frequent, closed, and maximal itemsets, highlighting lossless vs lossy compression characteristics.

11.14 Pattern Evaluation and Limitations of Confidence

Although support and confidence are widely adopted, confidence exhibits severe interpretive flaws because it **completely ignores the marginal support of the consequent** $P(Y)$ [2].

11.14.1 2×2 Contingency Tables

The bivariate statistical relationship between itemsets X and Y is captured in a 2×2 contingency table:

	Y	$\overline{Y}$ (No Y)	**Marginal Total**
X	$f_{11} = \sigma(X \cup Y)$	$f_{10} = \sigma(X \cup \overline{Y})$	$f_{1+} = \sigma(X)$
$\overline{X}$ (No X)	$f_{01} = \sigma(\overline{X} \cup Y)$	$f_{00} = \sigma(\overline{X} \cup \overline{Y})$	$f_{0+} = \sigma(\overline{X})$
Marginal Total	$f_{+1} = \sigma(Y)$	$f_{+0} = \sigma(\overline{Y})$	$N = \lvert T \rvert$

Table 11.6: 2×2 contingency table for joint association analysis.

In cell frequency terms:

$$s(X \longrightarrow Y) = \frac{f_{11}}{N}, \qquad c(X \longrightarrow Y) = \frac{f_{11}}{f_{1+}} \tag{11.49}$$

11.14.2 Case Study 1: High Confidence with Negative Correlation

Consider tea and coffee consumption across $N = 1\,000$ consumers:

	Coffee	$\overline{\text{Coffee}}$	**Total**
Tea	150	50	200
$\overline{\text{Tea}}$	650	150	800
Total	800	200	1 000

Table 11.7: Contingency table between Tea and Coffee consumption.

Evaluating rule Tea $\longrightarrow$ Coffee:

- Confidence:

$$c(\text{Tea} \longrightarrow \text{Coffee}) = P(\text{Coffee} \mid \text{Tea}) = \frac{150}{200} = 0.75 \quad (75\%) \tag{11.50}$$

  At 75% confidence, the rule appears strong.

- Marginal coffee probability:

$$P(\text{Coffee}) = \frac{800}{1\,000} = 0.80 \quad (80\%) \tag{11.51}$$

- Probability among non-tea drinkers:

$$P(\text{Coffee} \mid \overline{\text{Tea}}) = \frac{650}{800} = 0.8125 \quad (81.25\%) \tag{11.52}$$

Critical Paradox Interpretation

In the overall customer base, coffee consumption is 80%. Among tea drinkers, it drops to 75%. Knowing someone drinks tea actually **decreases** the probability of them drinking coffee.
The rule falsely implies positive association when Tea and Coffee are in fact **negatively correlated** (substitute goods).

	Honey	$\overline{\text{Honey}}$	**Total**
Tea	100	100	200
$\overline{\text{Tea}}$	20	780	800
Total	120	880	1 000

Table 11.8: Contingency table between Tea and Honey consumption.

11.14.3 Case Study 2: Low Confidence with Strong Positive Correlation

Consider tea and honey consumption across $N = 1\,000$ customers:

Evaluating rule Tea $\longrightarrow$ Honey:

$$c(\text{Tea} \longrightarrow \text{Honey}) = \frac{100}{200} = 0.50 \quad (50\%) \tag{11.53}$$

With minconf $= 0.60$, this rule is rejected. Yet baseline honey purchasing is only:

$$P(\text{Honey}) = \frac{120}{1\,000} = 0.12 \quad (12\%) \tag{11.54}$$

Drinking tea increases honey purchasing probability from 12% to 50% (over fourfold). The strong positive association is masked by fixed confidence filtering.

11.14.4 Statistical Dependence Condition

Genuine association requires satisfying:

$$c(X \longrightarrow Y) > s(Y) \quad \Longleftrightarrow \quad P(Y \mid X) > P(Y) \tag{11.55}$$

- $P(Y \mid X) = P(Y)$: statistical independence ($P(X \cap Y) = P(X)P(Y)$);
- $P(Y \mid X) > P(Y)$: positive correlation;
- $P(Y \mid X) < P(Y)$: negative correlation.

11.15 Objective Statistical Measures of Interest

To complement support and confidence, objective statistical measures quantify deviation from statistical independence [2, 15].

11.15.1 Lift and Interest Metrics

Lift (or **Interest**) measures the ratio of observed joint probability to the probability expected under stochastic independence:

$$\text{Lift}(X \longrightarrow Y) = \frac{P(Y \mid X)}{P(Y)} = \frac{c(X \longrightarrow Y)}{s(Y)} = \frac{P(X,Y)}{P(X)P(Y)} = \frac{Nf_{11}}{f_{1+}f_{+1}} \tag{11.56}$$

- Lift $= 1$: X and Y are independent;
- Lift > 1: X and Y are positively correlated;
- Lift < 1: X and Y are negatively correlated.

Checking Lift for the Tea and Coffee case (Table 11.7):

$$\text{Lift}(\text{Tea} \longrightarrow \text{Coffee}) = \frac{0.15}{0.20 \times 0.80} = \frac{0.15}{0.16} = 0.9375 < 1 \tag{11.57}$$

Lift < 1 correctly flags negative correlation, rectifying the misleading 75% confidence.

11.15.2 Piatetsky-Shapiro Measure and Pearson's ϕ

- **Piatetsky-Shapiro (PS / Leverage)**: Computes the difference between observed joint probability and expected probability under independence [15]:

$$PS = P(X,Y) - P(X)P(Y) = \frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2} \tag{11.58}$$

- **Pearson Correlation Coefficient (ϕ)**: Evaluates correlation across binary variables:

$$\phi = \frac{P(X,Y) - P(X)P(Y)}{\sqrt{P(X)(1-P(X))P(Y)(1-P(Y))}} = \frac{Nf_{11} - f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}} \tag{11.59}$$

  Bounded in $[-1, +1]$, with 0 indicating independence.

11.15.3 Comparative Summary of Objective Measures

Table 11.9 summarizes major objective association measures [2].

Measure	Symbol	2×2 Contingency Table Formulation
Correlation (ϕ)	ϕ	$\frac{Nf_{11} - f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}$
Odds Ratio	α	$\frac{f_{11}f_{00}}{f_{10}f_{01}}$
Cohen's Kappa	κ	$\frac{Nf_{11} + Nf_{00} - f_{1+}f_{+1} - f_{0+}f_{+0}}{N^2 - f_{1+}f_{+1} - f_{0+}f_{+0}}$
Interest / Lift	I	$\frac{Nf_{11}}{f_{1+}f_{+1}}$
Cosine / IS	IS	$\frac{f_{11}}{\sqrt{f_{1+}f_{+1}}}$
Piatetsky-Shapiro	PS	$\frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2}$
Jaccard	J	$\frac{f_{11}}{f_{1+} + f_{+1} - f_{11}}$
All-Confidence	h	$\min\left[\frac{f_{11}}{f_{1+}}, \frac{f_{11}}{f_{+1}}\right]$

Table 11.9: Analytical formulations of objective statistical interest measures.

11.16 Handling Categorical and Continuous Attributes

Real-world transaction datasets are not confined to symmetric boolean flags, frequently incorporating polytomous categorical features and continuous numeric attributes [2]. To integrate these dimensions into association mining, the binarization and discretization methodologies formalized in Chapter 4 (section 4.6) are applied, tailored to the requirements of the Apriori framework.

11.16.1 Polytomous Categorical Attributes

Discrete attributes (e.g., `Education` $\in$ {High School, College, Graduate}) undergo **One-Hot Encoding** (section 4.6):

- Distinct binary items are created for each pair (`Education=Graduate`);
- Transactions become sparse asymmetric binary vectors where absence (0) is uninformative and excluded from co-occurrence calculations.

The High-Cardinality Challenge

Categorical attributes with numerous categories (e.g., geographic zip codes) dilute individual category supports.

- **Effect**: Under standard minsup, almost no category survives pruning;
- **Mitigation**: Apply **conceptual roll-up**, aggregating fine-grained values into higher-level macro-categories (e.g., zip codes into regions).

11.16.2 Continuous Numeric Attributes

For continuous variables (e.g., age, income, online hours), three paradigms exist:

1. **Discretization-Based Methods**: Domain values are segmented into discrete bins (equal-width or equal-frequency, cf. section 4.6), each treated as a discrete item:

$$\{\text{Gender} = \text{M}, \text{Age} \in [21, 30)\} \longrightarrow \{\text{Online Hours} \geq 10\} \tag{11.60}$$

2. **Statistics-Based Methods**: Antecedents contain discrete items, while the consequent summarizes continuous distributions via mean μ and standard deviation σ:

$$\{\text{Age} \in [21, 30), \text{Chat} = \text{Yes}\} \longrightarrow \text{Hours: } [\mu = 14,\ \sigma = 4] \tag{11.61}$$

3. **Non-Discretization Methods (*minApriori / Fuzzy*)**: Continuous attributes normalized in $[0, 1]$ are mined directly using fuzzy operators (t-norms) replacing boolean set intersections.

11.17 Concept Taxonomies and Multi-Level Association Rules

Commercial items naturally organize into taxonomies (trees or DAGs), where specific leaf items aggregate into broader categories [2].

11.17.1 Motivations for Multi-Level Mining

1. **Insufficient Support at Leaf Levels**: Highly specific items (e.g., specialized organic milk brands) sell infrequently, failing minsup and facing premature pruning;
2. **Fragmentation at Leaf Levels**: Leaf-level mining creates fragmented rules across minor brand variants, obscuring general category associations;
3. **Overly Broad Rules at Top Levels**: Root-level rules (Food $\rightarrow$ Electronics) are often trivial or uninformative.

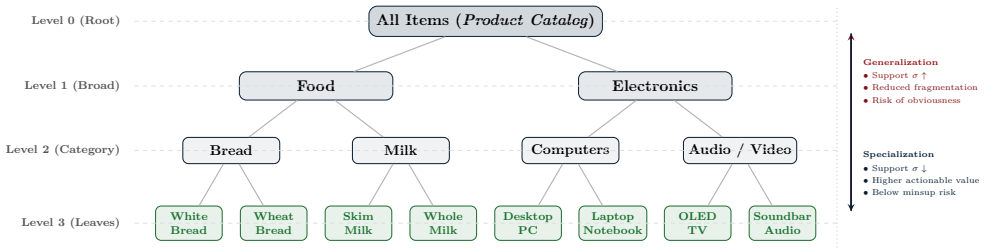


Figure 11.5: Multi-level conceptual taxonomy for hierarchical association rule mining, highlighting taxonomy levels and the generalization vs. specialization trade-off.

11.17.2 Support and Confidence Behavior Across Taxonomies

Let X_1 and X_2 be lower-level items sharing parent category X:

1. **Parent Support Sub-Additivity**: Parent transaction frequency is bounded above by child count sums:

$$\sigma(X) \leq \sigma(X_1) + \sigma(X_2) \tag{11.62}$$

   The inequality arises because a single transaction can contain both X_1 and X_2.

2. **Upward Support Propagation**: If a lower-level itemset is frequent, replacing any item with its taxonomic ancestor yields a frequent itemset:

$$\sigma(X_1 \cup Y_1) \geq \text{minsup} \times |T| \quad \Longrightarrow \quad \sigma(X \cup Y) \geq \text{minsup} \times |T| \tag{11.63}$$

3. **Confidence Propagation**: If a specific rule satisfies minimum confidence, generalizing the consequent also satisfies it:

$$c(X_1 \longrightarrow Y_1) \geq \text{minconf} \quad \Longrightarrow \quad c(X_1 \longrightarrow Y) \geq \text{minconf} \tag{11.64}$$

11.17.3 Multi-Level Implementation Strategies

- **Strategy 1: Transaction Augmentation**: Transactions are enriched by appending all taxonomic ancestors:

$$\begin{aligned} &\{\text{Skim Milk, Wheat Bread}\} \longrightarrow \\ &\quad \{\text{Skim Milk, Wheat Bread, Milk, Bread, Food}\} \end{aligned} \tag{11.65}$$

  Drawbacks: Significantly increases transaction width and database size, causing support explosion and trivial redundant rules (Skim Milk $\rightarrow$ Milk).

- **Strategy 2: Level-by-Level Generation**: Mine frequent itemsets at the top level first, then descend restricting child searches to descendants of frequent parent patterns. *Drawbacks*: Requires multiple database scans and misses cross-level associations without specialized traversals.

Chapter 12

Sequential Pattern Mining: Models, Definitions, and Exercises

In classic data mining, frequent pattern mining and market-basket analysis focus on discovering sets of items (*itemsets*) that co-occur within the same transaction [2, 14]. However, across numerous real-world operational domains and information systems, observations do not naturally present themselves as isolated, time-agnostic transactions, but rather as ordered streams of events distributed along a continuous or discrete timeline [16, 17].

Sequential Pattern Mining represents the foundational algorithmic extension developed to bridge this gap: it captures ordering, causal precedence, and temporal dependency relationships among collections of events by formalizing the concept of a transaction sequence over time [2, 16].

This chapter details the paradigm shift from time-invariant itemsets to sequential structures, the taxonomy of sequence data across diverse domains, the formal algebraic definitions of sequences, length, k-sequences, and subsequence containment, and the evaluation of support alongside multi-instance occurrence management (*embeddings*), culminating in fully worked analytical exercises.

12.1 Introduction and Motivations

In traditional association analysis, each record in the database represents an independent consumption event, and the order of items inside a basket holds neither semantic nor algorithmic significance [2]. In reality, however, human users, physical devices, and corporate workflows interact dynamically and

cumulatively over time.

12.1.1 Real-World Sequence Data Examples

The prevalence of sequential data is apparent across multiple heterogeneous domains:

1. **Consumer E-commerce Purchase Timelines**: Consider the purchase trajectory of a retail consumer spanning several months:

$$\langle \{\text{Digital Camera, iPad}\}\ \{\text{memory card}\}\ \{\text{headphone, cover}\} \rangle \quad (12.1)$$

   In this sequence:

   - At initial time t_1, the customer simultaneously purchases two high-value technological assets: {Digital Camera, iPad}.
   - At a subsequent time t_2, the customer acquires a compatible functional accessory: {memory card}.
   - Later at time t_3, the customer completes the cycle by purchasing accessories: {headphone, cover}.

   This scenario exemplifies both concurrent co-occurrence within the same shopping basket and delayed temporal dependencies across distinct shopping episodes.

2. **Three Mile Island Incident Cascading Events (1979)**: In industrial telemonitoring and fault diagnosis systems, events correspond to physical faults, sensor triggers, and alarm logs rather than commercial purchases:

$$\begin{aligned} &\langle \{\text{clogged resin}\}\ \{\text{outlet valve closure}\} \\ &\ \{\text{loss of feedwater}\}\ \{\text{condenser polisher valve shut}\} \\ &\ \{\text{booster pumps trip}\}\ \{\text{main waterpump trips}\} \\ &\ \{\text{main turbine trips}\}\ \{\text{reactor pressure increases}\}\rangle \end{aligned} \quad (12.2)$$

   The Three Mile Island nuclear incident evolved through a cascading chain of thermohydraulic and electromechanical failures (resin blockage, valve shutoffs, pump trips, turbine shutdown, and rapid critical pressurization of the reactor). Mining sequential patterns from telemetry logs enables predictive maintenance and timely automated interlocks before failures become catastrophic.

3. **Library Book Borrowing Trajectories**: Consider the borrowing history of a patron:

$$\langle \{\text{Fellowship of the Ring}\}\, \{\text{The Two Towers}\}\, \{\text{Return of the King}\} \rangle \tag{12.3}$$

Here the ordering is dictated by narrative and logical dependencies: the borrower reads the trilogy in its natural chronological order.

Fundamental Distinction: Instantaneous Co-occurrence vs Temporal Dependency

While standard association rules (Chapter 11) capture items purchased simultaneously within the same basket (e.g., $\{\text{Diapers}\} \rightarrow \{\text{Beer}\}$), **Sequential Pattern Mining** identifies delayed chronological progressions across distinct time steps:

$$\langle \{\text{Core Hardware}\} \rangle \xrightarrow{\Delta t} \langle \{\text{Accessories}\} \rangle \xrightarrow{\Delta t'} \langle \{\text{Maintenance}\} \rangle$$

12.1.2 Application Domains of Sequential Pattern Mining

Sequential pattern mining is widely utilized across multiple fields:

- **Retail and Dynamic Basket Analytics**: Investigating shifts in customer preferences, analyzing Customer Lifetime Value, and detecting early signs of churn.
- **Web Server Logs and Clickstream Analytics**: Tracing user navigational trajectories to optimize site layouts, personalize UX, and reduce checkout drop-off rates.
- **Social Media and Information Diffusion**: Modeling tweet cascades, retweet chains, and viral spread dynamics.
- **Process Mining and Industrial Workflows**: Monitoring stage progressions along assembly lines or enterprise ERP workflows to isolate bottlenecks and process deviations.
- **Bioinformatics and Computational Genomics**: Discovering conserved functional motifs and regulatory sequences within DNA, RNA, or protein primary structures.

12.2 From the Transactional Paradigm to the Sequential Paradigm

To understand the necessity of sequential algorithms, it is helpful to revisit the assumptions of traditional frequent itemset mining when applied to time-distributed transactions [2].

12.2.1 Recap: Frequent Itemsets and Market-Basket Analysis

In the classic framework (Chapter 11), given a finite universe of distinct items $I = \{i_1, i_2, \ldots, i_m\}$ and a database of transactions $D = \{T_1, T_2, \ldots, T_N\}$ with each $T_j \subseteq I$, the support of an itemset $X \subseteq I$ is defined as the fraction of transactions containing X:

$$\sigma(X) = \frac{\left|\{T_j \in D \mid X \subseteq T_j\}\right|}{|D|} \tag{12.4}$$

An itemset is declared frequent if $\sigma(X) \geq \text{minsup}$ (or if its raw count reaches Min_freq). In this model, transactions are treated as independent, unordered points in time.

Example 12.2.1 (Itemset Frequency on a 7-Transaction Database)**.** Consider the toy transactional database composed of $N = 7$ transactions shown in Table 12.1, evaluated with an absolute threshold Min_freq $= 5$ (minsup $= 5/7 \approx 71.4\%$).

TID	Steak	Pepper	Chicken	Fish	Ham	Bread	Sausages
1	1	1	1	1	0	0	0
2	1	1	1	1	0	0	1
3	1	1	1	1	1	1	1
4	1	1	1	1	1	1	0
5	0	0	1	1	1	1	0
6	0	0	1	1	1	1	0
7	1	0	0	0	1	0	0

Table 12.1: Toy binary transactional database ($N = 7$ transactions).

Evaluating the absolute occurrence counts for individual columns:

- **Steak**: appears in $\{1, 2, 3, 4, 7\} \implies \text{freq} = 5 \geq 5$ **(Frequent)**;
- **Pepper**: appears in $\{1, 2, 3, 4\} \implies \text{freq} = 4 < 5$ (Discarded);
- **Chicken**: appears in $\{1, 2, 3, 4, 5, 6\} \implies \text{freq} = 6 \geq 5$ **(Frequent)**;
- **Fish**: appears in $\{1, 2, 3, 4, 5, 6\} \implies \text{freq} = 6 \geq 5$ **(Frequent)**;
- **Ham**: appears in $\{3, 4, 5, 6, 7\} \implies \text{freq} = 5 \geq 5$ **(Frequent)**;
- **Bread**: appears in $\{3, 4, 5, 6\} \implies \text{freq} = 4 < 5$ (Discarded);
- **Sausages**: appears in $\{2, 3\} \implies \text{freq} = 2 < 5$ (Discarded).

Testing 2-itemset candidates, the pair {Chicken, Fish} occurs together in rows $\{1, 2, 3, 4, 5, 6\}$, giving an absolute frequency of $6 \geq 5$.

The complete set of frequent itemsets for Min_freq $= 5$ is:

$$\{\text{Steak}\}, \quad \{\text{Chicken}\}, \quad \{\text{Fish}\}, \quad \{\text{Ham}\}, \quad \{\text{Chicken, Fish}\} \tag{12.5}$$

12.2.2 Transition to Complex Multi-Basket Domains

In modern enterprise architectures, each customer generates a sequential stream of transactions over time. The historical record for an entity is therefore represented as a **temporal timeline of shopping carts**, where each cart may contain multiple simultaneous items.

Consider a database containing four customer accounts, whose sequential purchase histories are depicted in Figure 12.1.

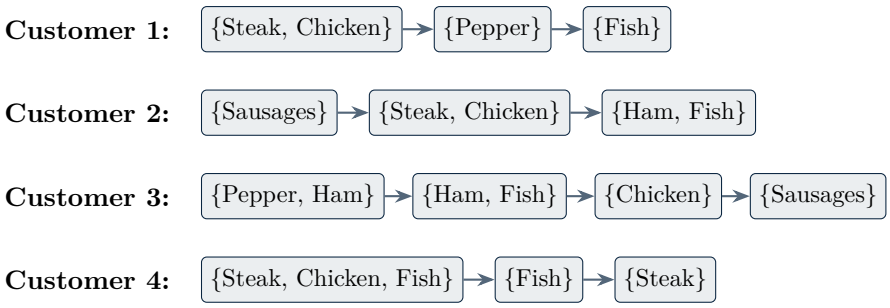


Figure 12.1: Temporal purchase histories for four customers in a multi-basket environment.

12.2.3 Objective of Sequential Pattern Mining

The overarching goal is to extract ordered temporal subsequences that recur across a substantial fraction of customer sequences, exceeding a user-defined threshold minsup.

Inspecting Figure 12.1, consider the candidate ordered sequence:

$$w = \langle \{\text{Steak, Chicken}\}\, \{\text{Fish}\} \rangle \tag{12.6}$$

This candidate dictates that a customer buys Steak and Chicken concurrently, and at a **later** point in time purchases Fish. Checking all four customers:

- **Customer 1**: purchases {Steak, Chicken} at cart 1, followed by {Fish} at cart 3 $\implies$ **Match**;
- **Customer 2**: purchases {Steak, Chicken} at cart 2, followed by cart {Ham, Fish} (containing Fish) at cart 3 $\implies$ **Match**;
- **Customer 3**: purchases Pepper, Ham, Fish, then Chicken. Steak and Chicken are never bought together prior to Fish $\implies$ **No Match**;
- **Customer 4**: purchases {Steak, Chicken, Fish} at cart 1 (which contains {Steak, Chicken}), followed by {Fish} at cart 2 $\implies$ **Match**.

The candidate pattern $w = \langle \{\text{Steak, Chicken}\}\, \{\text{Fish}\} \rangle$ is supported by 3 out of 4 customers:

$$\sigma(w) = \frac{3}{4} = 75\% \tag{12.7}$$

This captures a recurrent chronological consumption habit that isolated single-basket mining cannot detect.

12.2.4 Predictive Sequential Rules in Industrial and Commercial Systems

From discovered frequent sequential patterns, high-confidence predictive **sequential rules** $S_1 \implies S_2$ can be derived:

- **Telecommunication Networks (Alarm Management)**:

$$\langle \{\text{Line_Current}\} \rangle\, \langle \{\text{Rectifier_Alarm}\} \rangle \implies \langle \{\text{Fire_Alarm}\} \rangle \tag{12.8}$$

  In telecommunication facilities, abnormal line draw followed by rectifier thermal warnings accurately forewarns of facility fires, allowing automated load disconnection before catastrophic ignition.

- **Technical Computer Bookstore (Sequential Cross-Selling)**:

$$\begin{aligned} \langle \{\text{Intro_Visual_C}\} \rangle\, \langle \{\text{C++_Primer}\} \rangle \implies \\ \langle \{\text{Perl_for_dummies, Tcl_Tk}\} \rangle \end{aligned} \tag{12.9}$$

  Readers mastering foundational systems programming naturally transition toward system automation and scripting languages, enabling tailored post-purchase recommendations.

- **Athletic Apparel Store (Equipment Lifecycle)**:

$$\langle \{\text{Shoes}\} \rangle \langle \{\text{Racket, Racketball}\} \rangle \Longrightarrow \langle \{\text{Sports_Jacket}\} \rangle \quad (12.10)$$

  An initial footwear purchase followed by specialized court equipment predictably leads to protective outerwear purchases.

12.3 Taxonomy and Sequence Representation

Sequential pattern mining relies on a unified mathematical abstraction that applies identically across disparate operational domains [2].

12.3.1 Terminology Mapping Across Domains

Table 12.2 formalizes the conceptual correspondence between theoretical abstractions and concrete real-world contexts.

12.3.2 Tabular and Timeline Representations

In raw storage systems, sequential records are represented as tuples composed of an entity identifier (`Object`), a timestamp (`Timestamp`), and observed occurrences (`Events`).

Consider the raw logs listed in Table 12.3:

In Figure 12.2, these recordings are plotted along a continuous time axis from $t = 10$ to $t = 30$.

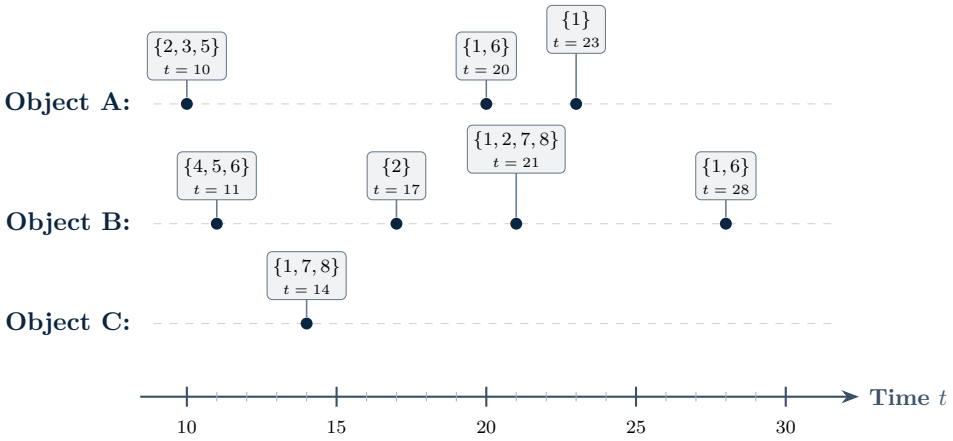


Figure 12.2: Continuous timeline projection of events for Objects A, B, and C.

Inspection of Figure 12.2 reveals two key properties:

Application Domain	**Sequence Database (D)**	**Sequence (s)**	**Element / Transaction (e_j)**	**Event / Item (i)**
Customer Retail	Complete customer transaction database	Full lifetime purchase history of one customer	Set of items purchased simultaneously at time t	Single distinct product (e.g., Milk, Book)
Web Clickstream	HTTP Web server access log repository	Navigational trajectory of a single visitor session	Web pages or resources retrieved upon a click	Individual URL, file, or script requested
Sensor Networks	Centralized telemetry archive	Historical time series recorded by a specific sensor	Set of alarms or status codes recorded at time t	Specific alarm type or threshold alert
Genomics / DNA	Molecular sequence database (e.g., GenBank)	Complete DNA strand for a biological organism	Specific position along the molecular sequence	Individual nucleotide base (A, C, G, T)

Table 12.2: Formal terminology mapping across primary application domains.

Object	**Timestamp (t)**	**Recorded Events**
A	10	$\{2, 3, 5\}$
	20	$\{1, 6\}$
	23	$\{1\}$
B	11	$\{4, 5, 6\}$
	17	$\{2\}$
	21	$\{1, 2, 7, 8\}$
	28	$\{1, 6\}$
C	14	$\{1, 7, 8\}$

Table 12.3: Raw tabular event logs for three distinct entities.

1. **Heterogeneous Time Intervals (Δt)**: The elapsed time between consecutive transactions is non-uniform. For Object B, $\Delta t_{1\to 2} = 17 - 11 = 6$, whereas $\Delta t_{2\to 3} = 21 - 17 = 4$, and $\Delta t_{3\to 4} = 28 - 21 = 7$.

2. **Cross-Object Asynchrony**: Events across independent objects do not align along synchronized clock ticks.

12.4 Formalization of Sequence Data

We formalize the core constructs of sequence mining mathematically [2, 16].

Definition 12.4.1 (Item and Universe Set)**.** Let $I = \{i_1, i_2, \ldots, i_m\}$ denote a finite set of distinct elements, called **items** or **events**.

Definition 12.4.2 (Element or Transaction)**.** An **element** (or transaction) $e \subseteq I$ is a non-empty subset of items:

$$e = \{i_{j_1}, i_{j_2}, \ldots, i_{j_k}\} \quad \text{with } e \subseteq I,\ e \neq \emptyset \tag{12.11}$$

Items belonging to the same element are enclosed in curly braces $\{\}$ and are unordered, signifying simultaneous co-occurrence.

Definition 12.4.3 (Sequence)**.** A **sequence** s is an ordered list of elements, enclosed within angle brackets $\langle\ \rangle$:

$$s = \langle e_1, e_2, \ldots, e_p \rangle \tag{12.12}$$

where each $e_j \subseteq I$ is an element. Positional indices denote chronological ordering such that event timestamps satisfy $t(e_j) < t(e_{j+1})$ for all $1 \leq j < p$.

Definition 12.4.4 (Sequence Length)**.** The **length** of a sequence s, denoted by $|s|$, is strictly defined as the total number of elements (distinct time steps) it contains:

$$|s| = p \tag{12.13}$$

Definition 12.4.5 (k-Sequence)**.** A sequence $s = \langle e_1, e_2, \ldots, e_p \rangle$ is called a **k-sequence** if the total number of items contained across all its constituent elements equals k:

$$k = \sum_{j=1}^{p} |e_j| \tag{12.14}$$

Critical Distinction: Sequence Length vs k-Value

Avoid conflating **length** $|s|$ with the size parameter k:

- **Length** $|s| = p$ counts the number of **discrete temporal steps** (baskets/elements);
- Parameter k sums the total volume of individual **items** across all baskets.

Example 12.4.1 (Calculating Length and k for a Sequence)**.** Consider the sequence:

$$s = \langle \{A, B\} \{B, E, F\} \{A\} \{E, F, H\} \rangle \tag{12.15}$$

- **Constituent elements**:

  $$e_1 = \{A, B\}, \quad e_2 = \{B, E, F\}, \quad e_3 = \{A\}, \quad e_4 = \{E, F, H\}$$

  The sequence contains $p = 4$ ordered elements $\implies$ **Length** $|s| = 4$.

- **Element cardinalities**:

  $$|e_1| = 2, \quad |e_2| = 3, \quad |e_3| = 1, \quad |e_4| = 3$$

  Summing item counts:

  $$k = |e_1| + |e_2| + |e_3| + |e_4| = 2 + 3 + 1 + 3 = 9$$

  Hence, s is a **9-sequence of length 4**.

- **Associated timestamps**: If elements occur at physical timestamps $t(e_1) = 0$, $t(e_2) = 120$, $t(e_3) = 130$, and $t(e_4) = 200$, intervals are heterogeneous: $\Delta t_{1\to2} = 120$, $\Delta t_{2\to3} = 10$, $\Delta t_{3\to4} = 70$.

12.4.1 Sequences Without Explicit Timestamps

When physical clock timestamps are omitted, standard ordinal indexing (zero-based or 1-based) is adopted as the implicit temporal dimension:

$$t(e_j) = j - 1 \quad \text{for } j = 1, 2, \ldots, p \tag{12.16}$$

For example, given:

$$s = \langle \{A, C\} \{E\} \{A, F\} \{E, G, H\} \rangle \tag{12.17}$$

the default assigned timestamps are:

$$t(\{A, C\}) = 0, \quad t(\{E\}) = 1, \quad t(\{A, F\}) = 2, \quad t(\{E, G, H\}) = 3$$

12.4.2 Singleton Sequences vs Complex Element Sequences

A fundamental categorization concerns the cardinality of elements e_j:

- **Singleton Element Sequences ($|e_j| = 1 \; \forall j$)**: Every time step comprises exactly one item. Here length strictly equals k ($|s| = k$).
  - *Web Clickstream*:

    $$\langle \{\text{Home}\}\, \{\text{Electronics}\}\, \{\text{Cameras}\}\, \{\text{Cart}\}\, \{\text{Checkout}\} \rangle$$

  - *Library Loans*:

    $$\langle \{\text{Book}_1\}\, \{\text{Book}_2\}\, \{\text{Book}_3\} \rangle$$

    Length = 3, 3-sequence.
- **Complex Element Sequences ($\exists j : |e_j| > 1$)**: Multiple items occur concurrently at one or more time steps, resulting in $k > |s|$.
  - *Industrial Incident*: $\langle \{\text{resin, valve}\}\, \{\text{pump}\}\, \{\text{turbine, pressure}\} \rangle$ (Length = 3, 5-sequence).

12.5 Containment Relationship and Subsequence Definition

The fundamental operation for frequency counting and sequential matching is the algebraic concept of a **subsequence** [2, 16].

Definition 12.5.1 (Subsequence / Containment)**.** Let a and b be two arbitrary sequences:

$$a = \langle a_1, a_2, \ldots, a_n \rangle \quad \text{and} \quad b = \langle b_1, b_2, \ldots, b_m \rangle \quad \text{with } m \geq n \tag{12.18}$$

We say that a is a **subsequence** of b (or that b contains a, formally denoted by $a \subseteq b$) if and only if there exists a strictly increasing index mapping of length n:

$$1 \leq i_1 < i_2 < \cdots < i_n \leq m \tag{12.19}$$

such that every element of a is a subset of the matched element in b:

$$a_1 \subseteq b_{i_1}, \quad a_2 \subseteq b_{i_2}, \quad \ldots, \quad a_n \subseteq b_{i_n} \tag{12.20}$$

If $a \subseteq b$, then b is called a **supersequence** of a, or b is said to support a.

Crucial Property: Non-Contiguity in Time

Elements of the candidate subsequence a **need not appear contiguously** inside b. Arbitrary intervening transactions may occur between b_{i_j} and $b_{i_{j+1}}$, provided that relative chronological order is strictly preserved $(i_1 < i_2 < \cdots < i_n)$.

12.5.1 Visual Subsequence Matching Example

Consider candidate subsequence a and data sequence b:

$$a = \langle \{A\}\, \{B, C\}\, \{D\} \rangle \quad (n = 3) \tag{12.21}$$

$$b = \langle \{D\}_{t=0}\, \{A, C\}_{t=1}\, \{A, B, C\}_{t=2}\, \{F\}_{t=3}\, \{B, E\}_{t=4}\, \{D\}_{t=5} \rangle \quad (m = 6) \tag{12.22}$$

Verifying $a \subseteq b$ through strictly increasing indices:

- For $a_1 = \{A\}$: at $t = 1$, $\{A\} \subseteq \{A, C\} = b_1 \implies i_1 = 1$;
- For $a_2 = \{B, C\}$: at $t = 2$, $\{B, C\} \subseteq \{A, B, C\} = b_2 \implies i_2 = 2$;
- For $a_3 = \{D\}$: at $t = 5$, $\{D\} \subseteq \{D\} = b_5 \implies i_3 = 5$.

Because $1 < 2 < 5$, the containment condition $a \subseteq b$ holds. Figure 12.3 illustrates this alignment.

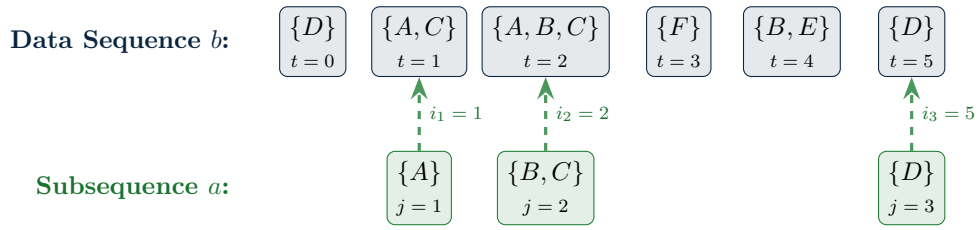


Figure 12.3: Alignment between candidate subsequence a and sequence b with indices $i_1 = 1 < i_2 = 2 < i_3 = 5$.

12.5.2 Containment Edge Cases and Counterexamples

To consolidate understanding of Definition 12.19, examine the three cases in Table 12.4.

Data Sequence (b)	**Candidate Subsequence** (a)	**Contained?**	**Analytical Justification**
$\langle\{2,4\}\,\{3,5,6\}\,\{8\}\rangle$	$\langle\{2\}\,\{3,5\}\rangle$	**YES**	$\{2\} \subseteq \{2,4\}$ at index 1; $\{3,5\} \subseteq \{3,5,6\}$ at index 2. Index ordering $i_1 = 1 < i_2 = 2$ holds.
$\langle\{1,2\}\,\{3,4\}\rangle$	$\langle\{1\}\,\{2\}\rangle$	**NO**	Items 1 and 2 appear in the **same** element b_1. Subsequence a strictly mandates 2 occurring **after** 1 ($i_1 < i_2$). Because no subsequent element contains 2, containment fails.
$\langle\{2,4\}\,\{2,4\}\,\{2,5\}\rangle$	$\langle\{2\}\,\{4\}\rangle$	**YES**	$\{2\} \subseteq b_1$ at index 1; $\{4\} \subseteq b_2$ at index 2. Index ordering $i_1 = 1 < i_2 = 2$ holds.

Table 12.4: Subsequence containment evaluation on sample sequences.

Difference Between $\{1, 2\}$ and $\langle\{1\}\{2\}\rangle$

The second row in Table 12.4 highlights the essential semantic boundary between concurrent co-occurrence and chronological ordering:

- If two items are purchased in the same basket, they form element $\{1, 2\}$;
- Sequence $\langle\{1\}\{2\}\rangle$ explicitly requires item 1 to precede item 2 ($t(1) < t(2)$).

Thus, $\{1, 2\}$ neither matches nor supports $\langle\{1\}\{2\}\rangle$.

12.6 Sequential Pattern Support and Multiple Instances

We formally define support counting for candidate sequences over a sequence database [2, 16].

12.6.1 Definition of Sequential Pattern Support

Definition 12.6.1 (Sequence Support)**.** Let $D = \{s_1, s_2, \ldots, s_{|D|}\}$ be a sequence database. The **support** (relative frequency) of a candidate sequence w is defined as the fraction of sequences in D that contain w as a subsequence:

$$\sigma(w) = \frac{\left|\{s \in D \mid w \subseteq s\}\right|}{|D|} \tag{12.23}$$

Definition 12.6.2 (Sequential Pattern)**.** A sequence w is declared a **Sequential Pattern** (or frequent sequence) if and only if its support meets or exceeds a user-specified minimum threshold:

$$\sigma(w) \geq \text{minsup} \tag{12.24}$$

Example 12.6.1 (Support Calculation Across 4 Sequences)**.** Consider candidate sequence:

$$w = \langle\{A\}\{B, C\}\{D\}\rangle \tag{12.25}$$

and database D comprising 4 customer sequences:

$$\begin{aligned}
s_1 &= \langle\{D\}\{A, C\}\{A, B, C\}\{F\}\{D\}\rangle \\
s_2 &= \langle\{D\}\{A\}\{A, B, D\}\{D\}\rangle \\
s_3 &= \langle\{D\}\{A, C\}\{B, C\}\{D\}\{B, E\}\rangle \\
s_4 &= \langle\{D\}\{B, C\}\{D\}\{F\}\rangle
\end{aligned}$$

Verifying $w \subseteq s_k$:

1. **Sequence** s_1: $\{A\} \subseteq \{A, C\}$ at $t = 1$, $\{B, C\} \subseteq \{A, B, C\}$ at $t = 2$, $\{D\} \subseteq \{D\}$ at $t = 4$. Since $1 < 2 < 4 \implies w \subseteq s_1$ **(MATCH, ✓)**.
2. **Sequence** s_2: Element $\{A, B, D\}$ lacks item C. Itemset $\{B, C\}$ appears nowhere in s_2. Thus $w \not\subseteq s_2$ **(NO MATCH, ×)**.
3. **Sequence** s_3: $\{A\} \subseteq \{A, C\}$ at $t = 1$, $\{B, C\} \subseteq \{B, C\}$ at $t = 2$, $\{D\} \subseteq \{D\}$ at $t = 3$. Since $1 < 2 < 3 \implies w \subseteq s_3$ **(MATCH, ✓)**.
4. **Sequence** s_4: Element $\{B, C\}$ is preceded solely by $\{D\}$ at $t = 0$. Item A does not appear prior to $\{B, C\}$. Thus $w \not\subseteq s_4$ **(NO MATCH, ×)**.

Support evaluation:

$$\sigma(w) = \frac{|\{s_1, s_3\}|}{|D|} = \frac{2}{4} = 0.50 \quad (50\%) \tag{12.26}$$

If minsup $\leq 50\%$, w is classified as a frequent sequential pattern.

12.6.2 Concept of Instance (Embedding) and Counting Convention

A candidate sequence w may map onto the same data sequence s via multiple distinct timestamp combinations.

Definition 12.6.3 (Instance / Embedding)**.** Given candidate subsequence $w = \langle w_1, \ldots, w_n \rangle$ and data sequence $s = \langle s_0, \ldots, s_{m-1} \rangle$, each ordered tuple of indices $\langle i_1, i_2, \ldots, i_n \rangle$ satisfying:

$$i_1 < i_2 < \cdots < i_n \quad \text{with } w_j \subseteq s_{i_j} \ \ \forall j \in \{1, \ldots, n\} \tag{12.27}$$

is defined as a distinct **instance** (or **embedding**) of w in s.

Example 12.6.2 (Multiple Instance Analysis)**.** Consider candidate pattern:

$$w = \langle \{A\}\, \{B\}\, \{D\} \rangle$$

and data sequence:

$$\begin{aligned} s = \langle &\{D\}_{t=0}\, \{A, C\}_{t=1}\, \{A, B, C\}_{t=2} \\ &\{F\}_{t=3}\, \{B, E\}_{t=4}\, \{D\}_{t=5} \rangle \end{aligned} \tag{12.28}$$

This sequence yields three distinct valid matching embeddings:

1. **Instance 1**: $\langle i_1 = 1, i_2 = 2, i_3 = 5 \rangle$ ($\{A\} \subseteq s_1$, $\{B\} \subseteq s_2$, $\{D\} \subseteq s_5$)
2. **Instance 2**: $\langle i_1 = 1, i_2 = 4, i_3 = 5 \rangle$ ($\{A\} \subseteq s_1$, $\{B\} \subseteq s_4$, $\{D\} \subseteq s_5$)
3. **Instance 3**: $\langle i_1 = 2, i_2 = 4, i_3 = 5 \rangle$ ($\{A\} \subseteq s_2$, $\{B\} \subseteq s_4$, $\{D\} \subseteq s_5$)

Standard Support Counting Convention

In standard Sequential Pattern Mining:

Each data sequence contributes a binary count increment of $+\mathbf{1}$

regardless of the number of valid internal embeddings it contains. Discovering **at least one valid embedding** is sufficient to count the sequence toward the numerator of $\sigma(w)$.

12.7 Guided Exercise: Full Matching and Instance Enumeration

We carry out a complete analytical walkthrough verifying candidate patterns and enumerating all distinct instances.

12.7.1 Problem Statement

Given the 8-element data sequence s spanning $t = 0$ to $t = 7$:

$$\begin{aligned} s = \langle &\{A, C\}_{t=0}\, \{C, D\}_{t=1}\, \{F, H\}_{t=2}\, \{A, B\}_{t=3} \\ &\{B, C, D\}_{t=4}\, \{E\}_{t=5}\, \{A, B, D\}_{t=6}\, \{F\}_{t=7} \rangle \end{aligned} \tag{12.29}$$

Determine whether the following four candidate patterns are contained in s, and if so, enumerate all distinct temporal instances $\langle t_1, \ldots, t_n \rangle$:

1. $P_1 = \langle \{C\}\, \{H\}\, \{C\} \rangle$
2. $P_2 = \langle \{A\}\, \{F\} \rangle$
3. $P_3 = \langle \{A\}\, \{A\}\, \{D\} \rangle$
4. $P_4 = \langle \{A\}\, \{A, B\}\, \{F\} \rangle$

12.7.2 Detailed Analytical Solution

Pattern 1: $P_1 = \langle \{C\}\, \{H\}\, \{C\} \rangle$

Requires identifying timestamps $t_1 < t_2 < t_3$ such that:

$$C \in s_{t_1}, \quad H \in s_{t_2}, \quad C \in s_{t_3}$$

- Item H is only present at $t = 2$ ($s_2 = \{F, H\}$). Thus $t_2 = 2$ is fixed.
- Occurrences of C prior to $t = 2$:
  - $t_1 = 0$: $C \in s_0 = \{A, C\}$;
  - $t_1 = 1$: $C \in s_1 = \{C, D\}$.
- Occurrences of C following $t = 2$:
  - At $t = 4$: $C \in s_4 = \{B, C, D\}$. In subsequent steps $(5, 6, 7)$ item C does not recur. Thus $t_3 = 4$.

Combining valid timestamps yields **2 distinct instances**:

1. $\langle t_1 = 0,\, t_2 = 2,\, t_3 = 4 \rangle$
2. $\langle t_1 = 1,\, t_2 = 2,\, t_3 = 4 \rangle$

Result: Pattern P_1 **OCCURS** (supported with 2 instances).

Pattern 2: $P_2 = \langle \{A\}\, \{F\} \rangle$

Requires finding timestamps $t_1 < t_2$ such that:

$$A \in s_{t_1} \quad \text{and} \quad F \in s_{t_2}$$

- Item A occurs at: $t \in \{0, 3, 6\}$ ($s_0 = \{A, C\}$, $s_3 = \{A, B\}$, $s_6 = \{A, B, D\}$).
- Item F occurs at: $t \in \{2, 7\}$ ($s_2 = \{F, H\}$, $s_7 = \{F\}$).

Evaluating pairs (t_1, t_2) with $t_1 < t_2$:

- For $t_1 = 0$: valid $t_2 > 0$ are $t_2 = 2$ and $t_2 = 7 \implies \langle 0, 2 \rangle, \langle 0, 7 \rangle$.
- For $t_1 = 3$: valid $t_2 > 3$ is $t_2 = 7 \implies \langle 3, 7 \rangle$.
- For $t_1 = 6$: valid $t_2 > 6$ is $t_2 = 7 \implies \langle 6, 7 \rangle$.

This produces **4 distinct instances**:

$$\langle 0, 2 \rangle, \quad \langle 0, 7 \rangle, \quad \langle 3, 7 \rangle, \quad \langle 6, 7 \rangle \tag{12.30}$$

Result: Pattern P_2 **OCCURS** (supported with 4 instances).

Pattern 3: $P_3 = \langle \{A\}\, \{A\}\, \{D\} \rangle$

Requires timestamps $t_1 < t_2 < t_3$ with:

$$A \in s_{t_1}, \quad A \in s_{t_2}, \quad D \in s_{t_3}$$

- Candidate pairs (t_1, t_2) for A: $(0, 3)$, $(0, 6)$, $(3, 6)$.
- Item D occurs at timestamps: $t \in \{1, 4, 6\}$ ($s_1 = \{C, D\}$, $s_4 = \{B, C, D\}$, $s_6 = \{A, B, D\}$).
- Testing each (t_1, t_2) pair:
  - For $(0, 3)$: require $t_3 > 3$ with $D \in s_{t_3}$. Valid timestamps are $t_3 = 4$ and $t_3 = 6$. Yields instances $\langle 0, 3, 4 \rangle$ and $\langle 0, 3, 6 \rangle$.
  - For $(0, 6)$: requires $t_3 > 6$ with $D \in s_{t_3}$. At $t = 7$, only $\{F\}$ is present; no D occurs after $t = 6$. No valid instance.
  - For $(3, 6)$: requires $t_3 > 6$, which fails for the same reason.

Yields **2 distinct instances**:

$$\langle 0, 3, 4 \rangle \quad \text{and} \quad \langle 0, 3, 6 \rangle \tag{12.31}$$

Result: Pattern P_3 **OCCURS** (supported with 2 instances).

Pattern 4: $P_4 = \langle \{A\}\, \{A, B\}\, \{F\} \rangle$

Requires timestamps $t_1 < t_2 < t_3$ such that:

$$A \in s_{t_1}, \quad \{A, B\} \subseteq s_{t_2}, \quad F \in s_{t_3}$$

- The joint set $\{A, B\}$ appears in:
  - $s_3 = \{A, B\} \implies t_2 = 3$;
  - $s_6 = \{A, B, D\} \implies t_2 = 6$.
- **Case** $t_2 = 3$:
  - For $t_1 < 3$ with $A \in s_{t_1}$: unique choice is $t_1 = 0$ ($s_0 = \{A, C\}$).
  - For $t_3 > 3$ with $F \in s_{t_3}$: unique choice is $t_3 = 7$ ($s_7 = \{F\}$).
  - Resulting instance: $\langle 0, 3, 7 \rangle$.
- **Case** $t_2 = 6$:
  - For $t_1 < 6$ with $A \in s_{t_1}$: choices are $t_1 = 0$ or $t_1 = 3$.

– For $t_3 > 6$ with $F \in s_{t_3}$: unique choice is $t_3 = 7$.
– Resulting instances: $\langle 0, 6, 7\rangle$ and $\langle 3, 6, 7\rangle$.

Yields **3 distinct instances**:

$$\langle 0, 3, 7\rangle, \quad \langle 0, 6, 7\rangle, \quad \langle 3, 6, 7\rangle \tag{12.32}$$

Result: Pattern P_4 **OCCURS** (supported with 3 instances).

12.7.3 Summary of Exercise Solutions

Table 12.5 summarizes the results for sequence s.

Pattern	**Structural Form**	**Result**	**Temporal Instances** $\langle t_1, \ldots, t_n\rangle$
P_1	$\langle\{C\}\{H\}\{C\}\rangle$	**OCCURS**	$\langle 0, 2, 4\rangle$, $\langle 1, 2, 4\rangle$ (2 instances)
P_2	$\langle\{A\}\{F\}\rangle$	**OCCURS**	$\langle 0, 2\rangle$, $\langle 0, 7\rangle$, $\langle 3, 7\rangle$, $\langle 6, 7\rangle$ (4 instances)
P_3	$\langle\{A\}\{A\}\{D\}\rangle$	**OCCURS**	$\langle 0, 3, 4\rangle$, $\langle 0, 3, 6\rangle$ (2 instances)
P_4	$\langle\{A\}\{A, B\}\{F\}\rangle$	**OCCURS**	$\langle 0, 3, 7\rangle$, $\langle 0, 6, 7\rangle$, $\langle 3, 6, 7\rangle$ (3 instances)

Table 12.5: Summary of matching results and instances for the four candidate patterns.

Methodological Takeaways

The analytical walkthrough verifies that:

1. Individual elements in a sequence pattern may contain single or multiple simultaneous items;
2. Temporal precedence strictly requires increasing index mappings ($t_1 < t_2 < \cdots < t_n$);
3. Identical items may repeat at distinct subsequent time steps within the same sequence;
4. Under standard Sequential Pattern Mining, each matched sequence contributes +1 to support, regardless of how many valid internal embeddings exist.

Bibliography

[1] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231. AAAI Press, 1996.

[2] Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, and Vipin Kumar. *Introduction to Data Mining*. Pearson, 2nd edition, 2018.

[3] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel, and Jörg Sander. OPTICS: Ordering points to identify the clustering structure. *ACM SIGMOD Record*, 28(2):49–60, 1999.

[4] Ricardo J. G. B. Campello, Davoud Moulavi, Arthur Zimek, and Jörg Sander. Hierarchical density estimates for data clustering, visualization, and outlier detection. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 10(1):5:1–5:51, 2015.

[5] Dan Pelleg and Andrew W. Moore. X-means: Extending K-means with efficient estimation of the number of clusters. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000)*, pages 727–734. Morgan Kaufmann, 2000.

[6] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.

[7] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–38, 1977.

[8] Anil K. Jain and Richard C. Dubes. *Algorithms for Clustering Data*. Prentice-Hall, 1988.

[9] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[10] Brian Hopkins and J. G. Skellam. A new method for determining the type of distribution of plant individuals. *Annals of Botany*, 18(2):213–227, 1954.

[11] David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.

[12] T. Caliński and J. Harabasz. A dendrite method for cluster analysis. *Communications in Statistics*, 3(1):1–27, 1974.

[13] Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. Mining association rules between sets of items in large databases. *ACM SIGMOD Record*, 22(2):207–216, 1993.

[14] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94)*, pages 487–499. Morgan Kaufmann, 1994.

[15] Gregory Piatetsky-Shapiro. Discovery, analysis, and presentation of strong rules. In *Knowledge Discovery in Databases*, pages 229–248. AAAI/MIT Press, 1991.

[16] Rakesh Agrawal and Ramakrishnan Srikant. Mining sequential patterns. In *Proceedings of the Eleventh International Conference on Data Engineering (ICDE '95)*, pages 3–14. IEEE Computer Society, 1995.

[17] Ramakrishnan Srikant and Rakesh Agrawal. Mining sequential patterns: Generalizations and performance improvements. In *Proceedings of the 5th International Conference on Extending Database Technology (EDBT '96)*, pages 3–17. Springer, 1996.